

Modul 13 — Moderne Datenbankkonzepte: Praktische Umsetzung (SCA)

Studiengang: DHBW Ravensburg · WDS224 · Data Science & Künstliche Intelligenz **Modul:** *Datenbanken 2* — Moderne Datenbankkonzepte, Praktische Umsetzung **Dozent:** **Andreas Scheub** (Dipl.-Betriebswirt (BA) Wirtschaftsinformatik, Dipl.-Wirtschaftsingenieur (FH); bis 12/2025 Product Owner „Master data and cross-functions“, Mercedes-Benz Group AG) **Stand der Folien:** © 2023–2026 Andreas Scheub · Vorlesungstermine 02./06.03.2026 **Quelle:** ZIP WDS224 - Moderne Datenbankkonzepte -Praktische Umsetzung (SCA).zip **Schwestermodul:** → 14_Moderne-Datenbankkonzepte-Theoretisch.md (BER)

0. Die vier Schwerpunkte der Vorlesung ★

#	Schwerpunkt	Unterthemen
1	Grundlagen	DBMS, 3-Schichten-Modell, ERM, relationales Modell
2	SQL	DDL, DML · DQL · sonstige Themen (View, Index, Rechte, Systemkatalog)
3	DB-Programmierung	DB-intern (Functions, Procedures, Trigger, Transaktionen) · Java (JDBC) , REST, JPA/ORM
4	Nicht-relationale Datenbankkonzepte	Erweiterungen RDBMS, verteilte DB, NoSQL (Key-Value, Document, Wide-Column, Graph)

0.1 Die Materialien im Archiv

Datei	Seite n	Inhalt
1 - Datenbanken 2 - Einführung.pdf	20	Grundbegriffe, ANSI-SPARC, ERM, Jobprofile
2 - Datenbanken 2 - SQL Teil 1 DDL DML.pdf	61	PostgreSQL/DBEaver, Datentypen, Constraints, referentielle Integrität, DML
3 - Datenbanken 2 - SQL Teil 2 DQL.pdf	70	SELECT, Aggregate, GROUP BY/HAVING, Subqueries, Joins, Mengenoperationen
4 - Datenbanken 2 - SQL Teil 3 Zusatzthemen.pdf	24	VIEW, INDEX, Rechteverwaltung, Systemkatalog
5 - Datenbanken 2 - Programmierung DB-seitig und Transaktionen.pdf	73	Functions, Procedures, Trigger, ACID, Isolation Level, 2PL/2PC
6 - Datenbanken 2 - Programmierung JDBC und JPA.pdf	67	JDBC, Prepared Statements, REST/3-Schichten, ORM/JPA
7 - Datenbanken 2 - Erweiterungen RDBMS und verteilte DB.pdf	47	OLAP/Cube, ORDBMS, SAP HANA, verteilte DB, CAP, BASE
8 - Datenbanken 2 - NoSQL Datenbanken.pdf	123	Redis, MongoDB, Cassandra, Neo4j, Polystores, Historie
Übungen (12 Dateien)		Chinook-Aufgaben mit vollständigen Musterlösungen
Beispiele Klausuraufgaben.pdf	7	★ Die Klausur-Aufgabentypen
SQL Befehlsübersicht (Auswahl) V1.0.pdf	5	Referenzkarte

0.2 ★★ ★ DIE KLAUSUR — Aufgabentypen (aus Beispiele Klausuraufgaben.pdf)

„exemplarisch, kein Indikator für Schwierigkeitsgrad, nicht vollständig!“

Block 1: SQL

Ty p	Aufgabenstellung
Ty p 1	Relationenschema gegeben, Schwerpunkt DDL — z. B.: „Schreiben Sie einen SQL-Befehl für das Erstellen der Tabelle <code>album</code> mit allen Integritätsbedingungen (Hinweis: <code>artist_id</code> ist ein Fremdschlüssel, <code>album_id</code> der Primärschlüssel).“ Weitere Varianten: Hinzufügen/Änderung von Integritätsbedingungen · View erstellen · Verständnisfragen : „Kann folgender Datensatz eingefügt werden ...?“
Ty p 2	Tabelle mit Inhalt gegeben — Ergebnis der SQL-Abfrage ermitteln (⚠ das Ergebnis ist hinzuschreiben!)
Ty p 3	Multiple Choice — „Es können mehrere Antworten richtig sein.“
Ty p 4	Abfragen erstellen — Relationenschema gegeben, SQL-Befehle für fachliche Anforderungen formulieren

Original-Beispiel für Typ 2:

Gegeben ist eine Tabelle `xcustomer`. SQL-Befehl:

```
SELECT country, count(country)
FROM xcustomer
GROUP BY country
HAVING count(country) > 1;
```

Lösung (ist hinzuschreiben):

country	count
Canada	2
Brazil	2
USA	2

Original-Beispiel für Typ 3 (Multiple Choice):

Welchen Hauptzweck erfüllt ein Index in einer relationalen Datenbanktabelle? ☐ Er stellt sicher, dass alle Werte in einer Spalte eindeutig sind. ☐ Er definiert die Struktur der Tabelle und die Datentypen der Spalten. ☒ **Er beschleunigt Such- und Abfragevorgänge, indem er einen schnelleren Zugriff auf die Daten ermöglicht.** ☐ Er erzwingt die referentielle Integrität zwischen zwei Tabellen.

Original-Beispiel für Typ 4:

„Finde die Titel aller Alben (aus der Tabelle `album`), die von dem Künstler mit dem Namen 'Metallica' (name aus der Tabelle `artist`) stammen.“

Block 2: Functions und Trigger

Typ p	Aufgabenstellung
Typ p 1	Funktion und Tabelle sind gegeben — was ist das Ergebnis eines SELECT mit dieser Funktion? z. B. <code>SELECT count(*) AS Gesamtanzahl, anzahl(10,100) AS imBereich FROM invoice;</code>
Typ p 2	Function und/oder Trigger schreiben bzw. ergänzen Original-Beispiel: „Erstellen Sie einen Trigger und eine Trigger-Funktion: Beim Einfügen eines neuen Genre in die Tabelle <code>genre</code> soll geprüft werden, ob der <code>name</code> belegt ist. Wenn der <code>name</code> NULL ist, soll er automatisch auf den Text <code>'Untitled Genre'</code> gesetzt werden.“
Typ p 3	Eventuell Multiple-Choice-Fragen

Block 3: JDBC

Typ	Aufgabenstellung
Typ 1	Java-Programmrumpf ist gegeben — Code zwischen START und ENDE ergänzen oder „Schlagen Sie eine andere Implementierung vor“
Typ 2	Eventuell Multiple-Choice-Fragen

★ Der Programmrumpf, der in der Klausur gestellt wird — auswendig kennen!

```
import java.util.Properties;
import java.sql.DriverManager;
import java.sql.Connection;
import java.sql.Statement;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.DatabaseMetaData;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse = "jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url, props);

        // START Code zur Lösung der Aufgaben a) und b)

        // ENDE: Code zur Lösung der Aufgaben a) und b)
        // Freigabe der Ressourcen
        conn.close();
        st.close();
        rs.close();
    }
}
```

Block 4: Andere Datenbankkonzepte

Typ	Aufgabenstellung
Typ 1	Praktische Aufgaben — werden in/nach der Vorlesung besprochen
Typ 2	Wissensfragen , z. B. zu BASE, ACID ...
Typ 3	Eventuell Multiple-Choice-Fragen

🔑 **Konsequenz für die Vorbereitung:** Man muss **SQL aktiv schreiben** können (nicht nur lesen), **Abfrageergebnisse im Kopf ermitteln** können, **Functions/Trigger von Hand formulieren** und **JDBC-Code ergänzen**. Reines Auswendiglernen genügt nicht.

Legende dieses Dokuments:

Symbol	Bedeutung
★	Prüfungsrelevant
⚠	Klassische Falle
📝	Beispiel / durchgerechnete Aufgabe
🔑	Kernaussage
?	Frage aus den Folien

TEIL A — GRUNDLAGEN

1. Daten, Information, Wissen ★

Begriff	Definition	Beispiel
Daten	isolierte und uninterpretierte Fakten und Kennwerte der Realitätsbeschreibung	Name = Meier, Straße = Hauptstraße 12, Ort = 72076 Tübingen
Informationen	verknüpfte und mit Bedeutung versehene Daten (Daten mit ihrer semantischen Bedeutung)	<i>Herr Meier wohnt in der Hauptstraße 12 in 72076 Tübingen.</i>
Wissen	Informationen, die individuell verarbeitet sind und einen Bezug zur eigenen Erfahrungswelt erhalten haben; die sinnvolle Verknüpfung von Informationen und die In-Beziehung-Setzung zu Erfahrungen	<i>Mein ehemaliger Schulfreund Peter Meier wohnt auch hier in Tübingen...</i>

2. Datenbank, DBMS, Datenbanksystem ★★

Begriff	Definition	Beispiel
Datenbank	eine logische Einheit zusammengehörender Daten und sämtlicher Meta-Informationen dazu	eine PostgreSQL- Datenbank
Datenbank-Management-System (DBMS)	stellt Werkzeuge bereit, mit welchen eine oder mehrere Datenbanken erstellt, mit Daten gefüllt und verwaltet werden können	die PostgreSQL- Installation auf einem Server
Datenbanksystem (DBS)	die funktionale Einheit aus der logischen Datenmenge (Datenbank) und der verwaltenden Software (DBMS)	PostgreSQL-Installation + PostgreSQL-Datenbank

★ Merkformel: DBS = DB + DBMS

2.1 Die zentralen Systemaufgaben des DBMS ★★

Aufgabe	Erläuterung
Exklusiver Datenzugriff	Es ist die einzige Komponente mit direktem Zugriff auf den physischen Speicher
Garant für Konsistenz & Integrität	Stellt sicher, dass Daten jederzeit korrekt, widerspruchsfrei und regelkonform bleiben
Zentrale Zugriffskontrolle („Türsteher“)	Verwaltet Berechtigungen und verhindert unautorisierte Zugriffe oder Systemkonflikte
Transparente Hintergrundprozesse	Optimierung, Backup, Logging laufen für den Anwender unsichtbar ab, sind aber essenziell

2.2 Anforderungen an ein DBMS ★★

Eine sichere Klausurfrage — die sechs Punkte auswendig:

1. Bereitstellung von Datenbanksprachen

- **DDL (Data Definition Language):** zur **Definition und Änderung der Datenstrukturen**
- **DML (Data Manipulation Language):** zur **Abfrage, Einfügung und Modifikation** von Daten

2. Sicherheit & Autorisierung: zentrale Zugangskontrolle (Wer darf welche Daten lesen oder verändern?)

3. Datenintegrität: Sicherstellung der **Korrektheit und Widerspruchsfreiheit** (semantische Integrität)

4. Robustheit & Recovery: Erhalt eines **konsistenten Zustands** auch bei Systemabstürzen oder Hardwarefehlern

5. Mehrbenutzer-Synchronisation: Koordination gleichzeitiger Zugriffe (Stichwort: **ACID**)

6. Effizienz: optimierter physischer Datenzugriff für hohe Performance

3. Das 3-Schichten-Modell (ANSI-SPARC, 1975) ★★★

EXTERNE SCHICHT
legt fest, **WELCHE DATEN** bestimmte Benutzer bzw. Programme **SEHEN** und **BEARBEITEN** können
→ Sichten (VIEWS), Benutzerschemata

LOGISCHE (konzeptionelle) SCHICHT
Gesamtstruktur der Datenbank
→ **UNABHÄNGIG** von der physischen Implementierung ★

PHYSISCHES (interne) SCHICHT
legt fest, welche **ZUGRIFFSMECHANISMEN** und **SPEICHERSTRUKTUREN** verwendet werden
→ Indizes, Dateiorganisation, Blöcke

🔑 **Die Kernaussage:** „Die **logische Schicht** ist **unabhängig von der physischen Implementierung**.“ Genau das ist die **physische Datenunabhängigkeit** — SQL-Abfragen funktionieren unverändert weiter, wenn sich die Speicherorganisation ändert.

4. Konzeptionelles Datenmodell und das ER-Modell ★★

4.1 Zielsetzung

- **Schaffung eines allgemeingültigen Verständnisses** der Datenobjekte und ihrer Beziehungen (**Abbild der Realität**)
- **Technologieunabhängigkeit:** Fokus rein auf den **Geschäftsprozessen** — ohne Bezug auf spezifische Datenbank-Software oder Hardware

4.2 Das Entity-Relationship-Modell (ERM) ★★

- **1976 von Peter Chen** eingeführt
- Ermöglicht die **grafische Darstellung abstrakter Datenstrukturen**
- Dient als **universelles Kommunikationsmittel** zwischen Anwendern, Analysten und Entwicklern
- **Weiterentwicklung:** moderne **UML-Klassendiagramme** greifen die Grundkonzepte des ERM auf und erweitern sie für die objektorientierte Modellierung

Die drei Grundelemente des ER-Modells (Chen-Notation) ★:

Element	Definition	Beispiele
Entitäten	abstrakte oder reale Objekte , die in der Datenbank gespeichert werden	Kunden, Produkte, Aufträge
Beziehungen	beschreiben, wie Entitäten miteinander verbunden sind	ein Kunde, der eine Bestellung aufgibt; ein Produkt, das von einem Hersteller produziert wird
Attribute	Eigenschaften von Entitäten	Name eines Kunden, Preis eines Produkts

5. Das relationale Datenmodell ★★★★★

5.1 Meilensteine der Entwicklung

Jahr	Ereignis
1970	Edgar Frank Codd veröffentlicht das theoretische Fundament: „ <i>A Relational Model of Data for Large Shared Data Banks</i> “
—	System R: erste praktische Implementierung im IBM-Forschungslabor
—	SEQUEL zu SQL: Entwicklung der Structured English QUery Language , Basis des heutigen ANSI-Standards SQL
1980er	Das relationale Modell wird zum „ Industriestandard “ der strukturierten Datenhaltung

5.2 Charakteristika von SQL ★★

Eigenschaft	Erläuterung
Deklarative Abfragesprache	Der Fokus liegt auf dem „ WAS “ (Ergebnismenge) statt auf dem „ WIE “ (algorithmischer Weg)
Physische Datenunabhängigkeit	SQL-Abfragen sind unabhängig von der konkreten Speicherorganisation (konform zum 3-Schichten-Modell)

5.3 Grundbausteine ★★★★★

Begriff	Synonym	Definition
Tabelle	Relation	die fundamentale Struktur zur Speicherung von Daten
Tupel	Datensatz, Zeile	eine Zeile der Tabelle; repräsentiert eine konkrete Instanz
Attribut	Merkmal, Spalte	die Spalten der Tabelle; definieren die Eigenschaften eines Tupels durch vordefinierte Wertebereiche

Stärken:

- **Einfachheit & Abstraktion:** Sämtliche Objekte und Beziehungen der realen Welt werden **einheitlich in Tabellenform** dargestellt
- Die **starre Definition von Spalten** sorgt für **hohe Datenqualität und Vorhersehbarkeit**

5.4 Die vier Eigenschaften einer Relation ★★★★★

Eine **Relation** ist eine Tabelle, bestehend aus einem **Kopf** und einem **Rumpf**, mit folgenden **vier Eigenschaften** (nach E. Schicker, „Datenbanken und SQL“, S. 26):

1. **Es gibt keine doppelten Tupel**
2. **Tupel sind nicht geordnet**
3. **Attribute sind nicht geordnet**
4. **Alle Attribute sind atomar**

Zusätzlich:

- Die **Eindeutigkeit des Primärschlüssels** wird **zwingend gefordert**
- Einzelne Einträge sind **nicht willkürlich wählbar**, sondern werden aus einer Menge gültiger Werte entnommen — der **Definitionsmenge**

⚠ Wichtige Konsequenz für Abfragen (Folie zum DISTINCT):

Eine SQL-Abfrage liefert **definitionsgemäß KEINE mathematische Menge (Set)**, sondern eine **MULTIMENGE (Bag)** zurück! Der Unterschied: Eine Multimenge **erlaubt das mehrfache Vorkommen identischer Elemente**. Dieses Standardverhalten (**implizites SELECT ALL**) ist auch aus **Performance-Gründen** gewählt, da es die zeitaufwendige **Duplikat-Eliminierung vermeidet**. Erst das explizite **DISTINCT** wandelt die Multimenge in eine mathematisch eindeutige Menge um.

6. Relationale Algebra ★★★★★

Die **Relationenalgebra** bildet den **formalen Rahmen** für relationale Datenbanksprachen. Sie umfasst einen **Satz algebraischer Operatoren**, die auf **Tabellen wirken und als Resultat eine Tabelle zurückgeben**.

Kategorie	Operatoren
Mengenorientierte Operatoren	Vereinigung, Durchschnitt, Differenz, Kartesisches Produkt
Relationenorientierte Operatoren	Projektion, Selektion, Division, Verbund

★★★★★ Die Zuordnung Relationenalgebra ↔ SQL — sichere Klausurfrage:

Operator der Relationenalgebra	SQL-Entsprechung	Was passiert?
Selektion (Restriktion)	WHERE	wählt bestimmte ZEILEN (Tupel) aus, die eine Bedingung erfüllen
Projektion	SELECT (Spaltenliste)	wählt bestimmte SPALTEN (Attribute) aus
Kartesisches Produkt	FROM t1, t2 bzw. CROSS JOIN	jede Zeile mit jeder kombiniert
Verbund (Join)	JOIN ... ON	Produkt mit Verknüpfungsbedingung
Vereinigung	UNION	Zeilen beider Ergebnisse
Durchschnitt	INTERSECT	Schnittmenge
Differenz	EXCEPT	Differenzmenge
Division	 kein eigenes Schlüsselwort!	—

 **Zur Division:** „Die Division der Relationenalgebra ist ein komplexeres Konzept, für das SQL kein eigenes Schlüsselwort besitzt. Sie wird in SQL typischerweise über **doppelt negierte Existenzquantoren (NOT EXISTS ... NOT EXISTS)** oder über **Gruppierung mit COUNT-Vergleichen** umgesetzt.“

 **Merkhilfe:** Selektion = Zeilen (horizontal schneiden) = WHERE · Projektion = Spalten (vertikal schneiden) = SELECT

7. SQL-Normierungen ★★

Standard	Jahr	Gremium	Wichtige Neuerungen
SQL-86 (SQL-1)	1986	ANSI , später ISO	Erster offizieller Standard, Basisfunktionen (SELECT, INSERT, UPDATE, DELETE)
SQL-89	1989	ANSI/ISO	Kleinere Erweiterungen, verbesserte Integritätsbedingungen
SQL-92 (SQL-2)	1992	ANSI/ISO	Sehr umfangreich : neue Datentypen, JOINS , Sichten, verbesserte Constraints; verpflichtend für Datenbankhersteller
SQL:1999 (SQL-3)	1999	ISO/IEC	Objektorientierte Erweiterungen , rekursive Abfragen (WITH RECURSIVE), Triggers , OLAP-Funktionen
SQL:2003	2003	ISO/IEC	XML-Unterstützung , Window Functions , BOOLEAN
SQL:2006	2006	ISO/IEC	Verbesserte XML-Funktionen, XQuery
SQL:2008	2008	ISO/IEC	MERGE , TRUNCATE , temporäre Tabellen
SQL:2011	2011	ISO/IEC	Temporal Tables (zeitbezogene Tabellen)
SQL:2016	2016	ISO/IEC	JSON-Datentypen , polymorphe Tabellenfunktionen
SQL:2019	2019	ISO/IEC	Verbesserte JSON-Unterstützung, Polymorphismus, analytische Abfragen
SQL:2023	2023	ISO/IEC	Neueste Version, Fokus auf moderne Datentypen (JSON)

★ Die drei Jahreszahlen, die man kennen sollte: 1986 (SQL-1) · 1992 (SQL-2 — JOINS, referentielle Integrität mit ON DELETE) · 1999 (SQL-3 — Trigger, OO, Rekursion)

8. Jobprofile im Umfeld Datenbank ★

Rolle	Aufgaben
Datenbankadministrator (DBA)	Installation, Betrieb, Wartung von DBMS; Performance-Tuning, Backup/Recovery, Benutzer- und Rechteverwaltung, Monitoring
Datenbankentwickler / Programmierer	Implementiert Datenbanklogik, Stored Procedures, Trigger, Funktionen ; entwickelt effiziente SQL-Abfragen
Datenbankdesigner / Data Architect	Modelliert Datenstrukturen (ER-Modelle, Normalisierung, Schema-Design); stellt Konsistenz, Integrität und Skalierbarkeit sicher
Datenintegrations-/ETL-Spezialist	Datenmigration, Schnittstellen, Extract-Transform-Load ; arbeitet mit Data Warehouses und BI-Systemen
Data-Warehouse-Architekt / BI-Entwickler	Analytische Systeme; Fokus auf Berichte, Dashboards, OLAP-Cubes
Data Engineer	Baut Datenpipelines , Speicherung und Verarbeitung großer Datenmengen; Hadoop, Spark, Cloud-Dienste
Data Scientist / Data Analyst	Nutzt Datenbanken für Analysen, Prognosen und Machine Learning ; SQL sowie Python, R
Sicherheits- und Compliance-Spezialist	Datenschutz (DSGVO) , Zugriffs- und Berechtigungskonzepte, Schutz vor Angriffen

★ Für Data-Science-Studierende relevant: **SQL ist die gemeinsame Sprache aller acht Rollen** — sie ist die Kernkompetenz, unabhängig davon, in welche Richtung man geht.

9. PostgreSQL und DBeaver ★

9.1 PostgreSQL — Historie

Jahr	Meilenstein
Mitte 1980er	Forschungsprojekt POSTGRES an der UC Berkeley unter Michael Stonebraker als Weiterentwicklung des Ingres-DBMS
1989	Fertigstellung der ersten Postgres-Version (erster lauffähiger Prototyp)
1994	Erweiterung um einen SQL-Interpreter , Freigabe als Postgres95 unter Open-Source-Lizenz
1996	Umbenennung in PostgreSQL (erste Version unter dem neuen Namen: 6.0)
seit 1997	kontinuierliche Weiterentwicklung durch eine weltweite Open-Source-Community

9.2 Technische Stärken ★

- **Hoch standardkonform** (nahe an ISO SQL)
- **Vollständige ACID-Transaktionen — inkl. DDL** ★ (eine Besonderheit!)
- **Extrem erweiterbar**: eigene **Datentypen, Funktionen und Operatoren**; Stored Procedures und vielfältige Prozedursprachen (**PL/pgSQL, Python, Java u. v. m.**)
- Native **JSON/JSONB-Datentypen** und **SQL/JSON-Funktionen** für semi-strukturierte Daten
- **Freie, BSD-ähnliche Lizenz** → **kein Vendor Lock-in**
- Hoch skalierbar: aktive Cluster verwalten **viele Terabytes bis Petabytes**
- Standard-Datenbank in **macOS seit Version 10.7**; **AWS, Google, Microsoft** engagieren sich aktiv in der Entwicklung
- Praxisbeispiele: **Finanzdaten-Analysen, Flugzeug- und Asteroiden-Tracking, medizinische Informationssysteme, Geodaten (PostGIS)**

9.3 DBeaver ★

- **Open-Source Universal-Datenbank-Client** (Java-basiert, **seit 2010**)
- Unterstützt u. a. **PostgreSQL, MySQL/MariaDB, Oracle, SQL Server, DB2, SQLite, MongoDB**
- **Community Edition** (kostenlos, Open Source) und **Enterprise Edition** (kommerziell)
- **Plattformunabhängig** (Windows, macOS, Linux)
- SQL-Editor mit **Syntax-Highlighting, Autovervollständigung, ER-Diagramme, Import/Export**

⚠️ ⚠️ **Wichtiger Hinweis des Dozenten:**

„Die Visualisierung der Datenbank in DBeaver wird zwar **als ER-Modell bezeichnet, ist aber ein RELATIONENSHEMA** (Beispiel: **m:n-Beziehungen sind als Tabellen dargestellt, Beziehungen werden mittels Fremdschlüssel abgebildet**).“

★ **Der Unterschied ER-Modell ↔ Relationenschema:**

	ER-Modell	Relationenschema
Ebene	konzeptionell	logisch
m:n-Beziehung	als Beziehung (Raute) dargestellt	als eigene Tabelle (Zwischentabelle)
Beziehungen	als Linien mit Kardinalität	über Fremdschlüssel
Beispiel Chinook	Playlist ↔ Track (m:n)	Tabelle <code>playlist_track</code>

TEIL B — SQL DATA DEFINITION LANGUAGE (DDL)

10. Die fünf SQL-Befehlskategorien ★★★

Eine sichere Klausurfrage.

Kategorie	Langform	Zweck	Befehle
DDL	Data Definition Language	Struktur der Datenbank definieren und verwalten	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Daten manipulieren	INSERT, UPDATE, DELETE
DQL	Data Query Language	Daten abfragen	SELECT
TCL	Transaction Control Language	Transaktionen steuern	COMMIT, ROLLBACK, SAVEPOINT, START TRANSACTION
DCL	Data Control Language	Zugriffsrechte verwalten	GRANT, REVOKE

⚠ In manchen Darstellungen wird DQL zur DML gezählt. Die Vorlesung führt sie **getrennt** auf — in der Klausur die Vorlesungssystematik verwenden.

11. SQL-Syntax und Konventionen ★★

11.1 Syntaxhinweise

- Anweisungen können über **mehrere Zeilen** verteilt werden; **Einrückungen sind sinnvoll** wegen der Lesbarkeit
- Das **Semikolon** zum Abschluss ist **wichtig**, insbesondere bei Eingabe im Terminal/DB-Frontend
- Kommentare** einzeilig (--) oder mehrzeilig (/* ... */) möglich
- String-Literale** werden in **einfache Hochkommas** gesetzt: 'Karl'
- Identifizierer** dürfen in **doppelte Anführungszeichen** gesetzt werden, wenn man **Groß-/Kleinschreibung erzwingen** oder **Sonderzeichen** verwenden will: "FirstName"
- ★ **SQL ist NICHT case-sensitiv** (aber Konventionen sind üblich)

11.2 Die Konventionen ★

Element	Konvention
SQL-Schlüsselwörter	GROSSSCHREIBUNG : SELECT, INSERT, UPDATE, DELETE, FROM, WHERE, JOIN
Tabellen- und Spaltennamen	kleingeschrieben oder CamelCase : employee, FirstName
Aliase	oft kleingeschrieben : e für employee
Formatierung	Einrücken für bessere Übersicht; jede SQL-Klausel in eigener Zeile

12. Schema und Tabelle erstellen ★★

12.1 Datenbank und Schema

Ein DBMS kann **mehrere Datenbanken** verwalten. Jede Datenbank ist eine **voneinander isolierte Umgebung** — erstellt mit CREATE DATABASE. Innerhalb jeder Datenbank können **Schemas** erstellt werden (z. B. public, employees, billing), um die Tabellen zu organisieren — erstellt mit CREATE SCHEMA.

```
-- Syntax
CREATE SCHEMA <Schemaname>;

-- Beispiel
CREATE SCHEMA employees;

-- Schema verwenden
SET SCHEMA 'employees';
```

★ Hierarchie: DATENBANK → SCHEMA → TABELLE → SPALTE

12.2 Tabelle erstellen ★★★

```
-- Syntax
CREATE TABLE <Tabellenname> (
    <Name> <Datentyp>,
    ...
    [PRIMARY KEY (<Spaltenname>)]
);

-- Beispiel
CREATE TABLE emp_employee (
    emp_id          INTEGER,
    emp_email       VARCHAR(50),
    emp_first_name  VARCHAR(50),
    emp_last_name   VARCHAR(50),
    PRIMARY KEY (emp_id)
);
```

12.3 ALTER und DROP ★★

```
-- LÖSCHEN
DROP SCHEMA <Schemaname>;
DROP TABLE <Tabellenname>;
DROP TABLE <Tabellenname> CASCADE;  -- inkl. abhängiger Objekte

-- ÄNDERN
ALTER SCHEMA <Schemaname> <Optionen>;
ALTER TABLE <Tabellenname> <Optionen>;
```

Die ALTER TABLE -Optionen ★★:

Option	Beispiel
Spalte hinzufügen	ALTER TABLE employee ADD COLUMN geburtsdatum DATE;
Spalte löschen	ALTER TABLE employee DROP COLUMN first_name;
Datentyp ändern	ALTER TABLE t ALTER COLUMN c TYPE VARCHAR(100);
Tabelle umbenennen	ALTER TABLE t1 RENAME TO t2;
Spalte umbenennen	ALTER TABLE t1 RENAME c1 TO c2;
Constraint hinzufügen	ALTER TABLE t ADD CONSTRAINT chk_value CHECK (c > 0);
Constraint löschen	ALTER TABLE t DROP CONSTRAINT chk_value;
DEFAULT setzen	ALTER TABLE t ALTER COLUMN c SET DEFAULT 'Hund';

🔑 Die drei DDL-Hauptbefehle:

- **CREATE** — Schemas, Tabellen **und alle weiteren DDL-Objekte** (USER, VIEWS, INDEX, CONSTRAINTS) anlegen. ⚠ **NICHT für das Einfügen konkreter Daten!**
- **ALTER** — DDL-Objekte **ändern**
- **DROP** — DDL-Objekte **löschen**

13. PostgreSQL-Datentypen ★★

13.1 Alphanumerische Datentypen (Text/Zeichenketten)

Typ	Beschreibung	Verwendung
CHAR(n)	Zeichenkette mit FESTER Länge von genau <i>n</i> Zeichen. Kürzere Werte werden mit Leerzeichen aufgefüllt	Daten, die immer die gleiche Länge haben (Abkürzungen, Postleitzahlen)
VARCHAR(n)	Zeichenkette mit VARIABLER Länge , maximal <i>n</i> Zeichen. Speicherplatz wird nur für die tatsächlich benötigten Zeichen verwendet	★ Der am häufigsten genutzte Typ — Namen, Adressen, E-Mail-Adressen
TEXT	Variable Länge, praktisch keine Längenbeschränkung	Beschreibungen, Blog-Artikel, Kommentare

⚠ ⚠ **Die klassische Klausurfrage CHAR vs. VARCHAR :**

CHAR(10) speichert **immer 10 Zeichen**, auch wenn nur "Hallo" eingefügt wird — die restlichen 5 Zeichen sind Leerzeichen. Das **verschwendet Speicherplatz**. VARCHAR(50) kann 1 bis 50 Zeichen speichern, **ohne unnötigen Speicher zu belegen** (zuzüglich geringen Overheads). → **VARCHAR ist die effizientere Wahl für Texte mit unterschiedlichen Längen.**

13.2 Numerische Datentypen — ganzzahlig ★★

Typ	Wertebereich	Speicher	Verwendung
SMALLINT	−32.768 bis +32.767	2 Bytes	kleinere Zähler oder IDs mit absehbarem Wertebereich
INTEGER	ca. −2,1 Mrd. bis +2,1 Mrd.	4 Bytes	★ Die häufigste Wahl — IDs (PRIMARY KEY), Mengen, Zählwerte
BIGINT	ca. $-9 \cdot 10^{18}$ bis $+9 \cdot 10^{18}$	8 Bytes	sehr große IDs oder Mengen
SERIAL	automatisch generierte eindeutige Ganzzahl		★ oft als Primärschlüssel

⚠ ⚠ **Die SERIAL-Falle:**

*„Kein echter Datentyp, sondern eine Vereinfachung für Spalten mit automatisch inkrementierenden Werten. **Vorsicht: SERIAL ist KEIN Standard-SQL-Typ!** Es erstellt **intern eine Sequenz**, die den Wert generiert, und setzt einen **NOT NULL-Constraint**.“*

Die Vorlesung weist mehrfach ausdrücklich darauf hin: „Hinweis: serial ist kein Standard SQL!“

13.3 Numerische Datentypen — Fließkomma und Festkomma ★★

Typ	Beschreibung	Verwendung
DECIMAL (p, s) / NUMERIC (p, s)	EXAKTE numerische Werte. p = Gesamtzahl der Ziffern (Präzision), s = Ziffern nach dem Komma (Skala)	★ Währungen , wissenschaftliche Daten — keine Rundungsfehler
REAL	ungenau, speichereffiziente Fließkommazahl, geringere Präzision	Berechnungen, bei denen Rundungen erlaubt sind
DOUBLE PRECISION (= FLOAT)	ungenau Fließkommazahl, höhere Präzision	dito

🔑 **Hinweis:** **DECIMAL** und **NUMERIC** sind in PostgreSQL **identisch**. Die Existenz beider Namen ist **rein historisch bedingt** — PostgreSQL implementiert beide zur Kompatibilität mit dem SQL-Standard und anderen Systemen.

📝 **DECIMAL**(10, 2) speichert Zahlen mit **bis zu 10 Ziffern, davon maximal 2 nach dem Komma** (z. B. 12345678,99).

⚠️ **Für Geldbeträge NIEMALS REAL / DOUBLE verwenden** — Rundungsfehler! Immer **NUMERIC / DECIMAL**. In der Chinook-DB: `total NUMERIC(10,2)`, `unit_price NUMERIC(10,2)`.

13.4 Datum und Uhrzeit ★★

Typ	Beschreibung
DATE	speichert nur ein Datum (Jahr, Monat, Tag)
TIME	speichert nur eine Uhrzeit
TIMESTAMP (... WITHOUT TIME ZONE)	Datum und Uhrzeit, ohne Zeitzoneinformation
TIMESTAMP WITH TIME ZONE (kurz TIMESTAMPZ)	Datum und Uhrzeit mit Zeitzone

★★ Wie **TIMESTAMPZ** intern funktioniert (prüfungsrelevant):

PostgreSQL **wandelt den eingegebenen Wert intern in die UTC-Zeitzone um** und speichert ihn in diesem Format. Beim Auslesen **konvertiert PostgreSQL ihn zurück in die Zeitzone des Clients**, der die Abfrage ausführt.

13.5 Weitere Datentypen

Typ	Beschreibung
BOOLEAN / BOOL	Wahrheitswerte (TRUE , FALSE) — ideal für Ja/Nein (<code>is_active</code> , <code>is_paid</code>)
BLOB (<i>Binary Large Object</i>)	binäre Daten wie Bilder, Audio, Video

⚠️ **Praxishinweis der Folie zu BLOB:** „Eignet sich für das Speichern von Dateien direkt in der Datenbank, obwohl dies aus Performance-Gründen oft vermieden und stattdessen nur der Dateipfad gespeichert wird.“

13.6 Zusammenfassung der Standardtypen (nicht PostgreSQL-spezifisch)

Datentyp	Beschreibung
CHAR(n)	Zeichenkette mit fester Länge (belegt Speicher immer)
VARCHAR(n)	Zeichenkette variabler Länge, maximal n
INTEGER	Ganzzahl
DECIMAL(n,m)	Kommazahl mit n Stellen (m Nachkommastellen)
DATE	Datum
DATETIME	Datum und Uhrzeit
BOOL	Wahrheitswert
TEXT, MEDIUMTEXT, LONGTEXT	große Textdaten
BLOB, MEDIUMBLOB, LONGBLOB	Binärdaten

14. Integritätsbedingungen (Constraints) ★★

🔑 **Integritätsbedingungen (Constraints)** sind **Regeln**, mit denen das DBMS die **Datenkonsistenz und Gültigkeit sicherstellt**, indem es bei Einfüge-, Änderungs- oder Löschvorgängen deren Einhaltung **erzwingt**. **Qualitätssicherung der Daten**. Bei Verstoß werden Befehle **NICHT ausgeführt**.

14.1 Domänen-Integrität (Domain Integrity)

Die Constraints **NOT NULL**, **UNIQUE** und **CHECK** definieren die **Domänen-Integrität** über eine **Einschränkung/Festlegung der zulässigen Wertemenge** für eine Spalte.

NOT NULL ★

- Bei Nutzung von **NOT NULL** **darf die Spalte nicht leer sein**
- Mit **DEFAULT** kann beim Leerlassen ein **Standardwert** gesetzt werden
- NOT NULL** und **DEFAULT** **können kombiniert werden**

```
CREATE TABLE emp_employee (
    emp_id          INTEGER,
    emp_email       VARCHAR(50) NOT NULL DEFAULT 'noemail@sth.de',
    emp_first_name  VARCHAR(50),
    emp_last_name   VARCHAR(50),
    PRIMARY KEY (emp_id)
);
```

UNIQUE ★

Die Werte einer Spalte **müssen eindeutig** sein.

```
CREATE TABLE emp_employees (
    emp_id          INTEGER,
    emp_email       VARCHAR(50) UNIQUE,
    PRIMARY KEY (emp_id)
);
```

→ Alle Mitarbeiter benötigen eine eindeutige E-Mail; keine zwei Mitarbeiter können dieselbe E-Mail haben.

CHECK ★★**Zusätzliche Regeln, die ein Spalteneintrag erfüllen muss.**

```
-- Inline-Variante
CREATE TABLE emp_employees (
    emp_id      INTEGER,
    emp_email   VARCHAR(50) UNIQUE,
    emp_age     INTEGER CHECK (emp_age >= 12),
    PRIMARY KEY (emp_id)
);

-- Benannte Variante (Constraint über den Namen "altercheck" referenzierbar)
CREATE TABLE emp_employees (
    emp_id      INTEGER,
    emp_age     INTEGER,
    CONSTRAINT altercheck CHECK (emp_age >= 16),
    PRIMARY KEY (emp_id)
);
```

PRIMARY KEY ★★★★★

Ein Primärschlüssel stellt automatisch ZWEI fundamentale Integritätsbedingungen sicher: 1. UNIQUE (eindeutig) und 2. NOT NULL (nicht leer)

(Und er ist zusätzlich automatisch **indiziert** — siehe Abschnitt 27.)

Zusammengesetzte Schlüssel (Composite Keys) ★

```
CREATE TABLE emp_employee (
    emp_first_name VARCHAR(50) NOT NULL,
    emp_last_name  VARCHAR(50) NOT NULL,
    PRIMARY KEY (emp_first_name, emp_last_name)
);
```

? Die Folie fragt: „→ fachlich keine gute Idee: Warum?“ ★ **Antwort:** Namen sind **nicht eindeutig** (mehrere „Hans Müller“), sie können sich **ändern** (Heirat), und ein Primärschlüssel sollte **fachlich neutral, stabil und kurz** sein. Deshalb verwendet man in der Praxis **künstliche Schlüssel (Surrogatschlüssel)** wie SERIAL -IDs.

 **Beispiel aus Chinook:** `playlist_track` hat einen sinnvollen zusammengesetzten Primärschlüssel: `PRIMARY KEY (playlist_id, track_id)` — die klassische Auflösung einer **m:n-Beziehung**.

14.2 Die drei Notationsvarianten für Constraints ★★

SQL erlaubt unterschiedliche Notationen und Stile aus **historischen, didaktischen und praktischen Gründen**:

- **Lesbarkeit & Klarheit:** Manchmal ist es übersichtlicher, die FOREIGN KEY -Definition **getrennt vom Spaltenblock** zu schreiben — insbesondere bei mehreren Constraints
- **Semantik & Erweiterbarkeit:** **Separate CONSTRAINT-Namen** ermöglichen spätere **gezielte Änderungen** oder das **Aktivieren/Deaktivieren** einzelner Constraints

Variante	Beispiel (FOREIGN KEY)	Vorteile	Nachteile
Inline (REFERENCES direkt an der Spalte)	<code>project_id INT REFERENCES project(id)</code>	kürzer , gut bei 1–2 FK	kein CONSTRAINT-Name , evtl. schwer zu warten
Separate Definition	<code>FOREIGN KEY (project_id) REFERENCES project(id)</code>	besser lesbar , flexibler bei vielen Keys	länger, weniger kompakt
Benannter Constraint	<code>CONSTRAINT fk_proj FOREIGN KEY (project_id) REFERENCES project(id)</code>	ermöglicht gezieltes Löschen/Deaktivieren ; Fehlermeldungen mit Namen	etwas komplexere Syntax

14.3 Welcher Constraint kann wie definiert werden? ★★★★★

Diese Tabelle ist eine wahrscheinliche Klausurfrage:

Constraint-Typ	Inline möglich	Separate Definition möglich	Benennbar (CONSTRAINT <name>)	Besonderheiten
PRIMARY KEY	✓	✓	✓	muss eindeutig + NOT NULL sein
FOREIGN KEY	✓	✓	✓	ON DELETE / ON UPDATE optional
UNIQUE	✓	✓	✓	kann mehrfach pro Tabelle vorkommen
CHECK	✓	✓	✓	logische Bedingungen (z. B. Alter ≥ 18)
NOT NULL	✓	✗	✗	⚠ nur inline, nicht benennbar
DEFAULT	✓	✗	✗	⚠ nur inline ; Vorgabewerte für Spalten

⚠ **Merken: NOT NULL und DEFAULT sind die beiden Ausnahmen — nur inline, nicht benennbar!**

15. Referentielle Integrität ★★★★★

15.1 Definition

- Um **Beziehungen (Assoziationen) zwischen Tabellen** darzustellen, werden im Relationenmodell **Fremdschlüssel („Foreign Key“)** verwendet
- Ein Fremdschlüssel ist eine Spalte, die einen **Bezug zu einer anderen Tabelle** herstellt. Ein **Fremdschlüssel ist IMMER ein Verweis auf einen PRIMÄRSCHLÜSSEL in einer anderen Tabelle**
- ★ **Fremdschlüsselwerte müssen NICHT eindeutig sein und dürfen auch den NULL-Wert enthalten**
- 🔑 Eine Datenbank besitzt **REFERENTIELLE INTEGRITÄT**, wenn Fremdschlüsselfelder nur gültige Werte enthalten — also Werte, die in der bezogenen Tabelle als Primärschlüsselwerte vorkommen, ODER NULL-Werte.

```

CREATE TABLE pro_project (
    pro_id    SERIAL,
    pro_name  VARCHAR(255),
    PRIMARY KEY (pro_id)
);

CREATE TABLE emp_employee (
    emp_id        SERIAL,
    emp_pro_id    INTEGER,
    emp_email     VARCHAR(50),
    emp_first_name VARCHAR(50),
    emp_last_name  VARCHAR(50),
    PRIMARY KEY (emp_id),
    FOREIGN KEY (emp_pro_id) REFERENCES pro_project(pro_id)
);

```

Mitarbeiter sind Projekten zugewiesen. Ein Mitarbeiter **kann** ein Projekt haben. **Durch NOT NULL kann eine Referenz ERZWUNGEN werden.**

15.2 Änderungen in der referenzierten Tabelle ★★★★★

Bislang haben wir betrachtet, wie die referentielle Integrität sichergestellt wird, **wenn in einer „abhängigen“ Tabelle ein Fremdschlüssel angelegt oder geändert wird. Allerdings können auch Änderungen in der REFERENZIERTEN Tabelle Auswirkungen haben** — z. B.: Wie wird mit einem Fremdschlüssel umgegangen, der auf einen Primärschlüssel referenziert, der **gelöscht oder geändert** werden soll?

Die drei Möglichkeiten: Änderungsweitergabe · Löschweitergabe · Aktion verhindern

1992 wurden mit der Norm SQL-2 neue Sprachkonstrukte eingeführt, um diese Probleme in den Griff zu bekommen. Die SQL-Norm gibt an, dass **jeder Fremdschlüssel GENAU EINE der drei folgenden Eigenschaften besitzen MUSS**:

Für DELETE des referenzierten Primärschlüssels	Für UPDATE des referenzierten Primärschlüssels
ON DELETE NO ACTION	ON UPDATE NO ACTION
ON DELETE CASCADE	ON UPDATE CASCADE
ON DELETE SET NULL	ON UPDATE SET NULL

```

CREATE TABLE emp_employee (
    ...
    emp_pro_id INTEGER REFERENCES pro_project (pro_id) ON DELETE <Verhalten>,
    ...
);

```

15.3 Die drei Verhaltensweisen im Detail ★★★★★

Verhalten	Wirkung beim Löschen	Wirkung beim Ändern
RESTRICT / NO ACTION	Löschen eines Projekts, welches einen Mitarbeiter hat, ist NICHT MÖGLICH — der Befehl wird mit einem Fehler abgewiesen	Änderung wird verhindert
CASCADE	Wird ein Projekt gelöscht, werden ALLE referenzierten Mitarbeiter GELÖSCHT	Änderung des PK wird an alle FK weitergegeben
SET NULL	Wird ein Projekt gelöscht, wird <code>emp_pro_id</code> des Mitarbeiters auf NULL gesetzt	FK-Werte werden auf NULL gesetzt

★ Die Eigenschaft wird **BEIM FREMDSCHLÜSSEL** definiert (also in der abhängigen Tabelle).

📝 Das Chinook-Beispiel der Folie (Tabelle `album` ist abhängig von `artist`):

Verhalten	Aktion in Tabelle <code>artist</code> (PRIMARY KEY)	Auswirkung in Tabelle <code>album</code> (FOREIGN KEY)
CASCADE	Änderung des Primärschlüssels 1 auf 999	Änderung der beiden Fremdschlüssel mit dem Wert 1 auf 999
CASCADE	Löschen des Tupels mit dem Primärschlüssel 1	Löschung der beiden Datensätze mit dem Fremdschlüssel 1
SET NULL	Änderung des Primärschlüssels 1 auf 999	Änderung der beiden Fremdschlüssel auf NULL
SET NULL	Löschen des Tupels mit dem Primärschlüssel 1	Änderung der beiden Fremdschlüssel auf NULL

⚠️ ⚠️ **Wichtiger Hinweis zur Mehrzeilen-Verarbeitung:**

*„Sollen mit einem einzigen SQL-Befehl gleich **mehrere Zeilen gelöscht** werden, so wird **Zeile für Zeile einzeln geprüft**. Gibt es für **MINDESTENS EINE Zeile** eine Zurückweisung (No Action), so wird der **GESAMTE SQL-Befehl nicht ausgeführt**.“*

TEIL C — SQL DATA MANIPULATION LANGUAGE (DML)

16. Daten einfügen: **INSERT** ★★★★★

```
-- Syntax
INSERT INTO <Tabellenname> ( <Spaltennamen> )
VALUES ( <Werte> );

-- Beispiel
INSERT INTO employee (id, email, first_name, last_name)
VALUES (1, 'worker@comp.de', 'Max', 'Muster');

-- Mehrere Zeilen einfügen
INSERT INTO t VALUES (...), (...), (...);

-- Daten aus anderer Tabelle einfügen
INSERT INTO t1 (spalte1, spalte2)
SELECT spalteA, spalteB FROM t2;

-- Ohne Angabe von Attributen ⚠️
INSERT INTO t1 SELECT * FROM t2;
```

⚠️ **Zur letzten Variante:** „funktioniert *nur, wenn Spaltenanzahl und -typen identisch sind*“

17. Daten ändern: UPDATE ★★

```
-- Syntax
UPDATE <Tabellenname>
SET <Spaltenname> = <Neuer Wert>, ...
[WHERE <Auswahlbedingung>];

-- Beispiel
UPDATE employee
SET last_name = 'Mustermann', first_name = 'Maxi'
WHERE email = 'worker@comp.de'
AND last_name = 'Muster';
```

- Bezieht sich auf **eine Tabelle**
- Mit **WHERE** können zu ändernde Daten **gefiltert** werden
- Auswahlbedingungen können mit **AND / OR** verknüpft werden
- **Es können mehrere Attribute gleichzeitig geändert werden**
- ★ **Bestehende Inhalte können für den Update verwendet werden** (z. B. für Berechnungen wie Preiserhöhungen):

```
UPDATE article SET unit_price = unit_price * 1.10; -- 10 % Preiserhöhung
```

!!! DIE Falle:

*„Was würde passieren, wenn Sie die **WHERE** -Klausel weglassen? → Die **UPDATE** -Anweisung würde auf **ALLE Zeilen der Tabelle** angewendet werden.“*

Das gilt genauso für **DELETE**. Immer erst mit **SELECT** testen, dann **UPDATE / DELETE** !

18. Daten löschen: DELETE ★★

```
-- Syntax
DELETE FROM <Tabellenname>
[WHERE <Auswahlbedingung>];

-- Beispiel
DELETE FROM employee
WHERE email = 'worker@comp.de';
```

⚠ Ohne **WHERE** werden **alle Zeilen** gelöscht (die Tabellenstruktur bleibt erhalten — das ist der Unterschied zu **DROP TABLE**).

★ DELETE vs. TRUNCATE vs. DROP :

Befehl	Kategorie	Wirkung	Rollback möglich?
DELETE FROM t	DML	löscht Zeilen (zeilenweise, mit Trigger und Log)	ja
TRUNCATE TABLE t	DDL	löscht alle Zeilen sehr schnell (ohne zeilenweise Verarbeitung)	eingeschränkt
DROP TABLE t	DDL	löscht die gesamte Tabelle inkl. Struktur	—

TEIL D — SQL DATA QUERY LANGUAGE (DQL): der SELECT-Befehl

19. Aufbau des SELECT-Befehls ★★★★★

SQL ist keine prozedurale, sondern eine datenorientierte DEKLARATIVE Programmiersprache. Der Benutzer muss **nicht wissen, WIE** man im Einzelnen auf die gespeicherten Daten zugreift, sondern **lediglich angeben, MIT WELCHEN Daten er arbeiten möchte und welche Bedingungen** diese erfüllen müssen.

19.1 Die Bestandteile ★★★★★

```
SELECT  [DISTINCT] <Spaltenliste>      -- Projektion ← PFLICHT
FROM    <Tabellen>                    -- Auswahl der Tabellen ← PFLICHT
[WHERE  <Bedingung>]                  -- Restriktion (einzelne Zeilen)
[GROUP BY <Spalten>]                  -- Gruppierung
[HAVING <Bedingung>]                  -- Auswahl von GRUPPEN
[ORDER BY <Spalten> [ASC|DESC]]        -- Sortierung
[LIMIT n [OFFSET m]];                  -- Begrenzung / Paginierung
```

Klausel	Funktion
SELECT	Projektion — welche Spalten zurückgegeben werden. Ein Stern (*) gibt alle zurück
FROM	Auswahl der Tabellen für den Abfragebefehl
WHERE	Auswahlbedingungen der auszuwertenden Datensätze (Restriktion)
GROUP BY	Bedingungen, nach welchen Gruppierungen vorgenommen werden
HAVING	Auswahl von GRUPPEN
ORDER BY	Auswahl der Sortierung

🔑 Für eine Abfrage sind **LEDIGLICH SELECT UND FROM PFLICHT**. ★★★★★

```
SELECT * FROM emp_employee;      -- alle Spalten, alle Zeilen
SELECT emp_id FROM emp_employee; -- Projektion
SELECT * FROM emp_employee, pro_project; -- ⚠ KARTESISCHES PRODUKT!
```

19.2 Die Abarbeitungsreihenfolge ★★★★★

Achtung: Die logische Auswertungsreihenfolge ist NICHT die Schreibreihenfolge!

- ① FROM → Tabellen laden, Joins bilden
- ② WHERE → einzelne ZEILEN filtern
- ③ GROUP BY → Gruppen bilden
- ④ HAVING → GRUPPEN filtern
- ⑤ SELECT → Spalten auswählen (Projektion), Aliase vergeben
- ⑥ DISTINCT → Duplikate entfernen
- ⑦ ORDER BY → sortieren
- ⑧ LIMIT/OFFSET → begrenzen

⚠ ⚠ Daraus folgen zwei prüfungsrelevante Konsequenzen:

1. Ein in `SELECT` vergebener **Alias kann in `WHERE` NICHT verwendet werden** (`WHERE` läuft vorher) — wohl aber in `ORDER BY` (läuft nachher)
2. In `WHERE` **dürfen keine Aggregatfunktionen** stehen (die Gruppen existieren noch nicht) — dafür ist `HAVING` da

20. Aliase ★★

Ein **Alias** ist ein **alternativer Name**, den man einer Tabelle oder Spalte in einer SQL-Abfrage **vorübergehend** gibt.

- Der Alias **gilt nur für die Dauer der Abfrage**
- Er **ändert nichts** am eigentlichen Tabellen- oder Spaltennamen in der Datenbank

```
SELECT last_name AS Nachname, first_name AS Vorname
FROM   customer AS c;
```

Wann verwendet man Aliase? ★

- **Lesbarkeit verbessern** (statt `last_name` → `Nachname`)
- **Ausgabe sprechender** machen
- Lange oder komplizierte Namen **abkürzen** (z. B. `c` statt `customer`)
- ★ **Mehrere Tabellen im Join unterscheiden** (wenn Spalten gleich heißen, z. B. `id`)
- **Berechnete Spalten benennen**

⚠ **Zwingend erforderlich sind Aliase bei:**

1. **Korrelierten Subqueries auf derselben Tabelle** (Self-Reference)
2. **Inline Views** (abgeleitete Tabellen in der `FROM`-Klausel)
3. **Self Joins**

21. DISTINCT ★★

```
SELECT DISTINCT first_name FROM emp_employee;
```

→ Gibt alle Vornamen zurück, wobei **jeder Vorname im Ergebnis eindeutig** ist.

(Zur *Multimenge/Bag-Semantik* siehe Abschnitt 5.4)

22. Aggregatfunktionen ★★★

Mit **Aggregatfunktionen** können **Gesamtergebnisse des Resultatsets** ermittelt werden. Sie können Bestandteil von `SELECT`, `HAVING` oder `ORDER BY` sein.

Die fünf Aggregatfunktionen: `AVG()` · `COUNT()` · `SUM()` · `MAX()` · `MIN()`

22.1 Die zwei Grundregeln ★★★

1. **Aggregatfunktionen führen IMMER zu EINER Ergebniszeile** (wenn kein `GROUP BY` verwendet wird)
2. ★★ **NULL-Werte werden bei der Berechnung IGNORIERT**

Mit einer `WHERE`-Klausel können die **Grunddaten für die Berechnung reduziert** werden (d. h. man schließt Datensätze für die Berechnung der Aggregatfunktion aus).

22.2 `COUNT(*)` vs. `COUNT(Spalte)` vs. `COUNT(DISTINCT ...)` ★★

```
SELECT COUNT(*)           FROM emp_employee; -- zählt ALLE Zeilen
SELECT COUNT(emp_id)      FROM emp_employee; -- zählt Zeilen OHNE NULL in emp_id
SELECT COUNT(DISTINCT emp_id) FROM emp_employee; -- zählt EINDEUTIGE Werte
```

Variante	Zählt
<code>COUNT(*)</code>	alle Datensätze in der Ergebnismenge
<code>COUNT(<Spalte>)</code>	nur Datensätze, bei denen die Spalte KEINEN NULL-Wert enthält
<code>COUNT(DISTINCT <Spalte>)</code>	wie oft ein eindeutiger (unterschiedlicher) Wert vorkommt

★ **Der Hinweis der Folie:** Bei `emp_id` als Primärschlüssel führen `COUNT(*)` und `COUNT(emp_id)` zum selben Ergebnis, weil ein PK keine NULL-Werte hat.

22.3 ⚠️ Die Regel, die man verstehen muss

```
SELECT COUNT(*), emp_id FROM emp_employee; -- ❌ GEHT NICHT!
```

❓ „Bitte überlegen warum?“ — ★ **Die Antwort:**

`COUNT(*)` liefert **genau EINE Zeile** (ein aggregierter Wert über die gesamte Tabelle). `emp_id` liefert dagegen **VIELE Zeilen** (einen Wert pro Datensatz). Das DBMS könnte nicht entscheiden, **welchen** `emp_id`-Wert es neben dem einen Gesamtzählwert ausgeben soll. **Es geht nur bei Verwendung von `GROUP BY / HAVING`** — denn dann gibt es je Gruppe genau einen Wert.

23. Rechnen mit Zeit und Datum ★★

23.1 Aktuelle Zeit

Funktion	Rückgabe	Typ
<code>CURRENT_TIMESTAMP</code>	aktueller Transaktionszeitpunkt	<code>TIMESTAMP WITH TIME ZONE</code>
<code>LOCALTIMESTAMP</code>	aktueller Transaktionszeitpunkt ohne Zeitzone	<code>TIMESTAMP WITHOUT TIME ZONE</code>
<code>CURRENT_DATE</code>	aktuelles Datum	<code>DATE</code>

⚠️ „`CURRENT_TIMESTAMP` ist ein SQL-Schlüsselwort/Ausdruck — **es wird KEINE Tabelle zur Abfrage benötigt**“:
`SELECT CURRENT_TIMESTAMP;`

23.2 `EXTRACT()` ★★

```
EXTRACT(field FROM quelle)      -- field: YEAR, MONTH, DAY, HOUR, WEEK
SELECT EXTRACT(MONTH FROM '2025-09-12'::DATE); -- Ergebnis: 9
```

23.3 Datumsarithmetik ★★

```
-- Differenz zwischen zwei Datums-/Zeitwerten
SELECT '2025-09-12' - '2025-07-01';           -- Ergebnis: 73 (Tage)
SELECT '2025-09-12 10:00:00' - '2025-09-12 08:30:00'; -- Ergebnis: 01:30:00

-- Intervalladdition/-subtraktion: Datum + INTERVAL 'Menge Einheit'
SELECT CURRENT_DATE + INTERVAL '7 days';    -- am 12.9.2025 → 2025-09-19
SELECT '2025-09-12' - INTERVAL '1 year';    -- → 2024-09-12
```

24. Zeichenketten-Funktionen ★★

Funktion	Beschreibung	Beispiel
UPPER(s) / LOWER(s)	Groß-/Kleinbuchstaben — nützlich für Standardisierung	SELECT UPPER('sql'), LOWER('SQL');
LENGTH(s)	Länge einer Zeichenkette in Zeichen	SELECT LENGTH('Hochschule'); → 10
TRIM(s)	entfernt führende und nachfolgende Leerzeichen — wichtig für Datenbereinigung	SELECT TRIM(' Text ');
SUBSTRING(s FROM start FOR length)	extrahiert einen Teil	SELECT SUBSTRING('Datenbank' FROM 6 FOR 4); → bank
CONCAT(s1, s2, ...) oder 	verbindet mehrere Zeichenketten	SELECT CONCAT('Max', ' ', 'Muster');
REPLACE(s, from, to)	ersetzt alle Vorkommen eines Teilstrings	SELECT REPLACE('Hallo Welt', 'Welt', 'Datenbank'); → Hallo Datenbank

25. Typumwandlungen (Type Cast) ★★

Die **Typumwandlung (Type Cast)** wandelt den Datentyp eines Wertes in einen anderen um. Anwendung z. B., wenn ein **String in eine Zahl** oder ein **Datum** umgewandelt werden muss, um damit **zu rechnen oder zu vergleichen**.

Methode	Syntax	Beispiel
Doppel-Doppelpunkt-Operator :: ★	wert::ziel_datentyp	SELECT '123'::INTEGER; • SELECT '2025-09-12'::DATE;
CAST() -Funktion (standardkonform)	CAST(wert AS ziel_datentyp)	SELECT CAST('123' AS INTEGER);

★ **:: ist in PostgreSQL bevorzugt; CAST() ist die standardkonforme SQL-Methode**, die in vielen Datenbanksystemen funktioniert.

26. Sortierung, LIMIT und OFFSET ★★

26.1 ORDER BY

```
SELECT * FROM emp_employee ORDER BY age;           -- ASC ist Default
SELECT * FROM emp_employee ORDER BY last_name DESC, first_name ASC;
```

Schlüsselwort	Bedeutung
ASC (ascending)	aufsteigend — ★ Default-Einstellung
DESC (descending)	absteigend

★★ **NULL-Behandlung in PostgreSQL:** NULL-Werte werden bei **ASC** standardmäßig ans **ENDE** (**NULLS LAST**) und bei **DESC** **an den ANFANG** (**NULLS FIRST**) sortiert.

26.2 LIMIT und OFFSET ★★

Klausel	Funktion
LIMIT n	begrenzt die Anzahl der zurückgegebenen Zeilen
OFFSET m	überspringt m Zeilen am Anfang der Ergebnismenge

Anwendungsfälle: Performance-Optimierung (Stichprobe) · nur die „**obersten**“ **Ergebnisse** einer sortierten Liste · ★ **Grundlage für PAGINIERUNG in Webanwendungen**

```
SELECT * FROM orderitem ORDER BY ordered_qty DESC LIMIT 1;
-- Das orderitem mit der größten ordered_qty

SELECT * FROM orderitem ORDER BY ordered_qty DESC LIMIT 1 OFFSET 10;
-- überspringt die ersten 10, gibt dann 1 zurück
```

⚠️⚠️ **Die Falle:** „Wenn **LIMIT** ohne **ORDER BY** verwendet wird, ist das Ergebnis **NICHT DETERMINISTISCH!**“* — Das DBMS darf die Zeilen in beliebiger Reihenfolge liefern (Relationen sind ungeordnet!).

27. GROUP BY und HAVING ★★★

27.1 GROUP BY

```
-- Anzahl der Mitarbeiter pro Projekt
SELECT emp_pro_id, COUNT(*)
FROM   emp_employee
GROUP BY emp_pro_id;
```

- Damit ist es möglich, **Aggregatfunktionen auf Teiltabellen (Gruppen)** anzuwenden
- Es können **mehrere Gruppierungskriterien** angegeben werden
- Attribute, nach denen gruppiert wird, **müssen nicht in der Attributliste im SELECT auftauchen**

27.2 ⚠️⚠️⚠️ DIE GOLDENE REGEL des GROUP BY

„Bei Anwendung des **GROUP BY**-Befehls können im **SELECT** **NUR** Attribute verwendet werden, nach denen **GRUPPIERT** wird (und zusätzlich Aggregatfunktionen).“

📝 **Die Beispiele der Folie** (Gruppierung nach **name**):

```
-- ✗ NICHT MÖGLICH:
SELECT name, SUM(wert1), wert2 FROM mitarbeiter GROUP BY name;
SELECT name, wert1, wert2      FROM mitarbeiter GROUP BY name;

-- ✓ MÖGLICH:
SELECT name, SUM(wert1), AVG(wert2) FROM mitarbeiter GROUP BY name;
SELECT name, SUM(wert1)             FROM mitarbeiter GROUP BY name;
```

★ **Die Begründung (bitte in der Klausur mitliefern):** Nach der Gruppierung existiert je Gruppe **nur noch EIN Wert pro gruppierter Spalte**. Für **nicht gruppierte Spalten** gibt es dagegen **viele verschiedene Werte** — das DBMS könnte nicht entscheiden, **welchen** es ausgeben soll. Deshalb sind nur **Aggregatfunktionen** erlaubt (die viele Werte auf einen reduzieren).

27.3 HAVING ★★★★★

```
-- IDs der Projekte, die mehr als einen Mitarbeiter haben
SELECT emp_pro_id, COUNT(*)
FROM   emp_employee
GROUP BY emp_pro_id
HAVING COUNT(emp_pro_id) >= 2;
```

Durch ein **HAVING** werden die Ergebnisse eines **GROUP BY** **VERWORFEN**, für welche die Bedingung nicht **TRUE** ergibt.

- **Hinweis (1):** Im Gegensatz zu **WHERE** werden die mit **HAVING** angegebenen Bedingungen **nicht auf einzelne Datensätze, sondern auf DATENSATZGRUPPEN** angewendet
- **Hinweis (2):** ★ Nach **HAVING** dürfen nur **GRUPPENFUNKTIONEN (Aggregatfunktionen)** stehen, da sich **HAVING** auf Datensatzgruppen und nicht auf einzelne Datensätze bezieht

27.4 ★★★★★ WHERE vs. HAVING — die Kernunterscheidung

	WHERE	HAVING
Wirkt auf	einzelne ZEILEN (Datensätze)	GRUPPEN von Datensätzen
Zeitpunkt	VOR der Gruppierung	NACH der Gruppierung
Aggregatfunktionen erlaubt?	✗ nein	✓ ja (und nur diese)
Ohne GROUP BY nutzbar?	✓ ja	⚠ nur in Kombination mit GROUP BY sinnvoll
Merksatz der Folie	schränkt die „ DATENGRUNDLAGE “ ein	schränkt das ERGEBNIS ein (bei Gruppierung)

★ **WHERE** findet auch bei **UPDATE** und **DELETE** Anwendung zur Selektion der betroffenen Datensätze.

28. WHERE-Bedingungen im Detail ★★★★★

28.1 Verknüpfungen

```
SELECT email FROM emp_employee
WHERE first_name <> 'Karl' AND age < 30;
```

- Bedingungen können mit **AND**, **OR** verknüpft werden
- **String-Literale in Hochkomma** ('Karl')

28.2 LIKE — Mustervergleich ★★☆☆

Wildcard	Bedeutung
%	beliebige Zeichen von beliebiger Länge (auch null Zeichen)
_	genau EIN beliebiges Zeichen

```
SELECT * FROM emp_employee WHERE first_name LIKE 'K%';    -- beginnt mit K
SELECT * FROM track      WHERE name LIKE '%Baby%';      -- enthält "Baby"
SELECT * FROM track      WHERE name LIKE 'M_ier';        -- Meier, Maier, Mzier ...
```

★ **Groß-/Kleinschreibung:** LIKE ist in PostgreSQL **case-sensitiv**. Für case-insensitive Suche: ILIKE oder LOWER(spalte) LIKE '%bach%' (so löst es die Musterlösung zu Aufgabe 24).

28.3 BETWEEN ★

```
SELECT * FROM emp_employee WHERE age BETWEEN 16 AND 30;
-- Hinweis: WHERE age >= 16 AND age <= 30 würde zum gleichen Ergebnis führen
```

★ BETWEEN ist **INKLUSIVE** beider Grenzen.

28.4 ! ! ! NULL-Behandlung — die wichtigste Falle

NULL ist NIE gleich oder ungleich. Deswegen liefert = NULL oder <> NULL KEINE Ergebnisse. Für die Überprüfung auf **unbekannte Werte** wird **IS NULL** oder **IS NOT NULL** verwendet.

```
-- ❌ FALSCH (liefert immer leer):
SELECT * FROM emp_employee WHERE first_name = NULL;

-- ✅ RICHTIG:
SELECT * FROM emp_employee WHERE first_name IS NOT NULL;
```

★ **Die Begründung:** NULL bedeutet „**unbekannt**“. Ein Vergleich mit einem unbekannten Wert kann weder TRUE noch FALSE sein — er ist **UNKNOWN**, und WHERE liefert nur Zeilen, für die die Bedingung **TRUE** ist.

28.5 IN ★★☆☆

```
SELECT * FROM emp_employee WHERE first_name IN ('Karl', 'Foo');
```

Mit dem Prädikat IN kann eine bestimmte Spalte **auf mehrere Werte hin überprüft** werden. IN prüft, ob der Wert **MINDESTENS EINMAL** in der Menge vorkommt.

28.6 ANY und ALL ★★☆☆

Operator	Bedeutung
ALL	prüft, ob eine Bedingung für ALLE Elemente erfüllt ist
ANY	prüft, ob eine Bedingung für MINDESTENS EIN Element erfüllt ist

```
SELECT * FROM emp_employee WHERE emp_age > ALL (27, 55, 70, 30);
-- älter als ALLE genannten Werte, also älter als 70
```

28.7 NOT ★

NOT **negiert die Bedingung und wird VOR den Operator geschrieben** — mit Ausnahme von IS NULL ; hier wird es **zwischengestellt**: IS NOT NULL

```
SELECT * FROM emp_employee WHERE first_name NOT IN ('Karl', 'Foo');
SELECT * FROM emp_employee WHERE name NOT LIKE 'A%';
SELECT * FROM emp_employee WHERE first_name IS NOT NULL;  -- ⚠ Sonderfall
```

28.8 EXISTS ★★

Mit dem Prädikat EXISTS wird geprüft, **ob das Ergebnis einer inneren Abfrage (Subquery) NICHT LEER ist**. Auch diese kann negiert werden mit NOT EXISTS und prüft dann, **ob das Ergebnis leer ist**.

```
SELECT * FROM emp_employee WHERE EXISTS ( ...Subquery... );
```

28.9 Komplexe Bedingungen ★

```
SELECT * FROM emp_employee
WHERE first_name LIKE '%K' OR (age > 30 AND first_name IS NOT NULL);
```

„Wähle alle Mitarbeiter aus, deren **Vorname mit K endet** ODER die **älter als 30 sind UND einen Vornamen haben**.“

⚠ **Klammern sind entscheidend!** AND bindet stärker als OR — ohne Klammern ergibt sich eine andere Bedeutung.

29. Unterabfragen (Subqueries) ★★★

29.1 Grundlagen

- Das **Ergebnis einer Abfrage wird als Teil einer neuen Abfrage verwendet**
- In WHERE -Klausel, FROM -Klausel und als Ausdruck möglich — **überall dort, wo ein Ausdruck zulässig ist**
- ★ **Verschachtelte Abfragen werden IMMER von unten nach oben bzw. von innen nach außen abgearbeitet** (Unterabfragen werden **vor** der Hauptabfrage ausgeführt — **Ausnahme: synchronisierte/korrelierte Unterabfrage!**)
- **Mehrfache Schachtelung** ist möglich

29.2 Syntaxregeln ★★★

- Unterabfragen werden in **runde Klammern** gesetzt
- Die Unterabfrage **kann sich auf dieselbe Tabelle beziehen** wie die Hauptabfrage
- ★ **Bei einem Vergleichsausdruck stehen Unterabfragen IMMER auf der RECHTEN Seite**

```
WHERE invoiceamount > (SELECT AVG(invoiceamount) FROM custorder)
```

★★ Die entscheidende Regel — welcher Operator wann?

Rückgabe der Unterabfrage	Erlaubte Operatoren
EIN atomarer (skalärer) Wert	Vergleichsoperatoren <code><</code> , <code>></code> , <code>=</code> — „Um ein atomares Ergebnis zu garantieren, werden in Unterabfragen oft Aggregate benutzt“
MEHRERE Ergebnisse	<code>IN</code> , <code>ANY</code> , <code>ALL</code> , <code>EXISTS</code>

29.3 Die vier Einsatzorte von Unterabfragen ★★★

(a) In der `SELECT`-Klausel (Skalare Unterabfrage)

Muss einen **einzelnen Wert (eine Zeile, eine Spalte)** zurückgeben.

```
SELECT *, (SELECT AVG(invoiceamount) FROM custorder) AS Durchschnitt
FROM custorder;
```

(b) In der `WHERE`-Klausel (Filterung) — der häufigste Fall

```
-- Mit skalarem Vergleichsoperator
SELECT * FROM custorder
WHERE invoiceamount > (SELECT AVG(invoiceamount) FROM custorder);

-- Mit Mengenoperatoren
-- IN prüft, ob ein Wert in der zurückgegebenen Liste enthalten ist
-- EXISTS prüft, ob die Unterabfrage mindestens eine Zeile zurückgibt
```

(c) In der `FROM`-Klausel (Abgeleitete Tabelle / Inline View) ★★

Unterabfragen in der `FROM`-Klausel werden als **abgeleitete Tabellen** oder **Inline Views** bezeichnet. Sie werden **wie eine temporäre Tabelle** behandelt. ⚠ **Die abgeleitete Tabelle MUSS einen Alias haben** (z. B. `t1`). Ermöglicht z. B. **Aggregationen oder Joins VOR der Hauptabfrage**.

(d) In DML-Befehlen (`INSERT` , `UPDATE` , `DELETE`)

Um die **Daten zu liefern, die eingefügt werden**, oder die **Zeilen zu identifizieren**, die aktualisiert oder gelöscht werden sollen.

(Zusätzlich: **in der `JOIN`-Klausel** — Spezialfall der Verwendung in der `FROM`-Klausel)

29.4 Korrelierte (abhängige/synchronisierte) Unterabfragen ★★★

Die vorherigen Beispiele können ausgewertet werden, indem die Unterabfrage **einmal** ausgeführt wird. **In Abfragen mit einer KORRELIERTEN Unterabfrage hängt die Unterabfrage für ihre Werte von der ÄUSSEREN Abfrage ab. Das bedeutet, dass die Unterabfrage WIEDERHOLT ausgeführt wird — und zwar EINMAL FÜR JEDE ZEILE, die von der äußeren Abfrage ausgewählt wurde.**

Das Beispiel der Folien:

„Geben Sie alle Angestellten aus, die weniger verdienen als jemand mit demselben Nachnamen.“

```
SELECT *
FROM   personen p1
WHERE  zuname IN
      (SELECT zuname
       FROM   personen p2
       WHERE  p1.pnr    <> p2.pnr
            AND  p1.gehalt < p2.gehalt);
```

- **⚠ Die innere Abfrage kann NICHT alleine durchgeführt werden** (abhängig von der Hauptabfrage)
- **⚠ Das Arbeiten mit Alias ist hier ZWINGEND notwendig** wegen der Unterscheidbarkeit (dieselbe Tabelle wird in Haupt- und Unterabfrage verwendet)
- **Übliche Operatoren:** WHERE , EXISTS (auch IN , ANY , ALL)

29.5 ★★★★★ Die Abarbeitungslogik — Analogie „geschachtelte Schleife“

Diese Erklärung der Folie ist prüfungsrelevant, weil sie das Verständnis abfragt:

- (1) HAUPTABFRAGE (p1)
 - Ausführen der Hauptabfrage, Selektion der ERSTEN Zeile in Tabelle personen (p1)
 - Belegung der Attribute p1.pnr, p1.gehalt

|
▼
- (2) UNTERABFRAGE (p2)
 - Unterabfrage ausführen, indem NACHEINANDER ALLE Tupel von p2 ausgewählt werden
 - Auswertung der WHERE-Klausel für JEDES Tupel der Tabelle p2
 - Es werden die jeweiligen Werte p2.pnr, p2.gehalt verwendet (für jede Zeile anders)
 - ★ Die Werte aus der Hauptabfrage p1.pnr, p1.gehalt BLEIBEN in diesem Durchlauf IMMER GLEICH (!)
 - Wenn die WHERE-Klausel TRUE wird → Übernahme des Attributs zuname in die Ergebnismenge der aktuellen Unterabfrage
 - Nach Durchlauf: In der Ergebnismenge sind die Nachnamen ALLER Mitarbeiter, die MEHR Gehalt haben als der aktuelle Mitarbeiter der Hauptabfrage und die nicht mit ihm identisch sind
 - Rücksprung in die Hauptabfrage

|
▼
- (3) HAUPTABFRAGE
 - Auswertung der WHERE-Klausel für die erste Zeile der Hauptabfrage
 - Wenn der zuname des aktuellen Tupels von p1 in der Ergebnismenge enthalten ist, dann gibt es einen Mitarbeiter mit gleichem Nachnamen, der mehr verdient

|
▼
- (4) Sprung zurück zu (1) – Selektion der NÄCHSTEN Zeile in der Hauptabfrage

⚠ Performance-Konsequenz: Bei n Zeilen in der Hauptabfrage wird die Unterabfrage **n-mal** ausgeführt → **$O(n^2)$** im schlechtesten Fall. Deshalb sind **Joins oft effizienter**.

29.6 Korrelierte Subquery vs. JOIN ★★

JOINS können teilweise genutzt werden, um Probleme zu lösen, die über korrelierte Subqueries gelöst werden. **⚠ DISTINCT ist notwendig, wenn 1:n-Beziehungen vorliegen und Duplikate vermieden werden sollen.**

Korrelierte Subquery	Als Join möglich?	Bemerkung
EXISTS , IN mit einfacher Bedingung	✓ ja	JOIN + DISTINCT oft effizienter
Vergleich mit Aggregatwert (z. B. SUM)	✓ ja	Join + GROUP BY + HAVING
Vergleich mit einem Skalarwert (z. B. MAX)	✓ ja	Join + GROUP BY
Komplexe Bedingungen in Subqueries	⚠ teilweise	abhängig vom Fall, u. U. nur mit Common Table Expressions
Subqueries mit mehrfacher Abhängigkeit	✗ schwer	nicht direkt mit Join ersetzbar

30. Abfragen auf mehreren Tabellen ★★★★★

30.1 Kartesisches Produkt ⚠

```
SELECT * FROM account, customer;
```

Jeder Datensatz der Tabelle account wird mit JEDEM Datensatz der Tabelle customer kombiniert, und daraus wird ein neuer Datensatz mit allen beteiligten Attributen gebildet.

⚠ Bei 1.000×1.000 Zeilen entstehen **1.000.000 Zeilen**. Fast immer ein **Fehler!**

30.2 Verknüpfung über **WHERE** (alte Schreibweise)

```
SELECT * FROM account, customer
WHERE account.customer_id = customer.customer_id;
```

Damit SQL weiß, **welches Attribut aus welcher Tabelle** gemeint ist, muss dem Attributnamen der **Tabellenname**, gefolgt von einem Punkt, vorangestellt werden. **Attribute mit unklarer oder mehrdeutiger Herkunft MÜSSEN so identifiziert werden.**

30.3 **JOIN ... ON** (seit SQL-92) ★★★★★

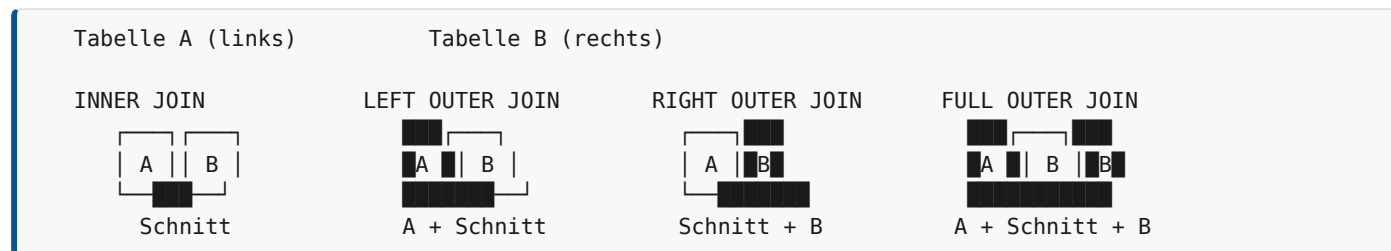
```
SELECT * FROM account INNER JOIN customer
ON account.customer_id = customer.customer_id;

-- Mit zusätzlichen Filterbedingungen
SELECT * FROM account INNER JOIN customer
ON account.customer_id = customer.customer_id
WHERE account.account_holder LIKE 'R%';
```

🔑 **Der Vorteil:** „Mit **JOIN** werden **keine WHERE-Bedingungen benötigt**, um Beziehungen abzubilden. Dies hat den Vorteil, dass **in der WHERE-Bedingung nur noch ECHTE FILTERKRITERIEN** stehen, was die **Lesbarkeit** von SQL-Anweisungen verbessert.“

★ „Die Verknüpfung über die **WHERE-Klausel entspricht einem INNER JOIN**. Vor dem SQL-92-Standard war dies die **einzige Möglichkeit**, einen Join zu erstellen. Auch wenn der Join über **WHERE** heute noch verwendet wird, ist die **Nutzung der JOIN ... ON-Klausel empfehlenswert.**“

30.4 Die JOIN-Arten ★★



JOIN-Art	Definition
INNER JOIN	„Der INNER JOIN zweier Tabellen ist die Teilmenge des kartesischen Produkts , für welche gilt, dass die Fremdschlüsselwerte zu den Primärschlüsselwerten passen .“
LEFT OUTER JOIN	INNER JOIN + Zeilen der LINKEN Tabelle, für die keine passende Paarung gefunden wurde
RIGHT OUTER JOIN	INNER JOIN + Zeilen der RECHTEN Tabelle ohne Paarung
FULL OUTER JOIN	INNER JOIN + Zeilen aus der LINKEN UND der RECHTEN Tabelle ohne Paarung

🔑 „Ein **LEFT/RIGHT OUTER JOIN** wird immer dann verwendet, wenn **nicht nur die Zeilen einer Tabelle interessant sind, für die es eine Paarung gibt**.“

★ **Typische Anwendung:** „Welche Kunden haben **keine** Rechnung?“ → `LEFT JOIN + WHERE invoice.invoice_id IS NULL`

30.5 Weitere JOIN-Typen ★★

Typ	Beschreibung	Beispiel
EQUI Join	Operator ist ein <code>=</code>	<code>JOIN Bestellungen ON Kunden.KundenID = Bestellungen.KundenID</code>
NON EQUI Join	Als JOIN-Operator wird kein <code>=</code> verwendet; meist mit BETWEEN	<code>JOIN Gehaltsspannen g ON m.Gehalt BETWEEN g.MinGehalt AND g.MaxGehalt</code>
Cross Join	Kartesisches Produkt — „selten sinnvoll“	<code>SELECT * FROM a CROSS JOIN b</code>
SELF JOIN ★	Eine Tabelle wird mit SICH SELBST verknüpft — rekursive Beziehung	Mitarbeiter und deren Vorgesetzte (Chinook: <code>employee.reports_to</code>)
NATURAL JOIN	Tabellen werden automatisch über gleichnamige Spalten verknüpft	⚠️⚠️ „ NICHT VERWENDEN! Intransparent, fehleranfällig “

30.6 JOIN mit mehr als 2 Tabellen ★★

★ „Ein **JOIN** mit n Tabellen erfordert **MINDESTENS** $(n - 1)$ **JOIN-Bedingungen**.“

📝 **Beispiel aus Chinook — alle Tracks mit Album- und Interpretenname:**

```
SELECT t.name AS track, al.title AS album, ar.name AS k nstler
FROM   track t
JOIN   album al ON t.album_id = al.album_id
JOIN   artist ar ON al.artist_id = ar.artist_id;
```

→ 3 Tabellen, **2 JOIN-Bedingungen** ✓

30.7 ★★★★★ JOIN mit Filterung: WHERE vs. ON — die anspruchsvollste Stelle

	Filterung im WHERE -Abschnitt (Restriktion)	Filterung im JOIN ... ON -Abschnitt (Verknüpfungsbedingung)
Zeitpunkt	NACHDEM der Join ausgeführt wurde, auf das gesamte Zwischenergebnis	VOR dem Join, als zusätzliche Einschränkung für die Zeilenkombination
Wirkung	globaler Filter für die Ergebniszeile	bestimmt, welche Datensätze überhaupt miteinander verknüpft werden

★★ „Bei INNER JOIN sind beide Methoden LOGISCH ÄQUIVALENT und führen zum gleichen Ergebnis. ANDERS beim LEFT oder RIGHT JOIN!“

 Das Beispiel der Folien (Kunden und Rechnungen aus 2021):

Variante A — Filterung im WHERE :

```
SELECT c.*, i.*
FROM   customer c
LEFT JOIN invoice i ON c.customer_id = i.customer_id
WHERE  EXTRACT(YEAR FROM i.invoice_date) = 2021;
```

Zeigt **alle Kunden an, die MINDESTENS EINE Rechnung im Jahr 2021 hatten**. Kunden ohne Rechnung aus diesem Jahr werden **aus der Liste entfernt**. ⚠ „Kunden ohne Rechnung im Jahr 2021 werden **NACH dem Join ausgefiltert**, weil *i.InvoiceDate* für sie **NULL** wäre und **NULL die WHERE-Bedingung nicht erfüllt**.“ ⚠⚠ „Durch die Platzierung der Rechnungs-Bedingung im WHERE-Abschnitt wird der **LEFT JOIN EFFEKTIV ZU EINEM INNER JOIN**“, da alle Zeilen mit NULL-Werten auf der rechten Seite nachträglich herausgefiltert werden.

Variante B — Filterung im ON :

```
SELECT c.*, i.*
FROM   customer c
LEFT JOIN invoice i
      ON c.customer_id = i.customer_id
      AND EXTRACT(YEAR FROM i.invoice_date) = 2021;
```

Zeigt **ALLE Kunden an** und (falls vorhanden) Rechnungen aus dem Jahr 2021. **Kunden ohne Rechnungen oder nur mit Rechnungen aus anderen Jahren erscheinen dennoch in der Liste**. „Die Einschränkung filtert **nur die Verknüpfung**; sie entscheidet, **welche Rechnungen an den Kunden angehängt werden**.“

🔑 **Merksatz:** Bei OUTER JOINS gehört eine Bedingung auf die „optionale“ Tabelle in die ON-Klausel — sonst wird der OUTER JOIN zum INNER JOIN.

31. Mengenoperationen ★★★★★

- Mengenoperatoren **verbinden die Teilergebnisse einzelner SELECTs**
- ★ Die Verbindung erfolgt **VERTIKAL (Join — HORIZONTAL)**

Operator	Bedeutung
UNION / UNION ALL	Vereinigung
INTERSECT / INTERSECT ALL	Schnitt
EXCEPT / EXCEPT ALL	Differenz (je nach DB-Hersteller teilweise abweichend — Oracle: MINUS)

★ Grundsätzlich werden **DUPLIKATE ENTFERNT**. Durch den Zusatz **ALL** werden Duplikate **BEIBEHALTEN**.

31.1 Die vier Voraussetzungen ★★★★★

1. Beide **SELECT**-Abfragen müssen **GLEICH VIELE SPALTEN** zurückgeben
2. Die **Spalten** auf den gleichen Positionen müssen **KOMPATIBLE** Datentypen haben (⚠ nicht zwingend gleich!)
3. Die **Spaltennamen** im Gesamtergebnis kommen **IMMER** von der **ERSTEN SELECT**-Abfrage
4. ★ **Sortierung** nur am Ende: **ORDER BY** darf nur **EINMAL**, ganz am Ende stehen

31.2 Praxishinweise der Folie ★

- „In der Praxis spielen **Schnitt** und **Differenz** kaum eine Rolle.“
- ★ „**UNION** kann durch **KEINEN** anderen Befehl ersetzt werden.“
- „** **EXCEPT** könnte über **LEFT JOIN*** gelöst werden. **INTERSECT** könnte über einen **INNER JOIN** gelöst werden.“*

📝 **Beispiel UNION** — Liste aller Nachnamen mit Ort aus `customer` und `employee`:

```
SELECT last_name, city FROM customer
UNION
SELECT last_name, city FROM employee
ORDER BY last_name;
```

📝 **Beispiel EXCEPT** — Nachnamen aus `employee`, die nicht in `customer` vorkommen:

```
SELECT last_name FROM employee
EXCEPT
SELECT last_name FROM customer;
```

Alternative über **JOIN** (mit **DISTINCT**, da **EXCEPT** Duplikate entfernt — nur so ist das Ergebnis identisch):

```
SELECT DISTINCT e.last_name
FROM   employee e
LEFT JOIN customer c ON e.last_name = c.last_name
WHERE  c.last_name IS NULL;
```

★★ Der Kernunterschied JOIN ↔ Mengenoperator (sichere Klausurfrage):

JOINS kombinieren Daten HORIZONTAL, indem sie **Spalten** aus mehreren Tabellen **nebeneinander** in einer neuen Zeile zusammenführen. **Mengenoperatoren kombinieren Daten VERTIKAL**, indem sie **Zeilen** einer Abfrage **unter die Zeilen** einer anderen Abfrage hängen.

32. Beispiele für komplexe Abfragen ★★

Die Vorlesung führt drei Aufgaben mit der **Vorgehensmethodik** vor. **Diese Methodik ist die Klausurstrategie:**

Start: Analyse, in welchen Tabellen die Informationen zu finden sind.

32.1 Beispiel 1 — „Nie gekauft“

Fachliche Fragestellung: „Listen Sie alle Kunden auf, die **noch nie** einen Track aus dem Genre 'Metal' gekauft haben.“

Analyse laut Folie:

- Tabelle `customer` enthält alle Kunden
- Tabelle `genre` enthält die Genres
- Tabelle `invoice_line`: hier sind **alle Käufe von Tracks** enthalten
- Über die Verknüpfung `invoice_line` → `track` → `genre` kann ermittelt werden, welches Genre der Kauf war

Lösung mit Subquery (nicht korreliert):

```
SELECT c.customer_id, c.first_name, c.last_name
FROM   customer c
WHERE  c.customer_id NOT IN (
    SELECT i.customer_id
    FROM   invoice i
    JOIN   invoice_line il ON i.invoice_id = il.invoice_id
    JOIN   track t         ON il.track_id  = t.track_id
    JOIN   genre g         ON t.genre_id   = g.genre_id
    WHERE  g.name = 'Metal'
);
```

Lösung mit Join (LEFT JOIN mit Inline View aller Käufe):

```
SELECT c.customer_id, c.first_name, c.last_name
FROM   customer c
LEFT JOIN (
    SELECT DISTINCT i.customer_id
    FROM   invoice i
    JOIN   invoice_line il ON i.invoice_id = il.invoice_id
    JOIN   track t         ON il.track_id  = t.track_id
    JOIN   genre g         ON t.genre_id   = g.genre_id
    WHERE  g.name = 'Metal'
) AS metalkaeufer ON c.customer_id = metalkaeufer.customer_id
WHERE  metalkaeufer.customer_id IS NULL;
```

★ **Das Muster „noch nie ...“ / „hat kein ...“** löst man **immer** auf eine von zwei Arten: **NOT IN** (Subquery) oder **LEFT JOIN ... WHERE fremdschlüssel IS NULL**.

⚠ **NOT IN und NULL:** Wenn die Subquery **NULL-Werte** liefern kann, gibt **NOT IN** **gar keine Zeilen** zurück! In diesem Fall **NOT EXISTS** oder den **LEFT JOIN** verwenden.

32.2 Beispiel 2 — Aggregation über mehrere Ebenen

Fachliche Fragestellung: „Erstellen Sie eine Übersicht für **jeden Künstler**, die seinen **Namen**, die **Gesamtzahl seiner Alben** sowie den **Gesamtumsatz**, der mit **all** seinen Tracks erzielt wurde, anzeigt.“

Analyse laut Folie:

- `artist` : die Künstler und ihr Name
- `album` : die Alben eines Künstlers
- `track` : die Tracks eines Albums (und über eine weitere Verknüpfung die Tracks eines Künstlers)
- `invoice_line` : enthält `unit_price` und `quantity` — **die Multiplikation ergibt den Umsatz**

Lösung mit JOIN:

```
SELECT ar.name                                AS künstler,
       COUNT(DISTINCT al.album_id)           AS anzahl_alben,
       COALESCE(SUM(il.unit_price * il.quantity), 0) AS gesamtumsatz
FROM   artist ar
LEFT JOIN album      al ON ar.artist_id = al.artist_id
LEFT JOIN track      t  ON al.album_id  = t.album_id
LEFT JOIN invoice_line il ON t.track_id  = il.track_id
GROUP BY ar.artist_id, ar.name
ORDER BY gesamtumsatz DESC;
```

Lösung mit 2 korrelierten Subqueries (in der `SELECT`-Klausel — skalare Unterabfragen):

```
SELECT ar.name,
       (SELECT COUNT(*) FROM album al WHERE al.artist_id = ar.artist_id)
       AS anzahl_alben,
       (SELECT COALESCE(SUM(il.unit_price * il.quantity), 0)
        FROM   invoice_line il
        JOIN   track t ON il.track_id = t.track_id
        JOIN   album a ON t.album_id  = a.album_id
        WHERE  a.artist_id = ar.artist_id)
       AS gesamtumsatz
FROM   artist ar;
```

⚠️ ⚠️ DIE FALLE bei Beispiel 2: Bei der JOIN-Lösung muss man `COUNT(DISTINCT al.album_id)` verwenden — ohne `DISTINCT` würde jedes Album so oft gezählt, wie es Tracks (bzw. Rechnungspositionen) hat! Das ist ein **klassischer Fehler bei Aggregation über mehrere 1:n-Ebenen**.

32.3 Beispiel 3 — Korrelierte Subquery mit Gruppenbezug ★★★★★

Fachliche Fragestellung: „Finden Sie alle Tracks, deren **Einzelpreis höher ist als der Durchschnittspreis aller Tracks IHRES EIGENEN GENRES**.“

Analyse laut Folie: „In der Tabelle `track` sind alle Informationen enthalten. (In der Aufgabenstellung steht nicht, dass der Name des Genre ausgegeben werden soll, somit **kein Join mit Tabelle Genre nötig**.)“

Lösung mit korrelierter Subquery:

```

SELECT t1.track_id, t1.name, t1.unit_price, t1.genre_id
FROM   track t1
WHERE  t1.unit_price > (
        SELECT AVG(t2.unit_price)
        FROM   track t2
        WHERE  t2.genre_id = t1.genre_id      -- ★ die Korrelation!
      );

```

Die Erklärung der Folie ★:

- Die **äußere Abfrage durchläuft die ganze Tabelle track** für jeden Tupel/Datensatz
- Die **innere Abfrage berechnet FÜR JEDEN dieser Tracks den Durchschnittspreis SEINES spezifischen Genres**, indem sie auf die `Genre_Id` der äußeren Abfrage zurückgreift (`t2.genre_id = t1.genre_id`). **Sie wird für jeden Tupel der Hauptabfrage aufgerufen.**

🔑 **Das Erkennungsmerkmal einer korrelierten Subquery:** Die innere Abfrage enthält einen **Verweis auf einen Alias der äußeren Abfrage** (hier `t1.genre_id`).

TEIL E — SQL ZUSATZTHEMEN

33. VIEWS (Sichten) ★★ ★

33.1 Definition

- Mit **VIEWS (Pseudotabellen)** besteht die Möglichkeit, **Abfragen zu speichern** und **wie Tabellen in andere Abfragen einzubinden**
- ★ **VIEWS präsentieren sich auch nach außen (gegenüber Programmen) WIE TABELLEN**
- Erstellt mit **CREATE VIEW** ; die View selbst wird mithilfe eines **SELECT -Befehls** definiert
- **Eine View repräsentiert die Ergebnismenge eines SELECTs als Tabelle**

```

-- Syntax
CREATE VIEW Sichtname AS <Select-Befehl>;
DROP VIEW Sichtname;

-- Beispiel 1: Sicht mit den Spaltennamen der ursprünglichen Tabelle
CREATE VIEW Mitarbeiter_namen AS
SELECT id, vornamen, nachname FROM mitarbeiter;

-- Beispiel 2: Sicht mit NEUEN Spaltennamen
CREATE VIEW Mitarbeiter_namen2 (Nummer, VNAME, NNAME) AS
SELECT id, vornamen, nachname FROM mitarbeiter;

```

33.2 Die drei Vorteile von VIEWS ★★★★★

#	Vorteil	Erläuterung
1	Datenunabhängigkeit / Abstraktionsschicht	„Wenn Programme über VIEWS anstelle von Tabellen zugreifen, hat dies den Vorteil, dass sich die Tabellen hinter der Sicht ändern können, OHNE dass die Programme davon betroffen sind “ (Bsp.: Einfügen weiterer Attribute)
2	Abbildung von Geschäftsregeln	„Mit Datensichten können Geschäftsregeln abgedeckt werden (z. B. Berechnungsformeln in Feldern). Solche Sichten können dann für verschiedene Auswertungen und Berichte verwendet werden“
3	★★★★ Mächtiges SICHERHEITSINSTRUMENT	„Man kann einem Benutzer den Zugriff auf eine Basistabelle verweigern , ihm aber den Zugriff auf eine View erlauben, die nur unkritische Spalten (z. B. keine Gehälter) oder nur bestimmte Zeilen (z. B. nur Mitarbeiter der eigenen Abteilung) anzeigt“

Zusätzlich: **Komplexe oder wiederholende Abfragen können in Views überführt werden.**

33.3 VIEW vs. Inline View ★★★★★

	(Persistente) VIEW	Inline View
Definition	ein benanntes Datenbankobjekt , das mit <code>CREATE VIEW</code> DAUERHAFT im Datenbankschema gespeichert wird	eine temporäre, einmalig verwendete SELECT-Anweisung , die direkt im FROM-Abschnitt der Hauptabfrage definiert wird
Zweck	wiederverwendbare Abstraktionsschicht , deren Code nicht bei jeder Abfrage neu definiert werden muss	Zwischentabelle innerhalb einer einzelnen Abfrage
Alias	nicht nötig	⚠ MUSS einen Alias bekommen

Anwendungen der Inline View:

- **Zwischentabelle zur Vorbereitung von Daten** (z. B. Aggregationen oder komplexe Filterungen)
- **Datenquelle für Joins**
- **Modularisierung komplexer Abfragen**

33.4 ★★★★★ Wann sind Änderungsoperationen über eine VIEW erlaubt?

Über Datensichten können auch **Daten in den zugrunde liegenden Tabellen verändert** werden, wenn die Berechtigungen dies zulassen und die Sicht **keine Einschränkungen** wie z. B. berechnete Spalten aufweist.

NUR wenn ALLE folgenden Bedingungen erfüllt sind, ist das Ändern, Löschen oder Einfügen in der Sicht erlaubt:

#	Bedingung
1	Die <code>FROM</code> -Klausel enthält NUR EINE Relation (⚠ kein JOIN!)
2	Es gibt KEINE GROUP BY -Klausel
3	Die <code>SELECT</code> -Klausel enthält KEIN DISTINCT
4	Die Spaltenliste besteht nur aus einzelnen Spaltennamen — also KEINE Berechnungen, Aggregationen oder konstante Werte
5	Die Operatoren <code>UNION</code> , <code>INTERSECT</code> oder <code>EXCEPT</code> kommen NICHT vor

★ **Diese Liste ist eine sehr wahrscheinliche Klausurfrage** (siehe auch Übungsaufgabe zu `top_kunden`, Teil K).

Zusätzliche Option: `CREATE VIEW v AS ... WITH CHECK OPTION;` — die View prüft bei `INSERT / UPDATE`, ob der neue Datensatz auch **durch die View sichtbar** wäre.

34. INDEX ★★★★★

(Hinweis der Folie: „Nur Auszug — Thema ist vielschichtiger“)

34.1 Zweck und Trade-off ★★★★★

- **Optimierung der Geschwindigkeit von Datenabfragen**
- ⚠️ „Indizes verbessern die LESE-Performance, verschlechtern aber die SCHREIB-Performance (Overhead bei INSERT/UPDATE/DELETE).“
- „Indizierung ist ein **wichtiges Thema für ein performanceorientiertes Datenbankdesign** und die **Akzeptanz eines Systems durch die Endanwender.**“

```
-- Syntax
CREATE INDEX <Indexname> ON <Tabellenname> (<Spaltenname>);
CREATE UNIQUE INDEX <Indexname> ON <Tabellenname> (<Spaltenname>);
DROP INDEX <Indexname>;

-- Beispiel
CREATE INDEX Index2 ON invoice(billing_country);
```

34.2 Die drei Merksätze ★★★★★

1. Ein **PRIMARY KEY** ist (automatisch) indiziert
2. **CREATE UNIQUE INDEX** erzeugt einen eindeutigen Index → Verhalten WIE EIN PRIMARY KEY (da indiziert **und** eindeutig)
3. **EXPLAIN <Abfrage>** kann erste Hinweise geben für sinnvolle Indizes. Darüber hinaus gibt es viele weitere Tools und Vorgehen zur Performance-Optimierung

⚠️ **Die typische Multiple-Choice-Frage (siehe Klausurbeispiel Typ 3):** Ein Index **beschleunigt Such- und Abfragevorgänge** — er erzwingt **weder Eindeutigkeit** (das tut `UNIQUE`) **noch referentielle Integrität** (das tut `FOREIGN KEY`) und **definiert nicht die Struktur** (das tut `CREATE TABLE`).

35. Rechteverwaltung ★★

(Hinweis der Folie: „Nur Auszug — Thema ist vielschichtiger“)

⚠️ „Der **Datenschutzteil von SQL ist wenig standardisiert**. Die Befehle in den einzelnen Datenbanksystemen können relativ stark voneinander abweichen. **Das Grundprinzip ist aber überall ähnlich:** Es gibt **User**, es gibt **Berechtigungen für Datenbankobjekte**, es gibt **Rollen** und **Zuweisungsbefehle.**“

35.1 Das Grundprinzip ★★

🔑 „Die Zugriffskontrolle bzw. Rechtevergabe sollte nach dem Prinzip ‚MINIMALE RECHTE‘ erfolgen – nur Zugriff auf relevante Daten erlauben.“

```
-- Useranlage
CREATE USER <user> WITH PASSWORD <password>;
CREATE USER martin WITH PASSWORD '123456789';
-- ★ Neu erstellte User haben KEINE Rechte
DROP USER <user>;

-- Rechte vergeben (DCL)
GRANT SELECT ON invoice TO martin;
GRANT SELECT (emp_first_name, emp_last_name) ON emp_employee TO martin; -- spaltenweise!
GRANT ALL ON ALL TABLES IN SCHEMA aufgabe2 TO martin;

-- Rechte entziehen
REVOKE SELECT ON aufgabe2.emp_employee FROM martin;
```

35.2 Rollenbasierte Zugriffsrechte (RBAC) ★★★

	Benutzerbasierte Zugriffsrechte	Rollenbasierte Zugriffsrechte (RBAC)
Vergabe	direkt an einzelne User	Es werden Rollen für verschiedene Nutzertypen definiert ; die Rechte werden an die ROLLE vergeben; User erhalten je nach Aufgabe verschiedene Rollen
Bewertung	⚠ „Dies wird schnell unübersichtlich und ist risikobehaftet “	✅ „Zugriffsberechtigungen müssen nicht jedem neuen Benutzer von neuem zugewiesen werden. Stattdessen wird der neue Benutzer einer Rolle zugewiesen, die diese Rechte bereits besitzt “

★ * „Grundsätzlich kann jeder User Mitglied bei beliebig vielen Rollen sein. Jede Rolle kann beliebig viele Mitglieder haben.“ * (m:n-Beziehung)

```
CREATE ROLE buchhaltung;
GRANT SELECT ON invoice TO buchhaltung; -- Rolle erhält die Rechte
GRANT buchhaltung TO martin; -- User erhält die Rolle
DROP ROLE buchhaltung;
```

- ★ **Neu erstellte Rollen haben keine Rechte**
- Die **Vergabe von Rechten an Rollen erfolgt in der gleichen Syntax** wie an User
- „Es ist je nach Datenbanksystem auch möglich, **bestehende Netzwerkrollen zu verwenden**. Dadurch entfällt das separate Anmelden beim Datenbankserver.“

⚠ ⚠ **Die Reihenfolge beim Löschen (siehe Übungsaufgabe 18):**

```
REVOKE SELECT ON customer FROM support_readonly; -- 1. Recht entziehen
DROP USER tim; -- 2. User löschen
DROP ROLE support_readonly; -- 3. Rolle löschen
```

36. System Catalog und INFORMATION_SCHEMA ★★

36.1 System Catalog

- **Implementierung teilweise von der Datenbank abhängig**
- „Im Systemkatalog legt ein **RDBMS die METADATEN** ab. Das sind z. B. Informationen über **Tabellen, Spalten** und weitere Informationen zur Verwaltung des DB-Systems.“
- ★ „Der Systemkatalog sind **REGULÄRE RELATIONALE TABELLEN**“ — man kann ihn also ganz normal mit `SELECT` abfragen!

```
SELECT * FROM pg_attribute WHERE attname LIKE 'billing_country';
-- Attribut billing_country suchen (PostgreSQL-spezifisch)
```

36.2 INFORMATION_SCHEMA ★★★

*„Mit **SQL-2** wurden diese Verwaltungsdaten **normiert**: Jede Datenbank enthält ein **vordefiniertes Schema mit dem Namen `Information_Schema`**. In diesem Schema werden die gesamten Verwaltungsinformationen **in Relationen abgelegt**.“*

★★ **Der entscheidende Vorteil (Klausurfrage!):**

Information_Schema ist ein ISO-STANDARD und somit fast identisch in SQL Server, MySQL oder PostgreSQL. Die `pg_*`-Tabellen hingegen sind POSTGRESQL-SPEZIFISCH.

```
SELECT * FROM Information_Schema.Tables;
-- alle Tabellen der Datenbank auflisten

SELECT * FROM information_schema.TABLES WHERE table_schema = 'chinook';
-- welche Tabellen gibt es im Schema chinook?

SELECT * FROM Information_Schema.Columns WHERE table_name = 'track';
-- alle Spalten der Tabelle track

SELECT * FROM Information_Schema.referential_constraints
WHERE constraint_schema = 'chinook';
-- welche referentiellen Integritätsbedingungen existieren?
```

TEIL F — DATENBANKSEITIGE PROGRAMMIERUNG

37. Business-Logik in der Datenbank? ★★★

Business-Logik kann auch in der Datenbank anstatt in der Anwendungsschicht implementiert werden. Dazu können **Stored Procedures, Trigger und Functions** verwendet werden.

37.1 Vorteile und Risiken ★★★★★

Vorteile	Risiken / Nachteile
Bessere Performance , da Datenbanken auf die Nutzung von Stored Procedures, Functions oder Triggern optimiert sind	Geschäftslogik ist auf mehrere Schichten verteilt (Komplexität)
Nur Ergebnisse werden dem Client übertragen (keine Zwischenergebnisse) → weniger Netzwerk-Traffic	Schlechte Portierbarkeit wegen proprietärer SQL-Dialekte
Teile der zentralen Anwendungslogik liegen auf der Datenbank und können in verschiedenen Anwendungen genutzt werden (⚠️ ABER: Architektur!)	Fehlende Integration in die Entwicklungsumgebung

🔑 * „Ob im jeweiligen Kontext datenbankseitige Logik eingesetzt werden kann oder darf, muss auf Basis der angestrebten **SOFTWARE-ARCHITEKTUR** beantwortet werden.“*

37.2 SQL/PSM und die DBMS-Sprachen ★★

„SQL wurde zu Beginn als **rein deklarative** Programmiersprache konzipiert. Im Laufe der Zeit wurden in die DBMS-Produkte und schließlich auch im ANSI-SQL-Standard die wichtigsten **prozeduralen Sprachelemente** übernommen. Es gibt einen Standard, **SQL/PSM (,Persistent Stored Modules‘)** seit **1996**. Der Standard ist ähnlich zu **Oracle's PL/SQL**. Diese Codekomponenten werden **in der Datenbank gespeichert und vom DBMS ausgeführt**.“

DBMS	Programmiersprachen
Oracle	PL/SQL, Java
PostgreSQL	PL/pgSQL, PL/Perl, PL/Python (auch PL/Tcl)
MS SQL Server	Transact-SQL
MySQL	SQL/PSM
DB2	SQL PL, Java

37.3 ★★★★★ Functions vs. Procedures — die Kernunterscheidung

„Funktionen werden für **BERECHNUNGEN** genutzt, die einen **WERT** liefern. Prozeduren sind für **PROZESSABLÄUFE** geeignet, die **TRANSAKTIONEN STEuern** können (z. B. ein Monatsabschluss, der mittendrin ein **COMMIT** braucht).“

	FUNCTION	PROCEDURE
Rückgabewert	✅ ja (Skalarwert oder Datensatz)	❌ nein (außer über Ausgabeparameter)
In SQL-Anweisungen verwendbar?	✅ ja (z. B. im <code>SELECT</code>)	❌ nein
Aufruf	direkt in SQL: <code>SELECT f(x);</code>	★ <code>CALL f(x);</code> (aus SQL-Skript oder JDBC mit <code>CallableStatement</code>)
Transaktionsbefehle (<code>BEGIN TRANSACTION</code> , <code>SAVEPOINT</code> , <code>ROLLBACK</code> , <code>COMMIT</code>)	❌ NICHT zulässig	✅ erlaubt
Parameter	Eingabeparameter	IN, OUT, INOUT
RETURN	gibt einen Wert zurück (bei PL/pgSQL zwingend)	★ beendet nur die Ausführung — liefert keinen Wert an den Aufrufer
Verfügbar in PostgreSQL	schon immer	★ erst seit Version 11

38. FUNCTIONS ★★★★★

38.1 Allgemeines

- Benutzerdefinierte SQL-Funktionen geben entweder einen **Skalarwert** oder einen **Datensatz** zurück. Der **Datentyp des Rückgabewertes wird während der Funktionsdeklaration festgelegt** (z. B. `RETURNS int`). Mittels `RETURNS void` wird definiert, dass die Funktion **keinen Rückgabewert** hat
- ★ **Funktionen in PostgreSQL dürfen auch Werte in der Datenbank ÄNDERN** („dies ist nicht bei allen DBMS so implementiert“)
- 🗝️ **„Benutzerdefinierte Funktionen können auch in SQL-Anfragen genutzt werden. Mit Funktionen kann man also die SPRACHE SQL ERWEITERN.“**

38.2 LANGUAGE SQL ★★★★★

```
CREATE FUNCTION function_name(param_list)    -- Parameter z. B. a int
RETURNS return_type                        -- Typ des Rückgabewerts, z. B. int
LANGUAGE sql                               -- Sprache: hier SQL
AS
$$
-- Rumpf: Liste von SQL-Statement(s)
-- ⚠️ OHNE Variablendeklarationen und Kontrollstrukturen
--   (nur bei PL/pgSQL möglich)
$$;
```

Input-Parameter: *, „Die Parameter-Namen haben die gleiche Syntax wie Bezeichner in SQL. **Wenn man keinen Parameter-Namen angibt (nur den Datentyp), heißen die Parameter \$1, \$2 usw.**“*

★★★★★ Der Rückgabewert bei `LANGUAGE SQL` — IMPLIZIT (sichere Klausurfrage):

„Der Rückgabewert wird **NICHT** wie in PL/pgSQL durch die explizite Anweisung `RETURN` festgelegt. **Implizite Rückgabe:** Der Rückgabewert ergibt sich **implizit aus dem Ergebnis der LETZTEN SQL-Anweisung** im Funktionskörper. Die letzte Anweisung definiert den Wert: 1) Die letzte Anweisung muss eine **SELECT-Anweisung** sein, oder 2) eine **DML-Anweisung** (`INSERT`, `UPDATE`, `DELETE`) — **mit RETURNING-Klausel.**“

38.3 Dollar-Quoting \$\$ ★★★★★

„Der **Rumpf einer Funktion**, also der eigentliche Programmcode, wird als **String-Konstante** definiert (mit Hochkomma). Die Nutzung von Hochkommas kann ein Problem sein, wenn der **Programmcode selbst String-Konstanten enthält** (' muss verdoppelt werden).“

★ *, „In PostgreSQL kann als Begrenzer auch das **„Dollar-Quoting“** verwendet werden: `$<Tag>$ ... $<Tag>$` — **Tag kann leer sein: \$\$**. Diese dienen dazu, den **Funktionskörper zu umgrenzen und zu kennzeichnen**, unabhängig von der verwendeten Sprache.“*

„String-Konstanten in SQL (egal ob '...' oder \$\$\$...) können **Zeilenumbrüche** enthalten. Somit kann der Code von Funktionen **mehrere Zeilen** enthalten.“

📝 **Beispiel mit Dollar-Quoting:**

```
CREATE FUNCTION addone1(a int)
RETURNS int
RETURNS NULL ON NULL INPUT
LANGUAGE SQL
AS
$$
    SELECT a + 1;
$$;

-- Aufruf aus einem SQL-Statement
SELECT stock, addone1(stock) FROM article;
```

Kurzform (ohne Dollar-Quoting) ★:

```
CREATE FUNCTION addone(a int)
RETURNS int
RETURNS NULL ON NULL INPUT
LANGUAGE SQL
RETURN a + 1;
```

„PostgreSQL ermöglicht eine **Abkürzung für LANGUAGE SQL -Funktionen, die nur aus einem Rückgabeausdruck bestehen**. Wenn das Schlüsselwort **AS** und der Body-Delimiter **\$\$** weggelassen wird, interpretiert PostgreSQL **die nächste Anweisung, die mit einem Semikolon endet, als den Funktionskörper**. Dann wird die Anweisung **RETURN a + 1; implizit als SELECT a + 1 interpretiert**. (⚠ Eigentlich ist **RETURN a+1** kein SQL-Befehl.)“

38.4 Vollständiges Beispiel: LANGUAGE SQL (Chinook) ★★★★★

Aufgabe: „Erstellen Sie eine Funktion, die den Namen eines Kunden basierend auf seiner `Customer_id` zurückgibt. Der Name soll als vollständiger Text ("Vorname Nachname") formatiert sein.“

Schritt 1 — Parameter festlegen und SQL-Befehl definieren:

- **Input:** `Customer_id`, Typ `INT`
- **Rückgabe (= Typ der Function):** `text` oder z. B. `varchar(100)`
- SQL-Befehl: 1) Identifikation über `WHERE "CustomerId" = cust_id;`
2. Ergebnis zusammensetzen: **Konkatenation von Vor- und Nachnamen (||)**

Schritt 2 — Funktion erstellen:

```
CREATE OR REPLACE FUNCTION kundenname(cust_id integer)
RETURNS varchar(100)
LANGUAGE SQL
AS $$
    SELECT "FirstName" || ' ' || "LastName"
    FROM   "Customer"
    WHERE  "CustomerId" = cust_id;
$$;
```

Schritt 3 — Funktion testen:

```
SELECT kundenname(5);
```

38.5 BEGIN ATOMIC ... END ★★

- **BEGIN ATOMIC ... END** ist ein Konstrukt, das in Funktionen der Sprache **LANGUAGE SQL** verwendet wird, um **mehrere SQL-Anweisungen (v. a. DMLs) zu einem logischen Block zusammenzufassen**
- ★ **Transaktionsgarantie:** „Es **garantiert die ATOMARITÄT** dieser Operationen, d. h. **entweder werden alle Schritte erfolgreich ausgeführt oder bei einem Fehler wird die gesamte Operation rückgängig gemacht.**“
- **Einsatz:** wenn die SQL-Funktion **mehr als einen einzelnen SELECT-Ausdruck** benötigt, typischerweise für komplexe **Datenmanipulationen (INSERT, UPDATE, DELETE)**

⚠️ Die Falle bei mehreren Anweisungen in **LANGUAGE SQL** :

```
CREATE FUNCTION addone2(a int)
RETURNS int
LANGUAGE SQL
AS
$$
    SELECT invoice_line_id FROM invoice_line;    -- ⚠️ Ergebnis wird VERWORFEN
    SELECT a + 1
$$;
```

*„**ACHTUNG:** Diese Funktion ist **gültige Syntax** in PostgreSQL, **folgt aber nicht der typischen oder empfohlenen Struktur.**

- PostgreSQL erwartet, dass der Rumpf einer **LANGUAGE SQL** -Funktion **nur einen einzigen Ausdruck** enthält, dessen Ergebnis der Rückgabewert ist
- Wenn mehrere Anweisungen durch Semikolon getrennt angegeben werden, werden **alle Anweisungen vor der letzten ausgeführt, aber ihre ERGEBNISSE WERDEN VERWORFEN.** Nur das Ergebnis der **allerletzten Anweisung** wird als Rückgabewert verwendet“*

🔑 Best Practice der Folie:

„Functions mit **LANGUAGE SQL** ⇒ **ENTWEDER** nur **EIN SELECT** ODER mehrere DMLs (mit **BEGIN ATOMIC/END**). **Ansonsten auf LANGUAGE PL/pgSQL wechseln.**“ (Hinweis: „Wir werden keine DML-Funktionen in Functions verwenden.“)

38.6 ★★★★★ LANGUAGE SQL vs. LANGUAGE PL/pgSQL — die Syntax-Übersicht

Element	LANGUAGE SQL	LANGUAGE PL/pgSQL
\$\$ (Dollar-Quoting)	als Body-Delimiter, um interne Semikola und einfache Anführungszeichen zu vermeiden	dito
AS	Trennung des Codekörpers als String-Literal von der Signatur (Name, Parameter, RETURNS, LANGUAGE)	dito
BEGIN ... END	⚠️ NICHT verwendet!	✅ Block-Struktur für den Codekörper , um Logik (IF, LOOP) zu gruppieren
BEGIN ATOMIC ... END	✅ wenn der Body mehrere DML-Anweisungen enthält (Atomarität)	nicht erforderlich/unüblich
DECLARE	❌ nicht möglich	✅ Variablendeklaration
RETURN	⚠️ nur als Sonderfall (ist kein SQL-Statement) — als Abkürzung für eine Einzelanweisung (implizites SELECT)	★ ZWINGEND ERFORDERLICH , um einen Wert an den Aufrufer zurückzugeben
Rückgabewert	implizit aus der letzten Anweisung	explizit über <code>RETURN</code> Ausdruck
Transaktionskontrolle (COMMIT)	❌ nicht zulässig	❌ nicht zulässig

★ **Der wichtigste Merksatz:** `COMMIT / ROLLBACK` sind in **KEINER** Function erlaubt — weder in `LANGUAGE SQL` noch in `PL/pgSQL`. Nur **PROCEDURES** dürfen das.

39. LANGUAGE PL/pgSQL ★★★★★

39.1 Warum PL/pgSQL?

*„PL/pgSQL ist **ähnlich zu PL/SQL (Oracle)**. Zuweisungen, Schleifen und Bedingungen sind gleich. **PL/pgSQL verbindet die deklarative Abfragesprache SQL mit einer prozeduralen Programmiersprache.**“*

Fähigkeit	Erläuterung
Logik-Kontrolle	Ermöglicht komplexe Ablaufsteuerung (<code>IF</code> , <code>LOOP</code> , <code>WHILE</code>), die in reinem SQL nicht möglich ist
Fehlerbehandlung	Implementierung robuster Fehlerbehandlung (<code>EXCEPTION</code>), um Transaktionen gezielt zu steuern und Fehler abzufangen
Variablen-Nutzung	Erlaubt die Definition und Zuweisung lokaler Variablen — essenziell für komplexe Berechnungen und iterative Verarbeitung

39.2 Die Struktur ★★★★★

```
CREATE FUNCTION function_name(param_list)    -- z. B. zahl1 int
RETURNS return_type                        -- z. B. int
LANGUAGE plpgsql
AS
$$
DECLARE
    -- Variablendeklaration
BEGIN
    -- Code / Logik
    RETURN ...;    -- ★ Rückgabewert EXPLIZIT festlegen
END;
$;
```

DECLARE ★:

„**Alle verwendeten Variablen müssen in dieser Sektion deklariert werden** (★ **Ausnahme: Übergabeparameter** — diese sind **implizit deklariert und in der Funktion verfügbar**). Mit der Deklaration können auch **Initialwerte** zugewiesen werden.“

```
DECLARE
  VAR1 int;
  VAR2 int := 0;      -- mit Initialwert
```

Mögliche Datentypen: `CHAR(n)`, `VARCHAR(n)`, `TEXT`, `INT`, `DATE`, `TIMESTAMP`, `BOOLEAN` etc.

39.3 Prozedurale Sprachelemente ★★

Element	Beschreibung
IF / CASE	legt fest, welche Anweisungen bei Eintreten einer bestimmten Bedingung ausgeführt werden
LOOP	fußgesteuert
WHILE	kopfgesteuert
FOR ... LOOP	Zählschleife
★ SELECT ... INTO <Variable>	Abfrageergebnisse in Variablen speichern (wenn das Ergebnis nur aus EINER Zeile besteht)

```
SELECT COUNT(*) FROM invoice INTO anzahl;
```

„SQL-Befehle, die **keine Werte zurückliefern** (außer Return Code), können **direkt als SQL-Befehl** eingegeben werden.“

39.4 📝 **Beispiel 1 — IF-Verschachtelung** ★★

```
CREATE OR REPLACE FUNCTION ustwert(a int)
  RETURNS int
  LANGUAGE plpgsql
  AS
  $$
  BEGIN
    IF a = 2
      THEN RETURN 7;
    ELSE IF a = 1 THEN RETURN 14;
      ELSE RETURN 0;
    END IF;
  END IF;
END
$$;

-- Abfrage: Aus den VAT-Schlüsseln werden Umsatzsteuersätze erzeugt
SELECT article_id, description, ustwert(vat) FROM article;
```

★ **HINWEIS der Folie:** „LANGUAGE ist `plpgsql`, deshalb **Nutzung von RETURN**! In PL/pgSQL ist das Schlüsselwort `RETURN` **zwingend erforderlich**, um einen Wert an den Aufrufer zurückzugeben.“*

39.5 Beispiel 2 — mit Variablendeklaration ★★★★★

Diese Funktion ist in **Beispiele Klausuraufgaben.pdf** als Typ-1-Aufgabe genannt!

```
-- Ermittelt die Anzahl der Rechnungen, die in einer Preisspanne liegen
CREATE OR REPLACE FUNCTION anzahl(untergrenze int, oberegrenze int)
RETURNS int
LANGUAGE plpgsql
AS
$$
DECLARE
    anzahl int;
BEGIN
    SELECT Count(*)
    FROM invoice
    INTO anzahl
    WHERE total BETWEEN untergrenze AND oberegrenze;
    RETURN anzahl;
END
$$;

-- ★ Der Aufruf aus den Klausurbeispielen:
SELECT count(*) AS Gesamtanzahl, anzahl(10,100) AS imBereich FROM invoice;
```

★★★★★ Was liefert dieser SELECT? — Genau EINE Zeile mit zwei Spalten:

- **Gesamtanzahl** = Anzahl **aller** Rechnungen in **invoice**
- **imBereich** = Anzahl der Rechnungen mit **total** **zwischen 10 und 100** Der Wert **imBereich** ist in **allen Zeilen identisch**, da die Funktion **nicht von der aktuellen Zeile abhängt** — und da **count(*)** aggregiert, bleibt genau eine Zeile übrig.

★ **Hinweis der Folie:** „In diesem Beispiel wäre es eigentlich **einfacher**, **LANGUAGE SQL** zu nehmen“:

```
CREATE OR REPLACE FUNCTION anzahl(untergrenze int, oberegrenze int)
RETURNS int
LANGUAGE SQL
AS $$
    SELECT Count(*) FROM invoice WHERE total BETWEEN untergrenze AND oberegrenze;
$$;
```

39.6 Beispiel 3 — Berechnung mit lokalen Variablen

```
-- Bruttobetrag eines Tracks (UnitPrice × 1,19)
CREATE OR REPLACE FUNCTION brutto(track_id_param integer)
RETURNS numeric
LANGUAGE plpgsql
AS $$
DECLARE
    mwst numeric := 0.19;
    netto numeric;
BEGIN
    -- 1. Nettopreis abfragen
    SELECT unit_price INTO netto
    FROM track
    WHERE track_id = track_id_param;
    -- 2. Bruttobetrag berechnen und zurückgeben
    RETURN netto * (1 + mwst);
END;
$$;

SELECT track_id, name, unit_price, brutto(track_id) FROM track;
```

40. PROCEDURES

„Lange Zeit hatte PostgreSQL nur CREATE FUNCTION. Seit Version 11 hat PostgreSQL auch CREATE PROCEDURE. Die Sprachauswahl ist gleich wie bei FUNCTIONS.“

Die wesentlichen Unterschiede zu FUNCTIONS (Wiederholung, Abschnitt 37.3):

- Prozeduren **können nicht innerhalb von SQL-Anweisungen verwendet werden** und **liefern keinen Wert zurück** (es sei denn über einen Ausgabeparameter)
- **Aufruf über CALL** (z. B. aus SQL-Skript) oder über **JDBC mit einem CallableStatement**
-  **Eine gespeicherte Prozedur KANN Transaktionsbefehle enthalten** (BEGIN TRANSACTION , SAVEPOINT , ROLLBACK , COMMIT) — **Funktionen können das nicht**
- **Parameter: IN** (Eingabe), **OUT** (Ausgabe), **INOUT** (beides)
-  **RETURN dient AUSSCHLIESSLICH dazu, eine PROCEDURE zu beenden** und liefert **keinen Wert** an den Aufrufer. **Werte werden immer über OUT-Parameter zurückgegeben**; RETURN dient nur der **Steuerung des Programmflusses**

 **Beispiel 1 — einfache Prozedur:**

```
CREATE OR REPLACE PROCEDURE change_support_staff(
    p_customer_id INT, -- Eingabeparameter Kundennummer
    p_new_support_id INT) -- Eingabeparameter neuer Supportmitarbeiter
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE Customer
    SET Support_Rep_Id = p_new_support_id
    WHERE Customer_Id = p_customer_id;
END;
$$;

CALL change_support_staff(1, 5);
```

Beispiel 2 — mit OUT-Parametern und COMMIT ★★:

```

CREATE OR REPLACE PROCEDURE add_genre(
    IN new_genre    text,    -- Eingabeparameter Name des Genre
    IN new_genre_id int,    -- Eingabeparameter ID des Genre
    OUT result_message text, -- Ausgabeparameter (1. Rückgabe)
    OUT new_count   int)    -- Ausgabeparameter (2. Rückgabe)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO genre (name, genre_id) VALUES (new_genre, new_genre_id);
    COMMIT;                                -- ★ Transaktion explizit beenden
    SELECT COUNT(genre_id) INTO new_count FROM genre;
    result_message := 'Datensatz erfolgreich hinzugefügt. ID: '
                    || new_genre_id || ', Name: ' || new_genre;
    RETURN;                                -- kein RETURN notwendig
END;
$$;

CALL add_genre('Heavy Metal', 97, NULL, NULL);

```

★ **Aufruf aus JDBC:** „Stored Procedures werden in JDBC mit einem `CallableStatement` aufgerufen. **OUT-Parameter müssen mit ihrem Typ REGISTRIERT werden.**“*

41. TRIGGER ★★★★★

41.1 Definition und Konzept

- **Trigger sind gespeicherte Prozeduren, die bei bestimmten TABELLENEREIGNISSEN (INSERT , UPDATE , DELETE) ausgeführt werden**
- ★ **Der Mechanismus ist eine Art von „HOOK“ („Haken“), um EIGENEN CODE IN DIE NORMALE ABARBEITUNG durch das DBMS EINZUFÜGEN**

Die drei Eigenschaften eines Triggers ★★:

1. **Ein Trigger hat KEINE Übergabeparameter**
2. **Ein Trigger liefert NICHTS zurück**
3. **Ein Trigger ist MIT EINER TABELLE VERBUNDEN**

Anwendungen von Triggern ★:

- **Die Überwachung komplexer Integritätsbedingungen** (wenn nicht über Angaben im `CREATE TABLE` - Statement möglich)
- **Änderungs-Propagierung zu redundanten Datenstrukturen**
- **Absicherung komplexer Geschäftsregeln direkt in der Datenbank**

41.2 ⚠️⚠️ Die Risiken

- „Die Verwendung von Triggern kann schnell zu **komplexen und unübersichtlichen Situationen** führen, da **Trigger weitere Trigger auslösen können**“
- „Für Entwickler **UNSIHTBAR’ AKTIV** (sehr problematisch, wenn der Anwendungsentwickler nicht auch der Datenbankentwickler ist)“

41.3 ★★★★★ Trigger und Trigger-Funktion — die zwingende Zweiteilung

🔑 „Trigger in PostgreSQL können NICHT DIREKT PROGRAMMCODE ENTHALTEN, sondern müssen eine ‚TRIGGER FUNCTION‘ AUFRUFEN. Trigger Functions sind dadurch gekennzeichnet, dass sie den ERGEBNISTYP `trigger` haben.“

„Der Code, der bei Auslösung des Triggers ausgeführt werden soll, muss in einer **SEPARATEN Trigger-Funktion gekapselt** werden. Der `CREATE TRIGGER`-Befehl dient **lediglich dazu, diese Funktion an ein bestimmtes Tabellenereignis zu BINDEN.**“

```
-- (1) DIE TRIGGER-FUNKTION
CREATE FUNCTION trigger_function()
RETURNS TRIGGER          -- ★ ZWINGEND: Rückgabotyp TRIGGER
LANGUAGE PLPGSQL
AS $$
BEGIN
    -- trigger logic
END;
$$;

-- (2) DER TRIGGER
CREATE TRIGGER trigger_name
{BEFORE | AFTER} { event }          -- Zeitpunkt + Ereignis
ON table_name                      -- Tabelle
[FOR [EACH] { ROW | STATEMENT }]    -- Granularität
EXECUTE FUNCTION trigger_function;
```

41.4 Die vier Dimensionen eines Triggers ★★★★★

Dimension	Optionen
Zeitpunkt	<code>BEFORE</code> oder <code>AFTER</code>
Ereignis	<code>INSERT</code> , <code>DELETE</code> , <code>UPDATE</code> (auch kombiniert mit <code>OR</code>)
Genereller Trigger vs. Spaltentrigger	<code>ON table</code> (generell) / <code>OF attribute ON table</code> (Spaltentrigger — ⚠️ nur bei <code>UPDATE</code>)
Granularität	<code>FOR EACH ROW</code> oder <code>FOR EACH STATEMENT</code>

★★★★★ BEFORE vs. AFTER :

BEFORE-Trigger werden **VOR der Operation** ausgeführt — sie können die Operation für die Zeile noch **STOPPEN** bzw. die einzufügende oder zu aktualisierende Zeile noch **VERÄNDERN**. **AFTER-Trigger** werden **NACH der Operation** ausgeführt.

★★★★★ FOR EACH ROW vs. FOR EACH STATEMENT :

Zeilen-Trigger (FOR EACH ROW) werden **EINMAL FÜR JEDE betroffene Zeile** aufgerufen.
Anweisungs-Trigger (FOR EACH STATEMENT) werden **NUR EINMAL für die ganze Anweisung** aufgerufen. ⚠ **In Zeilen-Triggern kann man auf die betroffenen Zeilen zugreifen, in Anweisungs-Triggern NICHT.**

Die beiden Beispiele der Folie:

```
-- Beispiel (1): Trigger auf Tabelle (ON) – genereller Trigger
CREATE TRIGGER log_employee_deletion
BEFORE DELETE ON Employee          -- Zeitpunkt und Ereignis: VOR der Löschung
FOR EACH ROW                        -- Granularität: für jede gelöschte Zeile
EXECUTE FUNCTION audit_employee_delete();

-- Beispiel (2): Trigger auf SPALTE (OF ... ON) → ⚠ nur bei UPDATE möglich
CREATE TRIGGER log_employee_deletion
AFTER UPDATE OF Email ON Employee -- Zeitpunkt und Ereignis: NACH Update
FOR EACH STATEMENT                -- Granularität: 1x für das Statement
EXECUTE FUNCTION audit_employee_delete();
```

41.5 OLD und NEW ★★★★★

- Bei **UPDATE** und **INSERT** kann mit **NEW.*** auf die **NEUEN Daten** zugegriffen werden
- Bei **UPDATE** und **DELETE** kann mit **OLD.*** auf die **ALTEN Daten** zugegriffen werden
- (Beachte: **UPDATE enthält NEUE UND ALTE Daten**)

Ereignis	OLD verfügbar?	NEW verfügbar?
INSERT	✗	✓
UPDATE	✓	✓
DELETE	✓	✗

41.6 Vordefinierte Variablen in Trigger-Funktionen ★★

„In einer Trigger Function sind eine ganze Reihe **lokaler Variablen AUTOMATISCH deklariert** — über diese kann man die Daten des Triggers abfragen.“

Variable	Bedeutung
OLD	Tupel VOR der Änderung (bei Zeilen-Triggern)
NEW	Tupel NACH der Änderung (bei Zeilen-Triggern)
TG_OP	Operation , z. B. "INSERT"
TG_NAME	Name des Triggers , der die Funktion aufruft
TG_TABLE_NAME	Name der Tabelle , zu der der Trigger gehört
TG_NARGS	Anzahl Argumente (vom Aufruf im Trigger)
TG_ARGV	Array mit Argumentwerten (Index ab 0)

★ „Trigger-Funktionen haben **IMMER eine LEERE Parameter-Liste**. Eingabewerte bekommt die Trigger-Funktion über diese **vordefinierten Variablen**.“

41.7 Syntaxhinweise für Trigger-Funktionen ★★

Element	Hinweis
LANGUAGE	★ Ohne Angabe wird standardmäßig PL/pgSQL angenommen
AS	trennt Funktionskopf vom Body
\$\$	umschließt den Body (da er BEGIN/END und interne Semikola enthält)
BEGIN ... END	definiert den prozeduralen Logikblock
DECLARE	wenn lokale Variablen benötigt werden
★★ RETURN NEW	„Die Anweisung <code>RETURN [NEW/NULL]</code> ist bei BEFORE-Triggern ZWINGEND ERFORDERLICH , da sie der Datenbank den (eventuell modifizierten) Datensatz (<code>NEW</code>) oder das ABBRUCHSIGNAL (<code>NULL</code>) für die nachfolgende DML-Operation liefert. Bei AFTER-Triggern ist eine Rückgabe technisch nicht relevant, häufig wird aber <code>RETURN NULL;</code> gesetzt.“

★★ Merke: `RETURN NULL` in einem BEFORE-Trigger BRICHT die Operation für diese Zeile ab!

41.8 Trigger-Beispiel 1: Lagerbestand sichern

```
-- Der TRIGGER
CREATE OR REPLACE TRIGGER sichernlagerbestand
BEFORE INSERT OR UPDATE OF stock
ON article
FOR EACH ROW
EXECUTE FUNCTION sichern();

-- Die TRIGGER-FUNKTION
CREATE OR REPLACE FUNCTION sichern()
RETURNS TRIGGER          -- ★ Trigger-Funktionen müssen immer TRIGGER zurückgeben
LANGUAGE plpgsql
AS
$$
BEGIN
    NEW.oldstock = OLD.stock;  -- alten Wert in die Spalte oldstock kopieren
    RETURN NEW;              -- ★ zwingend bei BEFORE-Trigger
END;
$$;
```

„Dieser Befehl bewirkt, dass **der alte Wert der Spalte `stock` (`OLD.stock`)**, so wie er vor der **Aktualisierung war, in die Spalte `oldstock` (`NEW.oldstock`) der Zeile kopiert wird, BEVOR die Änderung in der Datenbank festgeschrieben wird.**“

41.9 Trigger-Beispiel 2: Plausibilitätsprüfung mit EXCEPTION ★★★★★

```
-- Die TRIGGER-FUNKTION
CREATE OR REPLACE FUNCTION validate_hire_date()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    geburstag date;
    einstelltag date;
BEGIN
    geburstag := NEW.birth_date;    -- Daten aus der NEUEN Zeile in lokale Variablen
    einstelltag := NEW.hire_date;

    IF einstelltag <= geburstag THEN    -- Wurde die Person vor der Geburt eingestellt?
        RAISE EXCEPTION 'Fehler: Geburtstag nach Einstelldatum';    -- ★ Abbruch
    END IF;

    RETURN NEW;    -- Wenn die Prüfung bestanden wurde, die Operation fortsetzen
END;
$$;

-- Der TRIGGER
CREATE TRIGGER check_hire_date
BEFORE INSERT ON employee
FOR EACH ROW
EXECUTE FUNCTION validate_hire_date();
```

★ **RAISE EXCEPTION '...'** ist der Befehl zum **Abbruch mit Fehlermeldung** — sehr prüfungsrelevant, weil es in mehreren Übungsaufgaben verlangt wird.

41.10 ★★★★★ Die drei Einschränkungen für Trigger-Funktionen

#	Einschränkung
1	*„In Trigger-Funktionen können KEINE Kommandos ausgeführt werden, die ein Abfrageergebnis liefern (also vor allem kein SELECT). Die EINZIGE Ausnahme ist SELECT ... INTO var , um einen einzelnen Wert zu ermitteln und in einer Variablen zu speichern.“*
2	„Durch die Trigger-Funktion darf die Tabelle, für die der Trigger ausgeführt wird, NICHT VERÄNDERT werden . Der Code darf also keine INSERT -, UPDATE - und DELETE -Kommandos für die aktuelle Tabelle enthalten .“ ★ ABER: „Ein BEFORE-Trigger darf die Daten der eigenen Tabelle ändern über die Zuweisung zu den Variablen NEW oder OLD — nur eben nicht über einen expliziten SQL-DML-Befehl “ (siehe Beispiel <code>sichernlagerbestand</code>)
3	„In Trigger-Funktionen dürfen KEINE Transaktions- und Locking-Kommandos ausgeführt werden.“ ★ Begründung: „Da Trigger Teil der aufrufenden DML-Operation sind , dürfen sie die Transaktionskontrolle nicht eigenständig beeinflussen .“

TEIL G — TRANSAKTIONEN

42. Grundlagen ★★★★★

*„Nutzer/Systeme/Prozesse greifen **gleichzeitig auf dieselben Daten zu oder ändern sie sogar gleichzeitig (concurrent update)**. Das Datenbanksystem muss diese Zugriffe so organisieren, dass durch gleichzeitige Zugriffe **nicht irgendwelche Tupel in unerwünschter Weise geändert** werden.“*

42.1 Definition ★★★★★

🔑 „Als TRANSAKTIONEN werden FOLGEN VON DATENMANIPULATIONEN bezeichnet, die eine LOGISCHE EINHEIT bilden (englisch: LOGICAL UNIT OF WORK). Eine einzelne Operation könnte, für sich betrachtet, die Konsistenz der Datenbank ZERSTÖREN. Werden sie aber ALLE ausgeführt, geht die Datenbank wieder in einen konsistenten Zustand über.“

Die Aufgabe des Transaktionsmanagements:

Die **ordnungsgemäße Beendigung von Transaktionen zu kontrollieren**. Dabei muss es **Situationen erkennen und bewältigen, die die Konsistenz gefährden**. Falls der erfolgreiche Abschluss nicht möglich ist, muss es die Datenbank auf einen **bekanntermaßen konsistenten Zustand zurückführen**, d. h. **alle seit Beginn der Transaktion vorgenommenen Einzeländerungen rückgängig machen (über LOG-EINTRÄGE)**.

43. ACID ★★★★★

Die wichtigste Merkstruktur des gesamten Moduls.

Buchstabe	Englisch	Deutsch	Definition
A	Atomicity	Atomar	Transaktion wird ENTWEDER KOMPLETT ODER GAR NICHT durchgeführt
C	Consistency	Konsistenz	Nach einer Transaktion befinden sich alle Daten in einem KONSISTENTEN Zustand
I	Isolation	Isolation	Gleichzeitig ausgeführte Transaktionen BEEINFLUSSEN SICH NICHT gegenseitig
D	Durability	Dauerhaft	Änderungen von Transaktionen verbleiben DAUERHAFT in der Datenbank

🔑 „Einen Anweisungsblock, der den ACID-Anforderungen entspricht, nennt man TRANSAKTION.“

43.1 Vertiefung: Was bedeuten die vier Kriterien genau? ★★★★★

(aus der Musterlösung zu Übungsaufgabe 11 — sehr klausurnah formuliert)

Kriterium	Ausführliche Erläuterung
Atomizität	Stellt sicher, dass eine Transaktion entweder vollständig ausgeführt und umgesetzt wird, oder gar nicht . Der altgriechische Ursprung des Wortes soll ausdrücken, dass die Anweisungen der Transaktion nicht weiter aufteilbar sind. Bei einem Fehler oder Abbruch werden alle seit Beginn vorgenommenen Einzeländerungen rückgängig gemacht (ROLLBACK)
Konsistenz	Garantiert, dass die Datenbank nach erfolgreichem Abschluss alle vordefinierten Regeln, Integritätsbedingungen und Geschäftsvorgaben einhält . War die DB vorher konsistent, sorgt ein erfolgreiches COMMIT für einen konsistenten Zustand danach; bei Unterbrechung stellt der ROLLBACK die Ausgangskonsistenz wieder her
Isolation	Gleichzeitig ausgeführte Transaktionen beeinflussen sich nicht. Ergebnisse bleiben BIS ZUM COMMIT für andere Transaktionen UNSICHTBAR , sodass jede Transaktion so abläuft, als wäre sie die einzige im System . Der genaue Grad wird durch die ISOLATION LEVEL festgelegt
Dauerhaftigkeit	Nach erfolgreichem COMMIT müssen die Änderungen dauerhaft und permanent gespeichert sein — auch bei einem anschließenden Systemabsturz (über Log-Einträge gesichert)

★ **Warum müssen Zwischenzustände unsichtbar sein?** „Da die Konsistenz **WÄHREND** der Ausführung einer Transaktion **NICHT** garantiert werden kann, müssen Ergebnisse bis zum **COMMIT** für andere unsichtbar bleiben, damit diese **nicht auf inkonsistenten Zuständen arbeiten**.“

44. Transaktionen in SQL ★★

```
START TRANSACTION;      -- Transaktion beginnen
...                     -- DML-Anweisungen
COMMIT;                 -- dauerhaft festschreiben
-- oder
ROLLBACK;               -- alle Änderungen zurücknehmen
```

Die Kernaussagen:

- Mit Transaktionen können **Datenbankanweisungen gruppiert** werden
- Eine solche Gruppe wird **ganz oder gar nicht durchgeführt (atomar)**
- ★ „Ein RDBMS verliert **KEINE** abgeschlossenen Transaktionen!“
- Eine **Verzahnung von Transaktionen** kann durch geeignete **Isolation Level** eingestellt werden

44.1 AUTO COMMIT ★★★

„Verschiedene DBMS können über Systemvariablen so konfiguriert werden, dass sie **nach jeder UPDATE -, INSERT - oder DELETE -Anweisung AUTOMATISCH EIN COMMIT** durchführen (AUTO COMMIT).“

Vorteil	Nachteil
Sinnvoll, wenn alle Änderungen mit einer einzigen Anweisung erledigt werden	⚠ Sollen mehrere Änderungen im Zusammenhang betrachtet werden, MUSS dieses automatische COMMIT unterbunden werden
Andere Benutzer können möglichst bald mit dem aktualisierten Datenbestand arbeiten	

★ „**Standardmäßig arbeiten DBMS oft mit AUTO COMMIT**. Hier wird **jeder einzelne Befehl direkt auf der Datenbank ausgeführt**. Wird AUTO COMMIT deaktiviert, können Transaktionen mit **START TRANSACTION** gestartet und mit **COMMIT** (bzw. **ROLLBACK**) beendet werden.“

⚠ **Praxishinweis der Übungen:** „Bitte vorher in DBeaver Autocommit ausschalten“ — sonst funktionieren die Transaktionsübungen nicht!

45. Nebenläufigkeitsanomalien ★★★★★

🔑 **„GRUNDREGEL DER CONCURRENCY: JEDE TRANSAKTION HAT SO ABZULAUFEN, ALS SEI SIE ALLEIN IM SYSTEM.“** Aber wenn mehrere Transaktionen auf die gleichen Daten zugreifen, können sich **Nebenläufigkeitsanomalien** ergeben:

#	Problem	Beschreibung
1	Problem der verlorengegangenen Änderung (<i>Lost Update</i>)	
2	Problem der Abhängigkeit von noch nicht abgeschlossenen Transaktionen	
3	Problem der Inkonsistenz der Daten	

★ **„Nebenläufigkeitsanomalien sind VERLETZUNGEN DER ISOLATION (ACID).“**

45.1 Das Lost-Update-Problem im Detail ★★

(aus der Musterlösung zu Übungsaufgabe 3)

„Dieses Problem tritt auf, wenn zwei Transaktionen (TA1 und TA2) gleichzeitig denselben Datensatz LESEN, ihn MODIFIZIEREN und anschließend NACHEINANDER ZURÜCKSCHREIBEN. Da die zweite Transaktion das Ergebnis der ersten OHNE PRÜFUNG ÜBERSCHREIBT, geht der Update von TA1 VERLOREN. In der SQL-Norm wird dieses Phänomen auch als ‚DIRTY WRITE‘ bezeichnet und ist GRUNDSÄTZLICH VERBOTEN.“

Zeit	Transaktion 1	Transaktion 2
t1	LESE Kontostand = 1000	
t2		LESE Kontostand = 1000
t3	SCHREIBE $1000 + 100 = 1100$	
t4		SCHREIBE $1000 + 200 = 1200$
Ergebnis: 1200 – die +100 von TA1 sind VERLOREN (erwartet wären 1300)		

46. Synchronisationsstrategien ★★★★★

*„Es werden Mechanismen benötigt, die die Grundregel der Concurrency auch bei gleichzeitigem Zugriff absichern. **Pessimistische und optimistische Synchronisationsverfahren verhindern Nebenläufigkeitsanomalien** und tragen dazu bei, dass die Datenbank konsistent bleibt. Diese sichern die **ISOLATION (I in ACID) ab.**“*

🔑 **SERIALISIERUNG:** *„Das *Ergebnis mehrerer gleichzeitig ausgeführter Transaktionen ist IDENTISCH zu einem Ergebnis, das entstanden wäre, wenn die Transaktionen NACHEINANDER (SERIELL) ausgeführt worden wären.“

	Pessimistische Synchronisation	Optimistische Synchronisation
Grundannahme	Konflikte treten HÄUFIG auf	Konflikte treten SELTEN auf
Strategie	SPERREN SETZEN, BEVOR man liest oder schreibt	Transaktionen laufen OHNE Sperren. AM ENDE wird geprüft, ob Konflikte aufgetreten sind
Typisches Verfahren	Zweiphasen-Sperrprotokoll (2PL)	Validierung
Vorteil	sicher, verhindert Konflikte zuverlässig	hohe Parallelität
Nachteil	blockierend, DEADLOCKS möglich	Konflikte führen zu ABBRÜCHEN (Rollback)
Merksatz	„Pessimistisch = SPERREN, BEVOR etwas passiert.“ Serialisierung durch SPERREN.	„Optimistisch = ERST ARBEITEN, DANN PRÜFEN.“ Serialisierung durch VALIDIERUNG.

⚠ „Der **Implementierungsaufwand pessimistischer Strategien ist hoch**. Es ist eine **Sperrverwaltung** einzuführen, die die komplette Serialisierung miteinander konkurrierender Transaktionen übernimmt.“

47. Concurrency Level (die vier Anomalien) ★★

SQL definiert vier Concurrency-Level (welche Nebenläufigkeitsanomalien erlaubt werden bzw. akzeptiert wird):

#	Anomalie	Beschreibung
1	Dirty Write	Entspricht dem Problem der verlorengegangenen Änderung. ★ Die SQL-Norm verweist darauf, dass Dirty Writes GRUNDSÄTZLICH VERBOTEN sind — in allen Isolation Levels
2	Dirty Read	Problem der Abhängigkeit von noch nicht abgeschlossenen Transaktionen. In diesem Level können Daten gelesen werden, die so NIE GÜLTIG werden
3	Non Repeatable Read	Es wird nicht garantiert, dass ein einmal gelesener Eintrag innerhalb derselben Transaktion unverändert bleibt. Andere Transaktionen können Inhalte ändern → ein wiederholter Zugriff kann zu UNTERSCHIEDLICHEN ERGEBNISSEN führen
4	Phantom	Es wird erlaubt, dass beim wiederholten Lesen innerhalb einer Transaktion EINE ODER MEHRERE ZEILEN AUFTAUCHEN, die beim vorherigen Lesen noch nicht vorhanden waren. Eine andere Transaktion könnte neue Zeilen eingefügt haben — diese heißen Phantome

★★ Der Unterschied Non Repeatable Read ↔ Phantom:

- **Non Repeatable Read:** derselbe **Datensatz** hat beim zweiten Lesen einen **anderen Wert** (UPDATE durch eine andere Transaktion)
- **Phantom:** beim zweiten Lesen sind **zusätzliche Datensätze** da (INSERT durch eine andere Transaktion)

48. Isolation Level ★★☆☆

*„Der **Concurrency Level** beschreibt Nebenläufigkeitsanomalien, die auftreten **können**. Durch die Definition geeigneter **Isolation Level** können unerwünschte Probleme **verhindert** werden. **Der ISOLATION LEVEL legt fest, WIE STARK eine Transaktion von den Auswirkungen anderer, gleichzeitig laufender Transaktionen ABGESCHIRMT wird.** Der Isolation Level ist die Einstellung, **welche Nebenläufigkeitsprobleme in Kauf genommen werden dürfen, um eine höhere Parallelität zu erreichen.*“

Die SQL-Norm (SQL:2003) beschreibt vier Isolationslevel ★★☆☆:

Isolation Level	Dirty Write	Dirty Read	Non Repeatable Read	Phantom
READ UNCOMMITTED	✗ verboten	⚠ möglich	⚠ möglich	⚠ möglich
READ COMMITTED	✗ verboten	✓ verhindert	⚠ möglich	⚠ möglich
REPEATABLE READ	✗ verboten	✓ verhindert	✓ verhindert	⚠ möglich
SERIALIZABLE	✗ verboten	✓ verhindert	✓ verhindert	✓ verhindert

★★★ Der Trade-off (sichere Klausurfrage):

- „Um ein **höheres Isolationslevel** zu gewährleisten (mehr Probleme verhindern), muss die Datenbank **strengere Sperr- und Versionsmechanismen (Locking/Versioning)** anwenden. Dies führt zu **mehr Wartezeiten und Overhead.**“
- „Die Wahl eines **NIEDRIGEN** Isolation Levels (z. B. **READ UNCOMMITTED**) **MAXIMIERT die Concurrency** (viele Transaktionen laufen parallel), **lässt aber die meisten Concurrency-Probleme zu.**“
- *„Die Wahl eines **HOHEN** Isolation Levels (z. B. **SERIALIZABLE**) **MINIMIERT die Concurrency-Probleme, reduziert jedoch die allgemeine Parallelität (Performance)**, da Transaktionen häufiger warten müssen.“*

48.1 📝 Beispiel **READ COMMITTED** — das Problem

*„Die gleiche Anweisung innerhalb einer Transaktion kann zu **UNTERSCHIEDLICHEN ERGEBNISSEN** führen. Die Ergebnisse sind somit **nicht wiederholbar (non-repeatable)**, was zu unerwünschten Effekten innerhalb einer Transaktion führen kann.“* **Ein COMMIT in Session 1 macht den geänderten Wert für Session 2 sofort verfügbar.**

48.2 📝 Beispiel **REPEATABLE READ** — die Lösung ★★☆☆

*„Das **Ziel dieser Isolationsebene** ist, dass in einer Transaktion, die selbst die Daten nicht verändert, **die gleiche Auswertung das gleiche Ergebnis liefert.** ★ **Selbst die COMMIT -Anweisungen in Session 1 führen NICHT dazu, dass in beiden Transaktionen die gleichen Werte gelten** (keine Auswirkung auf Session 2!). ★ **Erst der COMMIT in Session 2 macht die Veränderungen von Session 1 in Session 2 sichtbar** (durch Beenden der vorherigen Transaktion in Session 2). Somit ist gewährleistet, dass **beide Sessions/Transaktionen mit IN SICH KONSISTENTEN Datenbeständen arbeiten.** Ein Repeatable Read wird gewährleistet.“*

📝 Der Versuchsaufbau (Übungsaufgabe 10) — muss man reproduzieren können:

```

-- SESSION 1
SET SCHEMA 'chinook';
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL
    REPEATABLE READ;

UPDATE genre SET name = 'RR'
    WHERE genre_id = 5;
COMMIT;

-- SESSION 2
SET SCHEMA 'chinook';
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL
    REPEATABLE READ;
SELECT * FROM genre WHERE genre_id = 5;    -- (1)

SELECT * FROM genre WHERE genre_id = 5;    -- (2)
-- ★ Änderung trotz COMMIT in Session 1
--   NICHT sichtbar!
COMMIT;
SELECT * FROM genre WHERE genre_id = 5;    -- (3)
-- ★ JETZT ist die Änderung sichtbar

```

49. Sperrmechanismen (Locks) und Deadlocks ★★★★★

49.1 Locks

*„Pessimistische Verfahren erfordern **Sperrmechanismen**. Diese Sperren werden als **LOCKS** bezeichnet. **Locks sind ein Schutz vor gleichzeitigen Zugriffen auf Tabellen.**“*

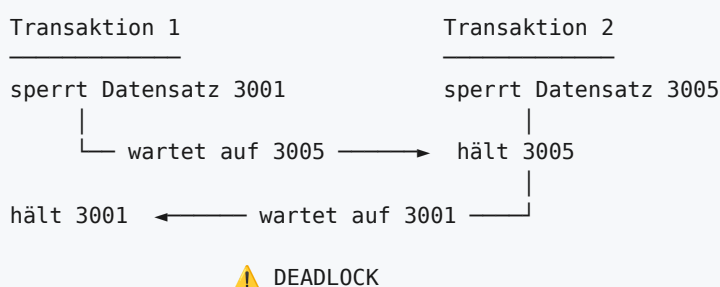
Die vier Regeln:

1. **Zu jeder Relation existiert ein Lock**
2. Eine Transaktion **holt vor jedem Zugriff auf eine Relation automatisch den dazugehörigen Lock**, falls sie diesen noch nicht besitzt
3. **Ist der Lock von einer anderen Transaktion belegt, wartet die Transaktion bis zur Lockfreigabe in einer WARTESCHLANGE**
4. **Bei Transaktionsende werden ALLE von dieser Transaktion gehaltenen Locks freigegeben**

★ „Dies führt dazu, dass sich **Transaktionen gegenseitig synchronisieren**, indem sie **nacheinander** auf Tabellen zugreifen.“

49.2 Deadlock ★★★★★

🔑 *„Ist die Freigabe einer Sperrung A von einer Sperrung B abhängig, die wiederum von der Freigabe der Sperrung A abhängt, spricht man von einem **DEADLOCK**.“* „Wenn mehrere Transaktionen **wechselseitig auf die Freigabe von Ressourcen warten**, spricht man von einer **VERKLEMMUNG oder einem DEADLOCK**.“



50. Sperrprotokolle und Commit-Protokolle ★★★★★

„**Concurrency** erzeugt potentiell Nebenläufigkeitsprobleme. **Isolation Levels** legen fest, welche Nebenläufigkeitsprobleme verhindert werden sollen. Um sie zu verhindern, verwenden pessimistische Verfahren **Sperren**. ► **Das wichtigste formale Sperrverfahren ist das ZWEIFHASSEN-SPERRPROTOKOLL (2PL): Es garantiert SERIALISIERBARKEIT (Isolation).** Isolation reicht in **verteilten** Datenbanksystemen **NICHT** aus: ► **Zusätzlich wird das ZWEIFHASSEN-COMMIT-PROTOKOLL (2PC) benötigt, um Transaktionen über MEHRERE KNOTEN ATOMAR abzuschließen.**“

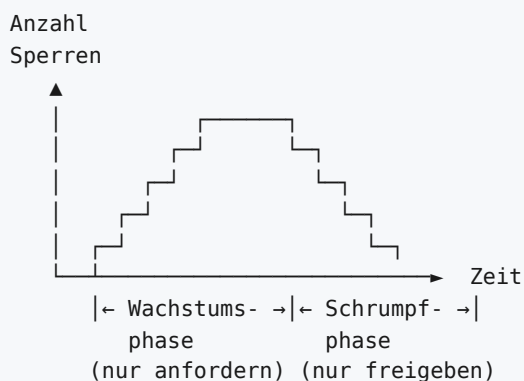
50.1 Zweiphasen-Sperrprotokoll (2PL) ★★★★★

Mechanismus zur Umsetzung von ISOLATION (I in ACID)

- Grundlage aller pessimistischen Locking-Verfahren
- Garantiert Serialisierbarkeit
- Gilt für zentrale UND verteilte Datenbanken

Es besteht aus ZWEI STRIKT GETRENNTEN Phasen:

Phase	Regel
1. Wachstumsphase (<i>Growing Phase</i>)	Die Transaktion darf Sperren ANFORDERN , aber KEINE EINZIGE Sperre freigeben
2. Schrumpfphase (<i>Shrinking Phase</i>)	Die Transaktion darf Sperren FREIGEBEN , aber KEINE NEUEN Sperren mehr anfordern



50.2 Zweiphasen-Commit-Protokoll (2PC) ★★★★★

Mechanismus zur Umsetzung von ATOMARITÄT im VERTEILTEN Fall (A in ACID)

- Wird **ZUSÄTZLICH** benötigt, wenn eine Transaktion über **MEHRERE KNOTEN** läuft
- Koordiniert Commit/Abort über alle beteiligten Systeme
- ⚠ „Hat **NICHTS** mit Isolation oder Sperren zu tun“

Der Ablauf:

- 1. Prepare-Phase:** „Zuerst fragt der **KOORDINATOR** alle beteiligten Knoten, ob sie die Transaktion **vorbereiten und dauerhaft ausführen können.**“
- 2. Commit-Phase:** „Danach entscheidet er — basierend auf den Antworten — **ob ALLE gemeinsam COMMITTEN oder ALLE gemeinsam ABBRECHEN.**“

🔑 „2PC sorgt für **ATOMARITÄT (A)** über mehrere Knoten und garantiert, dass **ALLE** Knoten gemeinsam committen.“

★★★★ Die Abgrenzung 2PL ↔ 2PC — sichere Klausurfrage:

	2PL (Two-Phase Locking)	2PC (Two-Phase Commit)
ACID-Kriterium	Isolation (I)	Atomarität (A)
Zweck	garantiert Serialisierbarkeit von Transaktionen	koordiniert den gemeinsamen Abschluss (COMMIT oder ABORT) über mehrere Knoten
Wo?	zentrale und verteilte DB	nur verteilte Systeme
Phasen	Wachstums-, Schrumpfphase	Prepare-, Commit-Phase
Sperren?	ja	⚠️ nein

⚠️ Die klassische Verwechslung: 2PL ≠ 2PC. Beide haben „zwei Phasen“ im Namen, aber sie lösen **völlig verschiedene Probleme**.

TEIL H — JDBC, REST UND JPA/ORM

51. Was ist JDBC? ★★★★★

- JDBC ist die Implementierung des **SQL/CLI (SQL Call-Level Interface)** für die Programmiersprache **Java**. JDBC ist also eine **Klassenbibliothek für den Zugriff auf SQL-Datenbanken in Java-Programmen**. Sie besteht aus einer Sammlung von **Klassen und Interfaces in den Paketen `java.sql` / `javax.sql`**
- **JDBC (Java Database Connectivity)** ist die **Standard-Schnittstelle** für den Zugriff auf Datenbanken über SQL aus Java-Anwendungen
- ★ **JDBC enthält KEINEN datenbankspezifischen Code**
- ★ **JDBC ist eine ABSTRAKTIONSSCHICHT und ermöglicht DATENBANKNEUTRALITÄT bzw. den Austausch des DBMS**

51.1 Eigenschaften ★★

- **Integrierter Bestandteil der Sprache Java**; enthalten in **J2SE- und J2EE-Releases**
- **Anwendung kann unabhängig von der Datenbank implementiert werden**
 - ★ „Write Once, Run Anywhere“
 - **SQL-Anweisungen werden als TEXT (Strings) übertragen**
 - ★ **JDBC-Treiber transformieren JDBC-SQL in DBMS-SQL**
- **DBMS-Anbieter implementieren und erweitern den Standard mit ihren eigenen JDBC-Treibern**
- **JDBC Driver API** für die Implementierung von Treibern

52. Die vier JDBC-Treibertypen ★★★★★

Eine sichere Klausurfrage.

Typ	Name	Merkmale
Typ 1	JDBC-ODBC-Bridge (Open Database Connection)	Ziel: unabhängiges Protokoll zwischen Datenbanken und Programm · ⚠ Deprecated in JDK 7 (JDBC 4.1) · ⚠ In JDK 8 (JDBC 4.2) ENTFERNT
Typ 2	Native-API (thick)	Spezielle Treiber des jeweiligen Datenbankherstellers · proprietär · betriebssystemabhängig · nicht alle Hersteller bieten native Treiber · Beispiel: Oracle OCI Treiber
Typ 3	Network-Protocol-Treiber / Middleware-Treiber	Komplett in Java geschrieben · keine spezielle Installation erforderlich · Treiber ist auf eine MIDDLEWARE angewiesen · DBMS kann problemlos ersetzt werden · Three-Tier-Architektur
Typ 4 ★	Database-Protokoll-Treiber (Pure)	Komplett in Java geschrieben · setzt JDBC-Calls DIREKT in das erforderliche Protokoll der jeweiligen Datenbank um · plattformunabhängig · DBMS-abhängig

★★★★ „Typ 4 — der Treiber, den WIR NUTZEN“ (PostgreSQL JDBC Driver von `jdbc.postgresql.org`)

52.1 Einbindung des Treibers ★

„Zwar wird mit der JDBC-API eine **einheitliche Schnittstelle** angeboten, jedoch werden die **spezifischen Klassen jeweils von den Datenbankanbietern realisiert**. Deshalb müssen diese Klassen, die in der Regel als **.jar -Dateien** zur Verfügung stehen, zunächst **beschafft und dann zur Laufzeit über den Java-CLASSPATH eingebunden** werden.“ In IntelliJ IDEA bei „Project Settings“ → „Libraries“ den Pfad zum JDBC-Treiber hinzufügen.

53. Der JDBC-Ablauf ★★★★★

Die sechs Schritte — auswendig:

- ① Laden des Treibers
- ② Herstellen der VERBINDUNG → `DriverManager.getConnection(url, props)`
- ③ Erzeugen eines STATEMENT-Objekts → `conn.createStatement()`
- ④ AUSFÜHREN der Anfrage/Updates → `st.executeQuery()` / `st.executeUpdate()`
- ⑤ AUSWERTUNG der Ergebnisse → `while (rs.next()) { ... }`
- ⑥ SCHLIESSEN der Verbindung → `rs.close(); st.close(); conn.close();`

53.1 Verbindungsaufbau ★★★★★

Was wird benötigt? Adresse der PostgreSQL-Datenbank · Name der Datenbank (Schema) · Username und Password

```
String adresse = "jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
String schema = "chinook";
String url = String.format(adresse, schema);

Properties props = new Properties();
props.setProperty("user", "postgres");
props.setProperty("password", "postgres");

// Die DriverManager-Klasse verfügt über die Methode getConnection
Connection conn = DriverManager.getConnection(url, props);
```

(Password und User könnten auch als separate Strings anstatt Properties übergeben werden:
`DriverManager.getConnection(url, user, password)`)

53.2 Statement vorbereiten und ausführen ★★★★★

- **SQL-Anweisungen werden über ein Objekt der Klasse `Statement`** an die Datenbank geschickt. Die Objekte werden mit der Methode `createStatement()` der Klasse `Connection` erzeugt
- **Die SQL-Anweisungen werden in einer String-Variable zwischengespeichert.** Dieser String wird z. B. mit der Methode `executeQuery` an die Datenbank übergeben
- **Als Antwortobjekt erhält man ein Objekt der Klasse `ResultSet`**

```
// Vorbereiten
Statement st = conn.createStatement();
String stcommand = "select * from chinook.genre;";
// Ausführen
ResultSet rs = st.executeQuery(stcommand);
```

53.3 ★★★★★ `executeQuery()` vs. `executeUpdate()`

Methode	Wofür?	Rückgabotyp
<code>executeQuery()</code>	SELECT -Abfragen (DQL)	<code>ResultSet</code>
<code>executeUpdate()</code>	INSERT, UPDATE, DELETE (DML) oder DDL-Anweisungen	<code>int</code> — ★ die Anzahl der betroffenen Zeilen

```
// UPDATE
Statement st = conn.createStatement();
String stcommand = "UPDATE genre SET name = 'Speed Metal' WHERE genre_id = 1;";
st.executeUpdate(stcommand);

// mit Ermittlung und Ausgabe der betroffenen Zeilen
int affectedrows = st.executeUpdate(stcommand);
System.out.print("Anzahl betroffener Zeilen: ");
System.out.println(affectedrows);

// INSERT
String stcommand = "INSERT INTO genre(genre_id,name) VALUES (81,'Old School Rock');";
st.executeUpdate(stcommand);

// DELETE
String stcommand = "DELETE FROM genre WHERE genre_id = 80;";
st.executeUpdate(stcommand);

// DDL (!) – ebenfalls executeUpdate
String stcommand = "ALTER TABLE genre ADD COLUMN test varchar(255);";
st.executeUpdate(stcommand);
```

53.4 Ergebnis auswerten und das Cursor-Konzept ★★★★★

- „Mit dem Antwortobjekt kann man nun arbeiten. Es kann auf die **einzelnen Spalten der Ergebnistabelle** zugegriffen werden. **Für jeden Datentyp gibt es eine entsprechende get -Methode**, die als Parameter die **Spaltennummer ODER den Spaltennamen** hat.“
- „In der `while` -Schleife wird mit `next()` so lange weitergesprungen, bis das Ende erreicht ist (dann gibt die Methode `false` zurück und die Schleife wird nicht mehr durchlaufen).“

```
while (rs.next()) {
    Integer ID = rs.getInt("genre_id");
    String Name = rs.getString("name");
    System.out.print(ID);
    System.out.println(Name);
}
```

★★ Das Cursor-Konzept — die Begründung:

- Das Resultat der DB wird mit einem CURSOR durchlaufen
- ★ **PROBLEM: Kopplung von SQL und Programmiersprache durch UNTERSCHIEDLICHE DATENSTRUKTUREN (RELATION vs. TUPEL)**
- ★ **LÖSUNG: Cursor als ITERATOR über die verschiedenen Tupel** (ein Tupel enthält eine Liste an Elementen)

```
int pos = rs.getRow();    // Cursorposition ermitteln
```

53.5 Ressourcenfreigabe ★★★★★

„Das ResultSet, die Anweisung (Statement) und die Verbindung SOLLTEN IMMER GESCHLOSSEN WERDEN. Ansonsten werden die Ressourcen NICHT DIREKT FREIGEgeben.“

```
rs.close();
st.close();
conn.close();
```

53.6 Fehlerbehandlung ★★

„Alle JDBC-relevanten Funktionen können Fehler werfen und müssen entsprechend abgefangen werden. Eine SQLException wird für ALLE SQL- und DBMS-Fehler geworfen und muss entsprechend behandelt werden. Wenn try / catch den Fehler abfängt, dann wird das Programm TROTZ SQLException FORTGESETZT.“

```
try {
    // JDBC Methoden
} catch (SQLException e) {
    e.printStackTrace();
}
```

53.7 Transaktionssteuerung in JDBC ★★★

```
conn.setAutoCommit(false); // Auto-Commit deaktivieren
...
conn.commit();             // Commit
...
conn.rollback();           // Rollback
```

54. SQL Injection und Prepared Statements ★★★

54.1 Das Problem ★★★

- „Mit `Statement.executeQuery(..)` kann **BÖSARTIGER CODE** in die Query gelangen“
- „Werden beispielsweise **Eingaben in einer Benutzeroberfläche in einen SQL-Code übernommen**, kann damit **die LOGIK des SQL-Befehls VERÄNDERT** werden“

Das Beispiel der Folie:

```
// Der SQL-Update-Befehl, der mit einer Eingabe zusammengefügt werden soll
String stcommand = "UPDATE genre SET name = 'Blues Rock' WHERE name = ";

// Eingabe emulieren
String eingabe = "'Blues' OR 1=1;"; // ⚠️ führt dazu, dass ALLE Datensätze selektiert werden

st.executeUpdate(stcommand + eingabe);
```

⚠️ „Im Ergebnis werden ALLE Datensätze geändert, da die Bedingung nun IMMER WAHR ist.“

Der resultierende Befehl lautet:

```
UPDATE genre SET name = 'Blues Rock' WHERE name = 'Blues' OR 1=1;
```

→ `1=1` ist **immer wahr** → **die gesamte Tabelle wird überschrieben.**

54.2 Prepared Statements ★★★★★

„Prepared Statements sind **VORÜBERSETZTE, PARAMETRISIERTE SQL-Anweisungen**, die eine **effiziente und sichere Kommunikation** zwischen Applikation und Datenbank ermöglichen.“

Funktionsprinzip — zweistufiger Prozess ★★:

1. **Vorbereitung (Prepare):** „Das **SQL-SKELETT** wird **deklariert, VORKOMPILIERT und vom DBMS OPTIMIERT**“
2. **Ausführung (Execute):** „Die **formalen Parameter** werden durch **aktuelle Werte ersetzt** und die fertige Anweisung ausgeführt“

Die zwei Eigenschaften ★★★★★:

Eigenschaft	Erläuterung
Sicherheit (SQL-Injection-Schutz)	„Da Parameter STRIKT von der Befehlslogik GETRENNT sind, prüft das System die Gültigkeit der Daten VOR der Verarbeitung — schädlicher Code wird als REINER TEXT behandelt “
Performance	„Die Anfrage muss nur EINMAL geparkt und optimiert werden. Bei wiederholter Ausführung mit unterschiedlichen Parametern entfällt der Overhead der Abfrageoptimierung “

Syntax ★★★★★:

- **Parameter werden in der Query mit ? gekennzeichnet**
- Diese werden nach der Erzeugung mit **setXXX()** **gesetzt**, wobei **XXX ein passender Datentyp** ist
- ★ **conn.prepareStatement(...)** **ANSTATT** **conn.createStatement()**

```
// Statement vorbereiten (mit SQL-Code)
PreparedStatement preparedStmt =
    conn.prepareStatement("SELECT * FROM relationaledb.article WHERE stock > ?");
preparedStmt.setInt(1, 100);           // Für Parameter 1 wird 100 übergeben
// SQL-Befehl ausführen
ResultSet rs = preparedStmt.executeQuery();
```

Der Injection-Anwendungsfall, umgesetzt mit Prepared Statement:

```
String eingabe = "'Blues' OR 1=1;";
PreparedStatement pst =
    conn.prepareStatement("UPDATE genre SET name = 'Blues Rock' WHERE name = ?");
pst.setString(1, eingabe);
pst.executeUpdate();
```

→ ★ **Die Eingabe wird als reiner Textwert behandelt.** Es wird nach einem Genre mit dem **Namen** `'Blues' OR 1=1;` gesucht — das existiert nicht, also passiert **nichts**.

⚠ **Die setXXX()-Methoden:** `setInt(i, wert)`, `setString(i, wert)`, `setDouble(i, wert)`, `setDate(i, wert)`, `setBoolean(i, wert)` ... Der **Index beginnt bei 1**, nicht bei 0!

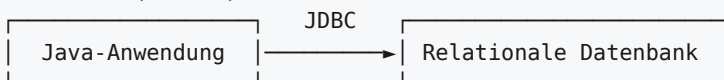
54.3 Exkurs: Zugriff aus Python ★

- `psycopg2` — „Treiber“ für PostgreSQL
- `sys` für Übergabe von Fehlermeldungen (Standard Error Output)
- Vorgehen: **Cursor erstellen** → **SQL-Befehl als Text in einer Variable speichern** → **SQL-Befehl ausführen**
- „Der Python-Skript nutzt die **DB-API 2.0-Schnittstelle**, die durch die Bibliothek `psycopg2` bereitgestellt wird, welche wiederum die **native C-Schnittstelle (libpq)** von PostgreSQL nutzt.“
- Installation: `pip install psycopg2-binary`

55. Datenzugriff über REST und die 3-Schichten-Architektur ★★★

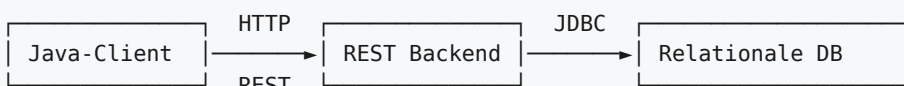
55.1 Der Architekturwandel

KLASSISCH (bisher):



- SQL-Anweisungen werden IM PROGRAMM formuliert und ausgeführt

MODERN:



- ★ Der Java-Client "spricht" HTTP (GET, POST...) ANSTATT SQL.
- ★ SQL ist GEKAPSELT im Backend.

55.2 REST ★★

REST = REpresentational State Transfer. Architekturprinzip für den Zugriff auf RESSOURCEN über das Web.

Kommunikation erfolgt über:

- HTTP
- eindeutige URLs (Ressourcen)
- Standardmethoden: `GET`, `POST`, `PUT`, `DELETE`
- Daten werden meist im JSON-Format übertragen

Beispiel:

Anfrage (HTTP): `GET /api/personen/1`
 Antwort (JSON): `{ "id": 1, "name": "Anna", "age": 28 }`

„Clients greifen über REST auf das Backend zu — NUR DAS BACKEND greift auf die Datenbank zu.“

55.3 Die 3-Schichten-Architektur ★★★

*„Eine moderne Anwendung wird in **drei klar getrennte Schichten** aufgeteilt. Jede Schicht hat eine **definierte Aufgabe** und kennt die anderen **NUR ÜBER SCHNITTSTELLEN**.“*

	1. Präsentationsschicht (Client)	2. Anwendungsschicht (Backend)	3. Datenschicht (Datenbank)
Beispiele	Webbrowser · Mobile App · Java-Client	Java Backend (Spring Boot, Jakarta EE) · REST-API	PostgreSQL · MySQL · Oracle
Aufgaben	Darstellung der Daten (UI) · Benutzerinteraktion · Senden von HTTP-Anfragen	★ GESCHÄFTSLOGIK · Validierung & Regeln · Sicherheit (Login, Rollen) · Transaktionssteuerung · Zugriff auf die Datenbank	Persistente Speicherung · Datenintegrität · ACID-Transaktionen · optimierte Abfragen
Technologien	—	REST / HTTP · JDBC / JPA / Hibernate	—
⚠ WICHTIG	★ KEIN SQL · ★ KEINE Datenbankzugangsdaten		

Die fünf Vorteile ★:

1. Klare Verantwortlichkeiten
2. Bessere Wartbarkeit
3. Höhere Sicherheit
4. Gute Skalierbarkeit
5. Austauschbarkeit einzelner Schichten

56. ORM und der Object-Relational Impedance Mismatch ★★★★★

56.1 Das Problem ★★★★★

„Der OBJECT-RELATIONAL IMPEDANCE MISMATCH beschreibt die FUNDAMENTALEN KONZEPTIONELLEN UNTERSCHIEDE zwischen dem OBJEKTMODELL (OO) und dem RELATIONALEN MODELL (RDBMS).“

Konflikt	Erläuterung
Struktur- und Typkonflikte	OO nutzt Vererbung, Polymorphie und komplexe Objektgraphen. RDBMS basiert auf flachen, normalisierten Tupeln und festen Tabellenstrukturen. Die direkte Abbildung ist schwierig
Granularitätskonflikt	Der Zugriff auf assoziierte Objekte erfolgt im Code über Referenzen (Objekt-Traversal), während die Datenbank zwingend Joins oder separate Abfragen erfordert
Synchronisationskonflikt	Diskrepanz zwischen flüchtigem RAM-Speicher (Objektzustand) und persistentem Datenbankspeicher

Die vier konkreten Mapping-Herausforderungen ★★:

Aspekt	Objektorientiertes Modell	Relationales Modell	Konflikt/Mismatch
Vererbung	natürliches Feature, Polymorphie	nicht nativ vorhanden (Simulation über Joins)	komplexes Mapping von Hierarchien
Identität	Identität basierend auf SPEICHERORT (Heap)	Identität basierend auf PRIMÄRSCHLÜSSEL (Daten)	Synchronisierung der Identitäten (Identity Map)
Navigatio n	direkte Referenzen / Objektgraphen	logische Verknüpfung über Fremdschlüssel	Übergang zwischen Objekt-Traversal und SQL-Joins
Set-Semantik	ungeordnete/geordnete Collections	Mengenoperationen (Relationale Algebra)	ineffiziente Abbildung von Collections

56.2 Der Lösungsansatz: ORM ★★

*„**ORM-Frameworks** helfen, diesen ‚Object-Relational Impedance Mismatch‘ zu überbrücken. *ORM agiert als **ABSTRAKTIONSSCHICHT** zwischen objektorientierten Domänenmodellen (z. B. Java-Klassen) und relationalen Datenbanken (RDBMS).“

Vorteil	Erläuterung
Automatisierung	Das ORM-Framework übernimmt die Übersetzung zwischen der Objektwelt und dem relationalen Schema (Klassen ↔ Tabellen, Objekte ↔ Zeilen)
Wartbarkeit	Der Verzicht auf manuelles SQL-Handling und manuelles Mapping von ResultSets reduziert den Codeaufwand erheblich

56.3 Evolution in Java ★★

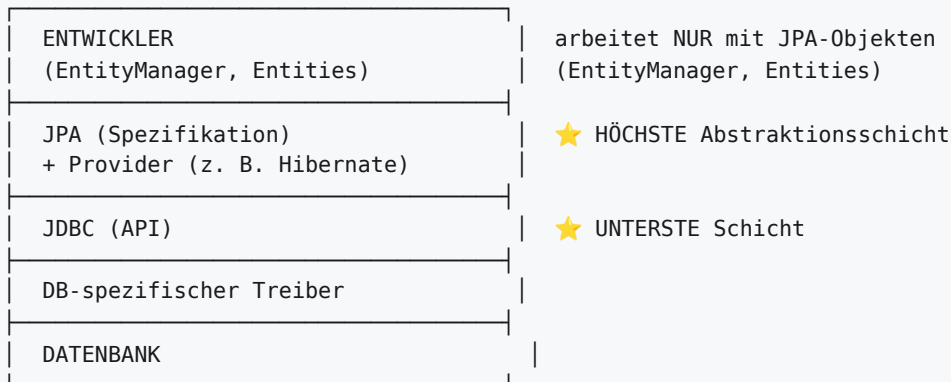
Jahr	Meilenstein
2001	Hibernate — Pionier der Open-Source-Frameworks; etablierte sich als De-facto-Standard für Java-ORM
2006	Einführung der Jakarta Persistence API (JPA) als offizieller Java-Standard (Teil von EJB 3.0), um den Wildwuchs proprietärer Lösungen zu vereinheitlichen

Ziel: „Konsolidierung proprietärer Lösungen unter einer **einheitlichen, herstellerunabhängigen Schnittstelle**.“

56.4 ★★★ Die drei Ebenen der Abstraktion

Ebene	Was ist es?	Erläuterung
Das KONZEPT: ORM	die generische Technik , Objekte einer Programmiersprache auf relationale Tabellen abzubilden	unabhängig von spezifischen Sprachen oder Datenbankprodukten
Die SPEZIFIKATION: JPA	der offizielle Java-Standard (Interface)	definiert Schnittstellen, Metadaten-Annotationen und die Abfragesprache JPQL · ⚠️ besitzt KEINEN eigenen ausführbaren Code für das Mapping (rein deklarativ)
Die IMPLEMENTIERUNG: Hibernate	das eigentliche Framework (Provider) , das die JPA-Spezifikation mit Leben füllt	bietet über den Standard hinausgehende Zusatzfunktionen (z. B. HQL)

56.5 JPA und JDBC ★★



- **JPA ist die HÖCHSTE Abstraktionsschicht.** Es definiert die API (Spezifikation), um mit **Java-Objekten zu arbeiten, OHNE sich um SQL kümmern zu müssen**
- **JDBC ist die UNTERSTE Schicht** in der Java-Datenbankkommunikation. Es ist die API, die es der JVM ermöglicht, **datenbankspezifische Treiber zu nutzen und SQL-Statements auszuführen**
- ★★ „Ein JPA-Provider (z. B. Hibernate) nutzt INTERN IMMER JDBC, um seine Aufgaben zu erfüllen.“
- * „Der Entwickler interagiert nur mit JPA-Objekten. **Die Details der JDBC-Verbindungen, SQL-Dialekte und Treiber werden vom JPA-Provider GEKAPSELT.**“*

57. Strategien zur Datenabfrage in JPA ★★

Strategie	Beschreibung	Vor-/Nachteil
JPQL (<i>Java Persistence Query Language</i>)	standardisierte, OBJEKTORIENTIERTE Sprache; arbeitet auf Entities und Attributen statt auf Tabellen	⚠ Nachteil: Syntaxfehler werden erst zur LAUFZEIT (Runtime) erkannt
Criteria API	programmatische Erstellung von Abfragen via Java-Code	✅ Vorteil: TYPISCHERHEIT (typesafe) — Fehler werden bereits beim KOMPIlieren erkannt . Ideal für komplexe, dynamisch aufgebaute Abfragen
HQL (<i>Hibernate Query Language</i>)	proprietäre Erweiterung von Hibernate mit Funktionen über den JPA-Standard hinaus	herstellerabhängig
EntityManager.find()	direktes Laden einer einzelnen Entität über ihren PRIMÄRSCHLÜSSEL (ID) — identitätsbasierter Zugriff ohne den Overhead einer Abfragesprache	✅ effizient für ID-basierte Zugriffe , typischere Rückgabe • ⚠ NICHT geeignet für Filterungen, Sortierungen oder Mengenoperationen ; keine dynamischen Bedingungen

58. JPA in der Praxis ★★★★★

58.1 Die zentralen Komponenten ★★★★★

Komponente	Rolle
EntityManagerFactory	*„Der ‚Bauplan-Leser‘. Sie liest einmalig die Konfiguration (<code>persistence.xml</code>) und stellt die grundlegende Verbindung zur Datenbank her.“* — eine Fabrik-Instanz , die bedarfsgerecht EntityManager erzeugt
EntityManager	*„Der eigentliche ‚ARBEITER‘. Über ihn laufen ALLE Operationen (Laden, Speichern, Löschen) auf der Datenbank.“* — zentrale Schnittstelle zur Verwaltung persistenter Objekte
Persistenzeinheit (Persistence Unit)	Definiert in der <code>persistence.xml</code> ; bündelt alle Konfigurationen für eine spezifische Datenbankverbindung
Persistenzkontext (Persistence Context)	★ Der „ARBEITSSPEICHER“ des Entity Managers; hier werden alle aktuell verwalteten Entities vorgehalten und überwacht
Transaktionen	„Änderungen an Daten müssen ZWINGEND in einer Transaktion (<code>begin</code> bis <code>commit</code>) gekapselt sein. “

58.2 Entities und Annotationen ★★★★★

„Ein **persistentes Objekt** wird **ENTITY** genannt („An entity is a lightweight persistent domain object.“).“

Annotation	Funktion
@Entity	★ macht aus der gewöhnlichen Java-Klasse ein PERSISTENTES Entity
@Id	★ kennzeichnet die Instanzvariable, die von JPA als PRIMÄRSCHLÜSSELVARIABLE verwendet wird
@GeneratedValue	„Wenn man diese Zeile weglässt, muss die ID MANUELL vergeben werden . Im Java-Code muss dann sichergestellt werden, dass jede ID eindeutig ist. “

★ „In JPA reicht die Angabe der @Entity - und @Id -Annotationen, um aus einem normalen Java-Objekt ein persistentes Java-Objekt zu machen.“

58.3 persistence.xml ★★★★★

*„Die `persistence.xml` definiert sogenannte ‚Persistence Units‘, die alle für die **Datenbankverbindung und das Object-Relational Mapping (ORM)** notwendigen Einstellungen enthalten. Dazu gehören:*

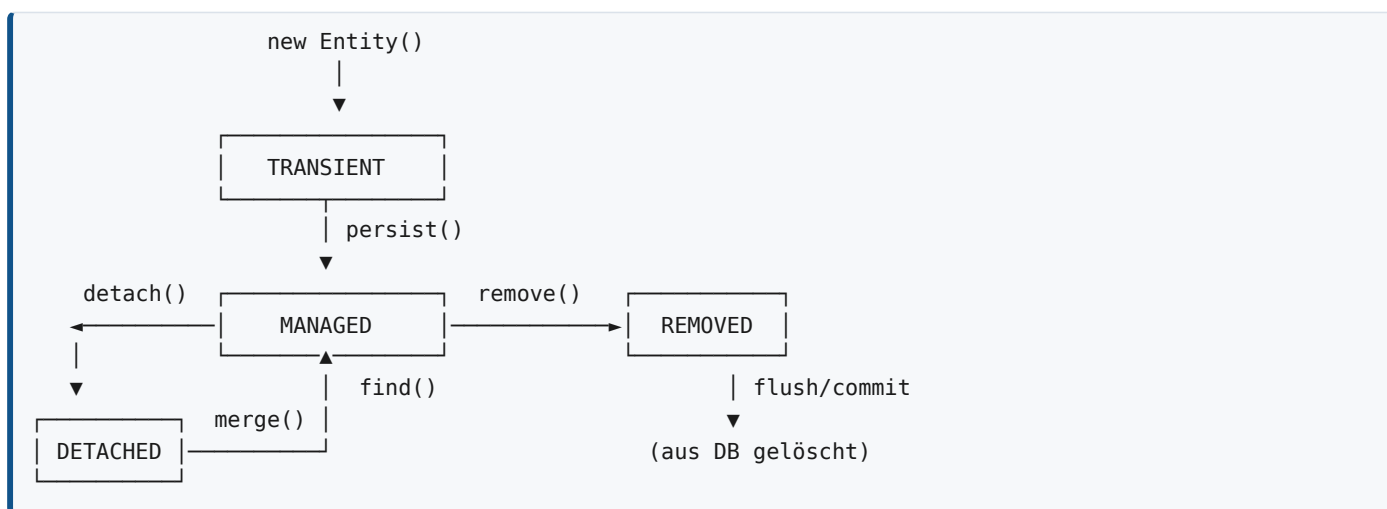
- **Datenbank-Zugangsdaten** (URL, User, Passwort)
- **der verwendete JPA-Provider** (z. B. Hibernate)
- **Verweise auf die zu verwaltenden Entitätsklassen**
- **Schema-Generierungsstrategien.**“

Standardeinstellungen ★:

*„Das Mapping von Objekten auf Tabellen bedingt, dass JPA eine ganze Reihe von **Standardeinstellungen** als gesetzt annimmt. Beispiele: **ein String-Property wird auf eine VARCHAR-Spalte abgebildet; alle Properties einer Klasse sollen persistent sein. Ausnahmen müssen EXPLIZIT angegeben werden.**“*

58.4 ★★★★★ Der Objektlebenszyklus einer Entity — die vier Zustände

Zustand	Deut sch	Beschreibung
Transient	Neu	Die Entity wurde instanziert (<code>new Entity()</code>) und existiert im Java-Heap , ist aber noch nicht mit einem Persistence Context assoziiert und besitzt keinen Datenbank-Identifikator . Sie ist NICHT VERWALTET
Managed	Verwaltet	Die Entity wurde durch <code>persist()</code> oder <code>find()</code> in den Persistence Context aufgenommen . ★ Änderungen werden VERFOLGT und beim COMMIT in die Datenbank SYNCHRONISIERT
Detached	Abgelöst	Die Entity war verwaltet , aber der Persistence Context wurde geschlossen oder die Entity wurde explizit abgelöst (<code>detach()</code>). Die Instanz existiert weiterhin im Java-Speicher, aber ⚠ Änderungen werden NICHT MEHR vom ORM verfolgt
Removed	Entfernt	Die Entity wurde mit <code>remove()</code> zum Löschen markiert . Sie ist weiterhin verwaltet , wird jedoch beim nächsten flush physisch aus der Datenbank gelöscht



58.5 JPA CRUD ★★★★★

Kategorie	Befehl	Beschreibung
C (Create)	<code>em.persist(person)</code>	Fügt ein neues (transientes) Objekt zum Persistence Context hinzu. ★ Beim Transaktions-Commit wird ein SQL INSERT generiert
R (Read) — nach ID	<code>Person p = em.find(Person.class, 2L);</code>	Liest die Entität anhand ihres Primärschlüssels
R (Read) — über Query	<code>List<Person> results = em.createQuery(...)</code>	Liest Entitäten basierend auf einer Abfrage — nutzt JPQL oder die Criteria API
U (Update)	<code>person.setVorname("Lena");</code>	★ Die Änderung erfolgt DIREKT auf der managed Entität (dem Objekt im Persistence Context) — kein expliziter Speicherbefehl nötig!
Merge (Update für Detached)	<code>Person mergedP = em.merge(detachedP);</code>	Wird verwendet, wenn eine detached (abgelöste) Entität mit geänderten Daten wieder in den Persistence Context integriert werden soll
D (Delete)	<code>em.remove(person);</code>	Markiert eine managed Entität zur Löschung . Beim Transaktions-Commit wird ein SQL DELETE ausgeführt

Transaktionen in JPA ★★:

Konzept	Befehle	Zweck
Transaktion (⚠️) zwingend für CUD	<pre>em.getTransaction() .begin(); em.getTransaction() .commit();</pre>	Definiert die logische Einheit der Arbeit ; stellt sicher, dass alle Operationen atomar ausgeführt werden (alles oder nichts)
Rollback (Fehlerbehandlung)	<pre>em.getTransaction() .rollback();</pre>	Macht alle Änderungen der aktuellen Transaktion rückgängig

★★ Der Kernmechanismus:

„**** em.persist(p) * nimmt das normale Java-Objekt p und stellt es unter die Beobachtung des EntityManagers. Beim commit() wird daraus dann AUTOMATISCH der SQL-Befehl INSERT INTO... erzeugt.** *** „Der Persistenzkontext wird erst AM ENDE einer Transaktion mit der Datenbank SYNCHRONISIERT.“

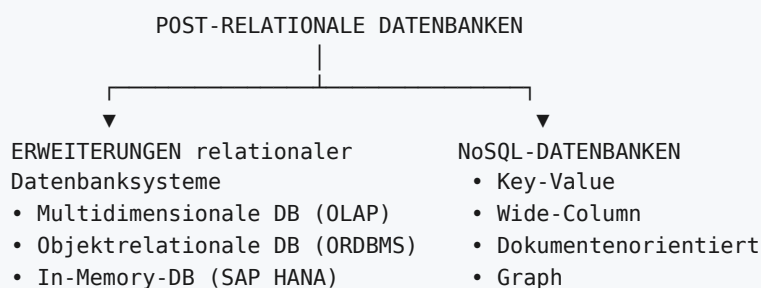
58.6 Maven ★

⚠️ *** „Maven ist KEIN Framework** wie Spring, JPA/Hibernate oder Apache Struts, das **zur Laufzeit** der Anwendung aktiv ist. **Es wird nur während der ENTWICKLUNG und des BUILDS verwendet.**“ — Die Abhängigkeiten (z. B. Hibernate, Jedis, MongoDB-Treiber) werden in der `pom.xml` deklariert.

TEIL I — ERWEITERUNGEN RELATIONALER DB UND VERTEILTE DATENBANKEN

59. Postrelationale Datenbanken ★

„Die relationale Datenbanktechnologie hat sich in den **1980er- und 1990er-Jahren** breit im Markt durchgesetzt. Relationale Datenbanken haben sich für die meisten betrieblichen Anwendungen als **De-facto-Standard** etabliert. **Es gibt allerdings Anwendungsszenarien, bei denen z. B. die TRANSAKTIONS- UND KONSISTENZORIENTIERUNG relationaler Datenbanken NACHTEILE bringen.**“



60. OLTP vs. OLAP und der Data Cube ★★★★★

60.1 Der Analyse-Konflikt

	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Fokus	schnelle, präzise Geschäftsabwicklung (z. B. Bestellungen schreiben)	Entscheidungsunterstützung durch multidimensionale Datenstrukturen
Struktur	hochgradig NORMALISIERT , um Redundanzen zu vermeiden	denormalisiert
Zweck	Abwicklung des Tagesgeschäfts (viele kurze Transaktionen, Schreib-Fokus)	Analyse historischer Datenbestände (komplexe Abfragen, Lese-Fokus)

⚠ „Analytische Abfragen über **MILLIONEN** von Datensätzen führen in OLTP-Systemen zu **MASSIVEN PERFORMANCE-EINBUSSEN** und **BLOCKIEREN DAS OPERATIVE GESCHÄFT**.“
Lösungsansatz: OLAP — **EIGENSTÄNDIGE** Systeme zur Entscheidungsunterstützung.

60.2 Der Data Cube ★★★★★

„Um Daten effizient analysieren zu können, werden Informationen in **MEHRDIMENSIONALEN WÜRFELN (Cubes)** dargestellt.“

Element	Beschreibung
Dimensionen (die Achsen)	Beschreiben die Auswertungskriterien (z. B. Zeit, Produkt, Region) · ★ Die Reihenfolge der Achsen ist beliebig · Aggregationsstufen : Dimensionen können Hierarchien bilden (z. B. Tag → Monat → Quartal)
Fakten & Kennzahlen (der Inhalt)	Quantitative Werte im Zentrum des Würfels (z. B. Umsatz, Stückzahl), die für die Entscheidungshilfe aggregiert werden

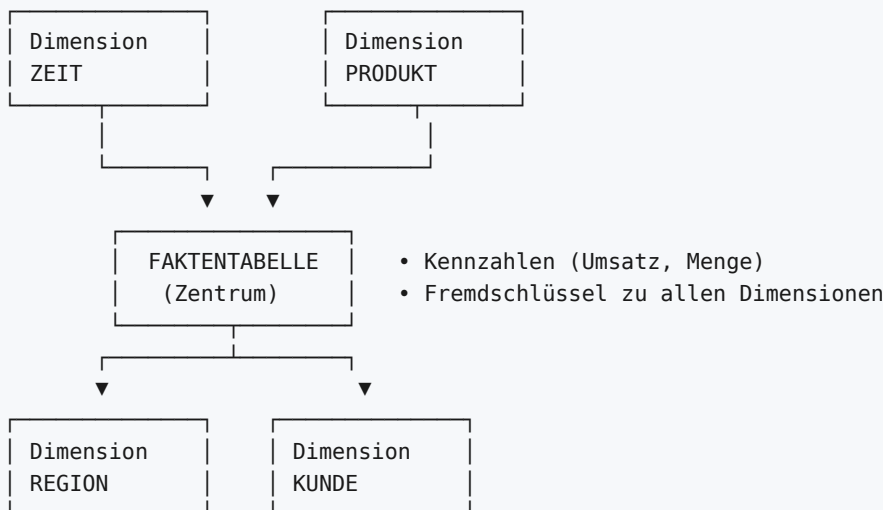
60.3 Die vier analytischen Operationen ★★★★★

Operation	Beschreibung
Slicing	Ausschneiden einer 2D-„SCHEIBE“ aus dem Würfel (z. B. Umsatz aller Produkte in Region „West“) — EINE Dimension wird fixiert
Dicing	Auswahl eines kleineren TEILWÜRFELS durch Einschränkung MEHRERER Dimensionen
Roll-up	VERDICHTEN der Daten auf HÖHERE Ebenen (z. B. von Monat auf Jahr)
Drill-down	AUFSCHLÜSSELN der Daten in DETAILLIERTE Ebenen (z. B. von Region auf Stadt)

★ **Merkhilfe**: Slice = eine Scheibe (1 Dimension fixiert) · Dice = ein Würfelchen (mehrere Dimensionen eingeschränkt) · Roll-up = hoch (aggregieren) · Drill-down = runter (aufschlüsseln)

61. ROLAP: Das Sternschema ★★★★★

*„Das **Sternschema** ist die **gängigste Methode**, um multidimensionale Datenstrukturen (Cubes) effizient in **RELATIONALEN** Datenbanksystemen abzubilden (**Relational OLAP, ROLAP**).“*



Element	Beschreibung
Faktentabelle (Zentrum)	Beinhaltet die quantitativen Kennzahlen (Fakten) sowie Fremdschlüssel zu allen assoziierten Dimensionstabellen
Dimensionstabellen (Strahlen)	Gruppieren deskriptive Attribute (Produktname, Region, Zeit), die sternförmig um die Faktentabelle angeordnet sind
Beziehung	Pro Dimension GENAU EINE Tabelle — das macht die Struktur übersichtlich und navigierbar

61.1 ★★★ Strategische Denormalisierung

- „Dimensionstabellen werden **BEWUSST DENORMALISIERT**.“
- „**Redundanzen und erhöhter Speicherbedarf werden IN KAUF GENOMMEN**, um die **Anzahl der notwendigen JOINS bei komplexen Analysen zu MINIMIEREN**.“
- „**Update-Anomalien werden AKZEPTIERT**, da **OLAP-Systeme primär für LEBENDE Zugriffe (Read-Only) optimiert sind**.“

⚠ **Die Grenzen:** „Die Leistungsfähigkeit eines relationalen Datenbanksystems **stößt bei umfangreichen multidimensionalen Datenbanken an Grenzen**. Auch das **Formulieren von Abfragen ist mit dem Sternschema AUFWENDIG und FEHLERANFÄLLIG**.“

62. MOLAP: Der native OLAP-Cube ★★

„MOLAP — Multidimensional Online Analytical Processing ist eine Architektur, die Daten **DIREKT in MULTIDIMENSIONALEN ARRAY-STRUKTUREN (Cubes)** speichert, anstatt sie mit zweidimensionalen Tabellen zu modellieren.“

Merkmal	Beschreibung
Direkte Adressierung	Daten werden NICHT über Joins gesucht, sondern über ihre KOORDINATEN im Raum (Dimensionen) DIREKT adressiert
Vorausberechnung (Pre-Aggregation)	Kennzahlen werden bereits beim LADEN des Cubes auf ALLEN HIERARCHIESTUFEN aggregiert und gespeichert
Antwortzeit	Ermöglicht KONSTANTE, extrem schnelle Zugriffszeiten, UNABHÄNGIG von der Komplexität der Abfrage
Slicing & Dicing	erfolgt nativ durch Einschränkung der ARRAY-INDIZES
Drill-down / Roll-up	Navigation durch vordefinierte Hierarchien OHNE Neuberechnung
Speichereffizienz	spezielle Kompressionsverfahren für „Sparse Data“ (leere Zellen im Würfel)

Produktbeispiele: Oracle Essbase („gilt oft als ‚Ur-Vater‘ der multidimensionalen Datenbanken“), IBM Planning Analytics (ehemals TM1)

★ * „Der Markt bewegt sich heute stark in Richtung schnelle IN-MEMORY-RELATIONALE Systeme wie SAP HANA.“*

63. Objektrelationale Datenbanken (ORDBMS) ★★

„ORDBMS sind eine DIREKTE WEITERENTWICKLUNG klassischer relationaler Systeme durch INTEGRATION OBJEKTORIENTIERTER PARADIGMEN.“

Merkmal	Beschreibung
Benutzerdefinierte Datentypen (UDT)	erlauben die Definition komplexer Strukturen über einfache Typen hinaus
Hierarchien	Unterstützung von Typ- und Tabellenhierarchien (Vererbung von Struktur und Operatoren)
Typisierte Objekte	typisierte Tabellen und Sichten für eine konsistente Objektstruktur
Standardisierung	Verankerung im SQL-Standard (ab SQL:1999/2003)
Objektidentität (Surrogate)	Jedes Objekt erhält einen systemdefinierten, UNVERÄNDERBAREN Schlüsselwert
Kapselung & Methoden	generische Operatoren (Methoden) direkt auf den Objekten; interne Darstellung bleibt nach außen verborgen
Strukturierte Vererbung	Unterklassen erben sowohl Attribute als auch Methoden
Komplexe Objekte	Typkonstruktoren , um Objekte aus anderen Objekttypen zusammenzusetzen

★ * „Fast alle großen relationalen Datenbanken haben heute objektrelationale Features integriert. Man nennt sie meist NICHT MEHR ORDBMS, sondern sie sind einfach TEIL MODERNER SQL-DATENBANKEN.“*

64. In-Memory-Datenbanken und SAP HANA ★★ ★

64.1 In-Memory-Technologie

„Unter IN-MEMORY-TECHNOLOGIE (auch In-Memory-Computing) versteht man Datenbanksysteme, bei denen der HAUPTSPEICHER (RAM) als PRIMÄRER SPEICHERORT für Daten genutzt wird.“

- * „Bei In-Memory-Datenbanken werden die Daten **entweder beim Systemstart vollständig in den Speicher geladen oder dynamisch während des Betriebs im Speicher gehalten.** ★ (Persistenter Speicher wird weiterhin zur Sicherstellung der DAUERHAFTIGKEIT genutzt — etwa durch LOGS oder SNAPSHTOTS.)“*
- „Durch die **deutliche Reduzierung von I/O-Zugriffen** auf den Massenspeicher ermöglichen In-Memory-Datenbanken **erheblich schnellere Datenzugriffe** sowie eine **effizientere Verarbeitung von Abfragen.**“

64.2 SAP HANA — der Ansatz ★★ ★

* „Klassische Datenbanken sind **festplattenbasiert**, und Datenzugriffe machten komplexe Analysen langsam. **Datenbanksysteme waren ursprünglich auf SPEICHERPLATZOPTIMIERUNG statt auf hohe RECHENGESCHWINDIGKEIT ausgelegt.**“*

🔑 „DER HANA-ANSATZ: Daten werden im ARBEITSSPEICHER (RAM) gehalten, um den FAKTOR 100.000 schnellere Zugriffszeiten im Vergleich zur Festplatte zu erreichen.“

Vorteil	Erläuterung
Wegfall des I/O-Bottlenecks	„Bei traditionellen SQL-Datenbanken werden laufend Daten vom permanenten Speicher in den Hauptspeicher geladen. Diese Operationen stellen meist einen Engpass im System dar“
Reduzierter Speicherplatzbedarf	durch Kompression und Wegfall redundanter Daten (Indizes/Aggregate)
Multi-Core-Architektur	„SAP HANA ist so konzipiert, dass Abfragen MASSIV PARALLEL über Hunderte von CPU-Kernen verteilt werden “

64.3 Vision und Entwicklung ★★

* „HANA ist entstanden aus der **Zusammenarbeit zwischen dem HASSO-PLATTNER-INSTITUT (HPI) und SAP**, um die **wachsende Kluft zwischen Transaktionsverarbeitung (OLTP) und Analyse (OLAP)** zu schließen. Mit HANA soll beides vereint werden: ★ „TRANSLYTICAL DATABASE“.“*

- **Echtzeit-Analysen auf produktiven Daten** (kein Warten auf nächtliche Ladeprozesse)
- **Schnelle Ad-hoc-Abfragen ohne vorberechnete Strukturen**

Jahr	Meilenstein
2010	Vorstellung der „ High-Performance Analytic Appliance “ (HANA) — damals primär als schneller „ Beschleuniger “ für das SAP Business Warehouse (BW)
2013	Freigabe für die Business Suite (ERP on HANA) — die Datenbank wird zum Fundament für geschäftskritische Prozesse
2015	Launch von SAP S/4HANA — das gesamte ERP-System wurde neu entwickelt , um die Vorteile der In-Memory-Technik (z. B. Wegfall von Aggregat-Tabellen) voll auszuschöpfen
Heute	Entwicklung zur SAP HANA Cloud — Cloud-native und für hybride Szenarien

64.4 Data Tiering ★

*„SAP HANA unterstützt **Daten-Tiering**, indem **geschäftskritische Daten im In-Memory-Speicher (HOT)**, **weniger genutzte Daten über die Native Storage Extension (WARM)** und **Archivdaten in externen Data-Lake- oder Cloud-Speichern (COLD)** abgelegt werden.“*

64.5 ★★★ Column-Store vs. Row-Store

	Row-Store	Column-Store
Speicherung	Daten werden ZEILENWEISE als vollständige Tupel gespeichert	Daten werden SPALTENWEISE gespeichert — Werte desselben Attributs liegen zusammen und werden tupelübergreifend verarbeitet
Lesen	Beim Lesen werden IMMER ALLE Attribute eines Datensatzes gelesen	Es wird AUSSCHLIESSLICH auf die BENÖTIGTE SPALTE zugegriffen
Geeignet für	★ transaktionale Systeme (OLTP) — da häufig einzelne Datensätze gelesen oder geändert werden	★ analytische Anwendungen (OLAP) — bei Abfragen über große Datenmengen werden nur die benötigten Spalten geladen und effizient aggregiert
Kompression	gering	★ EXTREME Kompression möglich , da Werte in einer Spalte oft identisch sind

★ *„HANA unterstützt **Row- und Column-Store (Hybrid-Ansatz)**, nutzt aber für Business-Daten **fast ausschließlich den COLUMN-STORE** (SAP-Empfehlung: **Column Store als Default, Row Store nur bei klarer Begründung**).“*

64.6 ★★★ Data-to-Code vs. Code-to-Data (Code Pushdown)

	DATA-TO-CODE (klassisch)	CODE-TO-DATA (Code Pushdown)
Ablauf	1. Die Anwendung fordert ALLE potenziell relevanten ROHDATEN von der Datenbank an 2. Die Daten werden über das NETZWERK zum Applikationsserver übertragen 3. Erst auf dem Applikationsserver findet die eigentliche Geschäftslogik statt (Sortieren, Aggregieren, Berechnen)	1. Die berechnungsintensive Logik wird „NACH UNTEN“ IN DIE DATENBANK VERLAGERT 2. Statt Millionen von Einzeldatensätzen zu übertragen, sendet die Anwendung nur noch den AUSFÜHRUNGSBEFEHL 3. Die Datenbank nutzt ihre Parallelverarbeitung und In-Memory-Geschwindigkeit, um das Ergebnis direkt vor Ort zu berechnen
Problem/Ergebnis	⚠ *„Das NETZWERK und der SPEICHER des Applikationsservers werden zum FLASCHENHALS (I/O-Bottleneck‘), besonders bei großen Datenmengen“*	✅ „ Nur das VERDICHTETE ENDERGEBNIS (z. B. eine einzelne Summe oder eine gefilterte Liste) wird über das Netzwerk zurückgegeben“

Umsetzung von Code-to-Data in SAP HANA ★

- **Stored Procedures (SQLScript):** Ausführung komplexer Logik **direkt in der HANA-Datenbank**
- **Core Data Services (CDS):** Definition **datenbanknaher Modelle (CDS-Views)** zur Abbildung von Beziehungen, Filtern und Berechnungen
- **Aggregationsfunktionen & SQL:** Nutzung komplexer **GROUP BY - und Join-Operationen statt Schleifen im Applikationscode (★ SQL statt ABAP-Schleifen)**

⚠ **Die wichtige Einschränkung:** „**ABER: Der Application Server BLEIBT TROTZDEM die Instanz für Geschäftslogik, die NICHT direkt mit Massendaten-Operationen zu tun hat** (z. B. Ablaufsteuerung, UI, Integration anderer Systeme).“

Die drei Vorteile: **Performance** (Reduktion der Laufzeit durch Vermeidung von Datentransporten) · **Netzlast** (minimale Belastung der Netzwerkinfrastruktur) · **Skalierbarkeit** (Nutzung der parallelen Multi-Core-In-Memory-Architektur)

64.7 HANA: SQL- und ACID-Kompatibilität ★★

- **SQL-Unterstützung:** „SAP HANA ist eine **VOLLWERTIGE SQL-DATENBANK**. Sie unterstützt den SQL-Standard (nahe am Standard, mit SAP-spezifischen Erweiterungen wie **SQLScript**)“
- ★ **ACID-Kompatibilität:** * „TROTZ In-Memory erfüllt HANA ALLE ACID-Kriterien.“*
- **Durability:** „wird durch **SYNCHRONES SCHREIBEN der Transaktionsänderungen in das REDO LOG beim COMMIT** sichergestellt“
- **Savepoints & Redo-Log:** „Savepoints werden **ZEITGESTEUERT erstellt** und schreiben den **In-Memory-Zustand der Daten auf Disk**. Sämtliche zwischenzeitlichen committeten Änderungen werden **als Logs in einer Logdatei auf einem NICHTFLÜCHTIGEN Speicher persistiert**“

64.8 ★★ Einordnung SAP HANA

„SAP HANA ist eine **RELATIONALE SQL-DATENBANK** mit **VOLLER ACID-KONFORMITÄT**, die Tabellen und SQL als Kern nutzt, aber dank **IN-MEMORY-TECHNOLOGIE** deutlich schnellere Abfragen ermöglicht als klassische SQL-Datenbanken. Darüber hinaus unterstützt HANA **MULTIDIMENSIONALE MODELLE** über **CALCULATION VIEWS** und bietet **MULTIMODALE Funktionalitäten** für **GRAPH- und DOKUMENTENDATEN (NoSQL)**, sodass **SQL-, Graph- und JSON-Abfragen KOMBINIERT** werden können. Damit geht HANA über traditionelle relationale Datenbanken hinaus.“

65. Verteilte Datenbanksysteme ★★★★★

65.1 Definition

- „Bei **verteilten Datenbanken** werden Daten **nicht auf einem einzelnen Server gespeichert, sondern auf mehrere Rechner (NODES) in einem Netzwerk verteilt**“
- „Jeder Node speichert einen **TEIL der Gesamtdaten ODER KOPIEN davon**“ → bessere Skalierbarkeit, höhere Verfügbarkeit
- ★ * „Für Anwendungen wirkt die Datenbank dennoch wie **EIN LOGISCHES GESAMTSYSTEM**.“*

🔑 „Eine Datenbank ist **DEZENTRAL** oder **VERTEILT**, wenn sie durch **EIN EINZIGES LOGISCHES DATENBANKSCHEMA** beschrieben wird, aber **MEHRERE PHYSISCHE FRAGMENTE** auf geographisch verteilten Rechnern hält.“ ★ „Fragmente bei RDBMS sind **HORIZONTAL** (Zeilen) oder **VERTIKAL** (Spalten).“

65.2 Das DDBMS ★★

„Ein **DDBMS** sorgt dafür, dass eine **physisch verteilte Infrastruktur** wie ein **LOKALES EINBENUTZERSYSTEM** erscheint.“

Eigenschaft	Beschreibung
Logische Einheit	Zusammenführung geografisch verteilter Rechenknoten zu einem gemeinsamen globalen Datenbankschema
Verteilungstransparenz	★ Maskierung der Komplexität ; der Anwender interagiert mit dem DDBMS OHNE Kenntnis über Datenfragmente oder Replikate (ORTSTRANSparenZ)
Atomarität & Konsistenz	Einsatz von Protokollen wie dem Zweiphasen-Freigabeprotokoll (2PC) zur Koordination verteilter Transaktionen

65.3 Partitionierung und Sharding ★★★

Begriff	Definition
Partitionierung	„die LOGISCHE AUFTEILUNG der Daten in logisch getrennte TEILMENGEN . Jede Partition enthält nur einen AUSSCHNITT der Gesamtdaten, z. B. basierend auf einem SCHLÜSSEL . Dadurch lassen sich große Datenmengen HORIZONTAL SKALIEREN . Anfragen müssen GEZIELT ZUR RICHTIGEN PARTITION geleitet werden.“ • Horizontal: Aufteilung nach ZEILEN (★ häufigster Fall) • Vertikal: Aufteilung nach SPALTEN (seltener)
Sharding	★ „eine SPEZIELLE FORM der Partitionierung, bei der Partitionen auf VERSCHIEDENE NODES verteilt werden (PHYSISCHE Verteilung).“ 📌 Beispiel: alle Mitarbeiter mit Nachnamen A-F auf Knoten 1 , G-L auf Knoten 2 usw.

★ **Merke: Partitionierung = logisch · Sharding = physisch (Partitionen auf verschiedenen Knoten)**

⚠ **Vertikale Fragmentierung:** „Die Tabelle wird nach **Spalten (Attributen)** aufgeteilt, wobei ★ **der PRIMÄRSCHLÜSSEL in JEDEM Fragment vorhanden sein MUSS**“ (sonst kann man die Fragmente nicht wieder zusammensetzen).

65.4 Replikation ★★★

„**REPLIKATION** bezeichnet die **REDUNDANTE SPEICHERUNG** von Daten auf **MEHREREN SERVERN**.“

- „Sie **erhöht die AUSFALLSICHERHEIT** und kann die **LESEPERFORMANCE verbessern**, da Anfragen auf verschiedene Replikate verteilt werden können“
- ⚠ „Die **KONSISTENZ zwischen den Replikaten** erfordert **KOORDINATIONSMECHANISMEN**. Klassische relationale Systeme können hierfür **Zwei-Phasen-Commit-Protokolle** einsetzen“

🔑 ⇒ „Replikation erhöht Verfügbarkeit — Konsistenz wird komplizierter.“

Verfahren	Beschreibung
Master-Slave-Replikation	Lesezugriffe über ALLE Nodes, Schreibzugriffe (Änderungen) NUR über den MASTER-Node · Bei Ausfall des Masters wird ein Slave zum Master
Master-Master-Replikation	ALLE Nodes haben Lese- UND Schreibzugriff

65.5 Skalierung ★★☆☆

	Vertikale Skalierung (<i>Scale-up</i>)	Horizontale Skalierung (<i>Scale-out</i>)
Prinzip	Aufrüstung des DB-Servers (mehr CPU, RAM)	Aufteilung der Daten auf verschiedene Server (Nodes), Lastenaufteilung
Verbreitung	★ bei RDBMS üblich	★ NoSQL-Datenbanken
Problem	physische Grenzen, teuer	⚠ „HERAUSFORDERUNG für RDBMS (Einhaltung ACID)“

65.6 Sharing als Herausforderung ★★☆☆

„SHARING bezeichnet das GLEICHZEITIGE NUTZEN VERTEILTER DATEN durch mehrere Anwendungen oder Benutzer.“

- „In verteilten Datenbanken greifen potentiell **viele Clients PARALLEL auf unterschiedliche Nodes** zu“
- „Dadurch entstehen Anforderungen an **Synchronisation, Sperrmechanismen und Transaktionskontrolle**“

⚠⚠ Warum relationale Datenbanken hier an Grenzen stoßen:

- „Relationale Datenbanken sind **ursprünglich für ZENTRALE, STARK KONSISTENTE Systeme konzipiert**“
- „In verteilten Umgebungen ist die **strikte ACID-Konsistenz nur mit HOHEM KOORDINATIONSAUFWAND** sicherzustellen“
- „Mechanismen wie **globale Transaktionen, Locks und Joins über mehrere Nodes** sind **TEUER und nur EINGESCHRÄNKT SKALIERBAR**“

66. Konsistenz: ACID vs. verteilte Systeme ★★★★★

66.1 ★★★★★ Die zwei Konsistenzbegriffe — DIE Verständnisfrage

	ACID-Konsistenz (Integrität)	Konsistenz in verteilten Systemen (Linearisierbarkeit)
Definition	„der Zustand widerspruchsfreier Daten “ — die Daten müssen nach jeder Änderung alle definierten Integritätsregeln erfüllen (Datentypen, Wertebereiche, referentielle Integrität)	„sichert die EXTERNE ÜBEREINSTIMMUNG ALLER DATENKOPIEN zum selben Zeitpunkt “
Bezugspunkt	INTERNE KORREKTHEIT der Daten gegenüber Geschäftsregeln	Einheitlichkeit über alle Knoten hinweg
Garantie	„Eine Transaktion überführt die Datenbank von einem gültigen Zustand in einen anderen gültigen Zustand “	„Jeder Lesevorgang liefert ENTWEDER das Ergebnis des JÜNGSTEN Schreibvorgangs ODER einen Fehler “
Ziel	Alles-oder-Nichts-Prinzip (Atomarität)	★ * „Das verteilte System soll sich nach außen hin so verhalten, ALS GÄBE ES NUR EINE EINZIGE DATENKOPIE (SINGLE SYSTEM IMAGE) “*
Sichtbarkeit	—	„Sobald ein Schreibvorgang auf EINEM Knoten bestätigt wurde, müssen ALLE nachfolgenden Lesevorgänge (egal auf welchem Knoten) diesen NEUEN WERT SEHEN “

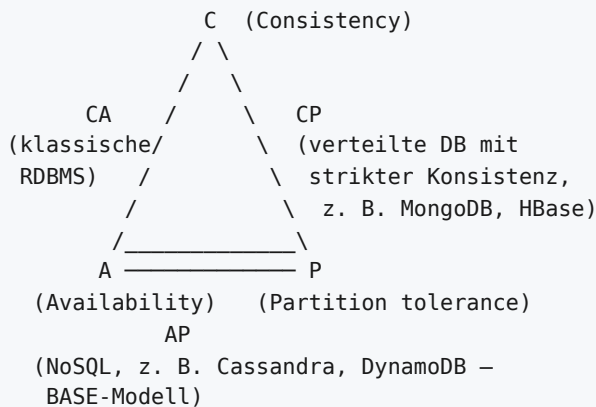
🔑 Für die verteilte Konsistenz gilt: „Keine veralteten Replikate, keine divergierenden Versionen, **SOFORTIGE, SYNCHRONE** Replikation.“

67. Das CAP-Theorem ★★★★★

„**Eric Brewer** von der Universität **Berkeley** stellte **2000** die Vermutung auf, dass die folgenden drei Eigenschaften **NICHT GLEICHZEITIG in einem MASSIV VERTEILTEN Rechnersystem gelten können.**“ „Die Vermutung wurde von **Wissenschaftlern des MIT BEWIESEN** und als **CAP-Theorem etabliert.**“

Buchstabe	Eigenschaft	Definition
C	Consistency — Konsistenz	(hier: Konsistenz in verteilten Systemen) — alle Knoten sehen gleichzeitig dieselben Daten
A	Availability — Verfügbarkeit	Alle Anfragen an das System werden STETS BEANTWORTET
P	Partition tolerance — Ausfalltoleranz	Das System arbeitet auch weiter, wenn Knoten NICHT MEHR MITEINANDER KOMMUNIZIEREN können (z. B. um die Daten untereinander konsistent zu halten)

🔑 „Das CAP-Theorem sagt aus, dass in einem massiv verteilten Datenhaltungssystem **JEWELLS NUR ZWEI** Eigenschaften aus den **DREI** der Konsistenz (C), Verfügbarkeit (A) und Ausfalltoleranz (P) **GARANTIERT** werden können.“



★★★★ Die entscheidende Erklärung (Klausurfrage!):

Warum kann ein verteiltes System bei Netzwerkpartitionierung nicht gleichzeitig konsistent (C) und verfügbar (A) sein? *„Bei einer **Netzwerktrennung (P)** können Knoten **NICHT KOMMUNIZIEREN**. Will man **KONSISTENZ (C)** wahren, muss man Anfragen **ABLEHNEN (kein A)**, bis die Daten synchron sind. Will man **VERFÜGBAR** bleiben (A), riskiert man, **VERALTETE** oder **UNTERSCHIEDLICHE** Daten zu liefern (kein C).“*

⚠ **In der Praxis ist P nicht wählbar** — Netzerkerausfälle passieren. Die reale Wahl ist also immer zwischen C und A.

68. Das BASE-Modell ★★★★★

68.1 Vom CAP-Theorem zum BASE-Modell

„Gemäß CAP-Theorem müssen massiv verteilte Systeme **bei gesetzter Ausfalltoleranz (P) zwischen STRIKTER KONSISTENZ (C) und HOHER VERFÜGBARKEIT (A) WÄHLEN**. Für globale Webanwendungen ist **VERFÜGBARKEIT (A) oft KRITISCHER als sofortige Konsistenz (C)**. Nutzer erwarten eine Antwort, auch wenn Daten eines Knotens noch nicht final aktualisiert sind. ⇒ **Ansatz: Man LOCKERT die strengen ACID-Anforderungen und AKZEPTIERT VERZÖGERTE KONSISTENZ zugunsten der SKALIERBARKEIT**. Dieser Ansatz wird durch das **BASE-Modell** formalisiert, welches das **GEGENSTÜCK** zum klassischen **ACID-Prinzip** bildet.“

68.2 BASE ★★★★★

BASE = Basically Available, Soft State, Eventually Consistent

Buchstabe(n)	Bedeutung	Erläuterung
BA	Basically Available	Das System garantiert eine grundlegende Verfügbarkeit , indem es Anfragen AUCH BEI TEILAUSFÄLLEN oder NETZWERKPARTITIONEN beantwortet
S	Soft State	Es wird akzeptiert, dass sich Daten ZEITWEILIG in einem ÜBERGANGSZUSTAND befinden , in dem die Replikate noch NICHT SYNCHRONISIERT sind
E	Eventually Consistent	Es wird lediglich zugesichert , dass das System OHNE WEITERE SCHREIBZUGRIFFE NACH EINER GEWISSEN ZEIT einen konsistenten Zustand ERREICHT

BASE als Gegensatz zu ACID (starke Konsistenz):

- „**Replizierte Rechnerknoten können ZWISCHENZEITLICH UNTERSCHIEDLICHE DATENVERSIONEN halten** und werden **erst ZEITLICH VERZÖGERT aktualisiert**“
- ★ „**Eventual Consistency** bedeutet, dass **Schreibaktionen NICHT UNMITTELBAR auf ALLEN Knoten durchgeführt werden müssen**, sondern dass **Aktualisierungen NACH UND NACH im Knotennetz verteilt** werden können“

68.3 ★★☆☆ ACID vs. BASE — die Gegenüberstellung

Kriterium	ACID	BASE
Konsistenz	stark, sofort (<i>strong consistency</i>)	schwach, verzögert (<i>eventual consistency</i>)
Verfügbarkeit	nachrangig	★ Vorrang
Zustand	immer konsistent	★ Soft State (Übergangszustände erlaubt)
Priorität	Korrektheit	★ Skalierbarkeit + Verfügbarkeit
CAP-Einordnung	CP (bzw. CA im zentralen Fall)	★ AP
Typische Systeme	RDBMS (PostgreSQL, Oracle, SAP HANA), Neo4j	NoSQL (Cassandra, DynamoDB, Riak)
Skalierung	vertikal	horizontal
Transaktionen	vollständige Transaktionen	oft keine Multi-Row-Transaktionen

TEIL J — NoSQL-DATENBANKEN**69. Der Paradigmenwechsel ★★☆☆****69.1 Konzept und Ursprung**

Aspekt	Erläuterung
Terminus (1998)	bewusster VERZICHT auf SQL-Schnittstellen
★ Präzisierung	„ NICHT-RELATIONAL “ ist oft TREFFENDER
Schema-flexibel	KEINE starren Tabellendefinitionen

★★ Warum ist „nicht-relational“ treffender? (Musterlösung der Folie)

„Weil **viele NoSQL-Systeme mittlerweile SQL-ÄHNLICHE SCHNITTSTELLEN** (z. B. **CQL**) besitzen, ist das **Fehlen von SQL NICHT MEHR das Hauptmerkmal**; das **abweichende DATENMODELL** hingegen schon.“

69.2 Treiber und Evolution ★

Treiber	Erläuterung
Web 2.0 (2000er)	Massives Wachstum der User-Zahlen
Big Data	Bewältigung unstrukturierter Datenmengen
Hochverfügbarkeit	„System darf NIEMALS stillstehen“

69.3 Architektur (BASE) ★

- **Verfügbarkeit: Vorrang vor strikter Konsistenz**
- **Replikation: Schutz vor Ausfall einzelner Knoten**
- **BASE-Modell:** Basically Available, Soft-state, Eventual consistency

69.4 Das NoSQL-Versprechen ★★

Versprechen	Beschreibung
Web-Scale	Millionen gleichzeitige Nutzer ohne Performance-Verlust
Verfügbarkeit	Kontinuierlicher Betrieb auch bei Knotenausfall (BASE)
Flexible Strukturen	Integration unstrukturierter oder polymorpher Daten
Massiver Durchsatz	Verarbeitung von Big Data mit extrem geringer Latenz

69.5 Die sechs Eigenschaften von NoSQL-Datenbanken ★★★

Eigenschaft	Beschreibung
Datenmodell	Nicht-relationaler Ansatz zur Abbildung komplexer Strukturen
Skalierung	unterstützt massiv verteilte Webanwendungen und HORIZONTALE Skalierung
Fokus auf Big Data (3V) ★★	erfüllt die Anforderungen für umfangreiche Datenbestände (VOLUME) , flexible Datenstrukturen (VARIETY) und Echtzeitverarbeitung (VELOCITY)
Schema	KEIN FIXES SCHEMA nötig — ermöglicht agile Entwicklung und schnelle Iterationen
Replikation	Hochverfügbarkeit durch Verteilung von Datenkopien über mehrere Knoten
Konsistenz	anpassbar über das CAP-Theorem (oft Weak/Eventual Consistency)

★★★ Die 3 V's von Big Data — auswendig: **VOLUME** (Menge) · **VARIETY** (Vielfalt/Strukturflexibilität) · **VELOCITY** (Geschwindigkeit).

69.6 Die vier Hauptkategorien ★★★

Kategorie	Kernidee	Fokus
Key-Value Stores	Einfachstes Modell: Wert wird über eindeutigen Schlüssel adressiert. Extrem schnell	Maximale Geschwindigkeit
Document Stores	Speicherung in strukturierten Dokumenten (JSON/BSON). Hohe Flexibilität	Flexibilität und tiefe Schachtelung
Wide-Column Stores	SPALTENFAMILIEN statt Tabellenzeilen. Optimiert für massive Datenverteilung	Massive Skalierbarkeit & Schreibdurchsatz
Graph-Datenbanken	Fokus auf BEZIEHUNGEN . Knoten und Kanten als primäre Speicherobjekte	Komplexe Beziehungen

Die Analogien der Folie ★★:

Datenbank-Typ	Analogie
Key-Value	ein einfaches WÖRTERBUCH
Document	ein ARCHIV für JSON/XML-AKTEN
Wide-Column	eine TABELLE mit MILLIARDEN Spalten/Zeilen
Graph	ein SOZIALES NETZWERK (Knoten/Kanten)

Architektur: „massiv verteilte Datenhaltungsarchitektur mit **Fokus auf HORIZONTALE Skalierbarkeit.**“

70. Key-Value-Datenbanken ★★★★★

70.1 Das Paradigma

Aspekt	Beschreibung
Definition	Einfachstes NoSQL-Datenmodell — Zuweisung: UNIQUE KEY → VALUE
Schemafrei	keine Tabellen-Vorabdefinition
⚠ Einschränkung	KEINE Fremdschlüssel / KEINE referentielle Integrität
Analogie	★ „ein riesiges, verteiltes DICTIONARY oder eine HASHMAP im Arbeitsspeicher“

70.2 Namespacing und Schlüssel-Design ★★★★★

- * „Technisch sind **ALLE Keys GLEICHBERECHTIGT** („FLACH“)“*
- ★ „Konvention: Nutzung von **TRENNZEICHEN (:)** zur **SIMULATION** von **HIERARCHIEN**“
- ⚠ ⚠ „**WICHTIG: Die Datenbank ‚VERSTEHT‘ die Logik NICHT. Die Struktur existiert NUR in der ANWENDUNGSLOGIK.**“

User:1001:vorname
 User:1001:email
 Product:A55:preis

„Eine Key-Value-Datenbank behandelt diese Keys als **SEPARATE STRINGS OHNE SEMANTISCHE VERBINDUNG.**“

70.3 Basis-Operationen und Systeme ★

Operation	Beschreibung
SET	Speichert/Überschreibt einen Wert
GET	Liest den Wert eines Keys aus
DEL	Entfernt Key und Wert

System	Charakteristik
Redis	In-Memory , unterstützt Datenstrukturen (Listen, Sets)
Memcached	reiner Cache , extrem simpel
Riak / DynamoDB	Fokus auf Hochverfügbarkeit

70.4 ★★★★★ Warum Key-Value? Die Performance-Begründung

Vorteil	Erläuterung
Massive Skalierbarkeit	Sharding: „Da Keys UNABHÄNGIG sind, lassen sie sich leicht über Knoten verteilen (Partitioning)“
★★★★ Konstante Zugriffszeit	„O(1) — UNABHÄNGIG davon, ob in der Datenbank 100 Datensätze ODER 100 MILLIARDEN Datensätze gespeichert sind. (★ GEGENSATZ zu relationalen Datenbanken — dort LOGARITHMISCH)“
Maximale Performance	häufig IN-MEMORY-Betrieb · KEINE Rechenlast durch JOINS oder ACID-Constraints
Ideal für	Session-Management, Caching und Real-Time Analytics

71. Redis ★★★★★

71.1 Hintergrund

Aspekt	Info
Entwickler	Salvatore Sanfilippo (2009)
★ Akronym	REmote DIctionary Server
Charakter	Open-Source & In-Memory fokussiert
Ziel	Maximale Lese-/Schreibgeschwindigkeit
Architektur	„Daten werden primär im RAM gehalten. Dies ermöglicht Antwortzeiten im MIKROSEKUNDEN-Bereich , erfordert aber Strategien gegen Datenverlust “
Einsatz	„häufig als Anwendungs-Cache (Session, Page, Object caching), kann aber auch als PRIMÄRE Datenbank für Messaging oder Echtzeit-Anwendungen dienen“

71.2 Die Redis-Datentypen ★★★★★

Typ	Beschreibung
Strings	der Basis-Typ (Text, Zahlen, Binärdaten)
Hashes	Maps von Feldern zu Werten — ★ ideal für Objekte
Lists	geordnete Ketten von Strings (Stacks/Queues)
Sets	ungeordnete Mengen (eindeutige Werte)
Streams	Append-only Log-Strukturen
RedisJSON	Modul für native JSON-Dokumente — ★ „Damit kann Redis auch als DOCUMENT STORE verwendet werden“

⚠️⚠️ „**WICHTIG:** Befehle sind **TYPESPEZIFISCH**. Ein **HGET** auf einem String-Key führt zum Fehler (WRONGTYPE-Error). Redis **ERZWINGT TYPESICHERHEIT AUF KEY-EBENE.**“

★★ Die Einordnung der Folie: „Redis ist **streng genommen KEINE reine ‚Key-Value‘-Datenbank**, sondern eine **‚KEY-DATA-STRUCTURE‘-Datenbank.**“

71.3 Persistenz ★★★★★

Verfahren	Beschreibung
RDB (Redis Database)	Kompakte SCHNAPPSCHÜSSE der Daten · Point-in-Time Recovery · geringer Performance-Impact
AOF (Append Only File)	Protokolliert JEDE Schreiboperation · bietet MAXIMALE SICHERHEIT gegen Abstürze · JOURNALING-Prinzip

★ „Die Daten werden **zyklisch in RDB-Dateien als Schnapsschüsse gespeichert** und **Veränderungen in einem JOURNAL festgehalten**, um **Systemausfälle OHNE Datenverlust** zu gewährleisten.“

71.4 Die Redis-Befehle ★★★★★

A) Strings (einfache Werte)

Befehl	Beschreibung
SET <Key> <Value>	Speichert einen Wert unter einem Schlüssel
GET <Key>	Ruft den Wert eines Schlüssels ab
INCR <Key>	★ Erhöht einen numerischen Wert ATOMAR um 1
EXISTS <Key>	Prüft, ob ein Schlüssel vorhanden ist
DEL <Key>	Löscht einen Schlüssel und seinen Wert

B) Hashes (strukturierte Objekte)

Befehl	Beschreibung
HSET <Key> <Feld> <Value>	Speichert einen Wert in einem spezifischen FELD eines Hashes
HGET <Key> <Feld>	Ruft den Wert eines EINZELNEN Feldes ab
HGETALL <Key>	★ Gibt ALLE Felder und Werte eines Hashes zurück
HDEL <Key> <Feld>	Löscht ein spezifisches FELD innerhalb eines Hashes

C) Allgemeines & Verwaltung

Befehl	Beschreibung
EXPIRE <Key> <Sekunden>	★ Setzt eine Zeitspanne, nach der ein Schlüssel AUTOMATISCH gelöscht wird
TTL <Key>	Zeigt die verbleibende Lebenszeit eines Schlüssels in Sekunden an

 Beispiel in der Redis-Konsole (redis-cli):

```
SET User:U17547:nachname Mueller
```

(Der Key ist User:U17547:nachname)

71.5 ★★★★★ String-Modell vs. Hash-Modell — die Kernunterscheidung

	String-Modell	Hash-Modell
Typ	Redis String	Redis Hash
Struktur	★ FLACH — jedes Attribut ist ein eigener Key (user:U17547:vorname)	★ ZWEISTUFIG — EIN Key (user:U17547) enthält viele „Felder“
Befehle (Jedis)	jedis.set() / jedis.get()	jedis.hset() / jedis.hgetAll()
Folge	⚠ Viele get() -Aufrufe nötig, um EINEN User zu laden	✅ NUR EIN hgetAll() -Aufruf nötig

Die drei Vorteile des Hash-Modells ★★:

1. „Alle Daten eines Users können mit **EINEM EINZIGEN Befehl** (**HGETALL**) gelesen werden“
2. **Logische Bündelung:** „Daten, die zusammengehören (ein User-Objekt), werden **unter EINEM Key** gespeichert“
3. **Übersichtlichkeit:** „Reduziert die **Anzahl der Keys** auf der **obersten Ebene**“

★ * „Die **String-Lösung** ist **technisch NÄHER** am Key-Value-Konzept. Die **Hash-Lösung** ist jedoch das **ÜBLICHERE, EFFIZIENTERE** und **EMPFOHLENE** Vorgehen, wenn man **OBJEKTE** in Redis speichert.“*

71.6 Redis in Java (Jedis) ★★

Jedis ist ein **Java-Client für Redis** (Bibliothek/Sammlung von Java-Klassen). Er wird als **Abhängigkeit (Dependency)** in der **pom.xml (Maven)** aufgenommen.

```
import redis.clients.jedis.Jedis;
import java.util.Map;

// Verbindung
Jedis jedis = new Jedis(REDIS_HOST, REDIS_PORT);

// — String-Modell —————
jedis.set("user:U17548:vorname", "Lisa");
String vorname = jedis.get("user:U17547" + ":vorname");

// — Hash-Modell —————
String key1 = "user:U17547";
Map<String, String> userData1 = Map.of(
    "vorname", "Max",
    "nachname", "Mustermann",
    "email", "max@test.de"
);
jedis.hset(key1, userData1); // schreibt den Hash
Map<String, String> geleseneDaten = jedis.hgetAll(key1); // liest den ganzen Hash
```

★★ Redis Hash vs. Java Map — die Abgrenzung (prüfungsrelevant):

	Redis Hash	Java Map
Was	NATIVER DATENTYP in der Redis-Datenbank	Datenstruktur im JAVA-CODE (<code>java.util.Map</code>)
Speichert	Sammlung von Feld-Wert-Paaren unter EINEM Hauptschlüssel	temporärer „Container“, um die User-Daten in Java zu bündeln
Rolle	ideal für strukturierte Objekte	„Die <code>jedis.hset</code> -Methode nutzt diese Java Map als EINGABE , um den Redis Hash zu befüllen“

71.7 Installation und Betrieb ★

#	Was	Wo	Befehl
1	Linux-Subsystem aktivieren	Windows PowerShell	<code>wsl --install</code>
2	Linux-Distribution installieren	Windows PowerShell	<code>wsl --install -d ubuntu</code>
3	Redis installieren	Ubuntu	<code>sudo apt update</code> <code>sudo apt install redis-server</code>
4	Redis starten	Ubuntu	<code>sudo service redis-server start</code>
5	Kontrolle, ob Redis läuft	Ubuntu	<code>redis-server status</code>
6	Kommandozeilentool starten	Redis	<code>redis-cli</code>

- „Redis erfordert für den Start **fast keine Konfiguration**“
- ★ * „Standardmäßig stellt Redis **16 LOGISCHE DATENBANKEN** zur Verfügung, **von 0 bis 15** nummeriert. **Bei einer Verbindung landet man automatisch in Datenbank 0.**“*
- * „In der Standardkonfiguration läuft Redis im sogenannten **„PROTECTED MODE“** — der Server akzeptiert **Verbindungen nur von localhost**“*
- **Verbindung in IntelliJ mit Java-Code:** `localhost:6379` ★ (Standard-Port merken!)

72. Dokumentenorientierte Datenbanken ★★★★★

72.1 Motivation und Einordnung

- Entwickelt für moderne Webdienste
- Nahtlose JavaScript/JSON-Integration
- Hohe Agilität durch Schemafreiheit
- Lineare horizontale Skalierbarkeit

⚠️ ⚠️ Die wichtigste Begriffsklärung:

„Dokument-Datenbanken speichern NICHT beliebige Dokumente (wie PDFs), sondern STRUKTURIERTE DATENSÄTZE, die ‚DOKUMENTE‘ genannt werden.“

72.2 Das zweistufige Datenmodell ★★★★★

„Dokument-Datenbanken sind AUF DER ERSTEN EBENE eine Art KEY-VALUE-Datenbank“

- **Schlüssel:** eindeutige Dokument-ID (z. B. 4711)
- **Wert:** ein strukturiertes Dokument (JSON/BSON)

Die ZWEITE Ebene definiert die interne Struktur:

- Rekursiv verschachtelte Attribut-Wert-Paare
- Listen und Arrays als Attributwerte möglich
- ★ In sich abgeschlossene Datensammlung (KEINE systemseitige referenzielle Integrität)

72.3 Schemafreiheit und ihre Kosten ★★★★★

Vorteile (Schema-on-Read)	Trade-offs (die „Kosten“)
Keine Definition vor Dateneingabe nötig	⚠ Keine referenzielle Integrität — Beziehungen müssen MANUELL gepflegt werden
Jedes Dokument kann INDIVIDUELLE Attribute haben	⚠ Keine Normalisierung — Datenredundanz wird für Performance IN KAUF GENOMMEN
Ideal für unstrukturierte oder sich schnell ändernde Daten	⚠ Komplexere Abfragen bei stark vernetzten Daten
★ Schemaverantwortung liegt bei der APPLIKATION	

Weitere Vertreter: BaseX, eXist, HCL Notes, OrientDB, Apache Jackrabbit, Couchbase

73. MongoDB ★★★★★

73.1 Historie und Motivation

Aspekt	Info
Entwicklungsbeginn	2007 durch 10gen (heute MongoDB Inc.)
Gründer	D. Merriman, E. Horowitz, K. Ryan (ehem. DoubleClick)
Release	erste stabile Version 2009

„Praxiserfahrungen bei DoubleClick zeigten **Skalierungs- und Flexibilitätsgrenzen klassischer RDBMS auf:**“

- **Skalierbarkeit:** Limitation RDBMS, z. B. bei **Ad-Tech-Mengen**
- **Fehlende Flexibilität:** **starre Schemata** machten die agile Entwicklung langsam
- ★ **Objekt-Relationale-Diskrepanz:** „Entwickler denken in **Objekten**, aber mussten diese in relationale Tabellen ‚übersetzen‘ (**O/R-Mapping**) — **komplex, langsam, viele JOINS**“

73.2 Das Datenmodell: BSON-Dokumente ★★★★★

Merkmal	Beschreibung
Dokumentenzentrierter Ansatz	Speicherung in FLEXIBLEN EINHEITEN statt in starren Zeilen/Tabellen-Gittern
Hierarchische Strukturen	Dokumente nutzen das JSON-Format, was die Abbildung komplexer Objektbeziehungen (Nested Documents, Arrays) OHNE JOINS erlaubt
Effizienz durch BSON	BINÄRE Repräsentation sorgt für performantes Parsen und erweitert JSON um TYPISIERTE Daten (z. B. IEEE 754 Decimal128, High-Precision Date, BinData)
Sammlungen (Collections)	Dokumente werden in Collections GRUPPIERT. ★ Eine Sammlung ist das ÄQUIVALENT zu einer TABELLE in SQL-Datenbanken

73.3 ★★★★★ Terminologie-Mapping RDBMS ↔ MongoDB

Sichere Klausurfrage!

RDBMS	MongoDB	Anmerkung
Datenbank	Datenbank	★ „identisches Konzept; dient in beiden Welten als OBERSTER CONTAINER“
Tabelle	COLLECTION	* „Gruppe von Dokumenten. Im Gegensatz zur Tabelle OHNE VORDEFINIERTER STRUKTUR (schemalos)“*
Datensatz / Zeile	DOCUMENT	* „eine Einheit aus benannten Feldern und Werten (JSON/BSON)“*
Spalte	FIELD (Feld)	„Schlüssel-Wert-Assoziation INNERHALB eines Dokuments“
SQL	MQL	MongoDB Query Language

73.4 Weitere Eigenschaften ★★

Eigenschaft	Beschreibung
Dynamisches Schema	„Dokumente in derselben Sammlung müssen NICHT DIESELBE STRUKTUR haben. Iterative, agile Entwicklung möglich, da die Datenbankstruktur jederzeit geändert werden kann, OHNE die Datenbank migrieren zu müssen“
Mächtige Abfragesprache (MQL)	Unterstützt Ad-hoc-Abfragen, feldbasierte Suchen, Range-Queries, Volltextsuche und komplexe Aggregationen (Aggregation Pipeline)
Replica Sets (Verfügbarkeit)	Daten werden auf mehrere Server GESPIEGELT · AUTOMATISCHES FAILOVER bei Ausfall des Primary
Sharding (Skalierung)	Horizontale Verteilung extrem großer Datenmengen · Daten werden über Shards verteilt · Verteilung basiert auf einem SHARD KEY (Partitionierungsschlüssel) · ermöglicht eine FAST LINEARE Skalierung von Lese- und Schreibvorgängen

73.5 Exkurs JSON und BSON ★★★★★

Format	Definition
JSON (JavaScript Object Notation)	TEXTBASIERTES, für Menschen LESBARES Datenaustauschformat
BSON (Binary JSON)	★ BINÄRE Repräsentation von JSON; wird von MongoDB zur effizienten Speicherung und Indizierung genutzt

Eigenschaften von JSON:

- **Schlüssel-Wert-Paare in geschweiften Klammern { }** sowie **Listen in eckigen Klammern []**
- **Abgeleitet aus dem JavaScript-Sprachstandard, jedoch SPRACHUNABHÄNGIG einsetzbar**
- **Kein vordefiniertes Schema nötig**
- ★ „Moderner Ersatz für XML zur Darstellung strukturierter Daten“

 **JSON-Beispiel mit allen Elementen:**

```
{
  "vorname":      "Max",           // String (Text)
  "alter":        30,              // Number (Zahl)
  "istStudent":   false,           // Boolean (wahr/falsch)
  "interessen":   ["Lesen", "Sport"], // Array (Liste)
  "adresse":      {                // verschachteltes Objekt (nested Object)
    "strasse":    "Musterstraße 1",
    "stadt":      "Berlin"
  },
  "firmenwagen":  null             // null = leeres/nicht existentes Feld
}
```

★★ Die zwei wichtigen Zusatzregeln:

1. „**MongoDB weist JEDEM Dokument ein EINDEUTIGES `_id`-Feld als PRIMÄRSCHLÜSSEL** zu. Wenn der Benutzer beim Einfügen kein `_id` angibt, **generiert MongoDB AUTOMATISCH eine ObjectId.**“
2. „**In der JSON-Variante von MongoDB (BSON) sind WEITERE DATENTYPEN erlaubt, z. B. Datum.**“*
3. Der **Schlüssel muss immer in DOPPELTEN Anführungszeichen** stehen

73.6 MQL — CRUD-Operationen ★★★★★

Zugriffsmuster: <database>.<collection_name>.<operation>

Befehl	Beschreibung
<code>db.<Coll>.insertOne({Dokument})</code>	Fügt EIN Dokument in eine Sammlung ein
<code>db.<Coll>.insertMany([{... }, {... }])</code>	Fügt mehrere Dokumente ein
<code>db.<Coll>.find({Filter})</code>	Sucht nach allen Dokumenten , die dem Filter entsprechen
<code>db.<Coll>.findOne({Filter})</code>	Gibt nur das ERSTE passende Dokument zurück
<code>db.<Coll>.countDocuments({Filter})</code>	Zählt die Anzahl der passenden Dokumente
<code>db.<Coll>.updateOne({Filter}, {Update})</code>	Ändert das ERSTE passende Dokument
<code>db.<Coll>.updateMany({Filter}, {Update})</code>	Ändert alle passenden Dokumente
<code>db.<Coll>.deleteOne({Filter})</code>	Löscht das ERSTE passende Dokument
<code>db.<Coll>.deleteMany({Filter})</code>	Löscht alle passenden Dokumente

Die wichtigen Update-Operatoren ★★★★★:

Operator	Beschreibung
<code>\$set</code>	Aktualisiert ein bestehendes Feld ODER FÜGT ES HINZU , falls es nicht existiert
<code>\$inc</code>	Erhöht (oder VERRINGERT bei negativen Zahlen) einen numerischen Wert
<code>\$unset</code>	Entfernt ein spezifisches Feld KOMPLETT aus einem Dokument

Die Filter-Operatoren (Vergleiche) ★★★★★:

Operator	Bedeutung
\$lt	kleiner als (<i>less than</i>)
\$gt	größer als (<i>greater than</i>)
\$lte	kleiner gleich
\$gte	größer gleich
\$or	logisches ODER

73.7 MQL-Beispiele

```
// — Datenbank/Collection wählen —————
use meineTestDatenbank
db.benutzer

// — 1. CREATE —————
db.benutzer.insertOne({
  name:      "Anna Schmidt",
  alter:     28,
  email:     "anna.schmidt@example.com",
  status:    "aktiv",
  interessen: ["Lesen", "Wandern", "Kochen"]
});

db.benutzer.insertMany([
  { name: "Peter Klein", alter: 35, email: "peter.klein@example.com",
    status: "inaktiv" },
  { name: "Lena Meyer",  alter: 22, email: "lena.meyer@example.com",
    status: "aktiv", interessen: ["Musik", "Gaming"] },
  { name: "Max Mustermann", alter: 30, email: "max@beispiel.com",
    registriert_am: new Date() }
]);

// — 2. READ —————
db.benutzer.find({}); // alle Dokumente
db.benutzer.find({ status: "aktiv" }); // alle aktiven Benutzer
db.benutzer.find({ alter: { $gt: 25 } }); // älter als 25
db.benutzer.find({ alter: { $lt: 30 } }); // jünger als 30
db.benutzer.find({ alter: { $gte: 30 } }); // Alter >= 30
db.benutzer.find({ alter: { $lte: 30 } }); // Alter <= 30
db.benutzer.find({ alter: 30, status: "aktiv" }); // UND-Verknüpfung
db.benutzer.find({ $or: [ { status: "aktiv" }, { alter: { $lt: 25 } } ] });
db.benutzer.find({ interessen: "Musik" }); // ★ auch in einem ARRAY suchbar!
db.benutzer.findOne({ name: "Anna Schmidt" }); // einzelnes Dokument

// ★ PROJEKTION: welche Felder ausgeben? (1 = anzeigen, 0 = ausblenden)
db.benutzer.find({}, { name: 1, email: 1 }); // nur Name und Email
db.benutzer.find({ status: "aktiv" }, { name: 1, _id: 0 }); // nur Namen, ohne _id

// — 3. UPDATE —————
db.benutzer.updateOne(
  { name: "Peter Klein" }, // Filter
  { $set: { status: "aktiv", letztes_update: new Date() }, // was ändern
    $inc: { anzahl_updates: 1 } } // Zähler erhöhen
);
db.benutzer.updateMany(
  { alter: { $lt: 25 } },
  { $set: { status: "student" } }
);

// — 4. DELETE —————
db.benutzer.deleteOne({ name: "Peter Klein" });
db.benutzer.deleteMany({ status: "inaktiv" });
```

★★ Wichtige Detailregeln:

- `$inc` **ERSTELLT** das Feld, falls es nicht existiert
- `db.benutzer.find({ interessen: "Musik" })` findet Dokumente, **auch wenn interessen ein ARRAY ist** — MongoDB sucht automatisch **innerhalb** des Arrays
- Bei der **Projektion** bedeutet **1 = anzeigen, 0 = ausblenden**; `_id` wird **immer mitgeliefert**, wenn man es nicht explizit mit `_id: 0` ausblendet

73.8 MongoDB in Java ★★

```
MongoClient mongoClient = MongoClient.create(connectionString);
MongoDatabase database = mongoClient.getDatabase("meineTestDatenbank");
MongoCollection<Document> collection = database.getCollection("benutzer");

// CREATE
Document neuerBenutzer = new Document("name", "Julia Schmidt (Java)")
    .append("alter", 29)
    .append("programmiersprache", "java")
    .append("status", "aktiv");
collection.insertOne(neuerBenutzer);

// READ
Document gefundenerBenutzer = collection.find(eq("name", "Julia Schmidt (Java)")).first();

// DELETE
collection.deleteOne(eq("name", "Julia Schmidt (Java)"));
```

★★ Der Unterschied zur MongoDB-Shell (prüfungsrelevant!):

- „Dieser Java-Code läuft in einer **Java-Umgebung (JVM)**, **NICHT in einer JavaScript-Shell.**“
- ★ „**Die Java-Sprache versteht die { ... } -Syntax NICHT als Definition für ein Datenbankdokument.** In Java müssen wir **JAVA-OBJEKTE** verwenden, um Daten zu repräsentieren.“
- * „Der MongoDB-Treiber gibt uns dafür die **Klasse org.bson.Document**. Um dieses Objekt zu erstellen und zu füllen, verwenden wir `.append()` (oder `.put()`).“*

74. Wide-Column-Datenbanken ★★★★★

74.1 Warum Wide-Column? ★★

Das Problem bei RDBMS:

- ★ **Sparse Data:** „**Viele ,NULL'-Werte VERSCHWENDEN Platz und Performance**“
- **Zeilenspeicherung:** „Um **nur EINE Spalte** zu lesen, muss oft **die GANZE Zeile** geladen werden“
- **Starrheit:** „**Jede Zeile muss die EXAKT GLEICHEN Spalten haben**“

🔑 Die Erkenntnis: * „Oft werden nur **BESTIMMTE GRUPPEN** von Spalten (**COLUMN FAMILIES**) **GEMEINSAM ABGEFRAGT**. Warum also nicht diese Gruppen als **SPEICHEREINHEIT** nutzen?“*

74.2 Das logische Modell ★★

„Ein Wide-Column Store ist eine MEHRDIMENSIONALE, SORTIERTE MAP:“

(RowKey, ColumnFamily, Column, Timestamp) → Value

Die Rolle des Timestamps ★★:

Zweck	Beschreibung
Versionierung	„Datenbanken wie HBase speichern standardmäßig z. B. die letzten 3 VERSIONEN eines Wertes“
Konfliktlösung	★ * „In verteilten Systemen GEWINNT DER SPÄTESTE ZEITSTEMPEL (LAST WRITE WINS) “*
Automatisches Cleanup	„Über TTL (Time To Live) können alte Versionen automatisch gelöscht werden“

74.3 Struktur und Spaltenfamilien ★★

- Daten werden in **COLUMN FAMILIES** (Spaltenfamilien) **GRUPPIERT**
- Innerhalb einer Zeile (identifiziert durch einen **ROW KEY**) können **BELIEBIG VIELE** Spalten existieren
- ★ „Unterschiedliche Zeilen in **DERSELBEN** Tabelle können **VÖLLIG UNTERSCHIEDLICHE** Spalten haben (Schema-Flexibilität)“

74.4 Physische Speicherung und Performance ★★

Column-Family Storage: „Daten einer Spaltenfamilie werden **physisch in einer EIGENEN DATEI (SSTable)** gespeichert.“

Vorteil	Erläuterung
I/O-Effizienz	Nur die BENÖTIGTEN Spaltenfamilien werden von der Festplatte gelesen
Kompression	Ähnliche Daten in Spalten lassen sich EXTREM GUT komprimieren
Skalierung	Milliarden Zeilen lassen sich über Partitionierung (Sharding) auf HUNDERTE Server verteilen

74.5 Historie und Vertreter ★★

System	Info
Google BigTable (2006/2008)	★ DER PIONIER — * „Das Whitepaper, das alles veränderte. Basis für Google Search, Maps und Gmail. “*
Apache HBase	„Die Open-Source-Implementierung von BigTable auf HADOOP-Basis “
Apache Cassandra	„Kombiniert Wide-Column-Datenmodell mit der verteilten Architektur von AMAZON DYNAMO “

⚠ * „**Wichtig:** Cassandra nutzt das Wide-Column-Modell, fügt aber eine **SQL-ÄHNLICHE Abfragesprache (CQL)** hinzu. **DENNOCH bleibt die PHYSISCH STRUKTUR eine Wide-Column Map!**“*

75. Apache Cassandra ★★

75.1 Entstehung und Geschichte ★★

Aspekt	Info
Ursprung	★ 2008 bei FACEBOOK entwickelt
Ziel	Realisierung des Facebook Inbox Search-Dienstes: sehr große Datenmengen · hohe Schreiblast · weltweite Verfügbarkeit · KEIN Single Point of Failure
Motivation	„Klassische relationale Datenbanken konnten diese Anforderungen NICHT erfüllen“
★ Technische Einflüsse	Amazon Dynamo: verteilte, fehlertolerante Key-Value-Architektur · Google Bigtable: Wide-Column-Datenmodell (Column Families)
Open Source	2009 als Open-Source-Projekt veröffentlicht; seit 2010 Top-Level-Projekt der Apache Software Foundation (ASF)
Kommerzieller Sponsor	DataStax (Support, Enterprise-Versionen)
Einsatzgebiete	große Web-Anwendungen · IoT- und Zeitreihendaten · Finanzsysteme · Streaming- und Logging-Systeme

Die vier zentralen Ziele ★★:

1. **Lineare horizontale Skalierbarkeit**
2. **Hohe Verfügbarkeit** (auch bei Node-Ausfällen)
3. **Eventual Consistency**
4. ★ **Verteilte Architektur OHNE MASTER-KNOTEN (Peer-to-Peer)**

🔑 „Apache Cassandra ist ein VERTEILTES, HOCHSKALIERBARES WIDE-COLUMN-STORE-System (P2P-Architektur, KEIN Single Point of Failure, WRITE-OPTIMIERT, FLEXIBLES SCHEMA).“

75.2 Einordnung und Vergleich mit RDBMS ★★

- * „Cassandra gehört zu den **Column-Family-Datenbanken (Wide-Column Stores)**. Anders als **dokumentenorientierte oder Key-Value-Stores** speichert es Daten in **TABELLEN mit ZEILEN, die VIELE DYNAMISCHE SPALTEN haben können**.“*
- * „Relationale DB nutzen **starres Schema und Normalformen**, unterstützen **JOINS/Transaktionen**. ★ **Cassandra VERZICHTET auf JOINS/Subqueries und relationale Integrität — Daten werden DENORMALISIERT (,ONE QUERY PER TABLE’)**.“*
- „**Schema-Freiheit**: Cassandra-Tabellen erfordern **NUR einen PRIMÄRSCHLÜSSEL**“
- * „Das Datenmodell ist **,dynamisch’**. Spalten können **OPTIONAL** sein. **NICHT jede Zeile muss ALLE Spalten besitzen**. Schemaänderungen (**ALTER TABLE**) sind **ONLINE** möglich.“*

75.3 Datenmodell: Keyspace und Tabelle ★★★★★

Begriff	Beschreibung
Keyspace	★ „ähnlich wie eine DATENBANK“ — definiert die Replikationsstrategie (SimpleStrategy , NetworkTopologyStrategy) und den Replikationsfaktor (RF) . „Jede Anwendung nutzt typischerweise EINEN Keyspace.“
Tabelle (Column Family)	Besteht aus Spalten und einem PRIMÄRSCHLÜSSEL . ★ Der Primärschlüssel gliedert sich in PARTITION KEY (für VERTEILUNG) und CLUSTERING COLUMNS (für SORTIERUNG innerhalb einer Partition)

```
-- Einfacher Primärschlüssel (Partition Key = id)
CREATE TABLE personen (
    id    UUID,
    name  text,
    alter int,
    PRIMARY KEY (id)
);

-- ★ Zusammengesetzter Primärschlüssel
-- (land) = Partition Key (erkennbar an der ZUSÄTZLICHEN KLAMMER)
-- stadt, strasse = Clustering Columns
CREATE TABLE adressen (
    land    text,
    stadt   text,
    strasse text,
    PRIMARY KEY ((land), stadt, strasse)
);
```

75.4 Partition Key, Clustering Columns, Replikation ★★★★★

Konzept	Beschreibung
Partition	*„ein CONTAINER für zusammengehörige Datensätze, die physisch GEMEINSAM auf der Festplatte EINES Servers (Nodes) gespeichert werden. ★ Alles, was denselben Partition Key hat, landet in DERSELBEN Partition. “*
Datenverteilung	*„Anhand eines HASHES (TOKEN) auf Basis des Partition Keys wird die Verteilung der Daten auf den Knoten gesteuert. Der Standard-Partitioner (MURMUR3) verteilt Daten gleichmäßig über alle Nodes. Neue Nodes lösen AUTOMATISCH ein REBALANCING aus. “*
Clustering Columns	„legen INNERHALB einer Partition die ZEILENREIHENFOLGE fest. Ermöglichen z. B. Sortierung nach Zeitstempel oder Namen “
Replikation	*„Jeder Keyspace definiert, wie viele KOPIEN (Replication Factor RF) von jeder Zeile über Nodes/Datenzentren erstellt werden. ★ ALLE Replikate sind GLEICHWERTIG (KEIN Master). “*

75.5 CQL — Cassandra Query Language ★★★★★

*„**SQL-ÄHNLICHE Sprache** für Cassandra. **Die Unterschiede zu SQL liegen WENIGER IN DER SYNTAX, sondern im ZUGRUNDE LIEGENDEN DATENMODELL und den ERLAUBTEN ABFRAGEN.**“*

Ähnlichkeiten zu SQL	Wesentliche Unterschiede zu SQL
Tabellen-, Zeilen- und Spaltenkonzept	⚠️ KEINE Joins oder Subqueries
DDL (CREATE / DROP) und DML (SELECT , INSERT , UPDATE , DELETE) existieren	Datenmodell erfordert Primärschlüssel bei Tabellen und OPTIMIERTE ABFRAGEN (Query für Primärschlüssel)
Daten werden in Tabellen gespeichert	Sekundärindizes sind EINGESCHRÄNKT (nur einfache Gleichheits-Filter)
Syntax in weiten Teilen ähnlich	

75.6 ★★★★★ SQL vs. CQL — die Gegenüberstellung

SQL (relational)	CQL (Cassandra)
Tabellen mit Zeilen	Tabellen mit PARTITIONEN
Normalisierung	★ DENORMALISIERUNG (GEWOLLTE Redundanz)
Fremdschlüssel	⚠️ KEINE Beziehungen
JOINS	⚠️ KEINE JOINS
Flexible Abfragen	⚠️ Abfragemöglichkeiten VORGEGEBEN (Primärschlüssel)

🔑★★★★★ DER Merksatz: „In SQL heißt es ‚MODEL FIRST‘ (erst Struktur, dann Abfrage), in Cassandra heißt es ‚QUERY FIRST‘ (erst Abfrage, dann Struktur).“

75.7 Die WHERE-Klausel in CQL ★★★★★

```
SELECT * FROM personen WHERE alter = 18;    -- Syntax analog zu SQL
```

⚠️⚠️ „Die WHERE-Klausel in CQL darf NUR TEILE DES PRIMÄRSCHLÜSSELS enthalten (PARTITION KEY ZWINGEND, Clustering Columns optional). NUR MIT DEM PARTITION KEY WEISS CASSANDRA, AUF WELCHEM NODE DIE DATEN LIEGEN.“

Warum keine JOINS? ★★

*„Joins benötigen **Beziehungen zwischen Tabellen, gemeinsame Schlüssel** und greifen auf **mehrere Tabellen** zu. Da Cassandra **VERTEILT** über viele Knoten ist, wären **JOINS SEHR TEUER** und **LANGSAM**. Cassandra hat **BEWUSST** auf JOINS **VERZICHTET**, um **SKALIERBARKEIT** und **VERFÜGBARKEIT** zu garantieren.“*

- Daten werden denormalisiert, Redundanz ist akzeptiert
- ★ „Oft **MEHRERE TABELLEN** für **VERSCHIEDENE ABFRAGEN** — ‚ONE QUERY – ONE TABLE‘-Prinzip“

Filtern auf Clustering Columns ⚠️:

„Man darf auf der **Clustering-Spalte NUR filtern**, wenn **GLEICHZEITIG AUCH auf den PARTITION KEY gefiltert** wird.“ Alternativen:

1. **Anlage eines SEKUNDÄREN INDEX** auf die Spalte — ⚠ „nur wenn **WENIG UNTERSCHIEDLICHE Werte**“
2. **Option ALLOW FILTERING :**

```
SELECT * FROM login_events WHERE ip = '192.168.1.1' ALLOW FILTERING;
```

⚠⚠ „NIE auf Produktivumgebung! Performance bricht ein.“

75.8 UPSERT ★★

„KEINE Unterscheidung zwischen **INSERT** und **UPDATE** .“

- „In klassischen SQL-Datenbanken sind **INSERT** und **UPDATE** **GETRENNTE Operationen.**“
- ★ „In Cassandra ist **JEDE SCHREIBOPERATION EIN UPSERT.**“

INSERT	UPDATE
INSERT INTO person (id, name) VALUES (1, 'Anna');	UPDATE person SET alter = 25 WHERE id = 1;
Existiert id = 1 → Werte werden überschrieben	Existiert die Zeile → Feld wird geändert
Existiert id = 1 NICHT → neue Zeile wird angelegt	Existiert Zeile NICHT → Zeile wird angelegt

Was passiert bei einem UPSERT? ★★

„Cassandra ist verteilt, hochskalierbar, **OHNE zentrale Sperren** und **OHNE globale Transaktionen.**“

- Cassandra **speichert JEDE Änderung als NEUEN SCHREIBVORGANG**
- ★ **Es wird NICHT GEPRÜFT, ob der Datensatz bereits existiert**
- Existiert der Schlüssel bereits, **wird der alte Wert ÜBERSCHRIEBEN**
- Existiert er nicht, **wird ein neuer Datensatz ANGELEGT**
- ★ **Der ZULETZT GESCHRIEBENE Wert ist gültig (Last Write Wins)**

Vorteil	Nachteil
hoher Schreibdurchsatz	⚠ KEIN Schutz vor „versehentlichem Überschreiben“
KEIN Locking	⚠ KEIN „INSERT ONLY“
einfache Semantik und hohe Verfügbarkeit	⚠ KEIN Fehler bei doppeltem Schlüssel

75.9 UUID ★★

*„**UUID** steht für **Universally Unique Identifier**. In Cassandra (und anderen verteilten Datenbanken) ist das der **Standard für IDs, ANSTATT der klassischen Nummern (1, 2, 3...)**. Eine UUID ist eine **128-Bit-Zahl**.“*

★★ Die Begründung (Klausurfrage!): *„Cassandra ist ein **VERTEILTES System**. Wenn **Server A** und **Server B GLEICHZEITIG** einen neuen Benutzer anlegen, wüssten sie **NICHT**, wer die **„Nummer 4“** und wer die **„Nummer 5“** bekommt, **OHNE sich ständig ABZUSPRECHEN**. Eine UUID wird **ZUFÄLLIG** generiert. Es ist **mathematisch so unwahrscheinlich**, dass zwei Computer dieselbe UUID generieren, dass man es in der Praxis als **„unmöglich“** betrachtet.“*

 In CQL erzeugt `uuid()` automatisch eine eindeutige ID.

75.10 Konsistenz und Transaktionen in Cassandra ★★★

Relationale Datenbanken	Cassandra
ACID-Transaktionen	⚠ KEINE klassischen Transaktionen Eventual Consistency (AP aus CAP-Theorem) ★ Konsistenz PRO ABFRAGE EINSTELLBAR

Die drei Consistency Level ★★★:

Level	Bedeutung
CONSISTENCY QUORUM	Die MEHRHEIT der Replikate muss zustimmen
CONSISTENCY ALL	ALLE Replikate müssen bestätigen
CONSISTENCY ONE	Es reicht, wenn EIN EINZIGER Node (Server) bestätigt

```
CONSISTENCY ALL;
SELECT * FROM gehalts_abrechnung WHERE monat = '2025-12';
```

Replikation, Konsistenz und CAP ★★★:

- **Replikation:** *„Jede Zeile wird gemäß Keyspace-Definition auf **k Nodes** kopiert (Replikationsfaktor, z. B. RF = 3). **ALLE Kopien sind gleichwertig; KEIN Master-Slave.**“*
- **Konsistenz-Level (CL):** *„**PRO ABFRAGE einstellbar**. Dadurch kann man **Availability vs. Consistency ABSTIMMEN**. Standard: **Eventual Consistency.**“*
- ★★ **CAP-Einordnung:** *„Cassandra wird typischerweise als **AP-SYSTEM** eingeordnet (Availability + Partition Tolerance). **STARKE KONSISTENZ (CP) IST OPTIONAL ERREICHBAR** über **HÖHERE CL** (z. B. **ALL** für Writes UND Reads).“*
- **Ausfallsicherheit:** „Durch **Mehrfachreplikation toleriert Cassandra Ausfälle einzelner Nodes ODER GANZER DATACENTERS**; der Cluster bleibt verfügbar (**kein Single Point of Failure**)“

75.11 ★★★★★ Cassandra vs. RDBMS — die Vergleichstabelle

Kriterium	Relationale DB	Cassandra
Abfragesprache	SQL	CQL
Syntax	relational	SQL-ähnlich
JOINS	Ja	⚠ Nein
Transaktionen	ACID	⚠ Nein
Schema	starr	flexibel
Abfragefreiheit	hoch	⚠ stark eingeschränkt
Skalierung	vertikal	★ horizontal
Konsistenz	stark	Eventual / einstellbar

75.12 Stärken und Schwächen ★★

Stärken	Schwächen
Hochverfügbarkeit & Fehlertoleranz: Peer-to-Peer-Architektur, kein Single Point of Failure ; automatische Replikation über Nodes/Datacenters	⚠ Konsistenz/Transaktionen: Im Standard Eventual Consistency ; KEINE Multi-Row-Transaktionen , geringere ACID-Garantien
Lineare Skalierbarkeit: horizontales Skalieren durch Hinzufügen von Nodes , quasi linear mehr Kapazität; kein Downtime beim Wachsen	⚠ Eingeschränkte Abfrageflexibilität: Abfragen müssen Primärschlüssel nutzen; Ad-hoc-Queries , Aggregationen und Joins wie in SQL sind NICHT vorgesehen
Hohe Schreibperformance: optimiert für viele parallele Schreib-Operationen	
Dynamische Schema-Erweiterung: Schemaänderungen online möglich	

76. Graphdatenbanken ★★★★★

76.1 Der Paradigmenwechsel: Beziehungen ★★★★★

Relationale Datenbanken	Graphdatenbanken
Beziehungen über STARRE FREMDSCHLÜSSEL-CONSTRAINTS	★ „Graphdatenbanken behandeln Beziehungen als ‚FIRST-CLASS CITIZENS‘“
⚠ Kostspielige JOIN-Operationen bei tief verschachtelten Abfragen (z. B. „Freunde von Freunden“)	★ „Beziehungen werden PHYSISCH als DIREKTE POINTER gespeichert, NICHT erst zur Laufzeit BERECHNET “
Starres Schema erschwert das Hinzufügen neuer Beziehungstypen	dynamisch erweiterbar

76.2 Das Property-Graph-Modell ★★★★★

Element	Beschreibung
Knoten (Nodes)	repräsentieren Objekte oder Entitäten
Kanten (Relationships)	Verbindungen mit TYP , RICHTUNG und EIGENSCHAFTEN
Eigenschaften (Properties)	zusätzliche Datenfelder (Key-Value) an Objekten UND Links
Labels	Markierungen zur SEMANTISCHEN GRUPPIERUNG

76.3 ★★★★★ Schema-Paradigmen: explizit vs. implizit

- ★ „Graphdatenbanken **BESITZEN** ein **STRUKTURIERENDES SCHEMA**.“
- * „**Unterschied zu Key-Value-, Wide-Column-Store- und Dokument-Datenbanken:** Diese **VERZICHTEN** zugunsten von einfacher **FRAGMENTIERUNG (Sharding)** auf **Datenbankschemas** und auf **REFERENZIELLE INTEGRITÄT**.“*
- ★★ * „**Im Gegensatz zu RDBMS handelt es sich hier um ein IMPLIZITES SCHEMA.** Neue Knoten- oder Kantentypen können ****DYNAMISCH ZUR LAUFZEIT** hinzugefügt werden, **OHNE** die bestehende Datenbankstruktur **VORAB** per DDL ändern zu müssen. **RDBMS haben ein EXPLIZITES SCHEMA (Definition VOR Dateneingabe)**.“*

76.4 Algorithmik und Analysepotenzial ★★

„Graphdatenbanken implementieren Prinzipien der **mathematischen GRAPHENTHEORIE** und bieten optimierte Algorithmen zur Berechnung **topologischer Eigenschaften**:“

Algorithmus	Beschreibung
Konnektivität (Zusammenhang)	Prüfung auf PFADEXISTENZ zwischen beliebigen Knotenmengen
Shortest Path	Effiziente Ermittlung des KOSTENGÜNSTIGSTEN Weges (z. B. DIJKSTRA- oder A*-Algorithmus)
Nachbarschaftsanalysen	Identifikation des „ Next Neighbors “ unter Berücksichtigung von Kantengewichten (wichtig für Routing und Empfehlungssysteme)
Strukturelles Matching	Lösung von Zuordnungsproblemen (Matching) durch Identifikation disjunkter Kantenmengen

76.5 Integrität und Konsistenz ★

„Graphdatenbanken ermöglichen eine **EXPLIZITE Prüfung von Konsistenzregeln**:“

- **Strukturelle Validierung:** „Die Integrität bezieht sich auf die **Einhaltung von Knoten- und Kantentypen, Attribut-Wertebereichen** sowie die **referenzielle Integrität der Beziehungen**“
- **Graph-Transformationen:** „Datenmanipulationen werden **NICHT als bloße Feldänderungen, sondern als OPERATIONEN AUF DER GRAPHSTRUKTUR** ausgedrückt (z. B. **Pfadsuche, Nachbarschaftsanalysen, Teilgraph-Extraktionen**)“

76.6 Bekannte Graphdatenbanken ★

System	Charakteristik
Neo4j	Technologieführer für NATIVE Graph-Speicherung und -Abfrage
OrientDB	flexibler Multi-Model-Ansatz (Kombination aus Dokumenten und Graphen)
Amazon Neptune	hochverfügbarer Cloud-Service für Graph-Workloads (unterstützt Gremlin und SPARQL)

77. Neo4j und Cypher ★★★★★

77.1 Entstehung und das Kernproblem ★★★★★

„Entwicklung **ab 2000** durch **Emil Eifrem (Neo4j Inc.)** als Reaktion auf die **Ineffizienz relationaler Systeme bei stark vernetzten Daten**.“ „**2007** wurde Neo4j erstmals **öffentlich vorgestellt**.“

Das Problem bei RDBMS ★★:

- * „**Beziehungen existieren NUR LOGISCH (Fremdschlüssel)**. Zur **ABFRAGEZEIT** müssen sie über **JOIN-Operationen BERECHNET** werden.“*
- „Join erfordert **Suchvorgänge mit LOGARITHMISCHER Komplexität**“
- ⚠️ „**JOIN BOMB**: Bei steigender Abfragetiefe (z. B. ‚Freunde von Freunden von...‘) wächst der Rechenaufwand **EXPONENTIELL**.“

77.2 ★★★★★ Index-free Adjacency — der Paradigmenwechsel

🔑 „Neo4j nutzt **INDEX-FREE ADJACENCY (INDEXFREIE NACHBARSCHAFT)**. Beziehungen werden **NICHT BERECHNET**, sondern **PHYSISCH PERSISTIERT (POINTER AM DATENSATZ)**. **TRAVERSIERUNG IN KONSTANTER ZEIT, UNABHÄNGIG VON DER ANZAHL DER DATENSÄTZE**.“

Traversierung: „Das **NAVIGIEREN entlang PERSISTIERTER KANTEN** von einem Knoten zum nächsten.“

★ Der Komplexitätsvergleich:

	RDBMS (JOIN)	Neo4j (Traversierung)
Beziehungen	zur Laufzeit berechnet	physisch als Pointer gespeichert
Komplexität	logarithmisch pro Join, bei Tiefe <i>d</i> exponentiell	★ KONSTANT (O(1)) pro Schritt
Abhängig von	Gesamtdatenmenge	★ NUR von der Anzahl der Nachbarn

77.3 Neo4j — Eigenschaften und Einsatz ★★

Eigenschaft	
Datenmodell	Property-Graph-Modell (Knoten, Kanten, Eigenschaften)
Charakter	★ „eine TRANSAKTIONALE Graph-Datenbank mit NATIVEM Graphspeicher “
Lizenz	duale Lizenz: Community Edition (Open Source) · Enterprise Edition (kommerziell)
Verbreitung	★ „die weltweit am weitesten verbreitete Graphdatenbank“

Einsatzgebiete ★: **Social Networks · Recommendation Engines · Fraud Detection · Knowledge Graphs · Netzwerk- und IT-Topologien · Supply-Chain-Analysen**

Die sechs Eigenschaften:

1. **Effiziente Verarbeitung hochgradig vernetzter Daten**
2. **Konstante Traversierungsperformance**
3. **Intuitive Modellierung realer Domänen als Graph**
4. **Cypher als deklarative Graphabfragesprache**
5. **Flexible Schemaoptionen (Constraints möglich)**
6. ★★ **ACID-Transaktionen**

77.4 Cypher ★★★★★

*„Von Neo4j entwickelt, um Graphen hocheffizient abzufragen. **Syntaktische Anlehnung an SQL (Struktur) und SPARQL (Pattern Matching)**. Seit 2015 durch das openCypher-Projekt als **OFFENER INDUSTRIESTANDARD** etabliert.“*

Eigenschaft	Beschreibung
Deklarativ	Fokus auf das WAS (Ergebnis-Muster) statt auf das WIE (technischer Suchweg) — genau wie SQL
★ ASCII-Art-Syntax	Intuitive Darstellung von KNOTEN in RUNDEN Klammern () und KANTEN in PFEILEN -->
Pattern Matching	Suche nach STRUKTURELLEN MUSTERN innerhalb des Netzwerks
Schema-Flexibilität	dynamisches Hinzufügen von Attributen OHNE vorherige Tabellendefinition

Die drei Grundelemente des Abfrageteils ★★★★★:

Element	Funktion
MATCH	Identifikation von Knoten und Kanten sowie Deklaration von SUCHMUSTERN
WHERE	Bedingungen zur FILTERUNG von Ergebnissen
RETURN	Bereitstellung des Resultats, bei Bedarf AGGREGIERT

Das Beispiel der Folie:

„Möchte man das **Erscheinungsjahr des Films ‚The Matrix‘** berechnen, so lautet die Anfrage in Cypher:“

```
MATCH (m: FILM {Titel: "The Matrix"})
RETURN m.Jahr
```

77.5 Cypher-Befehle im Überblick ★★★★★

```
-- KNOTEN erzeugen
CREATE (p:Person {name: "Anna", alter: 30, stadt: "Berlin"});
--      ▲      ▲
--      Label   Properties (Key-Value-Paare)

-- BEZIEHUNGEN erzeugen
MATCH (p:Person {name: "Anna"}), (f:Firma {name: "ACME"})
CREATE (p)-[:ARBEITET_BEI {seit: 2020}]->(f);
--      ▲              ▲              ▲
--      Kantentyp      Kanten-Attribute  Richtung

-- SUCHEN & AUSGEBEN
MATCH (p:Person)-[:ARBEITET_BEI]->(f:Firma)
RETURN p.name, f.name;

-- FILTERN
MATCH (p:Person)
WHERE p.alter > 25 AND p.stadt = 'Berlin'
RETURN p;

-- LÖSCHEN
MATCH (p:Person {name: "Anna"}) DELETE p;           -- nur den Knoten
MATCH (p:Person {name: "Anna"}) DETACH DELETE p;     -- Knoten + alle Beziehungen
```

Die Erläuterungen der Folien ★:

- **Knoten:** `Person` dient der **INDIZIERUNG** und **GRUPPIERUNG** · Attribute als Key-Value · **Schema-Flexibilität:** „Knoten desselben Labels können **unterschiedliche Properties** besitzen“
- **Beziehungen:** **Persistierung** (physisch gespeichert, **index-freie Adjazenz**) · **Kanten-Attribute** (auch Beziehungen können Properties tragen) · ★ **Richtung:** „Jede Kante ist **GERICHTET**, kann aber **BIDIREKTIONAL TRAVERSIERT** werden“
- **ASCII-Art:** Knoten in runden Klammern `( )`, Kanten in eckigen `[ ]` innerhalb eines Pfeils

★★★★ **DELETE vs. DETACH DELETE** — sichere Klausurfrage:

	DELETE	DETACH DELETE
Wirkung	Löscht NUR den Knoten	Löscht Knoten INKL. ALLER ein- und ausgehenden Beziehungen
Verhalten	⚠ SCHLÄGT FEHL , wenn noch Beziehungen existieren	funktioniert immer

🔑 „Sicherheitsmechanismus: Neo4j **VERHINDERT** das Löschen von Knoten mit ‚**VERWAISTEN**‘ Kanten (**REFERENZIELLE INTEGRITÄT**).“

77.6 Integrität in Neo4j ★★★★★

„Integrität muss in Neo4j **EXPLIZIT** über **CONSTRAINTS** **ERZWUNGEN** werden.“

Constraint	Wirkung
UNIQUE Constraint	verhindert DUBLETTEN und optimiert
NOT NULL Constraint	erzwingt die Existenz von PFLICHTFELDERN

⚠ ⚠ **Die zentrale Aussage:**

„Neo4j stellt **KEINEN** systemseitig definierten **PRIMÄRSCHLÜSSEL** bereit. **BEST PRACTICE:** **UNIQUE-Constraints** fungieren in der Praxis als **PRIMÄRSCHLÜSSEL**. ★ **DIE INTERNE NEO4J-ID IST NICHT STABIL UND SOLLTE NICHT FACHLICH GENUTZT WERDEN.**“

77.7 ★★★★★ Neo4j und ACID — die Besonderheit

„Neo4j ist **EINE DER WENIGEN** **NoSQL-DATENBANKEN**, **DIE VOLLSTÄNDIG ACID-KONFORM SIND**. Andere NoSQL-Datenbanken nutzen meist das **BASE-Modell**. Diese priorisieren **VERFÜGBARKEIT** und **HORIZONTALE SKALIERBARKEIT** gegenüber einer **STRIKTEN KONSISTENZ** (**CAP-Theorem**).“

ACID-Kriterium	Umsetzung in Neo4j
Atomarität	Operationen werden als eine UNTEILBARE Einheit behandelt
Isolation	Transaktionen beeinflussen sich nicht; Zwischenzustände sind für andere Nutzer UNSICHTBAR
Konsistenz	„Neo4j stellt sicher, dass der Graph nach JEDER Transaktion in einem VALIDEN Zustand bleibt“
Dauerhaftigkeit	„Nach einem Commit sind alle Änderungen PERMANENT auf dem Datenträger gespeichert“

77.8 ★★★★★ Neo4j vs. Relationale Datenbanken

Kriterium	Relationales DBMS	Neo4j
Datenmodell	Tabellen, Zeilen, Spalten	Knoten, Kanten, Eigenschaften
Spezialisierung	Datenintegrität, ACID, Speicherplatz (durch Normalisierung)	Konnektivität, Traversierungsgeschwindigkeit, TIEFE Beziehungen
Behandlung von Beziehungen	Fremdschlüssel (★ berechnet via JOIN zur LAUFZEIT)	★ First-Class Citizens (physisch als POINTER gespeichert)
Abfragesprache	SQL (mengenorientiert)	Cypher (musterorientiert)
Skalierungsstrategie	vertikal (Scale-up), komplexes Sharding	horizontales „Scale-out“ von LESEVORGÄNGEN, Fabric (Sharding) — horizontale Partitionierung von Daten

77.9 Stärken und Schwächen ★★

Stärken	Schwächen
Schnelle Abfragen über TIEFE Beziehungen OHNE teure Joins	⚠ Horizontale Skalierung von SCHREIBZUGRIFFEN durch ACID-Garantien ERSCHWERT
Transaktionssicherheit und SOFORTIGE Konsistenz (ACID-konform)	⚠ Benötigt VIEL ARBEITSSPEICHER für die effiziente Graph-Verarbeitung
Dynamische Schema-Erweiterungen im LAUFENDEN Betrieb möglich	⚠ UNGEEIGNET für MASSEN-AGGREGATIONEN
Intuitive und lesbare Cypher-Syntax	⚠ Performance hängt STARK von KORREKTEM GRAPH-DESIGN ab

78. ★★★★★ Einsatzszenarien im Vergleich — DIE Übersichtstabelle

NoSQL-Typ	Datenmodell	Typische Einsatzszenarien	Beispiele
Key-Value Store	Key-Value	Session-Management, Caching, High-Speed-Lookups, Feature-Flags	Redis, DynamoDB
Dokumentendatenbank	JSON-ähnliche Dokumente	CMS, E-Commerce, Produktkataloge, flexible Datenstrukturen	MongoDB, Couchbase
Wide-Column Store	Spaltenfamilien	Time-Series, Logging, IoT-Daten, Analytics	Cassandra, HBase
Graphdatenbank	Knoten + Kanten	Social Graphs, Recommendation Engines, Fraud Detection	Neo4j, Amazon Neptune

★ Diese Tabelle ist eine sichere Klausurfrage. Auswendig lernen!

79. Polystores und Multi-Model-Datenbanken ★★

79.1 Polystores ★★★★★

„POLYSTORES sind Datenbanksysteme oder Architekturen, die HETEROGENE SPEICHER-ENGINES (z. B. Relational, Graph, Array, Dokument) UNTER EINER GEMEINSAMEN ABFRAGESCHNITTSTELLE VEREINEN.“

Aspekt	Beschreibung
Ansatz	★ „ONE SIZE DOES NOT FIT ALL“
Prinzip	„Die Daten VERBLEIBEN in ihrer NATIVEN, SPEZIALISIERTEN Datenbank (dem ‚Store‘)“
Ziel	„Nutzung der jeweils EFFIZIENTESTEN Engine für das spezifische Datenformat bei GLEICHZEITIGER INTEGRATION“
Kernidee	★ „Polystores INTEGRIEREN spezialisierte DATENSILOS, STATT sie in EINE EINZIGE Datenbank zu ZWINGEN („BEST OF BREED“)

Das Beispiel-Szenario (E-Commerce) ★★:

Datenart	Datenbanktyp	Begründung
Kundendaten & Bestellungen	Relationale DB	wegen ACID-Transaktionen
Produktkatalog	Dokumenten-DB	wegen variabler Attribute/JSON
Empfehlungslogik („Kunden kauften auch“)	Graph-DB	wegen Traversierung
Log-Dateien/Klickpfade	Key-Value DB	Geschwindigkeit

★ * „Der Polystore erlaubt EINE EINZIGE SQL-Abfrage über ALLE diese Töpfe hinweg.“*

Die drei Herausforderungen ⚠:

1. **Komplexität: Wartung mehrerer Systeme**
2. **Performance-Engpass:** * „Der Datentransfer ZWISCHEN den Systemen erzeugt hohe Latenz und Netzwerklast (Overhead)“*
3. **Transaktionen:** „Globale Konsistenz (ACID) über Systemgrenzen hinweg ist oft NICHT MÖGLICH oder SEHR LANGSAM“

79.2 ★★★★★ Polystore vs. Multi-Model-Datenbank

Kriterium	Polystore	Multi-Model-Datenbank
Architektur	Verbund MEHRERER UNTERSCHIEDLICHER DB-Engines	★ EINE Engine unterstützt MEHRERE Modelle
Speicherort	Daten liegen PHYSISCH GETRENNT in SpeziaSystemen	Daten liegen in EINEM MONOLITHISCHEN System
Beispiele	BigDAWG, Presto/Trino (als Query Engine)	NoSQL: Azure CosmosDB, ArangoDB · Relational: PostgreSQL, Oracle

80. Self-Tuning-Datenbanken ★★

„SELF-TUNING-DATENBANKEN sind Systeme, die ihre KONFIGURATION und PHYSISCHE STRUKTUR AUTONOM ZUR LAUFZEIT ANPASSEN, um die Performance zu maximieren.“

Aspekt	Beschreibung
Ziel	Optimale Ressourcennutzung OHNE menschlichen DBA (Database Administrator)
Kernprinzip	★ „Ein KONTINUIERLICHER REGELKREIS (FEEDBACK LOOP) aus ÜBERWACHUNG und ANPASSUNG“
Historisch	„Früher STATISCHE REGELWERKE („Wenn Speicher voll, dann...“)"
Heute (modern)	★ „Einsatz von MACHINE LEARNING (ML) und KI . Das System „LERNT“ das Lastverhalten (Workload) und PROGNOSTIZIERT Lastspitzen “
Vision	★ „Die „SELF-DRIVING DATABASE“ (z. B. Oracle Autonomous DB, Peloton)“

Die Techniken ★:

Physical Design Tuning	Parameter-Tuning
Automatische Erstellung/Löschung von INDIZES	Anpassung von PUFFERGRÖSSEN (Buffer Pool)
Partitionierung von Tabellen (Sharding)	Steuerung des PARALLELITÄTSGRADES (Threads)
Materialized Views vorschlagen/anlegen	Checkpointing-Intervalle

🔑 * „Self-Tuning ERSETZT die manuelle Feinjustierung durch INTELLIGENTE REGELKREISE, die sich DYNAMISCH an den Workload anpassen.“ *

81. Historische nicht-relationale Datenbanksysteme ★★

„Vor der Beschreibung des Relationenmodells durch Ted Codd gab es nicht-relationale Datenbanken u. a. in der Form von **HIERARCHISCHEN oder NETZWERKARTIGEN Datenbanken**.“

Der gemeinsame Grundansatz:

- Daten sind in KNOTEN (RECORDS) gespeichert und stehen zueinander in BEZIEHUNG (1:n)
- ★ „Der Zugriff ist NAVIGIEREND (PROGRAMMIERT)“

„Auch nachdem sich relationale Datenbanken breit durchgesetzt haben, wurden nicht-relationale Ansätze weiterhin für **technische oder wissenschaftliche Anwendungen** genutzt (z. B. **CAD**).“

81.1 Hierarchische Datenbanken (z. B. IMS) ★★

- ★ **BAUMORIENTIERTES Datenmodell** (also eingeschränkt auf 1:n-Beziehungen)
- ★ „Ein ‚CHILD-Record‘ kann **NUR EXISTIEREN**, wenn der zugehörige ‚PARENT‘ EXISTIERT“
- „Die Verbindung zwischen Parent und Child wird **EXPLIZIT** hergestellt (also **NICHT** über Werte)“
- ⚠ „‚Kryptische‘ DB-Sprache. **NAVIGATION DURCH PROGRAMMIERUNG**“

81.2 Netzwerk-Datenbanken ★★

- **Strukturkomponenten:** RECORD (besteht aus **Feldern und Wiederholungsgruppen**) und SET (**OWNER-MEMBER-Beziehung** zwischen „Vater“ und „Sohn“)
- ★ „Das Netzwerkmodell erlaubt **n:m-BEZIEHUNGEN**.“ „Durch direkte Verbindungen der Knoten untereinander entsteht ein **NETZ**. Jeder Record-Typ kann **Owner ODER Member in beliebig vielen Set-Typen sein**, d. h. **ein Record kann MEHRERE VORGÄNGER haben**“
- „Es existieren meist **UNTERSCHIEDLICHE SUCHWEGE**, um zu einem bestimmten Datensatz zu kommen“
- ★ „Normiert durch **CODASYL (Conference on Data Systems Languages)**.“ Spezifiziert sind u. a. **DDL und DML**

★★ Der Vergleich (sichere Klausurfrage):

	Hierarchisch (IMS)	Netzwerk (CODASYL)	Relational
Struktur	Baum	Netz	Tabellen
Beziehungen	nur 1:n	★ auch n:m	alle, über Fremdschlüssel
Vorgänger	genau einer	★ mehrere möglich	—
Zugriff	navigierend (programmiert)	navigierend (programmiert)	★ deklarativ (SQL)

82. Ranking der Datenbanktechnologien ★

Die Folien verweisen auf <https://db-engines.com/de/ranking> — sowohl für das **aktuelle Ranking** als auch für den **Zeitverlauf**.

★ **Was man wissen sollte:** Die Top-Plätze belegen seit Jahren **Oracle, MySQL, Microsoft SQL Server und PostgreSQL** (alle relational). **MongoDB** ist die stärkste NoSQL-Datenbank (meist Platz 5), gefolgt von **Redis, Elasticsearch** und **Cassandra**. **Neo4j** führt das Graph-Segment an. Der langfristige Trend zeigt **PostgreSQL im Aufwind** und **relationale Systeme weiterhin dominant** — NoSQL hat sie **ergänzt, nicht ersetzt**.

TEIL K — DIE CHINOOK-DATENBANK UND ALLE ÜBUNGSAUFGABEN

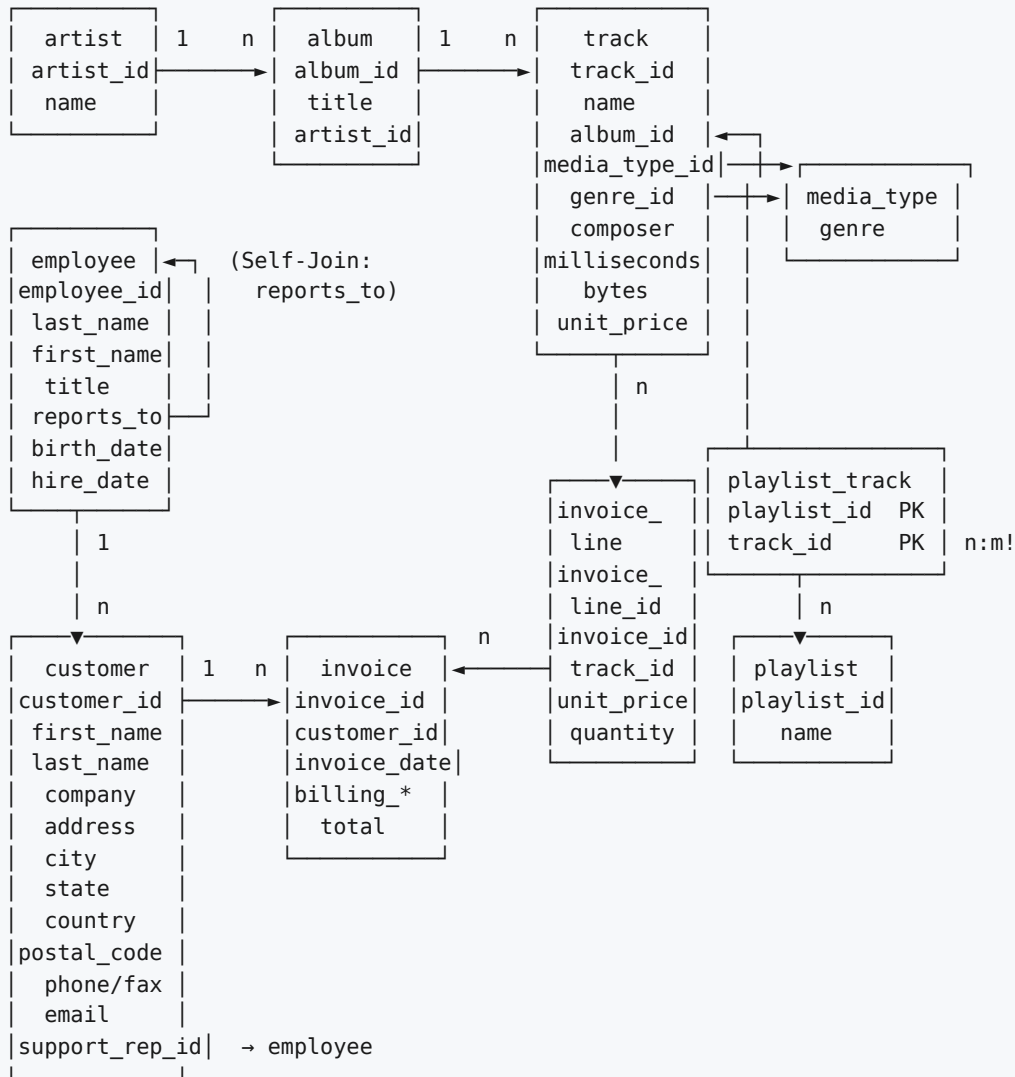
83. Die Chinook-Datenbank ★★★

83.1 Was ist Chinook?

„Die **Chinook-Datenbank** ist eine **Beispiel-Datenbank**, die speziell für das **Erlernen und Üben von SQL-Abfragen** entwickelt wurde. Sie wurde als **moderne Alternative zur klassischen NORTHWIND-Datenbank** geschaffen, mit Fokus auf **digitale Medien**. Das Szenario simuliert ein Unternehmen, das **digitale Musikdateien verkauft** — **ähnlich wie der ehemalige iTunes Store von Apple**.“

„Chinook eignet sich hervorragend, um praxisnahe Abfragen zu formulieren, da sie viele gängige Strukturen enthält: **1:n- und n:m-Beziehungen, HIERARCHIEN, Zeitstempel, Preisangaben** und mehr.“

83.2 ★★★★★ Das Relationenschema (11 Tabellen)



83.3 ★★★★★ Die Tabellen im Detail (aus dem Skript)

```

CREATE TABLE artist (
    artist_id INT NOT NULL,
    name      VARCHAR(120),
    CONSTRAINT artist_pkey PRIMARY KEY (artist_id)
);

CREATE TABLE album (
    album_id  INT NOT NULL,
    title     VARCHAR(160) NOT NULL,
    artist_id INT NOT NULL,
    CONSTRAINT album_pkey PRIMARY KEY (album_id)
);

CREATE TABLE genre (
    genre_id INT NOT NULL,
    name     VARCHAR(120),
    CONSTRAINT genre_pkey PRIMARY KEY (genre_id)
);

CREATE TABLE media_type (
    media_type_id INT NOT NULL,
    name          VARCHAR(120),
    CONSTRAINT media_type_pkey PRIMARY KEY (media_type_id)
);

CREATE TABLE track (
    track_id      INT NOT NULL,
    name          VARCHAR(200) NOT NULL,
    album_id      INT,                -- ⚠️ NULLABLE!
    media_type_id INT NOT NULL,
    genre_id      INT,                -- ⚠️ NULLABLE!
    composer      VARCHAR(220),       -- ⚠️ NULLABLE!
    milliseconds  INT NOT NULL,
    bytes         INT,
    unit_price    NUMERIC(10,2) NOT NULL,
    CONSTRAINT track_pkey PRIMARY KEY (track_id)
);

CREATE TABLE employee (
    employee_id INT NOT NULL,
    last_name   VARCHAR(20) NOT NULL,
    first_name  VARCHAR(20) NOT NULL,
    title       VARCHAR(30),
    reports_to  INT,                  -- ★ SELF-JOIN auf employee_id!
    birth_date  TIMESTAMP,
    hire_date   TIMESTAMP,
    address, city, state, country, postal_code, phone, fax, email,
    CONSTRAINT employee_pkey PRIMARY KEY (employee_id)
);

CREATE TABLE customer (
    customer_id  INT NOT NULL,
    first_name   VARCHAR(40) NOT NULL,
    last_name    VARCHAR(20) NOT NULL,
    company      VARCHAR(80),
    address, city, state, country, postal_code, phone, fax,
    email        VARCHAR(60) NOT NULL,
    support_rep_id INT,              -- ★ FK auf employee!
    CONSTRAINT customer_pkey PRIMARY KEY (customer_id)
);

```

```

CREATE TABLE invoice (
    invoice_id      INT NOT NULL,
    customer_id     INT NOT NULL,
    invoice_date    TIMESTAMP NOT NULL,
    billing_address, billing_city, billing_state, billing_country,
    billing_postal_code,
    total           NUMERIC(10,2) NOT NULL,
    CONSTRAINT invoice_pkey PRIMARY KEY (invoice_id)
);

CREATE TABLE invoice_line (
    invoice_line_id INT NOT NULL,
    invoice_id      INT NOT NULL,
    track_id        INT NOT NULL,
    unit_price      NUMERIC(10,2) NOT NULL,
    quantity        INT NOT NULL,
    CONSTRAINT invoice_line_pkey PRIMARY KEY (invoice_line_id)
);

CREATE TABLE playlist (
    playlist_id INT NOT NULL,
    name        VARCHAR(120),
    CONSTRAINT playlist_pkey PRIMARY KEY (playlist_id)
);

CREATE TABLE playlist_track (
    playlist_id INT NOT NULL,
    track_id    INT NOT NULL,
    CONSTRAINT playlist_track_pkey PRIMARY KEY (playlist_id, track_id)    -- ★ n:m!
);

```

83.4 Die Geschäftslogik ★★

*„Im Mittelpunkt stehen **Kunden (customer)**, die über den Shop Musikstücke kaufen. Diese Käufe werden durch **Rechnungen (invoice)** dokumentiert, wobei jede Rechnung aus einer oder mehreren **Rechnungspositionen (invoice_line)** besteht. Eine Rechnungsposition verweist auf einen konkreten **Track**.*

*Jedes Musikstück ist Teil eines **Albums**, das wiederum einem bestimmten **Künstler (artist)** zugeordnet ist. Zusätzlich besitzt jeder Track Angaben zum **Genre** sowie zum Dateiformat bzw. Medium (**media_type**).*

*Das Unternehmen beschäftigt zudem **Mitarbeiter (employee)**. ★ **Zwischen den Mitarbeitern besteht eine HIERARCHISCHE Beziehung, da jeder Mitarbeiter einem Vorgesetzten zugeordnet sein kann (SELF-JOIN)**. Kunden sind **einzelnen Mitarbeitern als Betreuer zugewiesen** (support_rep_id).*

*Ein weiteres interessantes Detail ist die **geographische Dimension**: Kunden besitzen Informationen zu Adresse, Stadt und Land → **regionale Auswertungen**. Gleichzeitig sind Rechnungen mit **Zeitstempeln** versehen → **Analysen über Zeiträume** (Trends, saisonale Schwankungen).“*

★★ Die vier wichtigsten Join-Ketten, die man auswendig können sollte:

Ziel	Join-Kette
Track → Künstler	track → album (album_id) → artist (artist_id)
Kunde → gekaufte Tracks	customer → invoice (customer_id) → invoice_line (invoice_id) → track (track_id)
Kunde → Betreuer	customer.support_rep_id → employee.employee_id
Mitarbeiter → Vorgesetzter	employee.reports_to → employee.employee_id ★ (Self-Join!)

84. Übungsblatt DDL 1: Tabellen und Schemata ★★

#	Aufgabe	Lösung
1	Schema <code>uebung</code> erstellen	<code>CREATE SCHEMA uebung;</code>
2	Tabelle <code>kunde_test</code> mit <code>kunde_id</code> (INTEGER, PK), <code>name</code> , <code>telefonnummer</code> , <code>adresse</code>	<code>CREATE TABLE uebung.kunde_test (kunde_id INTEGER PRIMARY KEY, name VARCHAR(255), telefonnummer VARCHAR(50), adresse VARCHAR(255));</code>
3	<code>name</code> in <code>vorname</code> / <code>nachname</code> aufteilen (je VARCHAR(100))	<code>ALTER TABLE uebung.kunde_test ADD COLUMN vorname VARCHAR(100), ADD COLUMN nachname VARCHAR(100);</code> <code>ALTER TABLE uebung.kunde_test DROP COLUMN name;</code>
4	Spalte <code>geburtsdatum</code> (DATE) hinzufügen	<code>ALTER TABLE uebung.kunde_test ADD COLUMN geburtsdatum DATE;</code>
5	Spalte <code>adresse</code> löschen	<code>ALTER TABLE uebung.kunde_test DROP COLUMN adresse;</code>
6	Tabelle <code>lieferant</code> erstellen	<code>CREATE TABLE uebung.lieferant (lieferant_id INTEGER PRIMARY KEY, firmenname VARCHAR(255), kontakt_email VARCHAR(255));</code>
7	Datentyp <code>kontakt_email</code> in TEXT ändern	<code>ALTER TABLE uebung.lieferant ALTER COLUMN kontakt_email TYPE TEXT;</code>
8	<code>lieferant</code> in <code>partner</code> umbenennen	<code>ALTER TABLE uebung.lieferant RENAME TO partner;</code>
9	Tabelle <code>partner</code> löschen	<code>DROP TABLE uebung.partner;</code>
10	Schema <code>uebung</code> löschen	<code>DROP SCHEMA uebung;</code>

⚠ **Hinweis der Musterlösung:** „Sollten Sie die Tabelle `kunde_test` noch nicht gelöscht haben, werden Sie einen **Fehler bekommen**. Den Fehler vermeiden Sie, indem Sie diese Tabelle löschen, **bevor** Sie das Schema löschen.“ (Alternativ: `DROP SCHEMA uebung CASCADE;`)

85. Übungsblatt DDL/DML 2: Integritätsbedingungen ★★ ★

```
-- 1. Tabelle mitarbeiter erstellen
CREATE TABLE mitarbeiter (
    email          VARCHAR(100) NOT NULL UNIQUE,
    vorname        VARCHAR(100),
    nachname       VARCHAR(100),
    geburtsdatum   DATE,
    Lieblingszahl   INTEGER,
    lieblingstier   VARCHAR(100)
);

-- 2. Spalte hinzufügen
ALTER TABLE mitarbeiter ADD COLUMN Lieblingsfarbe VARCHAR(50);

-- 3. CHECK-Bedingung
ALTER TABLE mitarbeiter
    ADD CONSTRAINT chk_Lieblingszahl CHECK (Lieblingszahl <= 100);

-- 4. Zusammengesetzter Primärschlüssel
ALTER TABLE mitarbeiter ADD PRIMARY KEY (vorname, nachname);

-- 5. Zweite Tabelle
CREATE TABLE standort (
    standort_id SERIAL PRIMARY KEY,
    stadt         VARCHAR(100)
);

-- 6. Fremdschlüsselbeziehung
ALTER TABLE mitarbeiter ADD COLUMN standort_id INTEGER;
ALTER TABLE mitarbeiter ADD CONSTRAINT fk_standort
    FOREIGN KEY (standort_id) REFERENCES standort(standort_id);

-- 7. ★ ON DELETE CASCADE – Constraint muss ERST GELÖSCHT und NEU ANGELEGT werden!
ALTER TABLE mitarbeiter DROP CONSTRAINT fk_standort;
ALTER TABLE mitarbeiter ADD CONSTRAINT fk_standort
    FOREIGN KEY (standort_id) REFERENCES standort(standort_id) ON DELETE CASCADE;

-- 8. Drei Mitarbeiter einfügen
INSERT INTO mitarbeiter (email, vorname, nachname, geburtsdatum,
    Lieblingszahl, lieblingstier, Lieblingsfarbe)
VALUES ('a@test.de', 'Anna', 'Müller', '1990-01-01', 7, 'Katze', 'Blau'),
    ('b@test.de', 'Ben', 'Schmidt', '1985-05-05', 13, 'Hund', 'Grün'),
    ('c@test.de', 'Clara', 'Schulz', '1992-07-20', 42, 'Vogel', 'Rot');

-- 9. Alle aktualisieren (⚠ ohne WHERE!)
UPDATE mitarbeiter SET Lieblingszahl = 3;

-- 10. Gezielt ändern
UPDATE mitarbeiter SET nachname = 'Mustermann' WHERE email = 'a@test.de';

-- 11. Löschen
DELETE FROM mitarbeiter WHERE email = 'b@test.de';
```

★★★ Die Constraint-Tests (Aufgaben 12–14) — sehr klausurnah:

#	Versuch	Was passiert?
1 2	Mitarbeiter mit leerer Email einfügen	⚠ Fehler: Spalte email ist NOT NULL . Ein Einfügen ohne Wert führt zu einem Fehler
1 3	Zwei Mitarbeiter mit identischer Email	⚠ Fehler: Spalte email ist UNIQUE . Zwei gleiche Werte verursachen einen Konflikt
1 4	Mitarbeiter mit lieblingszahl = 999	⚠ Fehler: CHECK-Constraint lieblingszahl <= 100 wird VERLETZT

```
-- 15. Spalte entfernen
ALTER TABLE mitarbeiter DROP COLUMN geburtsdatum;

-- 16. Spalte umbenennen
ALTER TABLE mitarbeiter RENAME COLUMN lieblingsfarbe TO augenfarbe;

-- 17. Tabelle projekt
CREATE TABLE projekt (
    projekt_id SERIAL PRIMARY KEY,
    projektname VARCHAR(255)
);

-- 18. FK mit ON DELETE SET NULL
ALTER TABLE mitarbeiter ADD COLUMN projekt_id INTEGER;
ALTER TABLE mitarbeiter ADD CONSTRAINT fk_projekt
    FOREIGN KEY (projekt_id) REFERENCES projekt(projekt_id) ON DELETE SET NULL;

-- 19. DEFAULT-Wert
ALTER TABLE mitarbeiter ALTER COLUMN lieblingstier SET DEFAULT 'Hund';

-- 20. ★ Löschen UNTER BERÜCKSICHTIGUNG der Fremdschlüsselabhängigkeiten
DROP TABLE mitarbeiter; -- ZUERST die abhängige Tabelle!
DROP TABLE projekt;
DROP TABLE standort;
```

★★ **Die Merkregel zur Löschreihenfolge: Immer zuerst die Tabelle löschen, die den FREMDSCHLÜSSEL enthält (die abhängige), dann die referenzierte Tabelle.**

86. Übungsblatt DQL 1: SELECT-Abfragen (20 Aufgaben) ★★★★★

#	Aufgabe	Lösung
1	Alle Künstler alphabetisch sortiert	<code>SELECT name FROM artist ORDER BY name;</code>
2	Einzigartige Musikgenres	<code>SELECT DISTINCT name FROM genre;</code>
3	Anzahl aller Alben	<code>SELECT COUNT(*) FROM album;</code>
4	Tracks länger als 5 Minuten	<code>SELECT name, milliseconds FROM track WHERE milliseconds > 300000;</code>
5	Tracks, deren Name mit "Love" beginnt	<code>SELECT name FROM track WHERE name LIKE 'Love%';</code>
6	Tracks, die "Baby" enthalten	<code>SELECT name FROM track WHERE name LIKE '%Baby%';</code>
7	Anzahl aller Tracks	<code>SELECT COUNT(*) FROM track;</code>
8	Anzahl unterschiedlicher Komponisten	<code>SELECT COUNT(DISTINCT composer) FROM track;</code>
9	Tracks ohne Komponisten	<code>SELECT name FROM track WHERE composer IS NULL;</code>
10	Kürzester und längster Track	<code>SELECT name FROM track ORDER BY milliseconds ASC LIMIT 1;</code> <code>SELECT name FROM track ORDER BY milliseconds DESC LIMIT 1;</code>
11	Tracks pro Genre, absteigend	<code>SELECT genre_id, COUNT(*) AS track_count FROM track GROUP BY genre_id ORDER BY track_count DESC;</code>
12	Tracks länger als der Durchschnitt	<code>SELECT name FROM track WHERE milliseconds > (SELECT AVG(milliseconds) FROM track);</code>
13	Dauer zwischen 3 und 5 Minuten	<code>SELECT name FROM track WHERE milliseconds BETWEEN 180000 AND 300000;</code>
14	Genre-ID 1 oder länger als 6 Minuten	<code>SELECT name FROM track WHERE genre_id = 1 OR milliseconds > 360000;</code>
15	Kunden aus USA oder Kanada	<code>SELECT first_name, last_name FROM customer WHERE country IN ('USA', 'Canada');</code>
16	Rechnungen über 10 USD	<code>SELECT invoice_id, total FROM invoice WHERE total > 10;</code>
17	Rechnungen pro Land	<code>SELECT billing_country, COUNT(*) AS invoice_count FROM invoice GROUP BY billing_country;</code>
18	Länder mit mehr als 5 Kunden	<code>SELECT country, COUNT(*) FROM customer GROUP BY country HAVING COUNT(*) > 5;</code>
19	Kunden, deren Nachname mit "S" beginnt	<code>SELECT first_name, last_name FROM customer WHERE last_name LIKE 'S%';</code>
20	Rechnungen ohne Bundesstaat	<code>SELECT invoice_id FROM invoice WHERE billing_state IS NULL;</code>

★ Die Millisekunden-Umrechnungen merken: 3 Min = **180.000 ms** · 5 Min = **300.000 ms** · 6 Min = **360.000 ms**

87. Übungsblatt DQL 2: Fortgeschrittene Abfragen (30 Aufgaben) ★★★★★



Das wichtigste Übungsblatt für die Klausur — alle 30 Lösungen:

```

-- 1. Tracks über dem Durchschnitt
SELECT * FROM track WHERE milliseconds > (SELECT AVG(milliseconds) FROM track);

-- 2. ★ Erweitert um die DIFFERENZ zur durchschnittlichen Tracklänge
SELECT *, Milliseconds - (SELECT AVG(Milliseconds) FROM track)
FROM track
WHERE Milliseconds > (SELECT AVG(Milliseconds) FROM track);

-- 3. ★★ Kunden mit mehr Rechnungen als der Durchschnitt (verschachtelte Aggregation!)
SELECT customer_id FROM invoice
GROUP BY customer_id
HAVING COUNT(*) > (SELECT AVG(c)
                   FROM (SELECT COUNT(*) AS c FROM invoice
                        GROUP BY customer_id) AS sub);

-- 4. Rechnung(en) mit dem höchsten Gesamtbetrag
SELECT * FROM invoice WHERE total = (SELECT MAX(total) FROM invoice);

-- 5. Tracks desselben Genres wie 'Headlong'
SELECT * FROM track
WHERE genre_id = (SELECT genre_id FROM track WHERE name = 'Headlong');

-- 6. Album + Künstlername
SELECT album.title, artist.name
FROM album JOIN artist ON album.artist_id = artist.artist_id;

-- 7. Kunden mit Land und Anzahl gekaufter Tracks (nur mit Käufen)
SELECT customer.first_name, customer.last_name, customer.country,
       COUNT(invoice_line.track_id) AS anzahl_käufe
FROM customer
JOIN invoice      ON customer.customer_id = invoice.customer_id
JOIN invoice_line ON invoice.invoice_id   = invoice_line.invoice_id
GROUP BY customer.customer_id, customer.first_name,
       customer.last_name, customer.country;

-- ⚠ Anmerkung der Musterlösung: "Aufgabe etwas unscharf formuliert.
--   Wenn ein Kauf eine Rechnung ist, dann ist kein Join zur invoice_line nötig."

-- 8. ★★ Kunden OHNE Rechnung – zwei Lösungswege!
SELECT customer.customer_id, customer.first_name, customer.last_name
FROM customer LEFT JOIN invoice ON customer.customer_id = invoice.customer_id
WHERE invoice.invoice_id IS NULL;
-- Alternative über Subquery:
SELECT * FROM customer AS Cus
WHERE Cus.customer_id NOT IN (SELECT Inv.customer_id FROM invoice Inv);

-- 9. Rechnungen inkl. Kundenname und Land
SELECT invoice.invoice_id, customer.first_name, customer.last_name, customer.country
FROM invoice JOIN customer ON invoice.customer_id = customer.customer_id;

-- 10. Tracks ohne Medienart
SELECT * FROM track WHERE media_type_id IS NULL;

-- 11. ★ Alben ohne Tracks
SELECT album.* FROM album
LEFT JOIN track ON album.album_id = track.album_id
WHERE track.album_id IS NULL;
-- Alternative:
SELECT * FROM album AS alb WHERE alb.album_id NOT IN (SELECT album_id FROM track);

-- 12. ★★ Tracks mit Genre, Albumtitel und Künstler (4-Tabellen-Join!)
SELECT track.name, genre.name AS genre, album.title AS album, artist.name AS künstler
FROM track

```

```

JOIN genre ON track.genre_id = genre.genre_id
JOIN album ON track.album_id = album.album_id
JOIN artist ON album.artist_id = artist.artist_id;

-- 13. ★★ Länder mit mind. 2 Kunden, die mehr als 3 Rechnungen haben
SELECT country FROM customer
WHERE customer_id IN (SELECT customer_id FROM invoice
                     GROUP BY customer_id HAVING COUNT(*) > 3)
GROUP BY country HAVING COUNT(*) >= 2;

-- 14. ★★★ Kürzeste Tracks je Genre (korrelierte Subquery mit ALL)
SELECT * FROM track t1
WHERE milliseconds < ALL (SELECT milliseconds FROM track t2
                        WHERE t1.genre_id = t2.genre_id
                        AND t1.track_id <> t2.track_id);

-- 15. Genres mit durchschnittlicher Tracklänge
SELECT genre.name, AVG(track.milliseconds) AS durchschnitt
FROM genre JOIN track ON genre.genre_id = track.genre_id
GROUP BY genre.name;

-- 16. Kunden mit Gesamtausgaben
SELECT customer.customer_id, SUM(invoice.total) AS gesamt
FROM customer JOIN invoice ON customer.customer_id = invoice.customer_id
GROUP BY customer.customer_id;

-- 17. Künstler mit mehr als einem Album
SELECT artist.name FROM artist
WHERE artist_id IN (SELECT artist_id FROM album
                  GROUP BY artist_id HAVING COUNT(*) > 1);

-- 18. ★ Berechnete Spalte: Preis pro Millisekunde
SELECT name, unit_price / milliseconds AS preis_pro_millis
FROM track WHERE milliseconds > 0
ORDER BY preis_pro_millis DESC;

-- 19. Kunden mit Käufen aus USA UND Kanada
SELECT customer.customer_id
FROM customer JOIN invoice ON customer.customer_id = invoice.customer_id
WHERE country IN ('USA', 'Canada')
GROUP BY customer.customer_id
HAVING COUNT(DISTINCT country) = 2;

-- 20. Rechnungen mit Kundenname, ohne Bundesstaat
SELECT invoice.invoice_id, customer.first_name, customer.last_name
FROM invoice JOIN customer ON invoice.customer_id = customer.customer_id
WHERE billing_state IS NULL;

-- 21. E-Mail beginnt mit 'lu'
SELECT * FROM customer WHERE email LIKE 'lu%';

-- 22. ★ Vorname GROSS, Nachname klein
SELECT UPPER(first_name) AS vorname_gross, LOWER(last_name) AS nachname_klein
FROM employee;

-- 23. ★ Albumtitel und Titellänge
SELECT title, LENGTH(title) AS titellänge FROM album;

-- 24. ★★★ Komponist enthält 'Bach' (case-insensitive!)
SELECT name, composer FROM track
WHERE composer IS NOT NULL AND LOWER(composer) LIKE '%bach%';

-- 25. ★ Rechnungsjahr extrahieren

```

```

SELECT invoice_id, EXTRACT(YEAR FROM invoice_date) AS rechnungsjahr FROM invoice;

-- 26. 5 neueste Rechnungen
SELECT * FROM invoice ORDER BY invoice_date DESC LIMIT 5;

-- 27. ★★ Dauer in Tagen bis heute – TYPUMWANDLUNG nötig!
SELECT invoice_id, (CURRENT_DATE - invoice_date::DATE) AS dauer_in_tagen FROM invoice;
-- ⚠ Typumwandlung wird benötigt, da invoice_date vom Typ TIMESTAMP ist
-- Alternative in Standard-SQL:
SELECT invoice_id, (CURRENT_DATE - CAST(invoice_date AS DATE)) AS dauer_in_tagen
FROM invoice;

-- 28. ★★ Die zwei ältesten Mitarbeiter mit Alter in Jahren
SELECT first_name, last_name,
       EXTRACT(YEAR FROM AGE(CURRENT_DATE, birth_date)) AS alter_in_jahren
FROM employee ORDER BY birth_date ASC LIMIT 2;

-- 29. ★ UNION: alle Städte aus customer und employee
SELECT city FROM customer UNION SELECT city FROM employee ORDER BY city;

-- 30. ★ EXCEPT: Mitarbeiter-Nachnamen, die keine Kunden sind
SELECT last_name FROM employee EXCEPT SELECT last_name FROM customer;

```

★★★ Die fünf Muster, die man aus diesem Übungsblatt mitnehmen muss:

Muster	Lösung
„... ohne ...“ / „... nie ...“	LEFT JOIN ... WHERE fk IS NULL oder NOT IN (Subquery)
„... mehr als der Durchschnitt ...“	Subquery mit AVG() — bei Gruppen: verschachtelte Subquery mit Alias (Aufg. 3)
„... je Gruppe der/die ...“	korrelierte Subquery mit Bezug auf die Gruppenspalte (Aufg. 14)
„... sowohl ... als auch ...“	GROUP BY ... HAVING COUNT(DISTINCT spalte) = n (Aufg. 19)
Datumsrechnung mit TIMESTAMP	⚠ ::DATE bzw. CAST(... AS DATE) nicht vergessen (Aufg. 27)

88. Übungsblatt Zusatzthemen: VIEWS, Systemkatalog, Rechte



```
-- — VIEWS —————
-- 1. SELECT mit Rechnungen + Kundendaten
SELECT i.*, c.first_name, c.last_name, c.country
FROM   invoice i JOIN customer c ON i.customer_id = c.customer_id;

-- 2. Daraus eine View machen
CREATE VIEW rechnung_mit_kunde AS
SELECT i.*, c.first_name, c.last_name, c.country
FROM   invoice i JOIN customer c ON i.customer_id = c.customer_id;

-- 3. View rock_tracks
CREATE VIEW rock_tracks AS
SELECT t.name, t.composer, a.title AS album_title
FROM   track t
JOIN   genre g ON t.genre_id = g.genre_id
JOIN   album a ON t.album_id = a.album_id
WHERE  g.name = 'Rock';

-- 4. ★ Die View wie eine Tabelle verwenden
SELECT * FROM rock_tracks WHERE name LIKE 'Love%';

-- 5. View top_kunden
CREATE VIEW top_kunden AS
SELECT DISTINCT c.*
FROM   customer c JOIN invoice i ON c.customer_id = i.customer_id
WHERE  i.total > 20;

-- 7. View löschen
DROP VIEW rock_tracks;
```

★★★★ Aufgabe 6 — die wichtigste Verständnisfrage: Kann man über `top_kunden` einfügen?

NEIN, ein INSERT über die View `top_kunden` ist NICHT erlaubt, weil:

- Sie basiert auf einem JOIN
- Es existiert eine WHERE-Klausel
- DISTINCT ist im SELECT enthalten

→ Diese Kombination verhindert, dass PostgreSQL die Änderungen SAUBER auf die ORIGINALTABELLEN ABBILDEN kann.

```
-- — Systemkatalog & Indexe —————
-- 9. Alle Tabellen im Schema chinook
SELECT * FROM Information_Schema.Tables WHERE table_schema = 'chinook';

-- 10. Alle Spalten der Tabelle track
SELECT * FROM Information_Schema.Columns WHERE table_name = 'track';

-- 11. Referentielle Integritätsbedingungen im Schema chinook
SELECT * FROM Information_Schema.referential_constraints
WHERE constraint_schema = 'chinook';

-- 12. Index auf last_name
CREATE INDEX i_last_name ON chinook.customer (last_name);

-- 13. Index löschen
DROP INDEX chinook.i_last_name;

-- 14. ★ UNIQUE INDEX auf last_name
CREATE UNIQUE INDEX i_last_name ON chinook.customer (last_name);
```

★★ Aufgabe 14 — warum erzeugt das einen Fehler?

„Ein NACHNAME ist oftmals NICHT EINDEUTIG. Wenn dieser nicht eindeutig ist, dann KANN KEIN UNIQUE INDEX ANGELEGT WERDEN.“

```
-- — Rechte & Rollen —————
-- 15. Rolle erstellen
CREATE ROLE support_readonly;

-- 16. SELECT-Zugriff gewähren
GRANT SELECT ON customer TO support_readonly;

-- 17. Benutzer anlegen und Rolle zuordnen
CREATE USER tim WITH PASSWORD 'securePass123';
GRANT support_readonly TO tim;

-- 18. ★★ AUFRÄUMEN – die REIHENFOLGE ist entscheidend!
REVOKE SELECT ON customer FROM support_readonly; -- 1. Leserecht entziehen
DROP USER tim; -- 2. User löschen
DROP ROLE support_readonly; -- 3. Rolle löschen
```

★ **„Als ERSTER Schritt muss das LESERECHT auf die Tabelle customer ENTZOGEN werden.“**

89. Übungsblatt DB-interne Programmierung ★★

Aufgaben 1–4: LANGUAGE SQL

```
-- 1. Durchschnittliche Tracklänge eines Albums
CREATE OR REPLACE FUNCTION get_avg_track_length(id integer)
RETURNS int
LANGUAGE SQL
AS $$
    SELECT AVG(milliseconds) FROM track WHERE album_id = id;
$$;
SELECT get_avg_track_length(1);

-- 2. ★ Anzahl Tracks eines Künstlers (nach NAME) – 3-Tabellen-Join in der Funktion
CREATE OR REPLACE FUNCTION count_tracks_by_artistname(suchname TEXT)
RETURNS int
LANGUAGE SQL
AS $$
    SELECT COUNT(t.track_id)
    FROM   track AS t
    JOIN   album AS a ON t.album_id = a.album_id
    JOIN   artist AS r ON a.artist_id = r.artist_id
    WHERE  r.name = suchname;
$$;
SELECT count_tracks_by_artistname('Accept');

-- 3. Einstellungsdatum eines Mitarbeiters
CREATE OR REPLACE FUNCTION get_employee_hire_date(emp_id int)
RETURNS date
LANGUAGE SQL
AS $$
    SELECT hire_date FROM employee WHERE employee_id = emp_id;
$$;

-- 4. ★★ Rechnungszeilen eines Betreuers (4-Tabellen-Join!)
CREATE OR REPLACE FUNCTION count_invoicelines_by_employee(employee_id int)
RETURNS int
LANGUAGE SQL
AS $$
    SELECT COUNT(il.invoice_line_id)
    FROM   employee AS e
    JOIN   customer AS c ON e.employee_id = c.support_rep_id
    JOIN   invoice AS i ON c.customer_id = i.customer_id
    JOIN   invoice_line AS il ON i.invoice_id = il.invoice_id
    WHERE  e.employee_id = employee_id;
$$;
```

⚠️ Ein sehr wichtiger Hinweis der Musterlösung zu Aufgabe 2:

„Wenn der Übergabeparameter `suchname` mit dem **Typ** `varchar(120)` definiert wird, dann **funktioniert das Join-Kriterium NICHT und ALLE Datensätze werden selektiert**. ★ **Lösung: Nutzung von `TEXT` als Typ des Übergabeparameters**. Es ist unklar, warum sich PostgreSQL so verhält ...“

Aufgaben 5-7: LANGUAGE PL/pgSQL

```
-- 5. ★ Existenzprüfung mit IF EXISTS
CREATE OR REPLACE FUNCTION KundeRechnung(cust_id integer)
RETURNS VARCHAR(20)
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (SELECT * FROM invoice WHERE customer_id = cust_id) THEN
        RETURN 'hat Rechnung';
    ELSE
        RETURN 'hat keine Rechnung';
    END IF;
END;
$$;
SELECT customer_id, first_name, last_name, kunderechnung(customer_id) FROM customer;

-- 6. ★ E-Mail des Betreuers – mit lokaler Variable und SELECT ... INTO
CREATE OR REPLACE FUNCTION supportemail(cust_id integer)
RETURNS text
LANGUAGE plpgsql
AS $$
DECLARE
    support_email text;
BEGIN
    SELECT e.email INTO support_email
    FROM   customer AS c
    JOIN   employee AS e ON c.support_rep_id = e.employee_id
    WHERE  c.customer_id = cust_id;
    RETURN support_email;
END;
$$;

-- 7. ★★ Rabattberechnung mit IF/ELSE
CREATE OR REPLACE FUNCTION discount(invoice_id_param integer)
RETURNS numeric
LANGUAGE plpgsql
AS $$
DECLARE
    invoice_total    numeric;
    discount_amount numeric := 0.00;
BEGIN
    SELECT total INTO invoice_total FROM invoice WHERE invoice_id = invoice_id_param;
    IF invoice_total > 10.00 THEN
        discount_amount := invoice_total * 0.10;
    ELSE
        discount_amount := invoice_total * 0.05;
    END IF;
    RETURN discount_amount;
END;
$$;
SELECT invoice_id, total, discount(invoice_id) AS Rabatt FROM invoice;
```

Aufgaben 8–9: TRIGGER ★★☆☆

```
-- 8. ★★ Prüfung beim UPDATE von hire_date (mit OLD und NEW)
CREATE OR REPLACE FUNCTION check_hire_date()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.hire_date < OLD.hire_date THEN
        RAISE EXCEPTION 'Das Einstellungsdatum kann nicht ...';
    END IF;
    RETURN NEW;
END;
$$;

-- ★ Spaltenspezifischer Trigger!
CREATE TRIGGER check_date_forward
BEFORE UPDATE OF hire_date ON employee
FOR EACH ROW
EXECUTE FUNCTION check_hire_date();

-- Test:
UPDATE employee SET hire_date = '01.01.1900' WHERE employee_id = 1;

-- 9. ★★☆☆ Default-Wert aus einer Aggregation setzen
CREATE OR REPLACE FUNCTION default_media_type()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    most_used_media_id int;
BEGIN
    -- Nur ausführen, wenn der Wert nicht gesetzt wurde
    IF NEW.media_type_id IS NULL THEN
        -- Ermittelt die am HÄUFIGSTEN verwendete Media Type ID
        SELECT media_type_id INTO most_used_media_id
        FROM track
        GROUP BY media_type_id
        ORDER BY COUNT(*) DESC
        LIMIT 1;
        NEW.media_type_id := most_used_media_id;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER default_media_type_trigger
BEFORE INSERT ON track
FOR EACH ROW
EXECUTE FUNCTION default_media_type();
```

★★★ Aufgabe 9 ist die Blaupause für die Klausuraufgabe aus [Beispiele Klausuraufgaben.pdf](#) („Wenn der `name` NULL ist, soll er automatisch auf `'Untitled Genre'` gesetzt werden“):

```
-- ★ Die Klausur-Musterlösung
CREATE OR REPLACE FUNCTION default_genre_name()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.name IS NULL THEN
        NEW.name := 'Untitled Genre';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER default_genre_name_trigger
BEFORE INSERT ON genre
FOR EACH ROW
EXECUTE FUNCTION default_genre_name();
```

Das Muster in vier Schritten: ① Trigger-Funktion mit `RETURNS TRIGGER` · ② `IF NEW.spalte IS NULL THEN` · ③ `NEW.spalte := 'Wert';` · ④ `RETURN NEW;` — und dann der `BEFORE INSERT ... FOR EACH ROW`-Trigger.

90. Übungsblatt JDBC ★★★★★

Aufgabe 1 — Tabelle auslesen (der Grundbaustein)

```
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse = "jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url, props);

        // SQL-Statement vorbereiten
        Statement st = conn.createStatement();
        String stcommand = "select * from chinook.customer;";

        // SQL-Befehl ausführen
        ResultSet rs = st.executeQuery(stcommand);

        // Auswertung
        while (rs.next()) {
            Integer ID = rs.getInt("customer_id");
            String Name = rs.getString("last_name");
            System.out.print(" "); System.out.print(ID);
            System.out.print(" "); System.out.println(Name);
        }

        // Freigabe der Ressourcen
        conn.close(); st.close(); rs.close();
    }
}
```

Aufgabe 2 — INSERT und UPDATE

```
// a) Dateneingabe
String stcommand = "INSERT INTO genre(genre_id,name) VALUES (77,'Pop 70er');";
st.executeUpdate(stcommand);

// b) Datenänderung
String stcommand = "UPDATE genre SET name = 'Pop Siebziger' WHERE genre_id = 77;";
st.executeUpdate(stcommand);
```

Aufgabe 3 — Referentielle Integrität und Fehlerbehandlung ★★★★★

a) Datensatz in employee mit reports_to = 555 einfügen (employee_id = 77, last_name = 'Napf', first_name = 'Karl') → **⚠️ Schlägt fehl, weil reports_to ein Fremdschlüssel auf employee_id ist und Mitarbeiter 555 nicht existiert** (Verletzung der referentiellen Integrität)

b) try ... catch -Block einfügen

```
try {
    st.executeUpdate(stcommand);
} catch (SQLException e) {
    e.printStackTrace();
}
```

→ ★ **Das Programm läuft trotz Fehler weiter**

c) ★ **Erst Mitarbeiter 555 anlegen, DANN Mitarbeiter 77 → funktioniert**, weil die referenzierte Zeile jetzt existiert

Aufgabe 4 — SQL Injection demonstrieren und verhindern ★★★★★

```
// a) Verwundbare Variante
String lastname = "King";
String stcommand = "SELECT * FROM employee WHERE last_name = ";
ResultSet rs = st.executeQuery(stcommand + "'" + lastname + "'");

// b) ⚠️ Die Injection
String lastname = "'King' OR 1=1"; // Ausdruck, der IMMER wahr ist
// → gibt ALLE Vornamen der Tabelle employee aus!
// ★ Potenzielles Risiko, falls die Eingabe über eine UI erfolgen würde

// c) ✅ Die sichere Variante mit PreparedStatement
PreparedStatement pst =
    conn.prepareStatement("SELECT * FROM employee WHERE last_name = ?");
pst.setString(1, lastname);
ResultSet rs = pst.executeQuery();
```

Aufgabe 5 — UPDATE, INSERT, SELECT kombiniert

```
// a) Mitarbeiter 2 scheidet aus
st.executeUpdate("UPDATE employee SET last_name = 'inactive' WHERE employee_id = 2;");

// b) Neuer Mitarbeiter
st.executeUpdate("INSERT INTO employee (employee_id, last_name, first_name) "
    + "VALUES (192, 'Mayer', 'Patrick');");

// c) Alle Mitarbeiter ausgeben
ResultSet rs = st.executeQuery(
    "SELECT employee_id, last_name, first_name FROM employee;");
while (rs.next()) {
    System.out.println(rs.getInt("employee_id") + " "
        + rs.getString("last_name") + " "
        + rs.getString("first_name"));
}
```

91. Übungsblatt Redis und MongoDB ★★★★★

91.1 Redis-Aufgaben mit Lösungen

#	Aufgabe	Lösung
1	Namen "Max Mustermann" unter benutzer:name speichern	SET benutzer:name "Max Mustermann"
2	Wert auslesen	GET benutzer:name
3	Existenz prüfen (ohne den Wert zu lesen)	EXISTS benutzer:name
4	Wert überschreiben	SET benutzer:name "Erika Musterfrau"
5	Zähler auf 0 setzen und um 1 erhöhen	SET besucher:anzahl 0 dann INCR besucher:anzahl
6	Schlüssel löschen	DEL benutzer:name
7	Hash produkt:100 mit name="Laptop" und preis="999"	HSET produkt:100 name "Laptop" preis "999"
8	Nur den preis anzeigen	HGET produkt:100 preis
9	Alle Felder und Werte anzeigen	HGETALL produkt:100
10	Feld lagerbestand="50" hinzufügen	HSET produkt:100 lagerbestand "50"
11	Nur das Feld preis löschen	HDEL produkt:100 preis
12	Ablaufzeit 60 Sekunden für den Hash	EXPIRE produkt:100 60

91.2 MongoDB-Aufgaben mit Lösungen

★ *, „Diese Collections **müssen vorher NICHT angelegt werden**. Anders als bei SQL-Datenbanken, wo man zwingend erst CREATE TABLE ausführen muss, arbeitet **MongoDB ,DYNAMISCH'**: Sobald ein Befehl zum Speichern ausgeführt wird, prüft MongoDB im Hintergrund, ob die Collection existiert — **wenn nein, erstellt es die Collection SOFORT AUTOMATISCH.**“*

#	Aufgabe	Lösung
1	Dokument mit <code>name = "Max Mustermann"</code> einfügen	<code>db.benutzer.insertOne({ name: "Max Mustermann" })</code>
2	Dokument suchen	<code>db.benutzer.find({ name: "Max Mustermann" })</code>
3	Existenz prüfen (zählen)	<code>db.benutzer.countDocuments({ name: "Max Mustermann" })</code>
4	Namen ändern (mit <code>\$set</code>)	<code>db.benutzer.updateOne({ name: "Max Mustermann" }, { \$set: { name: "Erika Musterfrau" } })</code>
5	Zähler anlegen und erhöhen	<code>db.statistik.insertOne({ art: "besucher", anzahl: 0 })</code> <code>db.statistik.updateOne({ art: "besucher" }, { \$inc: { anzahl: 1 } })</code>
6	Dokument löschen	<code>db.benutzer.deleteOne({ name: "Erika Musterfrau" })</code>
7	Produkt mit <code>manueller _id</code> anlegen	<code>db.produkte.insertOne({ _id: 100, name: "Laptop", preis: 999 })</code>
8	★ Nur den Preis anzeigen (Projektion, ID ausblenden)	<code>db.produkte.findOne({ _id: 100 }, { preis: 1, _id: 0 })</code> (★ 1 = anzeigen, 0 = ausblenden)
9	Ganzes Dokument lesen	<code>db.produkte.findOne({ _id: 100 })</code>
10	Feld <code>lagerbestand: 50</code> hinzufügen	<code>db.produkte.updateOne({ _id: 100 }, { \$set: { lagerbestand: 50 } })</code>
11	★ Feld <code>preis</code> entfernen	<code>db.produkte.updateOne({ _id: 100 }, { \$unset: { preis: "" } })</code>
12	Alle Dokumente anzeigen	<code>db.produkte.find()</code>

91.3 Challenge „The Sound Store“ ★★

Redis:

```
# A) Globaler Besucherzähler
SET shop:besucher:total 150
INCR shop:besucher:total
GET shop:besucher:total

# B) Angebot des Tages mit Zeitlimit
SET aktion:gutschein "ROCK2025"
EXPIRE aktion:gutschein 300
TTL aktion:gutschein

# C) Produktdatenbank (Hash)
HSET produkt:500 name "E-Gitarre" marke "Fender" preis 1200 lager 5
HSET produkt:500 lager 4 # ★ HSET überschreibt ein bestehendes Feld
HGETALL produkt:500

# D) Live-Warenkorb
HSET warenkorb:user:12 produkt:500 1
HSET warenkorb:user:12 produkt:202 10
HDEL warenkorb:user:12 produkt:202
```

★ **Hinweis der Musterlösung:** „Warum nur 1 bzw. 10 für die Menge im Warenkorb ohne zusätzlich beschreibendes Label? In Redis-Hashes wird die Menge DIREKT ALS WERT zum FELD (Produkt-ID) gespeichert, da die Struktur FLACH ist und KEINE zusätzlichen beschreibenden Label innerhalb eines einzelnen Feldes unterstützt.“

MongoDB:

```
// A) Kundenprofile mit VERSCHACHELTEM Dokument
db.kunden.insertOne({
  name: "Max Mustermann",
  adresse: { strasse: "Musterstraße 1", stadt: "Berlin" } // ★ nested Object
});
db.kunden.find({ name: "Max Mustermann" });

// B) Inventarverwaltung
db.produkte.insertOne({ _id: 500, name: "E-Gitarre", preis: 1200, bestand: 5 });
db.produkte.updateOne({ _id: 500 }, { $inc: { bestand: -1 } }); // ★ $inc mit NEGATIVEM Wert
db.produkte.find({ preis: { $lt: 1500 } }); // $lt = less than

// C) Warenkorb-Logik mit ARRAY
db.kunden.updateOne({ name: "Max Mustermann" },
  { $set: { warenkorb: ["E-Gitarre", "Verstärker"] } });
db.kunden.updateOne({ name: "Max Mustermann" }, { $unset: { adresse: "" } });
db.kunden.deleteOne({ name: "Max Mustermann" });
```

★★ Die drei MongoDB-Kernmuster aus der Challenge:

1. **Verschachtelte Objekte** (adresse: { ... }) — das, was in SQL eine zweite Tabelle wäre
2. **\$inc mit negativem Wert** verringert einen Zähler
3. **Arrays als Feldwert** (warenkorb: [...]) — das, was in SQL eine n:m-Tabelle wäre

TEIL L — SELBSTTEST

Die **Wiederholungs- und Transferfragen aus den Foliensätzen** (mit den Original-Beispiel- Lösungen des Dozenten) sowie ergänzende Fragen.

Block 1 — Grundlagen, DDL, DML (1–25)

▼ 1. ★★ Grenzen Sie Datenbank, DBMS und Datenbanksystem ab.

- **Datenbank:** eine **logische Einheit zusammengehörender Daten** und sämtlicher **Meta-Informationen** dazu
- **DBMS:** stellt **Werkzeuge** bereit, mit denen Datenbanken **erstellt, gefüllt und verwaltet** werden
- **Datenbanksystem (DBS):** die **funktionale Einheit aus DB + DBMS**

★ DBS = DB + DBMS

▼ 2. ★★★ Nennen Sie die sechs Anforderungen an ein DBMS.

1. **Bereitstellung von Datenbanksprachen (DDL, DML)**
2. **Sicherheit & Autorisierung**
3. **Datenintegrität** (semantische Integrität)
4. **Robustheit & Recovery**
5. **Mehrbenutzer-Synchronisation** (Stichwort **ACID**)
6. **Effizienz**

▼ 3. ★★★★★ **Beschreiben Sie das 3-Schichten-Modell nach ANSI-SPARC.**

- **Externe Schicht:** legt fest, **welche Daten bestimmte Benutzer/Programme sehen und bearbeiten** können (Views)
- **Logische (konzeptionelle) Schicht:** die Gesamtstruktur — ★ **unabhängig von der physischen Implementierung**
- **Physische Schicht:** legt **Zugriffsmechanismen und Speicherstrukturen** fest

Jahr: 1975

▼ 4. ★★★★★ **Nennen Sie die drei Elemente des ER-Modells und seinen Erfinder.**

Entitäten (abstrakte oder reale Objekte) · **Beziehungen** (wie Entitäten verbunden sind) · **Attribute** (Eigenschaften von Entitäten) **Peter Chen, 1976**. Weiterentwicklung: **UML-Klassendiagramme**

▼ 5. ★★★★★ **Was ist der grundlegende Unterschied zwischen Projektion und Selektion? Welche SQL-Schlüsselwörter entsprechen ihnen?**

(Original-Beispiellösung der Folie)

Die **SELEKTION (oder Restriktion)** wählt bestimmte **ZEILEN (Tupel)** aus einer Tabelle aus, die eine Bedingung erfüllen. Dies entspricht der **WHERE -Klausel**. Die **PROJEKTION** wählt dagegen bestimmte **SPALTEN (Attribute)** aus. Dies entspricht der **SELECT -Klausel**.

▼ 6. ★★★★★ **Erklären Sie den Hauptzweck der DDL im Gegensatz zur DML.**

(Original-Beispiellösung)

Die **DDL** wird verwendet, um die **STRUKTUR der Datenbank zu definieren und zu verwalten** (Erstellen, Ändern, Löschen von Tabellen oder Schemas). Die **DML** wird genutzt, um die **in den Tabellen gespeicherten DATEN zu manipulieren** — Datensätze einfügen, ändern oder löschen.

▼ 7. ★★★★★ **SQL ist nicht case-sensitive. Was bedeutet das, und welche Konvention wird empfohlen?**

(Original-Beispiellösung)

„Case-sensitive“ bedeutet, dass **nicht zwischen Groß- und Kleinschreibung unterschieden** wird. Der Befehl **SELECT** ist für die Datenbank **identisch mit select**. Es wird empfohlen, **SQL-Schlüsselwörter wie SELECT, FROM oder WHERE zur besseren Lesbarkeit immer GROSSZUSCHREIBEN**.

▼ 8. ★★★★★ Was ist der praktische Unterschied zwischen CHAR(n) und VARCHAR(n)?

(Original-Beispiellösung)

CHAR(n) speichert eine Zeichenkette mit **FESTER Länge** von n Zeichen. Wird ein kürzerer Text gespeichert, **wird dieser mit Leerzeichen aufgefüllt, was Speicherplatz verschwenden kann.** **VARCHAR(n)** speichert mit **VARIABLER Länge bis zu einem Maximum** und belegt **nur den tatsächlich benötigten Speicherplatz.** **VARCHAR(n)** ist daher die **effizientere Wahl für Texte mit unterschiedlichen Längen** wie Namen oder Adressen.

▼ 9. ★★★★★ Welche zwei Integritätsbedingungen werden durch einen Primärschlüssel automatisch sichergestellt?

(Original-Beispiellösung)

Ein Primärschlüssel stellt sicher, dass jeder Wert in dieser Spalte **EINDEUTIG (UNIQUE)** ist und **NICHT LEER (NOT NULL)** sein darf.

(Zusätzlich: Er ist automatisch **indiziert.**)

▼ 10. ★★★★★ Nennen Sie die drei DDL-Hauptbefehle mit ihrer Funktion.

(Original-Beispiellösung)

CREATE zum **Anlegen** neuer Datenbankobjekte wie Schemas und Tabellen **ALTER** zum **Ändern der Struktur** bestehender Objekte (Spalten hinzufügen/löschen) **DROP** zum **endgültigen Löschen** ganzer Datenbankobjekte

▼ 11. ★★★★★ Was versteht man unter referentieller Integrität?

(Original-Beispiellösung)

Referentielle Integrität stellt sicher, dass **Beziehungen zwischen Tabellen GÜLTIG bleiben.** Ein Wert in einer Fremdschlüssel-Spalte muss **immer einem EXISTIERENDEN Primärschlüsselwert in der referenzierten Tabelle entsprechen ODER NULL sein.**

▼ 12. ★★★★★ Erklären Sie den Unterschied zwischen ON DELETE CASCADE und ON DELETE NO ACTION.

(Original-Beispiellösung)

ON DELETE CASCADE : Wenn der referenzierte Datensatz (z. B. ein Projekt) gelöscht wird, werden **alle Datensätze, die auf ihn verweisen** (z. B. die Mitarbeiter dieses Projekts), **ebenfalls AUTOMATISCH GELÖSCHT.** **ON DELETE NO ACTION :** Die Datenbank **VERHINDERT das Löschen** des Datensatzes, solange noch andere Datensätze über einen Fremdschlüssel auf ihn verweisen. **Der Löschvorgang wird mit einem FEHLER ABGEWIESEN.**

▼ 13. ★★ Was ist der Zweck der CHECK-Integritätsbedingung? Geben Sie ein Beispiel.

(Original-Beispiellösung)

Der CHECK -Constraint dient dazu, **zusätzliche Regeln festzulegen, die ein Spaltenwert erfüllen muss**. Beispiel: CHECK (emp_age >= 18)

▼ 14. ★★★ Was bewirkt die WHERE-Klausel in einem UPDATE? Was passiert, wenn man sie weglässt?

(Original-Beispiellösung)

Die WHERE -Klausel **filtert die Datensätze, die geändert werden sollen**. Lässt man sie weg, **würde die UPDATE -Anweisung auf ALLE ZEILEN der Tabelle angewendet werden**.

▼ 15. ★★ Nennen Sie die vier Eigenschaften einer Relation.

1. Es gibt **keine doppelten Tupel**
2. **Tupel sind nicht geordnet**
3. **Attribute sind nicht geordnet**
4. **Alle Attribute sind atomar**

▼ 16. ★★ Warum liefert eine SQL-Abfrage keine Menge, sondern eine Multimenge?

Eine **Multimenge (Bag)** erlaubt das **mehrfache Vorkommen identischer Elemente**, da **jedes Ergebnis eine separate Zeile der Quelltable repräsentiert**. Dieses **implizite SELECT ALL** ist auch aus **Performance-Gründen** gewählt (**vermeidet die zeitaufwendige Duplikat-Eliminierung**). Erst **DISTINCT** wandelt sie in eine **mathematisch eindeutige Menge** um.

▼ 17. ★ Nennen Sie die fünf SQL-Befehlskategorien.

DDL (Data Definition) · **DML** (Data Manipulation) · **DQL** (Data Query) · **TCL** (Transaction Control) · **DCL** (Data Control)

▼ 18. ★★ Was ist SERIAL, und warum ist Vorsicht geboten?

Ein Typ, der **automatisch eine eindeutige Ganzzahl generiert**. ⚠ **Kein echter Datentyp**, sondern eine Vereinfachung. ⚠ **KEIN Standard-SQL-Typ!** Er **erstellt intern eine SEQUENZ** und setzt einen **NOT NULL -Constraint**.

▼ 19. ★★★ Welche Constraints können NICHT benannt werden?

NOT NULL und **DEFAULT** — sie können **nur INLINE** definiert werden und sind **nicht benennbar**. Alle anderen (**PRIMARY KEY** , **FOREIGN KEY** , **UNIQUE** , **CHECK**) können inline, separat und benannt definiert werden.

▼ 20. ★★ Was passiert, wenn ein DELETE mehrere Zeilen betrifft und für EINE Zeile eine NO-ACTION-Zurückweisung erfolgt?

„Es wird Zeile für Zeile einzeln geprüft. Gibt es für MINDESTENS EINE Zeile eine Zurückweisung, so wird DER GESAMTE SQL-BEFEHL NICHT AUSGEFÜHRT.“

▼ 21. ★ Wofür wird TIMESTAMP WITH TIME ZONE intern verwendet?

PostgreSQL wandelt den Wert intern in UTC um und speichert ihn so. Beim Auslesen wird er zurück in die Zeitzone des Clients konvertiert.

▼ 22. ★ Welchen Datentyp verwendet man für Währungen und warum?

NUMERIC(p,s) / DECIMAL(p,s) — exakte numerische Werte, keine Rundungsfehler. ⚠ NIEMALS REAL / DOUBLE PRECISION (Fließkomma → Rundungsfehler).

▼ 23. ★★ Grenzen Sie DELETE, TRUNCATE und DROP ab.

- DELETE FROM t — DML, löscht Zeilen (zeilenweise, mit Trigger und Log)
- TRUNCATE TABLE t — DDL, löscht alle Zeilen sehr schnell
- DROP TABLE t — DDL, löscht die gesamte Tabelle inkl. Struktur

▼ 24. ★ Was ist der Unterschied zwischen dem ER-Modell und einem Relationenschema?

Das ER-Modell ist konzeptionell und stellt m:n-Beziehungen als Beziehung dar. Das Relationenschema ist logisch: m:n-Beziehungen werden als eigene Tabelle aufgelöst, Beziehungen über Fremdschlüssel abgebildet. ⚠ DBeaver zeigt ein Relationenschema, nennt es aber „ER-Modell“.

▼ 25. ★ Wie hieß der erste PostgreSQL-Vorläufer, und wer entwickelte ihn?

POSTGRES an der UC Berkeley unter Michael Stonebraker (Mitte 1980er, erster Prototyp 1989). 1994 → Postgres95 (Open Source), 1996 → PostgreSQL (Version 6.0).

Block 2 — DQL (26–50)

▼ 26. ★★★ Welche zwei Klauseln sind die einzigen Pflichtbestandteile einer SELECT-Anweisung?

(Original-Beispiellösung) Die SELECT - und die FROM -Klausel.

▼ 27. ★★ Nennen Sie zwei Gründe für die Verwendung eines Alias.

(Original-Beispiellösung) Verbesserung der Lesbarkeit (z. B. AS Nachname), Abkürzen langer Tabellennamen (z. B. customer AS c), Benennen berechneter Spalten.

▼ 28. ★★ Was ist die Funktion von DISTINCT?

(Original-Beispiellösung) **DISTINCT** entfernt Duplikate aus der Ergebnismenge, sodass **jeder Wert bzw. jede Zeilenkombination nur EINMAL** vorkommt.

▼ 29. ★★ Wie verhalten sich Aggregatfunktionen ohne GROUP BY?

(Original-Beispiellösung) Sie fassen **ALLE** ausgewählten Zeilen zu **EINER EINZIGEN ERGEBNISZEILE** zusammen.

▼ 30. ★★★ Wie behandeln Aggregatfunktionen NULL-Werte?

(Original-Beispiellösung) **NULL-Werte** werden bei der Berechnung **IGNORIERT**.

▼ 31. ★★★ Worin liegt der Unterschied zwischen COUNT(*) und COUNT(Spalte)?

(Original-Beispiellösung) **COUNT(*)** zählt **ALLE** Datensätze in der Ergebnismenge.
COUNT(<Spaltenname>) zählt **NUR** die Datensätze, bei denen die angegebene Spalte **KEINEN NULL-Wert** enthält.

▼ 32. ★ In welcher Reihenfolge sortiert ORDER BY standardmäßig?

(Original-Beispiellösung) Die Standardeinstellung ist **ASC (ascending)**, also **AUFSTEIGEND**. (Zusatz: In PostgreSQL landen NULLs bei ASC am Ende, bei DESC am Anfang.)

▼ 33. ★★ Welchen Zweck erfüllen LIMIT und OFFSET?

(Original-Beispiellösung) **LIMIT** begrenzt die Anzahl der zurückgegebenen Zeilen, **OFFSET** überspringt eine bestimmte Anzahl von Zeilen am Anfang. Sie bilden die Grundlage für die **PAGINIERUNG (Seitenblätterfunktion)** in Webanwendungen. ⚠️ Ohne **ORDER BY** ist das Ergebnis nicht deterministisch!

▼ 34. ★★★ Erklären Sie den fundamentalen Unterschied zwischen WHERE und HAVING.

(Original-Beispiellösung) Die **WHERE**-Klausel filtert **EINZELNE ZEILEN VOR** der Gruppierung. Die **HAVING**-Klausel filtert **DATENSATZGRUPPEN NACH** der Gruppierung durch **GROUP BY**.

▼ 35. ★★★ Was passiert, wenn man im SELECT eine Spalte angibt, die weder in GROUP BY steht noch aggregiert ist?

(Original-Beispiellösung) Das ist **NICHT ERLAUBT**. In der **SELECT**-Liste dürfen bei Verwendung von **GROUP BY** **NUR** die Attribute stehen, nach denen gruppiert wird, **ODER** Aggregatfunktionen.

▼ 36. ★★★ Warum kann man nicht WHERE spalte = NULL schreiben?

(Original-Beispiellösung) **NULL** repräsentiert einen **UNBEKANNTEN** Wert, daher kann er **NICHT** mit **=** verglichen werden. Stattdessen muss die Syntax **IS NULL** oder **IS NOT NULL** verwendet werden.

▼ 37. ★★ Was ist der Unterschied zwischen den Wildcards % und _ ?

(Original-Beispiellösung) Das Prozentzeichen % steht für eine BELIEBIGE ANZAHL beliebiger Zeichen. Der Unterstrich _ steht für GENAU EIN beliebiges Zeichen.

▼ 38. ★★ In welcher Reihenfolge werden Hauptabfrage und einfache Unterabfrage ausgeführt?

(Original-Beispiellösung) Die Datenbank führt ZUERST die INNERE Unterabfrage aus und verwendet deren Ergebnis dann, um die ÄUSSERE Hauptabfrage auszuführen.

▼ 39. ★★★ Was unterscheidet eine korrelierte Unterabfrage von einer einfachen?

(Original-Beispiellösung) Eine korrelierte Unterabfrage ist VON DER ÄUSSEREN ABFRAGE ABHÄNGIG und wird FÜR JEDE ZEILE der äußeren Abfrage ERNEUT AUSGEFÜHRT.

▼ 40. ★★ Was versteht man unter einem kartesischen Produkt, und wann entsteht es?

(Original-Beispiellösung) Es entsteht, wenn man Tabellen in der FROM -Klausel auflistet, OHNE sie über eine WHERE - oder JOIN -Bedingung zu verknüpfen. Dabei wird JEDER Datensatz der einen Tabelle mit JEDEM Datensatz der anderen KOMBINIERT.

▼ 41. ★★★ Welches Ergebnis liefert ein INNER JOIN?

(Original-Beispiellösung) Ein INNER JOIN liefert NUR die Datensätze, bei denen es IN BEIDEN Tabellen PASSENDE WERTE gemäß der ON -Bedingung gibt.

▼ 42. ★★★ Was ist der Hauptunterschied zwischen INNER JOIN und LEFT OUTER JOIN?

(Original-Beispiellösung) Ein LEFT OUTER JOIN gibt ALLE Zeilen der LINKEN Tabelle zurück, AUCH WENN es in der rechten Tabelle KEINEN passenden Partner gibt. Beim INNER JOIN würden diese Zeilen ohne Partner WEGFALLEN.

▼ 43. ★★ In welchem Szenario wird ein SELF JOIN verwendet?

(Original-Beispiellösung) Um eine REKURSIVE Beziehung INNERHALB einer Tabelle abzufragen — zum Beispiel um Mitarbeiter und deren VORGESETZTE aus DERSELBEN Tabelle aufzulisten.

(Chinook: `employee.reports_to`)

▼ 44. ★★★ Welche zwei Voraussetzungen müssen für UNION erfüllt sein?

(Original-Beispiellösung) Beide SELECT-Abfragen müssen die GLEICHE ANZAHL an Spalten liefern, und die Spalten an den jeweiligen Positionen müssen KOMPATIBLE Datentypen haben. (Zusatz: Spaltennamen kommen von der ERSTEN Abfrage; ORDER BY nur EINMAL, ganz am Ende.)

▼ 45. ★★★★★ **Wie unterscheiden sich Mengenoperatoren von JOINS?**

(Original-Beispiellösung) JOINS kombinieren Daten HORIZONTAL, indem sie SPALTEN aus mehreren Tabellen NEBENEINANDER in einer neuen Zeile zusammenführen. Mengenoperatoren kombinieren Daten VERTIKAL, indem sie ZEILEN einer Abfrage UNTER die Zeilen einer anderen hängen.

▼ 46. ★★★★★ **Bei einem LEFT JOIN: Was ist der Unterschied, ob eine Bedingung im WHERE oder im ON steht?**

- Im WHERE : Der Filter wirkt **NACH** dem Join auf das gesamte Zwischenergebnis → ⚠ der LEFT JOIN wird effektiv zu einem INNER JOIN, weil Zeilen mit NULL auf der rechten Seite herausgefiltert werden
- Im ON : Die Bedingung wirkt **VOR** dem Join und bestimmt nur, welche Datensätze verknüpft werden → alle Zeilen der linken Tabelle bleiben erhalten

★ Bei INNER JOIN sind beide Varianten logisch äquivalent.

▼ 47. ★★ **Wie viele JOIN-Bedingungen braucht ein Join über n Tabellen?**

Mindestens $(n - 1)$ JOIN-Bedingungen.

▼ 48. ★ **Was ist ein NON EQUI JOIN, und warum sollte man NATURAL JOIN meiden?**

NON EQUI JOIN: Als Operator wird **kein** = verwendet, **meist** BETWEEN (z. B. Gehaltsspannen).
NATURAL JOIN: verknüpft automatisch über gleichnamige Spalten — ⚠ „NICHT verwenden!
Intransparent, fehleranfällig“

▼ 49. ★★ **Welche Operatoren muss man verwenden, wenn eine Subquery mehrere Werte liefert?**

IN, ANY, ALL, EXISTS. Bei einem atomaren (skalaren) Wert darf man <, >, = verwenden — um ein atomares Ergebnis zu garantieren, nutzt man oft Aggregate.

▼ 50. ★★ **Was ist eine Inline View, und was muss sie zwingend haben?**

Eine Unterabfrage in der FROM -Klausel (auch abgeleitete Tabelle). Sie wird wie eine temporäre Tabelle behandelt und ⚠ MUSS einen ALIAS haben.

Block 3 — Zusatzthemen, Programmierung, Transaktionen (51–85)

▼ 51. ★★★★★ **Was ist eine VIEW, und wie verhält sie sich gegenüber Programmen?**

(Original-Beispiellösung) Eine VIEW ist eine „PSEUDOTABELLE“ (gespeicherte Abfrage), die die Ergebnismenge eines SELECTs repräsentiert. NACH AUSSEN präsentiert sie sich gegenüber zugreifenden Programmen WIE EINE ECHTE TABELLE.

▼ 52. ★★ Welchen Vorteil bieten VIEWS bei Änderungen der Tabellenstruktur?

(Original-Beispiellösung) Programme müssen NICHT ANGEPASST werden, wenn sich die Tabellen hinter der Sicht ändern, da die VIEW als ABSTRAKTIONSSCHICHT STABIL bleibt.

▼ 53. ★★★★★ Definieren Sie den Unterschied zwischen persistenter VIEW und Inline View.

(Original-Beispiellösung) Eine persistente VIEW ist ein BENANNTES Datenbankobjekt, das DAUERHAFT im Schema gespeichert wird. Eine Inline View ist eine TEMPORÄRE SELECT-Anweisung, die DIREKT IM FROM -Abschnitt einer Abfrage definiert wird.

▼ 54. ★★★★★ Nennen Sie drei Bedingungen, damit über eine VIEW Daten geändert werden dürfen.

(Original-Beispiellösung) Die FROM -Klausel enthält NUR EINE Relation · es gibt KEINE GROUP BY -Klausel · die SELECT -Klausel enthält KEIN DISTINCT · es werden KEINE Berechnungen oder Aggregationen verwendet (und: keine UNION / INTERSECT / EXCEPT).

▼ 55. ★★★★★ Warum sollte man nicht wahllos Indizes erstellen?

(Original-Beispiellösung) Indizes verbessern zwar die LESE-Performance, VERSCHLECHTERN aber die SCHREIB-Performance, da bei jedem INSERT , UPDATE oder DELETE ein zusätzlicher OVERHEAD entsteht.

▼ 56. ★★ Wie erzwingt man mit einem Index ein Primary-Key-Verhalten?

(Original-Beispiellösung) Durch das Erstellen eines UNIQUE INDEX . Da dieser INDIZIERT und EINDEUTIG ist, verhält er sich WIE EIN PRIMARY KEY.

▼ 57. ★★ Welches Grundprinzip gilt bei der Rechtevergabe?

(Original-Beispiellösung) Das Prinzip der „MINIMALEN RECHTE“ — Benutzern sollte NUR Zugriff auf die FÜR SIE RELEVANTEN Daten gewährt werden.

▼ 58. ★★ Warum ist RBAC der direkten Rechtevergabe vorzuziehen?

(Original-Beispiellösung) RBAC ist ÜBERSICHTLICHER und WENIGER RISIKOBEHAFTET. Neue Benutzer müssen lediglich EINER BESTEHENDEN ROLLE ZUGEWIESEN werden, statt alle Berechtigungen einzeln neu zu vergeben.

▼ 59. ★★★★★ Was ist das Information_Schema, und welchen Vorteil bietet es?

(Original-Beispiellösung) Das Information_Schema ist ein ISO-STANDARD für Verwaltungsdaten (Metadaten). Es ist SYSTEMÜBERGREIFEND FAST IDENTISCH (z. B. in SQL Server, MySQL, PostgreSQL), während pg_* -Tabellen POSTGRESQL-SPEZIFISCH sind.

▼ 60. ★★★★★ **Nennen Sie zwei Vorteile und zwei Risiken von Business-Logik in der Datenbank.**

(Original-Beispiellösung)

Vorteile: Weniger Netzwerk-Traffic (keine Zwischenergebnisse zum Client) und **bessere Performance**, da Datenbanken auf die Nutzung von Stored Procedures, Functions oder Triggern optimiert sind. **Risiken:** Höhere Komplexität durch verteilte Logik und **schlechte Portierbarkeit** aufgrund proprietärer Funktionalitäten.

▼ 61. ★★★★★ **Wie unterscheiden sich FUNCTIONS und PROCEDURES?**

(Original-Beispiellösung) Funktionen LIEFERN EINEN WERT ZURÜCK und werden für **BERECHNUNGEN** genutzt. Prozeduren steuern **PROZESSABLÄUFE** und können **IM GEGENSATZ** zu Funktionen **EIGENE TRANSAKTIONSBEFEHLE** (wie `COMMIT`) **ENTHALTEN**.

Zusatz: Funktionen sind **in SQL-Anweisungen verwendbar**, Prozeduren werden mit `CALL` aufgerufen.

▼ 62. ★★★★★ **Wie wird der Rückgabewert in einer LANGUAGE-SQL-Funktion festgelegt?**

(Original-Beispiellösung) Der Rückgabewert ergibt sich **IMPLIZIT** aus dem Ergebnis der **ALLERLETZTEN SQL-Anweisung** im Funktionskörper.

▼ 63. ★★★★★ **Wozu dient das Dollar-Quoting (\$\$)?**

(Original-Beispiellösung) Es dient als **BODY-DELIMITER**, um den Programmcode zu umschließen und Probleme mit **EINFACHEN ANFÜHRUNGSZEICHEN** (') oder **ZEILENUMBRÜCHEN** innerhalb des Codes zu vermeiden.

▼ 64. ★★★★★ **Welche Bedeutung hat BEGIN ATOMIC ... END?**

(Original-Beispiellösung) Es garantiert die **ATOMARITÄT** mehrerer DML-Operationen innerhalb einer SQL-Funktion; **ENTWEDER** werden alle Schritte erfolgreich ausgeführt **ODER** alle zurückgesetzt.

▼ 65. ★★★★★ **Welche prozeduralen Elemente bietet PL/pgSQL, die reines SQL nicht hat?**

(Original-Beispiellösung) **Ablaufsteuerung** durch Kontrollstrukturen wie `IF`, `LOOP` oder `WHILE` sowie eine **ROBUSTE FEHLERBEHANDLUNG** mittels `EXCEPTION`.

▼ 66. ★★★★★ **Wo werden Variablen in PL/pgSQL deklariert, und welche Ausnahme gibt es?**

(Original-Beispiellösung) Variablen müssen in der `DECLARE`-Sektion definiert werden. **AUSNAHME** sind die **ÜBERGABEPARAMETER**, die **IMPLIZIT** deklariert und **SOFORT VERFÜGBAR** sind.

▼ 67. ★★★★★ **Mit welcher Anweisung speichert man Abfrageergebnisse in Variablen?**

(Original-Beispiellösung) Durch die Anweisung `SELECT ... INTO <Variable>`.

▼ 68. ★★ Erklären Sie IN-, OUT- und INOUT-Parameter.

(Original-Beispiellösung) **IN** sind reine EINGABEWERTE, **OUT** dienen der RÜCKGABE von Werten an den Aufrufer, und **INOUT** können SOWOHL als Eingabe empfangen ALS AUCH als Ausgabe verändert zurückgegeben werden.

▼ 69. ★★★ Was versteht man unter einem „Hook“ im Zusammenhang mit Triggern?

(Original-Beispiellösung) Ein Trigger dient als „HAKEN“, um EIGENEN CODE AUTOMATISCH in die NORMALE ABARBEITUNG von Ereignissen (**INSERT** , **UPDATE** , **DELETE**) durch das DBMS EINZUFÜGEN.

▼ 70. ★★★ Warum braucht man in PostgreSQL zusätzlich eine Trigger Function?

(Original-Beispiellösung) Der **CREATE TRIGGER** -Befehl dient in PostgreSQL NUR dazu, das EREIGNIS an eine TABELLE zu BINDEN. Die EIGENTLICHE LOGIK muss in einer Funktion mit dem Rückgabotyp **RETURNS TRIGGER** GEKAPSELT sein.

▼ 71. ★★★ Erklären Sie den Unterschied zwischen BEFORE- und AFTER-Trigger.

(Original-Beispiellösung) **BEFORE**-Trigger laufen VOR der Operation ab und können diese noch MODIFIZIEREN oder STOPPEN. **AFTER**-Trigger laufen ERST NACH der erfolgreichen Durchführung ab.

▼ 72. ★★★ Was repräsentieren OLD und NEW bei einer UPDATE-Operation?

(Original-Beispiellösung) **OLD** enthält den Zustand des Datensatzes VOR der Änderung, **NEW** enthält den Zustand NACH der Änderung.

▼ 73. ★★ Warum dürfen in Trigger-Funktionen keine COMMIT/ROLLBACK ausgeführt werden?

(Original-Beispiellösung) Da Trigger TEIL DER AUFRUFENDEN DML-OPERATION sind, dürfen sie die TRANSAKTIONSKONTROLLE NICHT EIGENSTÄNDIG BEEINFLUSSEN.

▼ 74. ★★ Was bewirkt RETURN NULL in einem BEFORE-Trigger?

Es liefert der Datenbank das **ABBRUCHSIGNAL** — die nachfolgende **DML-Operation** für diese Zeile wird **NICHT ausgeführt**. **RETURN NEW** setzt sie dagegen (ggf. mit modifizierten Werten) fort.

▼ 75. ★★ Was ist der Unterschied zwischen FOR EACH ROW und FOR EACH STATEMENT?

Zeilen-Trigger (**FOR EACH ROW**) werden einmal für JEDE betroffene Zeile aufgerufen; **Anweisungs-Trigger** (**FOR EACH STATEMENT**) nur EINMAL für die ganze Anweisung. ⚠ In Zeilen-Triggern kann man auf die betroffenen Zeilen zugreifen, in Anweisungs-Triggern nicht.

▼ 76. ★★★★★ **Erläutern Sie das Kriterium der Isolation und warum Zwischenzustände unsichtbar sein müssen.**

(Original-Beispiellösung) Isolation garantiert, dass sich GLEICHZEITIG AUSGEFÜHRTE Transaktionen NICHT GEGENSEITIG BEEINFLUSSEN. Da die KONSISTENZ WÄHREND der Ausführung einer Transaktion NICHT GARANTIERTE werden kann, müssen Ergebnisse BIS ZUM COMMIT für andere UNSICHTBAR bleiben, damit diese NICHT AUF INKONSISTENTEN ZUSTÄNDEN ARBEITEN.

▼ 77. ★★★★★ **Wie verhält sich ein DBMS im Auto-Commit-Modus, und wann muss er unterbunden werden?**

(Original-Beispiellösung) Im Auto-Commit-Modus führt das DBMS NACH JEDER EINZELNEN INSERT -, UPDATE - oder DELETE -Anweisung AUTOMATISCH ein COMMIT durch. Dies muss DEAKTIVIERT werden, wenn MEHRERE ÄNDERUNGEN als EINE ZUSAMMENHÄNGENDE LOGISCHE EINHEIT (Transaktion) betrachtet werden sollen.

▼ 78. ★★★★★ **Beschreiben Sie das Lost-Update-Problem.**

(Original-Beispiellösung) Es tritt auf, wenn zwei Transaktionen GLEICHZEITIG DENSELben Datensatz LESEN, ihn MODIFIZIEREN und anschließend NACHEINANDER ZURÜCKSCHREIBEN. Da die zweite Transaktion das Ergebnis der ersten OHNE PRÜFUNG ÜBERSCHREIBT, geht der Update von TA1 VERLOREN. In der SQL-Norm wird dieses Phänomen auch als „DIRTY WRITE“ bezeichnet und ist GRUNDSÄTZLICH VERBOTEN.

▼ 79. ★★★★★ **Vergleichen Sie optimistische und pessimistische Synchronisation.**

(Original-Beispiellösung)

Pessimistisch: Geht davon aus, dass Konflikte HÄUFIG auftreten; daher werden SPERREN GESETZT, BEVOR Daten gelesen oder geschrieben werden. Sperren werden erst bei Transaktionsende freigegeben. ⚠️ Wartezeiten, Gefahr von DEADLOCKS. **Optimistisch:** Geht davon aus, dass Konflikte SELTEN sind; Transaktionen laufen OHNE Sperren ab und werden erst am Ende (VALIDIERUNG kurz vor dem COMMIT) auf Konflikte geprüft. ⚠️ Bei Konflikt: ROLLBACK und NEUSTART.

▼ 80. ★★★★★ **Welches Problem verhindert SERIALIZABLE, das REPEATABLE READ noch zulässt?**

(Original-Beispiellösung) Im Level SERIALIZABLE werden PHANTOME verhindert. Ein Phantom entsteht, wenn bei einer WIEDERHOLTEN ABFRAGE innerhalb einer Transaktion NEUE ZEILEN AUFTAUCHEN, die eine andere Transaktion in der Zwischenzeit EINGEFÜGT hat.

▼ 81. ★★★★★ **Was ist ein Deadlock, und wie entsteht er?**

(Original-Beispiellösung) Ein Deadlock ist eine WECHSELSEITIGE BLOCKIERUNG, bei der mehrere Transaktionen auf die FREIGABE von Ressourcen WARTEN, die jeweils von der ANDEREN Transaktion GEHALTEN werden. Beispiel: Transaktion 1 sperrt Datensatz A und wartet auf B; Transaktion 2 sperrt B und wartet auf A.

▼ 82. ★★★ Unterscheiden Sie 2PL und 2PC.

(Original-Beispiellösung)

2PL (Two-Phase Locking): Dient der Umsetzung der **ISOLATION (I)** und **garantiert die SERIALISIERBARKEIT** von Transaktionen. Phasen: **Wachstumsphase** (nur anfordern) und **Schrumpfphase** (nur freigeben). **2PC (Two-Phase Commit):** Dient der Umsetzung der **ATOMARITÄT (A)** in **VERTEILTEN Systemen** und **koordiniert den GEMEINSAMEN ABSCHLUSS (COMMIT oder ABORT)** über **MEHRERE KNOTEN**. Phasen: **Prepare** und **Commit**.

▼ 83. ★★★ Erklären Sie Dirty Read, Non Repeatable Read und Phantom.

(Original-Beispiellösung)

Dirty Read: Transaktion A liest Daten, die von Transaktion B zwar **geändert, aber noch NICHT durch COMMIT festgeschrieben** wurden. Bei einem **ROLLBACK** von B hat A auf **UNGÜLTIGEN Daten** gearbeitet. **Non Repeatable Read:** Ein **einmal gelesener Eintrag bleibt innerhalb derselben Transaktion NICHT unverändert** — beim erneuten Lesen ein **anderes Ergebnis** (UPDATE durch eine andere Transaktion). **Phantom:** Bei einer **wiederholten Bereichsabfrage tauchen NEUE ZEILEN auf**, die eine zwischenzeitlich abgeschlossene Transaktion **eingefügt** hat.

▼ 84. ★★★ Welches Problem ist von der SQL-Norm in ALLEN Isolation Leveln verboten?

Das **DIRTY WRITE** (= das Lost-Update-Problem).

▼ 85. ★★★ Definieren Sie „Serialisierung“.

„Das Ergebnis **MEHRERER GLEICHZEITIG** ausgeführter Transaktionen ist **IDENTISCH** zu einem Ergebnis, das entstanden wäre, wenn die Transaktionen **NACHEINANDER (SERIELL)** ausgeführt worden wären.“

Block 4 — JDBC, JPA, verteilte DB, NoSQL (86–125)

▼ 86. ★★★ Was ist JDBC, und welche Rolle spielt es als Abstraktionsschicht?

(Original-Beispiellösung) **JDBC** ist die **JAVA-IMPLEMENTIERUNG** des **SQL/CLI-Standards** und dient als **KLASSENBIBLIOTHEK** für den Zugriff auf **SQL-Datenbanken**. Als **ABSTRAKTIONSSCHICHT** ermöglicht es **DATENBANKNEUTRALITÄT**, sodass das **DBMS OHNE GROSSE CODEÄNDERUNGEN AUSGETAUSCHT** werden kann.

▼ 87. ★★★ Nennen Sie die vier JDBC-Treibertypen. Welcher wird in der Vorlesung genutzt?

(Original-Beispiellösung) **Typ 1 (JDBC-ODBC Bridge)**, **Typ 2 (Native-API)**, **Typ 3 (Middleware-Treiber)**, **Typ 4 (Pure Java / Database-Protocol)**. ★ Im Kurs wird der **TYP-4-Treiber** genutzt, der **JDBC-Calls DIREKT** in das Protokoll der Datenbank umsetzt.

▼ 88. ★★ Erläutern Sie „Write Once, Run Anywhere“ im JDBC-Kontext.

(Original-Beispiellösung) Die Anwendung kann UNABHÄNGIG von der konkreten Datenbank implementiert werden. JDBC-TREIBER TRANSFORMIEREN das einheitliche JDBC-SQL in das jeweilige PROPRIETÄRE DBMS-SQL. Der Code kann somit mit ALLEN relationalen Datenbanken verwendet werden, die einen Standard-JDBC-Treiber bereitstellen.

▼ 89. ★★★★★ Skizzieren Sie die typischen Schritte eines JDBC-Programms.

(Original-Beispiellösung) 1. Laden des Treibers · 2. Herstellen der Verbindung · 3. Erzeugen eines Statement-Objekts · 4. Ausführen der Anfrage/Updates · 5. Auswertung der Ergebnisse (bei Abfragen) · 6. Schließen der Verbindung.

▼ 90. ★★★★★ Wann executeQuery(), wann executeUpdate()? Was ist der Rückgabotyp?

(Original-Beispiellösung) executeQuery() wird für SELECT-Abfragen genutzt und liefert ein ResultSet zurück. executeUpdate() dient für DML- (INSERT, UPDATE, DELETE) ODER DDL-Anweisungen und gibt die ANZAHL DER BETROFFENEN ZEILEN (int) zurück.

▼ 91. ★★ Erklären Sie das Cursor-Konzept beim ResultSet.

(Original-Beispiellösung) Ein Cursor fungiert als ITERATOR über die Tupel der Ergebnismenge. Mit der Methode next() springt man zum jeweils nächsten Datensatz, bis das Ende erreicht ist (false). ★ Grund: Kopplung von SQL und Programmiersprache durch unterschiedliche Datenstrukturen (Relation vs. Tupel).

▼ 92. ★★ Warum muss man ResultSet, Statement und Connection schließen?

(Original-Beispiellösung) Werden diese Ressourcen NICHT MANUELL GESCHLOSSEN, werden sie NICHT DIREKT FREIGEgeben, was zu UNNÖTIGEM RESSOURCENVERBRAUCH führen kann.

▼ 93. ★★ Wie deaktiviert man Auto-Commit in JDBC?

(Original-Beispiellösung) Mit conn.setAutoCommit(false) . Manuelle Abschlüsse erfolgen über conn.commit() oder im Fehlerfall über conn.rollback() .

▼ 94. ★★★★★ Was ist SQL Injection, und wie gelangt bösartiger Code ins System?

(Original-Beispiellösung) Bei SQL Injection wird die LOGIK eines SQL-Befehls durch BENUTZEREINGABEN MANIPULIERT (z. B. durch Anhängen von OR 1=1), um unbefugten Zugriff auf Daten zu erhalten oder FILTER ZU UMGEHEN.

▼ 95. ★★★★★ Warum gelten PreparedStatement als sicherer und performanter?

(Original-Beispiellösung) SICHERER: Die Datenbank PRÜFT DIE GÜLTIGKEIT DER PARAMETER VOR DER VERARBEITUNG und erschwert so Injections (schädlicher Code wird als REINER TEXT behandelt). PERFORMANTER: Die Anfrage muss NUR EINMAL optimiert („geparsed“) werden und kann dann MEHRFACH MIT VERSCHIEDENEN PARAMETERN ausgeführt werden.

▼ 96. ★★★★★ **Wie unterscheidet sich der Datenzugriff über REST vom direkten JDBC-Zugriff?**

(Original-Beispiellösung) Beim klassischen Zugriff kommuniziert die Java-Anwendung **DIREKT** via **JDBC** mit der Datenbank und **FORMULIERT SQL SELBST**. Bei REST erfolgt der Zugriff über ein **BACKEND (Webservice)** via **HTTP**; der Client nutzt Methoden wie **GET** oder **POST**, während **SQL IM BACKEND GEKAPSELT** ist.

▼ 97. ★★★★★ **Nennen Sie die drei Schichten der modernen Architektur und ordnen Sie die Geschäftslogik zu.**

(Original-Beispiellösung) **1. PRÄSENTATIONSSCHICHT (Client) · 2. ANWENDUNGSSCHICHT (Backend) · 3. DATENSCHICHT (Datenbank)**. Die **GESCHÄFTSLOGIK** gehört zur **ANWENDUNGSSCHICHT**. ⚠ Der Client hat **KEIN SQL** und **KEINE** Datenbankzugangsdaten.

▼ 98. ★★★★★ **Was ist das Ziel von ORM, und welchen Vorteil bietet es gegenüber JDBC?**

(Original-Beispiellösung) ORM dient als **ABSTRAKTIONSSCHICHT** zwischen Objekten (Java) und **Relationen (RDBMS)**. Der Vorteil ist, dass Entwickler Daten **ALS NATIVE OBJEKTE** behandeln können, **OHNE MANUELLE SQL-STATEMENTS** zu schreiben oder **RESULT-SETS HÄNDISCH ZU MAPPEN**.

▼ 99. ★★★★★ **Erläutern Sie das Verhältnis zwischen JPA und Hibernate.**

(Original-Beispiellösung) **JPA ist die SPEZIFIKATION (der API-Standard)**, während **HIBERNATE die IMPLEMENTIERUNG (der Provider) ist**, die diesen Standard umsetzt. ★ Die dritte Ebene ist das **KONZEPT: ORM** (generische Technik).

▼ 100. ★★★★★ **Was beschreibt der Object-Relational Impedance Mismatch? Nennen Sie zwei Konflikte.**

(Original-Beispiellösung) Er beschreibt **FUNDAMENTALE KONZEPTIONELLE UNTERSCHIEDE** zwischen **Objektmodellen (OO)** und **relationalen Modellen**. Beispiele: der **VERERBUNGSKONFLIKT** (OO nutzt Vererbung, RDBMS nicht nativ) und der **IDENTITÄTSKONFLIKT** (**SPEICHERORT** vs. **PRIMÄRSCHLÜSSEL**). (Weitere: Navigation, Set-Semantik, Granularität, Synchronisation.)

▼ 101. ★★★★★ **Wie interagieren JPA, JDBC und die Datenbank technisch? Wer ruft wen auf?**

(Original-Beispiellösung) **JPA ist die HÖCHSTE Schicht** für den Entwickler. Ein **JPA-PROVIDER** (wie **Hibernate**) **GENERIERT DARAUS SQL** und **NUTZT INTERN IMMER JDBC**, um die Befehle an die Datenbank zu senden.

▼ 102. ★★ **Was ist der Hauptunterschied in der Typsicherheit zwischen JPQL und Criteria API?**

(Original-Beispiellösung) Fehler in **JPQL** werden **ERST ZUR LAUFZEIT (Runtime)** erkannt. Die **Criteria API** bietet hingegen **TYPISICHERHEIT**, wodurch **Syntaxfehler BEREITS ZUR KOMPILIERZEIT** abgefangen werden können.

▼ 103. ★★ Welche Funktion haben @Entity und @Id?

(Original-Beispiellösung) @Entity MARKIERT EINE KLASSE ALS PERSISTENTES DOMÄNENOBJEKT. @Id KENNZEICHNET DIE INSTANZVARIABLE, die von JPA als PRIMÄRSCHLÜSSEL verwendet werden soll.

▼ 104. ★★ Welche zentralen Informationen enthält die persistence.xml?

(Original-Beispiellösung) Sie definiert die „PERSISTENCE UNITS“, welche DATENBANK-ZUGANGSDATEN (URL, User, Passwort), den JPA-PROVIDER, die ZU VERWALTENDEN KLASSEN und STRATEGIEN ZUR SCHEMA-GENERIERUNG enthalten.

▼ 105. ★★★ Beschreiben Sie den Zustand „Managed“ einer Entity.

(Original-Beispiellösung) Im Zustand MANAGED ist die Entity IN DEN PERSISTENCE CONTEXT AUFGENOMMEN worden. ALLE ÄNDERUNGEN am Objekt werden AUTOMATISCH VERFOLGT und BEIM TRANSAKTIONS-COMMIT MIT DER DATENBANK SYNCHRONISIERT. (Die vier Zustände: Transient, Managed, Detached, Removed.)

▼ 106. ★★★ Wann gilt eine Datenbank als verteilte Datenbank?

(Original-Beispiellösung) Eine verteilte DB besteht aus einer LOGISCH ZUSAMMENGEHÖRIGEN Datenbank, deren Daten PHYSISCH AUF MEHRERE, ÜBER EIN NETZWERK VERBUNDENE RECHNER VERTEILT sind.

▼ 107. ★★★ Erklären Sie horizontale und vertikale Fragmentierung.

(Original-Beispiellösung) HORIZONTAL: Die Tabelle wird NACH ZEILEN (Datensätzen) aufgeteilt. VERTIKAL: Die Tabelle wird NACH SPALTEN (Attributen) aufgeteilt, wobei ★ DER PRIMÄRSCHLÜSSEL IN JEDEM FRAGMENT VORHANDEN SEIN MUSS.

▼ 108. ★★ Was ist der Unterschied zwischen Partitionierung und Sharding?

Partitionierung = LOGISCHE Aufteilung der Daten in Teilmengen. Sharding = SPEZIELLE Form der Partitionierung, bei der Partitionen auf VERSCHIEDENE NODES verteilt werden (PHYSISCHE Verteilung).

▼ 109. ★★★ Unterscheiden Sie „Consistency“ in ACID von „Consistency“ im CAP-Theorem.

(Original-Beispiellösung) ACID-Konsistenz bezieht sich auf die INTEGRITÄT: Eine Transaktion überführt die DB von einem GÜLTIGEN Zustand in einen anderen GÜLTIGEN Zustand unter Einhaltung aller Regeln. CAP-Konsistenz bezieht sich auf die EINHEITLICHKEIT: JEDER LESEVORGANG LIEFERT DEN WERT DES JÜNGSTEN SCHREIBVORGANGS (atomare Konsistenz).

▼ 110. ★★★★★ **Benennen und beschreiben Sie die drei Komponenten des CAP-Theorems.**

(Original-Beispiellösung) **Consistency (C): ALLE KNOTEN SEHEN GLEICHZEITIG DIESELBEN DATEN. Availability (A): JEDE ANFRAGE WIRD BEANTWORTET. Partition Tolerance (P): DAS SYSTEM FUNKTIONIERT TROTZ NETZWERKAUSFÄLLEN WEITER.** Aufgestellt 2000 von Eric Brewer (Berkeley), vom MIT bewiesen.

▼ 111. ★★★★★ **Warum kann ein System bei Netzwerkpartitionierung nicht gleichzeitig C und A sein?**

(Original-Beispiellösung) Bei einer Netzwerktrennung (P) können Knoten NICHT KOMMUNIZIEREN. Will man KONSISTENZ (C) wahren, MUSS MAN ANFRAGEN ABLEHNEN (kein A), bis die Daten synchron sind. Will man VERFÜGBAR bleiben (A), RISIKIERT MAN, VERALTETE oder UNTERSCHIEDLICHE DATEN ZU LIEFERN.

▼ 112. ★★★★★ **Vergleichen Sie ACID mit BASE.**

(Original-Beispiellösung) **ACID** garantiert SOFORTIGE, STRIKTE KONSISTENZ nach jeder Transaktion. **BASE** akzeptiert einen VORÜBERGEHEND INKONSISTENTEN Zustand („SOFT STATE“), um eine HÖHERE VERFÜGBARKEIT und SKALIERBARKEIT zu erreichen. BASE = Basically Available, Soft state, Eventually consistent.

▼ 113. ★★★★★ **Was versteht man unter „Eventual Consistency“?**

(Original-Beispiellösung) Das System garantiert NICHT die SOFORTIGE SICHTBARKEIT eines Schreibvorgangs auf allen Knoten. Es wird lediglich garantiert, dass BEI AUSBLEIBEN NEUER UPDATES IRGENDWANN („EVENTUALLY“) ALLE KOPIEN DENSELben WERT AUFWEISEN.

▼ 114. ★★★★★ **Unterscheiden Sie OLTP und OLAP.**

(Original-Beispiellösung) **OLTP** dient der ABWICKLUNG DES TAGESGESCHÄFTS (viele kurze Transaktionen, SCHREIB-Fokus). **OLAP** dient der ENTSCHEIDUNGSUNTERSTÜTZUNG durch Analyse HISTORISCHER Datenbestände (komplexe Abfragen, LESE-Fokus).

▼ 115. ★★★★★ **Warum ermöglicht In-Memory in SAP HANA Echtzeit-Analysen ohne ETL?**

(Original-Beispiellösung) Da die Daten im SCHNELLEN HAUPTSPEICHER (In-Memory) vorliegen und der COLUMN STORE EXTREM SCHNELLE SCAN-VORGÄNGE erlaubt, können Analysen DIREKT AUF DEN AKTUELLEN OPERATIVEN DATEN ausgeführt werden. Dies ELIMINIERT die Notwendigkeit für ZEITAUFWENDIGE ETL-PROZESSE in separate OLAP-Systeme.

▼ 116. ★★★★★ **Nennen Sie die vier analytischen Operationen im Data Cube.**

Slicing (2D-Scheibe, eine Dimension fixiert) · **Dicing** (Teilwürfel, mehrere Dimensionen eingeschränkt) · **Roll-up** (verdichten auf höhere Ebene) · **Drill-down** (aufschlüsseln)

▼ 117. ★★ Warum werden Dimensionstabellen im Sternschema denormalisiert?

Um die **Anzahl der notwendigen JOINS** bei komplexen Analysen zu **MINIMIEREN**. Redundanzen und erhöhter Speicherbedarf werden in Kauf genommen; Update-Anomalien werden akzeptiert, da OLAP-Systeme primär für **lesende Zugriffe** optimiert sind.

▼ 118. ★★★ Erläutern Sie Data-to-Code vs. Code-to-Data.

Data-to-Code (klassisch): Rohdaten werden über das Netz zum Applikationsserver übertragen, dort findet die Logik statt → ⚠ **I/O-Bottleneck**. **Code-to-Data (Code Pushdown):** Die **berechnungsintensive Logik** wird in die Datenbank verlagert; nur das **verdichtete Endergebnis** geht über das Netz. ⚠ **ABER:** Der Application Server bleibt die Instanz für Logik, die nicht mit **Massendaten** zu tun hat.

▼ 119. ★★★ Warum ist „nicht-relational“ fachlich treffender als „NoSQL“?

(Original-Beispiellösung) Weil viele NoSQL-Systeme mittlerweile **SQL-ÄHNLICHE SCHNITTSTELLEN** (z. B. CQL) besitzen, ist **DAS FEHLEN VON SQL NICHT MEHR DAS HAUPTMERKMAL**; **DAS ABWEICHENDE DATENMODELL** HINGEGEN SCHON.

▼ 120. ★★★ Erläutern Sie die drei „Vs“ von Big Data.

(Original-Beispiellösung) **VOLUME** (Menge), **VARIETY** (Vielfalt/Strukturflexibilität), **VELOCITY** (Geschwindigkeit).

▼ 121. ★★ Wie simuliert man im flachen Key-Space eine Hierarchie?

(Original-Beispiellösung) Nutzung von **SONDERZEICHEN** (z. B. User:123:Name), um **LOGISCHE EBENEN FÜR DEN ENTWICKLER SICHTBAR** zu machen. ⚠ Die Datenbank versteht die Logik **NICHT** — die Struktur existiert nur in der Anwendungslogik.

▼ 122. ★★★ Was ist der Vorteil von In-Memory bei Redis, und wie wird Persistenz gewährleistet?

(Original-Beispiellösung) **EXTREM SCHNELL** durch RAM-Nutzung; Persistenz via **RDB-SNAPSHOTS** und **AOF-JOURNALING** auf Festplatte.

▼ 123. ★★★ Warum ist ein Redis Hash performanter als mehrere separate Strings?

(Original-Beispiellösung) **BÜNDELUNG:** Ein Hash erlaubt den **ABRUF ALLER FELDER** eines Objekts **MIT EINEM BEFEHL** (HGETALL), STATT VIELER EINZELNER GET -Aufrufe.

▼ 124. ★★★ Welches Format nutzt MongoDB intern, und was ist der Unterschied zu JSON?

(Original-Beispiellösung) **BSON:** **BINARY JSON**; **EFFIZIENTER ZU VERARBEITEN** und **UNTERSTÜTZT MEHR DATENTYPEN** (z. B. Date).

▼ 125. ★★★★★ Nennen Sie die MongoDB-Entsprechungen für Tabelle, Zeile und Spalte.

(Original-Beispiellösung) **Tabelle → COLLECTION · Zeile → DOCUMENT · Spalte → FIELD.**

Block 5 — NoSQL vertieft (126–140)

▼ 126. ★★★★★ Wie stellt MongoDB die Eindeutigkeit sicher, wenn kein Primärschlüssel definiert ist?

(Original-Beispiellösung) **OBJECTID: AUTOMATISCHE GENERIERUNG eines eindeutigen `_id`-Feldes durch MongoDB.**

▼ 127. ★★★★★ Was versteht man unter einer „Column Family“?

(Original-Beispiellösung) **Eine GRUPPE VON SPALTEN, die PHYSIKALISCH ZUSAMMEN GESPEICHERT werden, um den ZUGRIFF ZU OPTIMIEREN.**

▼ 128. ★★★★★ Erklären Sie den Unterschied zwischen „Model First“ (SQL) und „Query First“ (Cassandra).

(Original-Beispiellösung) **QUERY FIRST: Das TABELLENDESIGN wird STRIKT NACH DEN BENÖTIGTEN ABFRAGEN entworfen, DA JOINS FEHLEN. (In SQL: erst die Struktur normalisieren, dann beliebig abfragen.)**

▼ 129. ★★★★★ Wie erreicht Cassandra lineare horizontale Skalierbarkeit?

(Original-Beispiellösung) **SHARDING/HASHING: DATENVERTEILUNG ÜBER KNOTEN via PARTITION KEY und HASHING (MURMUR3); EINFACHES HINZUFÜGEN VON NODES möglich. (Neue Nodes lösen automatisch ein Rebalancing aus.)**

▼ 130. ★★★★★ Was ist ein UPSERT?

(Original-Beispiellösung) **JEDE SCHREIBOPERATION WIRD EINFACH GESPEICHERT; EXISTIERT DER KEY, WIRD ER ÜBERSCHRIEBEN, SONST NEU ANGELEGT (optimiert Schreibdurchsatz). ⚠️ Nachteil: kein Schutz vor versehentlichem Überschreiben, kein Fehler bei doppeltem Schlüssel.**

▼ 131. ★★★★★ Wann ist eine Graphdatenbank anderen NoSQL-Typen vorzuziehen?

(Original-Beispiellösung) **VERNETZUNG: Wenn die BEZIEHUNGEN zwischen Datensätzen WICHTIGER SIND ALS DIE DATENSÄTZE SELBST (Social Media, Routing).**

▼ 132. ★★★★★ Welches Performance-Problem lösen Graphdatenbanken durch „Indexfreie Adjazenz“?

(Original-Beispiellösung) **PERSISTIERTE BEZIEHUNGEN: Beziehungen werden PHYSISCH ALS POINTER GESPEICHERT, NICHT ERST ZUR LAUFZEIT VIA JOIN BERECHNET. ★ Damit entfällt die „JOIN BOMB“ — bei steigender Abfragetiefe wächst der Rechenaufwand in RDBMS exponentiell, in Neo4j bleibt die Traversierung konstant.**

▼ 133. ★★★ Inwiefern unterscheidet sich Neo4j bezüglich ACID von anderen NoSQL-Systemen?

(Original-Beispiellösung) **VOLL ACID-KONFORM**: Im Gegensatz zu vielen NoSQL-Systemen garantiert Neo4j **STRIKTE TRANSAKTIONSSICHERHEIT**. (Andere nutzen meist BASE.)

▼ 134. ★★★ Was ist das Kernprinzip eines Polystores?

(Original-Beispiellösung) **HETEROGENITÄT**: Daten **BLEIBEN** in **SPEZIALISIERTEN ENGINES** (SQL für Transaktionen, Graph für Beziehungen) und werden **ÜBER EINE SCHNITTSTELLE INTEGRIERT** („BEST OF BREED“). Ansatz: „One size does not fit all“.

▼ 135. ★★★ Was ist der Unterschied zwischen einem Polystore und einer Multi-Model-Datenbank?

Polystore: Verbund **MEHRERER** unterschiedlicher DB-Engines, Daten liegen **physisch getrennt** (BigDAWG, Presto/Trino). **Multi-Model-DB**: **EINE Engine unterstützt mehrere Modelle**, Daten in **einem monolithischen System** (Azure CosmosDB, ArangoDB, PostgreSQL).

▼ 136. ★★★ Was sind die drei Cassandra-Consistency-Level?

CONSISTENCY ONE (ein Node reicht) · **CONSISTENCY QUORUM** (Mehrheit) · **CONSISTENCY ALL** (alle Replikate). ★ **Damit ist starke Konsistenz (CP) auch in einem AP-System optional erreichbar**.

▼ 137. ★★★ Warum verwendet Cassandra UUIDs statt fortlaufender Nummern?

Weil in einem **verteilten System** zwei Server **nicht wüssten**, wer die **Nummer 4** und wer die **Nummer 5** bekommt, ohne sich **ständig abzusprechen**. Eine **UUID** wird **zufällig generiert** (128 Bit) — die Kollisionswahrscheinlichkeit ist praktisch null.

▼ 138. ★★★ Welches sind die drei Cypher-Grundelemente?

MATCH (Suchmuster deklarieren) · **WHERE** (filtern) · **RETURN** (Ergebnis bereitstellen, ggf. aggregiert). Syntax: **Knoten in ()**, **Kanten in []** innerhalb von Pfeilen **-->** (ASCII-Art).

▼ 139. ★★★ Was ist der Unterschied zwischen DELETE und DETACH DELETE in Neo4j?

DELETE löscht **nur den Knoten** und **schlägt fehl**, wenn noch **Beziehungen existieren**. **DETACH DELETE** löscht den **Knoten inkl. aller Beziehungen**. ★ **Sicherheitsmechanismus**: Neo4j verhindert „verwaiste“ Kanten (referenzielle Integrität).

▼ 140. ★★★ Unterscheiden Sie hierarchische und Netzwerk-Datenbanken.

Hierarchisch (IMS): **baumorientiert**, **nur 1:n**, ein Child-Record kann nur existieren, wenn der Parent existiert; Verbindung **explizit** (nicht über Werte). **Netzwerk (CODASYL)**: Strukturkomponenten **RECORD** und **SET** (Owner-Member); erlaubt **n:m-Beziehungen** — ein Record kann **MEHRERE Vorgänger haben**; **unterschiedliche Suchwege**. ★ **Beide: navigierender (programmierter) Zugriff**.

TEIL M — KLAUSUR-SPICKZETTEL

M.1 Die SQL-Befehlsübersicht (aus dem Modulmaterial) ★★

Querying Data

```
SELECT c1, c2 FROM t;           -- Spalten abfragen
SELECT * FROM t;               -- alle Spalten/Zeilen
SELECT c1, c2 FROM t WHERE condition; -- Zeilen filtern
SELECT DISTINCT c1 FROM t WHERE condition; -- nur eindeutige Werte
SELECT c1, aggregate(c2) FROM t GROUP BY c1; -- gruppieren
SELECT c1, aggregate(c2) FROM t GROUP BY c1
HAVING condition;             -- Gruppen filtern
SELECT c1, c2 FROM t ORDER BY c1 ASC [DESC]; -- sortieren
SELECT c1, c2 FROM t ORDER BY c1 LIMIT n OFFSET off; -- Pagination
```

Joins & Multiple Tables

```
SELECT c1, c2 FROM t1 INNER JOIN t2 ON condition; -- Inner Join
SELECT c1, c2 FROM t1 LEFT JOIN t2 ON condition; -- Left Join
SELECT c1, c2 FROM t1 RIGHT JOIN t2 ON condition; -- Right Join
SELECT c1, c2 FROM t1 FULL OUTER JOIN t2 ON condition; -- Full Outer Join
```

Set-Operatoren

```
... UNION [ALL] ...           -- Ergebnismengen vereinen
... INTERSECT ...             -- Schnittmenge
... EXCEPT ...              -- Differenz
```

Where-Operatoren

```
c1 [NOT] LIKE pattern         -- Mustervergleich (% = beliebig viele, _ = genau eins)
c1 [NOT] IN (...)             -- Listenprüfung
c1 BETWEEN low AND high       -- Bereich (inklusive!)
c1 IS [NOT] NULL              -- Null-Wertprüfung
```

Managing Tables

```
CREATE TABLE t (...);
ALTER TABLE t ADD column;
ALTER TABLE t DROP COLUMN c;
ALTER TABLE t ALTER COLUMN c TYPE VARCHAR(100);
ALTER TABLE t ADD CONSTRAINT chk_value CHECK (c > 0);
ALTER TABLE t1 RENAME TO t2;
ALTER TABLE t1 RENAME c1 TO c2;
DROP TABLE t CASCADE;
```

Modifying Data

```
INSERT INTO t (...) VALUES (...);
INSERT INTO t VALUES (...), (...);           -- mehrere Zeilen
INSERT INTO t1 SELECT ... FROM t2;           -- aus anderer Tabelle
UPDATE t SET ... WHERE condition;
DELETE FROM t WHERE ...;
```

Constraints

```
PRIMARY KEY (c1, c2)
FOREIGN KEY (c2) REFERENCES t2(c2)
UNIQUE (c2, c3)
CHECK (c1 > 0 AND c1 >= c2)
c2 VARCHAR NOT NULL
```

Indexes / Views / Triggers

```
CREATE INDEX      idx_name ON t(c1, c2);
CREATE UNIQUE INDEX idx_name ON t(c3, c4);
DROP INDEX idx_name;

CREATE VIEW v AS SELECT ...;
CREATE VIEW v AS ... WITH CHECK OPTION;
DROP VIEW view_name;

CREATE TRIGGER ... BEFORE INSERT ON t ...;
DROP TRIGGER trigger_name;
```

Aggregate Functions

```
AVG()   COUNT()   SUM()   MAX()   MIN()
```

M.2 Die Merkstrukturen ★★

Thema	Merkstruktur
DBS	DBS = DB + DBMS
3-Schichten (ANSI-SPARC, 1975)	extern · logisch · physisch — die logische Schicht ist unabhängig von der physischen
ER-Modell (Chen, 1976)	Entitäten · Beziehungen · Attribute
Relation	keine doppelten Tupel · Tupel ungeordnet · Attribute ungeordnet · alle Attribute atomar
SQL-Kategorien	DDL · DML · DQL · TCL · DCL
Rel. Algebra ↔ SQL	Selektion = WHERE (Zeilen) · Projektion = SELECT (Spalten)
Primärschlüssel	UNIQUE + NOT NULL (+ automatisch indiziert)
ON DELETE	NO ACTION/RESTRICT · CASCADE · SET NULL
SELECT-Reihenfolge	FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
JOIN-Bedingungen	n Tabellen → mindestens (n-1) Bedingungen
View änderbar?	1 Relation · kein GROUP BY · kein DISTINCT · keine Berechnungen · kein UNION/INTERSECT/EXCEPT
Function vs. Procedure	Function: Wert zurück, in SQL nutzbar, KEIN COMMIT · Procedure: CALL, OUT-Parameter, COMMIT erlaubt
Trigger	BEFORE/AFTER · INSERT/UPDATE/DELETE · FOR EACH ROW/STATEMENT · EXECUTE FUNCTION
ACID	Atomicity · Consistency · Isolation · Durability
Isolation Level	READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE
Anomalien	Dirty Write (immer verboten) · Dirty Read · Non Repeatable Read · Phantom
2PL	Wachstumsphase (nur anfordern) → Schrumpfphase (nur freigeben) → Isolation
2PC	Prepare-Phase → Commit-Phase → Atomarität im verteilten Fall
JDBC-Schritte	Treiber · Verbindung · Statement · Ausführen · Auswerten · Schließen
JDBC-Treibertypen	1: ODBC-Bridge · 2: Native-API · 3: Middleware · 4: Pure Java ★
3-Schichten-Architektur	Präsentation (Client) · Anwendung (Backend, Geschäftslogik) · Daten (DB)
ORM-Ebenen	Konzept ORM · Spezifikation JPA · Implementierung Hibernate
Entity-Lebenszyklus	Transient · Managed · Detached · Removed
Data Cube	Slicing · Dicing · Roll-up · Drill-down
CAP	Consistency · Availability · Partition tolerance — nur zwei von drei
BASE	Basically Available · Soft state · Eventually consistent
Big Data 3V	Volume · Variety · Velocity
NoSQL-Typen	Key-Value · Document · Wide-Column · Graph

M.3 Die Vergleichstabellen, die man auswendig braucht ★★ ★

NoSQL-Typen und ihre Einsatzszenarien

Typ	Modell	Einsatz	Beispiele
Key-Value	Key-Value	Session-Management, Caching, High-Speed-Lookups	Redis, DynamoDB
Document	JSON/BSON	CMS, E-Commerce, Produktkataloge	MongoDB, Couchbase
Wide-Column	Spaltenfamilien	Time-Series, Logging, IoT, Analytics	Cassandra, HBase
Graph	Knoten + Kanten	Social Graphs, Recommendations, Fraud Detection	Neo4j, Amazon Neptune

RDBMS ↔ MongoDB ↔ Cassandra ↔ Neo4j

RDBMS	MongoDB	Cassandra	Neo4j
Datenbank	Datenbank	Keyspace	Datenbank
Tabelle	Collection	Tabelle (Column Family)	(Label)
Zeile	Document	Zeile (in Partition)	Knoten
Spalte	Field	Spalte	Property
SQL	MQL	CQL	Cypher
JOIN	(Nested Docs)	⚠ keine	Traversierung
ACID	eingeschränkt	⚠ nein (BASE)	★ ja, voll

ACID vs. BASE

	ACID	BASE
Konsistenz	stark, sofort	eventual
Priorität	Korrektheit	Verfügbarkeit + Skalierbarkeit
CAP	CP	AP
Skalierung	vertikal	horizontal
Systeme	RDBMS, SAP HANA, Neo4j	Cassandra, DynamoDB, Riak

Row-Store vs. Column-Store

	Row-Store	Column-Store
Speicherung	zeilenweise	spaltenweise
Lesen	immer alle Attribute	nur die benötigte Spalte
Geeignet für	OLTP	OLAP
Kompression	gering	★ extrem hoch

M.4 Die 20 klassischen Fallen ⚠️⚠️

#	Falle	Richtig
1	<code>WHERE spalte = NULL</code>	<code>IS NULL / IS NOT NULL</code>
2	<code>UPDATE / DELETE</code> ohne <code>WHERE</code>	betrifft ALLE Zeilen!
3	Nicht gruppierte Spalte im <code>SELECT</code> bei <code>GROUP BY</code>	nur gruppierte Spalten + Aggregate
4	Aggregatfunktion in <code>WHERE</code>	gehört in <code>HAVING</code>
5	<code>LIMIT</code> ohne <code>ORDER BY</code>	nicht deterministisch!
6	Alias in <code>WHERE</code> verwenden	geht nur in ORDER BY (Reihenfolge!)
7	Filterbedingung im <code>WHERE</code> statt <code>ON</code> beim <code>OUTER JOIN</code>	macht den OUTER JOIN zum INNER JOIN
8	<code>COUNT(*) = COUNT(spalte)</code> gleichsetzen	<code>COUNT(spalte)</code> ignoriert NULLs
9	Aggregation über mehrere 1:n-Ebenen ohne <code>DISTINCT</code>	<code>COUNT(DISTINCT ...)</code> verwenden
10	<code>NOT IN</code> mit möglichen NULLs in der Subquery	<code>NOT EXISTS</code> oder <code>LEFT JOIN ... IS NULL</code>
11	<code>SERIAL</code> für Standard-SQL halten	PostgreSQL-spezifisch!
12	<code>REAL / DOUBLE</code> für Geldbeträge	<code>NUMERIC / DECIMAL</code>
13	Datumsdifferenz mit <code>TIMESTAMP</code> ohne Cast	<code>::DATE</code> bzw. <code>CAST(... AS DATE)</code>
14	Trigger-Funktion ohne <code>RETURNS TRIGGER</code>	zwingend erforderlich
15	<code>RETURN NEW</code> im <code>BEFORE</code> -Trigger vergessen	Operation wird abgebrochen
16	<code>COMMIT</code> in einer Function	⚠️ nur in PROCEDURES erlaubt
17	<code>RETURN</code> in <code>LANGUAGE SQL</code> erwarten	dort implizit über die letzte Anweisung
18	Statement statt PreparedStatement bei Nutzereingaben	⚠️ SQL Injection!
19	2PL und 2PC verwechseln	2PL = Isolation · 2PC = Atomarität (verteilt)
20	ACID-Consistency mit CAP-Consistency gleichsetzen	Integrität vs. Einheitlichkeit der Replikate

M.5 Die Klausur-Kochrezepte ★★★★★

Rezept: „Erstellen Sie die Tabelle X mit allen Integritätsbedingungen“

```
CREATE TABLE album (
    album_id INT NOT NULL,
    title VARCHAR(160) NOT NULL,
    artist_id INT NOT NULL,
    CONSTRAINT album_pkey PRIMARY KEY (album_id),
    CONSTRAINT album_fk FOREIGN KEY (artist_id) REFERENCES artist(artist_id)
);
```

Checkliste: ① Alle Spalten mit **Datentyp** · ② **NOT NULL** wo nötig · ③ **PRIMARY KEY** · ④ **FOREIGN KEY ... REFERENCES** · ⑤ ggf. **UNIQUE / CHECK / DEFAULT**

Rezept: „Was ist das Ergebnis dieser Abfrage?“

1. **FROM** — welche Tabelle(n)? Bei Join: welche Zeilen passen zusammen?
2. **WHERE** — welche Zeilen bleiben? (⚠️ NULL-Regeln!)
3. **GROUP BY** — welche Gruppen entstehen?
4. **Aggregate berechnen** (⚠️ NULLs werden ignoriert!)
5. **HAVING** — welche Gruppen bleiben?
6. **SELECT** — welche Spalten werden ausgegeben?
7. **ORDER BY / LIMIT** — Reihenfolge und Anzahl
8. ★ **Das Ergebnis als TABELLE mit Spaltenüberschriften hinschreiben!**

Rezept: „Erstellen Sie einen Trigger, der ...“

```
-- ① Trigger-Funktion
CREATE OR REPLACE FUNCTION mein_trigger_fn()
RETURNS TRIGGER          -- ★ zwingend
LANGUAGE plpgsql
AS $$
DECLARE
    hilfsvariable typ;      -- nur wenn nötig
BEGIN
    IF <Bedingung> THEN
        NEW.spalte := <Wert>;    -- Wert setzen
        -- oder: RAISE EXCEPTION 'Fehlermeldung';
    END IF;
    RETURN NEW;              -- ★ zwingend bei BEFORE
END;
$$;

-- ② Trigger binden
CREATE TRIGGER mein_trigger
BEFORE INSERT ON tabelle    -- Zeitpunkt + Ereignis + Tabelle
FOR EACH ROW               -- Granularität
EXECUTE FUNCTION mein_trigger_fn();
```

Rezept: „Ergänzen Sie das Java-Programm“

```
// SELECT
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery("SELECT spalte FROM tabelle;");
while (rs.next()) {
    System.out.println(rs.getInt("id") + " " + rs.getString("name"));
}

// INSERT / UPDATE / DELETE / DDL
int rows = st.executeUpdate("UPDATE tabelle SET spalte = 'wert' WHERE id = 1;");

// Sicher gegen SQL Injection
PreparedStatement pst = conn.prepareStatement("SELECT * FROM t WHERE name = ?");
pst.setString(1, eingabe);
ResultSet rs = pst.executeQuery();

// Transaktion
conn.setAutoCommit(false);
...
conn.commit();    // bzw. conn.rollback();
```

Rezept: „Formulieren Sie eine Abfrage für ...“

Formulierung in der Aufgabe	SQL-Baustein
„alle ...“	SELECT * FROM t
„... sortiert nach ...“	ORDER BY
„einzigartige/unterschiedliche ...“	DISTINCT bzw. COUNT(DISTINCT ...)
„wie viele ...“	COUNT(*)
„... pro ...“	GROUP BY
„... mit mehr als n ...“	GROUP BY ... HAVING COUNT(*) > n
„... beginnt mit / enthält ...“	LIKE 'X%' / LIKE '%X%'
„... zwischen ... und ...“	BETWEEN ... AND ...
„... ohne / kein ...“	IS NULL bzw. LEFT JOIN ... IS NULL bzw. NOT IN
„... über dem Durchschnitt ...“	> (SELECT AVG(...) FROM ...)
„... je Gruppe der/die ...“	korrelierte Subquery
„... sowohl ... als auch ...“	GROUP BY ... HAVING COUNT(DISTINCT ...) = 2
„vereinigte Liste aus zwei Tabellen“	UNION
„... die nicht in ... sind“	EXCEPT
„die n neuesten/größten ...“	ORDER BY ... DESC LIMIT n

M.6 Die wichtigsten Zahlen und Jahreszahlen

Zahl	Bedeutung
1970	Codd — <i>A Relational Model of Data for Large Shared Data Banks</i>
1975	ANSI-SPARC 3-Schichten-Modell
1976	Chen — ER-Modell
1986 / 1992 / 1999	SQL-1 / SQL-2 (JOINS, ON DELETE) / SQL-3 (Trigger, OO, Rekursion)
1996	SQL/PSM · Umbenennung in PostgreSQL
1998	Terminus NoSQL
2000	Brewer — CAP-Vermutung · Beginn Neo4j
2001 / 2006	Hibernate / JPA
2006/2008	Google BigTable
2007 / 2009	MongoDB (10gen) / Redis (Sanfilippo)
2008 / 2010	Cassandra (Facebook) / SAP HANA
4	JDBC-Treibertypen (★ Typ 4 wird genutzt) · Isolation Level · Entity-Zustände · NoSQL-Hauptkategorien · Eigenschaften einer Relation
3	Schichten (ANSI-SPARC und Anwendungsarchitektur) · V's von Big Data · ORM-Abstraktionsebenen
5	SQL-Befehlskategorien · Aggregatfunktionen
6379	★ Redis-Standardport
5432	★ PostgreSQL-Standardport
16	logische Redis-Datenbanken (0–15)
11	Tabellen der Chinook-Datenbank

Quellenverzeichnis

Quelle: ZIP-Archiv WDS224 - Moderne Datenbankkonzepte -Praktische Umsetzung (SCA).zip **Autor der Folien:** © 2023–2026 **Andreas Scheub**

Literaturverzeichnis der Vorlesung:

- A. Meier, *Werkzeuge der digitalen Wirtschaft: Big Data, NoSQL & Co., essentials*, DOI 10.1007/978-3-658-20337-5_2
- A. Meier, M. Kaufmann (Hrsg.), *SQL- & NoSQL-Datenbanken*, eXamen.press, DOI 10.1007/978-3-662-47664-2_1
- M. Unterstein, G. Matthiessen, *Anwendungsentwicklung mit Datenbanken*, eXamen.press, DOI 10.1007/978-3-642-39003-6_1
- S. Cordts et al., *Datenbanken für Wirtschaftsinformatiker*, DOI 10.1007/978-3-8348-8192-2_1
- **E. Schicker, *Datenbanken und SQL, Informatik & Praxis***, DOI 10.1007/978-3-8348-2185-0_1 (*die am häufigsten zitierte Quelle: S. 26, 45 ff., 104, 108 ff., 132, 138, 290, 291, 301, 313*)
- R. Steiner, *Grundkurs Relationale Datenbanken*, DOI 10.1007/978-3-658-17979-3_1
- M. Unterstein, G. Matthiessen, *Relationale Datenbanken und SQL in Theorie und Praxis*, DOI 10.1007/978-3-642-28986-6_1
- Ralf Adams, *SQL — Eine Einführung mit vertiefenden Exkursen*, ISBN 978-3-446-43278-9
- Farkisch (2011), *Data-Warehouse-Systeme kompakt*, S. 29, 40 f.
- Vey/Bachmeier/Krutov, *SAP In-Memory Computing on IBM eX5 Systems*, 2012, S. 12, 14
- Ranking: <https://db-engines.com/de/ranking>

Anmerkungen zur Vollständigkeit:

- Alle **Wiederholungs- und Transferfragen** der sieben Foliensätze sind mit den **Original-Beispiellösungen des Dozenten** in Teil L enthalten.
- Alle **Übungsblätter mit Musterlösungen** (DDL 1 und 2, DQL 1 und 2, Zusatzthemen, DB-interne Programmierung, JDBC, Redis/MongoDB samt Challenge) sind in Teil K vollständig wiedergegeben.
- Einige Folien bestehen ausschließlich aus **Screenshots/Grafiken** (DBeaver-Oberflächen, Cassandra-Demo-Ausgaben, Neo4j-Visualisierungen, Ranking-Diagramme). Deren Inhalt ist aus dem Kontext rekonstruiert und im Text als solcher kenntlich gemacht.
- Zum Übungsblatt **JDBC** sind in den Musterlösungen die vollständigen Java-Programme mit Imports enthalten; hier sind die **relevanten Codeblöcke** ohne die wiederholten Imports und Verbindungsaufbauten aufgeführt.

Ende Modul 13 — Moderne Datenbankkonzepte: Praktische Umsetzung (SCA) · 8 Foliensätze · 12 Übungsblätter mit Musterlösungen · 140 Selbsttestfragen